

Computer Programming I (Java)

A complete study manual, from your first line of code to a perfect score

Every idea is built from zero and nothing is used before it is explained. Every program in this book was **compiled and run**, and every output you see printed here is what the machine actually printed. You will not need any other book, any website, or any documentation.

```
C:\java> javac Hello.java
C:\java> java Hello
Hello, world!
C:\java> _
```

Contents

Page numbers are generated from the rendered book, not typed by hand. Every line below is a link.

START HERE · HOW TO USE THIS BOOK	4
FOUNDATIONS	7
F.1 · WHAT A COMPUTER ACTUALLY DOES	7
F.2 · SOURCE CODE	8
F.3 · FROM YOUR TYPING TO A RUNNING PROGRAM	8
F.4 · GETTING JAVA ONTO YOUR COMPUTER	10
F.5 · YOUR FIRST PROGRAM	11
F.6 · THE ANATOMY OF A PROGRAM	12
F.7 · YOUR FIRST ERRORS	14
F.8 · VARIABLES	15
F.9 · THE DRY RUN	16
F.10 · ARRAYS, EARLY AND HONEST	18
F.11 · JOPTIONPANE, EARLY AND HONEST	20
MODULE ONE INTRODUCTION TO COMPUTER PROGRAMMING	23
UNIT 1.1 · OVERVIEW OF COMPUTER PROGRAMMING	23
UNIT 1.2 · PROGRAMMING PARADIGMS	24
UNIT 1.3 · CLASSES, OBJECTS, ATTRIBUTES AND METHODS	26
MODULE TWO JAVA SETUP	29
UNIT 2.1 · INTRODUCTION TO JAVA	29
UNIT 2.2 · THE JAVA DEVELOPMENT ENVIRONMENT	30
MODULE THREE JAVA FUNDAMENTALS	33
UNIT 3.1 · THE FOUR BUILDING BLOCKS	33
UNIT 3.2 · VARIABLES AND THE RULES FOR NAMING THEM	34
UNIT 3.3 · DATA TYPES	35
UNIT 3.4 · CONSTANTS	41
UNIT 3.5 · OPERATORS · THE EIGHT FAMILIES	42
UNIT 3.6 · ARITHMETIC OPERATORS	43
UNIT 3.7 · RELATIONAL OPERATORS	48
UNIT 3.8 · LOGICAL OPERATORS	49
UNIT 3.9 · OPERATOR PRECEDENCE	51
UNIT 3.10 · JAVA INPUT	52
UNIT 3.11 · JAVA OUTPUT	55
MODULE FOUR JAVA CONTROL STRUCTURES	60
UNIT 4.1 · WHAT CONTROL STRUCTURES ARE	60
UNIT 4.2 · IF, IF-ELSE, AND THE LADDER	61
UNIT 4.3 · THE SWITCH STATEMENT	63
UNIT 4.4 · NESTED IF · THE ONE THE EXAM DEMANDS	66
UNIT 4.5 · THE THREE LOOPS	69
UNIT 4.6 · CONVERTING BETWEEN LOOPS · WORTH 10 MARKS	71
UNIT 4.7 · BRANCHING · BREAK, CONTINUE, RETURN	74
MODULE FIVE METHODS	78
UNIT 5.1 · WHAT A METHOD IS	78
UNIT 5.2 · VOID AND NON-VOID METHODS	80
UNIT 5.3 · PARAMETERS AND ARGUMENTS	82
UNIT 5.4 · PARAMETER PASSING · READ THIS TWICE	83
UNIT 5.5 · STATIC AND INSTANCE METHODS	85

UNIT 5.6 · METHOD OVERLOADING	86
UNIT 5.7 · TYPE PROMOTION	88
MODULE SIX OBJECT-ORIENTED PROGRAMMING	90
UNIT 6.1 · WHAT OOP IS	90
UNIT 6.2 · CLASSES AND OBJECTS	91
UNIT 6.3 · CONSTRUCTORS	92
UNIT 6.4 · ENCAPSULATION	95
UNIT 6.5 · ABSTRACTION	97
UNIT 6.6 · INHERITANCE	100
UNIT 6.7 · POLYMORPHISM	106
MODULE SEVEN STRINGS AND STRING PROCESSING	109
UNIT 7.1 · WHAT A STRING IS	109
UNIT 7.2 · THE STRING METHODS	111
UNIT 7.3 · == VERSUS EQUALS · THE FAMOUS ONE	112
UNIT 7.4 · STRINGBUILDER AND STRINGBUFFER	114
MODULE EIGHT ARRAYS AND ARRAY MANIPULATION	117
UNIT 8.1 · ARRAYS, PROPERLY	117
UNIT 8.2 · TWO-DIMENSIONAL ARRAYS	119
UNIT 8.3 · THE ARRAYS TOOLKIT	120
UNIT 8.4 · THE EXAM'S ARRAY TECHNIQUES	122
MODULE NINE RECURSION	129
UNIT 9.1 · WHAT RECURSION IS	129
UNIT 9.2 · THE STANDARD RECURSIONS	132
UNIT 9.3 · RECURSION VERSUS ITERATION	133
MODULE TEN EXCEPTION HANDLING AND FILE I/O	136
UNIT 10.1 · EXCEPTIONS	136
UNIT 10.2 · TRY, CATCH, FINALLY	138
UNIT 10.3 · FILE I/O	142
SOLVED PAST PAPERS	150
S.1 · 2025/26 · HOW THIS PAPER WORKS	150
S.2 · 2025/26 · QUESTION ONE · 20 MARKS	151
S.3 · 2025/26 · QUESTION TWO · 20 MARKS	157
S.4 · 2025/26 · QUESTION THREE · 25 MARKS	161
S.4 · 2025/26 · QUESTION THREE B	164
S.5 · 2025/26 · QUESTION FOUR · 25 MARKS	167
S.6 · 2025/26 · QUESTION FIVE · 25 MARKS	173
S.6 · 2025/26 · QUESTION FIVE B	176
S.7 · 2025/26 · QUESTION SIX · 25 MARKS	179
S.7 · 2025/26 · QUESTION SIX B	183
S.8 · 2024/25 · HOW THIS PAPER WORKS	187
S.9 · 2024/25 · QUESTION ONE · 17.5 MARKS	187
S.10 · 2024/25 · QUESTION TWO · 17.5 MARKS	191
S.11 · 2024/25 · QUESTION THREE · 17.5 MARKS	196
S.12 · 2024/25 · QUESTION FOUR · 17.5 MARKS	202
S.13 · 2024/25 · QUESTION FIVE · 17.5 MARKS	206
S.14 · 2024/25 · QUESTION SIX · 17.5 MARKS	210
MOCK PAPERS	216
M.1 · MOCK A · BEFORE YOU START	216
M.2 · MOCK A · QUESTION ONE · 17.5 MARKS	217
M.3 · MOCK A · QUESTION TWO · 17.5 MARKS	218

M.4 · MOCK A · QUESTION THREE · 17.5 MARKS	219
M.5 · MOCK A · QUESTION FOUR · 17.5 MARKS	220
M.6 · MOCK A · QUESTION FIVE · 17.5 MARKS	221
M.7 · MOCK A · QUESTION SIX · 17.5 MARKS	222
M.8 · MOCK B · BEFORE YOU START	223
M.9 · MOCK B · SECTION A · QUESTION ONE · 20 MARKS	224
M.10 · MOCK B · SECTION A · QUESTION TWO · 20 MARKS	225
M.11 · MOCK B · SECTION B · QUESTION THREE · 25 MARKS	226
M.12 · MOCK B · SECTION B · QUESTION FOUR · 25 MARKS	227
M.13 · MOCK B · SECTION C · QUESTION FIVE · 25 MARKS	228
M.14 · MOCK B · SECTION C · QUESTION SIX · 25 MARKS	230
MOCK PAPER SOLUTIONS	232
MS.1 · MOCK A SOLUTIONS · MARK YOUR OWN PAPER	232
MS.2 · MOCK A SOLUTIONS · QUESTION ONE	233
MS.3 · MOCK A SOLUTIONS · QUESTION TWO	236
MS.4 · MOCK A SOLUTIONS · QUESTION THREE	241
MS.5 · MOCK A SOLUTIONS · QUESTION FOUR	246
MS.6 · MOCK A SOLUTIONS · QUESTION FIVE	250
MS.7 · MOCK A SOLUTIONS · QUESTION SIX	254
MS.8 · MOCK B SOLUTIONS · SECTION A, QUESTION ONE	258
MS.9 · MOCK B SOLUTIONS · SECTION A, QUESTION TWO	261
MS.10 · MOCK B SOLUTIONS · SECTION B, QUESTION THREE	264
MS.11 · MOCK B SOLUTIONS · SECTION B, QUESTION FOUR	269
MS.12 · MOCK B SOLUTIONS · SECTION C, QUESTION FIVE	272
MS.13 · MOCK B SOLUTIONS · SECTION C, QUESTION SIX	277
REFERENCE	286
R.1 · REFERENCE · THE EXAM API CARD	286
R.2 · REFERENCE · THE METHOD REFERENCE	288
R.3 · REFERENCE · THE KEYWORD INDEX	293
R.4 · REFERENCE · THE GLOSSARY	295

START HERE · HOW TO USE THIS BOOK

How to use this book

Read this page. It is two minutes long and it is the difference between finishing this book able to write Java and finishing it able to recognise Java.

What this book promises

You may never have written a line of code in your life. That is the reader this book is written for. It starts from what a computer actually does when it runs a program, and it ends with you sitting a three hour paper and writing correct Java on paper, by hand, under time pressure.

Two promises follow from that, and they shape every page:

- ▶ **Nothing is used before it is explained.** No keyword, no symbol, no method appears in this book before the page that introduces it. If you meet something you do not recognise, that is a defect in the book, not a gap in you.
- ▶ **You will not need anything else.** No other textbook, no website, no official documentation, no lecturer. Every library method this course uses is documented in here, in full, at the point you need it, and again in the reference at the back.

TEACH · READING IS NOT STUDYING

Reading code feels productive and teaches you almost nothing. You will read a worked example, think "yes, obviously", turn the page, and be unable to reproduce a line of it. This is the single most common way people fail a programming exam: they have *seen* everything and can *write* nothing.

The cure is retrieval: closing the book and producing the answer from your own head. Every device in this book exists to force that. It will feel slower and less pleasant than reading. It is roughly three times more effective per hour, and it is the only thing the exam actually measures.

The boxes, and what each one wants from you

Every box has exactly one job. Learn the colours once and you can navigate this book by shape and colour alone.

BOX	ITS ONE JOB	WHAT YOU DO
TEACH	One idea, in plain language.	Read it once, slowly. If it has two ideas in it, that is a bug.
CODE	A real listing, with numbered lines.	Read every line. Every listing here was compiled and run.
DRY RUN	A trace table: the program executed by hand, one row per step.	Cover the table and trace it yourself first. The exam asks for exactly this.
WORKED EXAMPLE	A full problem solved, with the reasoning on every line.	Then redo it on blank paper with the book shut.
SYNTAX	The exact shape of a construct, every part named.	Copy it out by hand once. Muscle memory.
TRAP	The mistake made every single year.	Read it twice. These are free marks, won or lost.
RECALL AND CHECK	A question, with the answer hidden below a dashed line.	Answer out loud <i>before</i> your eye crosses the line.
FILL IN THE GAP	Code with holes in it.	Write the missing part. In pen. Guessing is allowed; skipping is not.
MUST-MEMORISE	The small number of things you simply have to know cold.	Memorise. Each one carries a memory hook.

● MUST-MEMORISE · THE COLOUR THAT MEANS "LEARN THIS"

This cyan is reserved. It appears in this book **only** on MUST-MEMORISE boxes and nowhere else, ever. If you are short of time, flick through and read only the cyan.

MEMORY HOOK: cyan is the colour of a cursor blinking at you, waiting. It is the book asking you a question.

How the book is arranged

The **Foundations** part comes first and owes nothing to the module order. It builds the machine underneath everything else: what a program is, what the compiler does, what a variable really is, and how to get a program running on your own laptop. Do not skip it, even if you have programmed before, because the modules assume it.

Then **Modules 1 to 10**, numbered exactly as your lecturer numbers them, so that this book and your course are always talking about the same Module 4. Then the back matter: every past exam question solved in full, mock papers in both formats, and a reference section you can revise from the night before.

⚠ TRAP · THE MODULE ORDER IS NOT A LEARNING ORDER

Your course teaches arrays in Module 8, near the end. But both Section A questions on the most recent paper are array problems, and arrays are the natural way to understand loops back in Module 4. The course also demands `JOptionPane` for input and output long before it explains what an object is.

This book fixes that without renumbering anything. Foundations teaches you, honestly and from zero, the few things the early modules need early, and tells you plainly where each one is developed in full. So when Module 8 arrives you are deepening something you already understand, not meeting it for the first time with three weeks to go.

If you have two weeks

1. Foundations, properly, doing every RECALL AND CHECK. One day.
2. Modules 3, 4, 5 and 8 (types, control, methods, arrays). These four carry most of the marks on every paper we have. Four days, and write out every worked example by hand.
3. Every solved past question in the back, with the solution covered. Three days.
4. Modules 6, 7, 9, 10. Three days.
5. The mock papers, timed, on paper, with the book shut. Three days.

Step 5 is not optional. You do not find out whether you can write Java under time pressure by reading about writing Java under time pressure.

PART I

Foundations

Everything the modules assume you already know, built from nothing. What a computer is really doing when it runs your program, how your typing becomes something a machine can execute, and how to get all of it working on your own laptop before the first module begins.



A machine that follows instructions, very fast, very literally

Before any Java, one idea. Everything else in this book is built on it.

TEACH · A COMPUTER DOES ONE THING

A computer takes a list of instructions and performs them, one at a time, in order, exactly as written. That is the whole of it. It is not clever, it does not know what you meant, and it will not fix your mistake for you. It is simply very fast and perfectly obedient.

Those two properties together are why programming is hard. The machine will do precisely what you said, several billion times a second, including the part you got wrong.

TEACH · A PROGRAM IS A RECIPE YOU CANNOT BE PRESENT FOR

Think of writing instructions for someone making tea in a kitchen you will never visit, for a person who has no common sense and will not ask questions. "Add milk" is not good enough. How much? Before or after the water? What if there is no milk?

A **program** is that recipe, written for that person. The skill you are learning is not typing. It is thinking precisely enough that a literal machine cannot misunderstand you.

RECALL AND CHECK

Your program produces the wrong answer. Before you look at anything else, what is the single most likely explanation?

ANSWER

That the machine did exactly what you told it to do, and you told it the wrong thing. Suspect your instructions before you suspect the computer. In a whole degree you will almost never meet a genuine hardware or compiler fault; you will meet your own logic errors daily.

The instructions are just text

TEACH · SOURCE CODE

Source code is the text you write, in a language with rules strict enough that a machine can read it without guessing. It is ordinary text: you could type it in Notepad. There is nothing magical in the file.

A **programming language** is that set of rules. Java is one. Python, C and C++ are others. They differ the way natural languages differ: same ideas, different grammar and vocabulary.

TEACH · WHY NOT JUST WRITE ENGLISH?

Because English is ambiguous, and a machine cannot resolve ambiguity. "Take the number and add one to it, then double it" has two meanings. Java's grammar exists to make every statement mean exactly one thing.

The price is that Java is **unforgiving**. A missing semicolon is not a small slip it can overlook; it is a sentence with no full stop, and the compiler refuses to continue. Beginners read this as hostility. It is the opposite: every complaint at compile time is a bug that cannot reach your exam script.

The compiler, bytecode, and the virtual machine

This section explains the three words your course uses constantly and never defines: **compile**, **bytecode**, and **JVM**. Every one of them is examinable.

TEACH · A PROCESSOR CANNOT READ JAVA

The chip in your laptop understands only its own **machine code**: numbers meaning "add these two", "put this there". It has never heard of `System.out.println`. Something must translate.

That translator is a **compiler**: a program whose input is your source code and whose output is something closer to what a machine can run. Java's compiler is called `javac`, and it comes with the kit you will install in Module 2.

TEACH · JAVA COMPILES TO BYTECODE, NOT TO MACHINE CODE

Here is where Java differs from C, and where most exam confusion lives. `javac` does **not** produce machine code for your particular chip. It produces **bytecode**: instructions for an imaginary computer that does not physically exist.

Bytecode lands in a `.class` file. It is not readable text and it is not something your processor can run.

TEACH · THE JVM IS THE IMAGINARY COMPUTER, MADE REAL

The **Java Virtual Machine** (JVM) is a program that pretends to be that imaginary computer. It reads bytecode instructions and carries them out using the real chip underneath. It is a translator standing between your compiled program and the actual hardware, working while your program runs.

So a Java program runs on a machine that is itself a program. That sounds wasteful, and it costs a little speed. What you buy with it is the single most famous property of the language.

● MUST-MEMORISE · WORA, AND WHY IT IS TRUE

Write Once, Run Anywhere (WORA). You compile your source to bytecode **once**. That same `.class` file then runs on Windows, macOS or Linux without being changed or recompiled, because each platform has its own JVM that knows how to translate the same bytecode for its own hardware.

The bytecode is identical everywhere. The JVM is what differs. That single sentence is the answer to "what is the role of the JVM?", which both papers we have ask in some form.

MEMORY HOOK: bytecode is a document written in one language; the JVM is the local interpreter hired in each country. One document, many interpreters, same meaning everywhere.

TEACH · THE WHOLE JOURNEY, IN FIVE STEPS

STEP	WHAT EXISTS AFTERWARDS	WHO DOES IT
You type	<code>Hello.java</code> , plain text	You, in any editor
Compile	<code>Hello.class</code> , bytecode	<code>javac Hello.java</code>
Load	bytecode in memory, checked for safety	the JVM
Execute	your program running	the JVM, using the real chip
Output	text on your screen	your program

You only ever type two of these: `javac` to compile, then `java` to run. Everything else happens for you.

⚠ TRAP · JAVAC AND JAVA ARE DIFFERENT PROGRAMS

javac compiles, and its name ends in **c** for compiler. It takes a **file name**, with the `.java` on the end:
`javac Hello.java`.

java runs, and it takes a **class name**, with no extension at all: `java Hello`. Not `java Hello.java`, and not `java Hello.class`. Both of those fail, and both are typed by almost every beginner in their first hour.

RECALL AND CHECK

Your friend compiles a Java program on a Mac and emails you the `.class` file. You are on Windows. Does it run, and why?

ANSWER

Yes. The `.class` file holds bytecode, which is not specific to any machine or operating system. Your Windows JVM reads that same bytecode and translates it for your hardware. That is WORA, and it is exactly why Java compiles to bytecode instead of machine code.

F.4 · GETTING JAVA ONTO YOUR COMPUTER

Installing the kit, and telling your computer where it is

Do this once, now, before Module 1. Nothing else in this book can be practised until it works, and it is worth marks directly: the setup is examinable.

● MUST-MEMORISE · JDK, JRE, JVM

JVM: the virtual machine that runs bytecode. Runs programs.

JRE: the JVM plus the standard library. Everything needed to *run* a Java program, but not to write one.

JDK: the JRE plus the development tools, above all the compiler `javac`. Everything needed to *write* Java.

You install the **JDK**, because you are a developer, not just a user. Installing the JDK gives you the JRE and the JVM inside it.

MEMORY HOOK: nested Russian dolls, biggest first: **JDK > JRE > JVM**. The kit contains the environment contains the machine. Developers need the outermost doll.

WORKED EXAMPLE · INSTALLING THE JDK

DO THIS NOW

- 1 Download the JDK installer for your operating system from Oracle's Java downloads page. Any version from 17 upward suits this course.

Your course notes point at Oracle. Any build of the same version behaves identically for everything in this book.

- 2 Run the installer and accept the default location. On Windows that is `C:\Program Files\Java\jdk-<version>`.

Write that path down. You need it in the next step, and <version> is whatever number you actually installed.

- 3 Open a terminal. On Windows press `Win + R`, type `cmd`, press Enter.

- 4 Type `java --version` and press Enter.

If you see a version number, the installer already set your `PATH` and you are done. If you see "not recognised", do F.4.1 below.

Now redo it: close the terminal, open a new one, and run `javac --version` as well. Both commands must answer. If `java` answers and `javac` does not, you have installed the JRE instead of the JDK, and you cannot compile anything.

F.4.1 Setting the environment variables

TEACH · WHAT PATH IS, AND WHY "NOT RECOGNISED" HAPPENS

When you type `javac`, your terminal does not search your whole hard disk for it. It looks only in a specific list of folders. That list is an **environment variable** called `PATH`: a setting that belongs to your operating system rather than to any one program.

"javac is not recognised" almost never means Java is missing. It means Java is installed somewhere your `PATH` does not mention, so the terminal looked in its usual places, did not find it, and gave up.

`JAVA_HOME` is a second variable holding the folder where the JDK lives. Java tools read it to find each other, so set it too.

WORKED EXAMPLE · SETTING JAVA_HOME AND PATH ON WINDOWS

- 1 Open File Explorer and confirm the JDK folder really exists, for example `C:\Program Files\Java\jdk-17`. Look inside it for a `bin` folder containing `javac.exe`.
Never set a path you have not looked at. A typo here produces the exact error you are trying to fix.
- 2 Right-click **This PC**, choose **Properties**, then **Advanced system settings**, then **Environment Variables**.
- 3 Under **System variables** click **New**. Variable name: `JAVA_HOME`. Variable value: the JDK folder, `C:\Program Files\Java\jdk-17`.
The folder itself, not its bin. No trailing backslash, no quotes.
- 4 In the same list select **Path**, click **Edit**, click **New**, and add `%JAVA_HOME%\bin`.
This is the bin, because that is the folder holding the commands. `%JAVA_HOME%` means "whatever `JAVA_HOME` says", so upgrading Java later means changing one setting, not two.
- 5 Click OK on all three dialogs. Close every open terminal and open a new one.
A terminal reads `PATH` once, when it starts. This is why it "didn't work" for you and did for the person next to you.
- 6 Verify.

TERMINAL

```
C:\> java --version
openjdk 17.0.16 2025-07-15
OpenJDK Runtime Environment Temurin-17.0.16+8 (build 17.0.16+8)
OpenJDK 64-Bit Server VM Temurin-17.0.16+8 (build 17.0.16+8, mixed mode, sharing)

C:\> javac --version
javac 17.0.16
```

Now redo it: from memory, name the two variables, say which one ends in `\bin` and why, and say why a terminal that was already open still fails.

⚠ TRAP · THE THREE THAT CATCH EVERYONE

- ▶ **Old terminal.** Changing `PATH` does not affect terminals that are already open. Close and reopen.
- ▶ **JAVA_HOME pointing at bin.** `JAVA_HOME` is the JDK folder; `PATH` gets the `\bin`. Put `bin` in both and `PATH` becomes `...\bin\bin`, which exists nowhere.
- ▶ **JRE not JDK.** `java` works, `javac` does not. You installed the thing that runs Java, not the thing that compiles it.

Hello, world

Type this yourself. Do not copy and paste. The typing is the point: your fingers need to learn the shape of a Java program, and you will be writing it on paper in the exam.

CODE · HELLO.JAVA

TYPE IT

```
1 public class Hello {
2     public static void main(String[] args) {
3         System.out.println("Hello, world!");
4     }
5 }
```

OUTPUT

```
Hello, world!
```

WORKED EXAMPLE · GETTING IT TO RUN

- 1 Make a folder, say C:\java, and save the file inside it as Hello.java.
The file name must be Hello.java, matching the class name exactly, capital H included. F.6 explains why this is a rule and not a preference.
- 2 In your terminal, move into that folder: `cd C:\java`
cd means change directory. The terminal is always "in" one folder, and it compiles what is in front of it.
- 3 Compile: `javac Hello.java`
Silence means success. Unix-descended tools say nothing when all is well; this unnerves beginners, but no news is good news.
- 4 Look: `dir`. There is now a Hello.class beside your Hello.java. That is the bytecode from F.3.
- 5 Run: `java Hello`
Class name, no extension.

TERMINAL

```
C:\java> javac Hello.java

C:\java> dir
Hello.class
Hello.java

C:\java> java Hello
Hello, world!
```

Now redo it: delete Hello.class, close the book, and get from an empty folder to the words on screen without looking. Do it until it is boring. This loop is the floor of everything else.

Every word in Hello.java, explained

You just typed nine words you were not told the meaning of. That is the last time this book does that. Here is every one.

SYNTAX · THE SMALLEST JAVA PROGRAM THAT DOES ANYTHING

```
public class NAME {                                <- the file must be NAME.java
    public static void main(String[] args) {        <- where running begins
        your statements, in order
    }
}
```

TEACH · CLASS, AND WHY YOUR FILE HAS THAT NAME

A **class** is a named container for code. For now that is all you need: Java has nowhere to put a statement except inside a class, so every program has at least one. Module 6 will show you what classes are really for.

public means anything may use it. The rule that catches beginners: **a public class must live in a file with exactly its own name**. Class Hello goes in Hello.java. Capitals count. Rename the file to hello.java and the compiler refuses it, which is one of the planted errors in your past papers.

TEACH · MAIN, THE DOOR IN

When you type `java Hello`, the JVM must decide which line to run first. It does not guess or start at the top. It looks for one exact method:

```
public static void main(String[] args)
```

Every word is load-bearing, and the JVM matches it letter for letter:

WORD	WHAT IT MEANS
public	Visible to everything, so the JVM (outside your class) is allowed to call it.
static	Belongs to the class itself, so it can be called without creating an object first. Since nothing exists yet when your program starts, it must be static.
void	Hands nothing back when it finishes.
main	The name the JVM looks for. Not Main, not start.
String[] args	A list of text values typed after the class name on the command line. You will rarely use it, and it must still be there.

Get any of it wrong and the program compiles perfectly, then refuses to start: the class is fine, it simply has no door.

● MUST-MEMORISE · THE MAIN METHOD SIGNATURE

```
public static void main(String[] args)
```

Write it from memory until you cannot get it wrong. You will write it on every program in the exam, and a mangled main costs marks in a question you otherwise answered perfectly.

MEMORY HOOK: say it aloud as one word, in rhythm: **"public-static-void-main-string-args"**. It is a chant, not five decisions.

TEACH · SYSTEM.OUT.PRINTLN, AND THE SEMICOLON

`System.out.println("Hello, world!");` reads right to left as: inside the class System there is out, which represents your screen, and you are asking it to println (print line) the text you gave it.

The double quotes mark a **String literal**: text meant literally rather than as an instruction. Without them, Java would read Hello as the name of something and complain that it does not exist.

The **semicolon** ends a statement, as a full stop ends a sentence. Java ignores line breaks entirely, so the semicolon is the only way it knows one instruction has finished. A missing one is the most common compiler error in the world, and it appears as a planted bug on both of your past papers.

⚠ TRAP · PRINTLN VERSUS PRINT

println prints and then moves to a new line. print prints and stays put. Use print and the next output continues on the same line, which is exactly what a past paper's PrintWriter question turns on. When an exam asks for the precise output, this is worth a mark on its own.

CODE · THE DIFFERENCE, RUN FOR REAL

```
1 public class PrintDemo {
2     public static void main(String[] args) {
3         System.out.print("A");
4         System.out.print("B");
5         System.out.println("C");
6         System.out.println("D");
7     }
8 }
```

OUTPUT

```
ABC
D
```

FILL IN THE GAP · THE SKELETON

Cover the page above. Fill these in with a pen.

1. A public class named Bank must be saved in a file called _____.
2. The JVM starts your program by looking for the method public _____ void main(String[] _____).
3. javac takes a _____ name; java takes a _____ name.
4. Compiling Hello.java produces a file called _____, which contains _____.
5. Every statement in Java ends with a _____.

ANSWERS

1. Bank.java · 2. static, args · 3. file, class · 4. Hello.class, bytecode · 5. semicolon

F.7 · YOUR FIRST ERRORS

Reading what the compiler is telling you

Both past papers ask you to find errors in code. That is not a trick: it is the actual daily work. Here is how to read a complaint.

● MUST-MEMORISE · THE TWO KINDS OF ERROR

Compile-time error: the code is not legal Java. Nothing runs at all. The compiler tells you the file and line. These are the cheap ones.

Run-time error: the code is legal and the program starts, then hits something impossible and stops. Java calls this **throwing an exception**. Module 10 handles them properly.

And the dangerous third case, which is not an error at all: a **logic error**. It compiles, it runs, it finishes, and the answer is wrong. Nothing warns you. Only a dry run (F.9) finds these.

MEMORY HOOK: compile-time is a letter rejected by the post office. Run-time is a letter delivered to a house that has burned down. A logic error is a letter delivered perfectly, to the wrong person, who never tells you.

```

1 public class Broken {
2     public static void main(String[] args) {
3         System.out.println("Hello")
4     }
5 }

```

WHAT THE COMPILER SAYS

```

Broken.java:3: error: ';' expected
    System.out.println("Hello")
                        ^

```

```

1 error

```

Read it as four facts: the file (`Broken.java`), the line (3), the complaint (`';' expected`), and the caret pointing at the exact column. The compiler has told you precisely what to type and where. Most beginners panic at the word "error" and never read the sentence.

⚠ TRAP · THE LINE NUMBER CAN LIE, BUT ONLY FORWARDS

A missing semicolon on line 3 is often reported on line 4, because the compiler only realises the sentence never ended when it meets the next word. If line *N* looks perfect, look at *N*-1. The compiler is never wrong about *that something is wrong*; it is sometimes late about *where*.

RECALL AND CHECK

A program compiles with no errors and runs to completion, but prints 141 when the right answer is 142. Which of the three kinds is this, and which tool finds it?

ANSWER

A logic error. No tool finds it. The compiler is satisfied and the JVM never complained, because nothing illegal or impossible happened: you simply told the machine to compute the wrong thing. Only tracing the program by hand, a dry run, will locate it. This is exactly what the exam's "find the errors" questions are testing, and why they hand you the correct output.

F.8 · VARIABLES

A named box that holds one value

Module 3 covers types in full. You need enough now to read the rest of Foundations.

TEACH · VARIABLES

A **variable** is a named box in memory holding one value at a time. You must say what kind of thing goes in the box before you use it, which is called **declaring** it:

```
int score = 0;
```

reads as: make a box called `score`, allow whole numbers in it, and put 0 in it now. `int` is the **type**, `score` the **name**, 0 the initial **value**.

The box holds exactly one value. Putting a new one in destroys the old one.

⚠ TRAP · = DOES NOT MEAN EQUALS

In mathematics $x = x + 1$ is a false statement. In Java it is an instruction, and a perfectly normal one.

`=` means **put the value on the right into the box on the left**. So `x = x + 1` says: work out what is in `x`, add one, put the result back in `x`. Right side first, always.

Java writes "is equal to" as `==`. Confusing the two is the most common single-character bug in the language, and both papers plant it.

TEACH · THE FOUR TYPES YOU NEED RIGHT NOW

TYPE	HOLDS	EXAMPLE
int	a whole number	int n = 42;
double	a number with a decimal point	double p = 15.5;
boolean	true or false, nothing else	boolean done = false;
String	text	String s = "Ken";

String is capitalised and the other three are not. That is not a typo, and Module 6 explains the real reason: the first three are built into the language, while String is a class.

⚠ TRAP · INT DIVISION THROWS AWAY THE REMAINDER

Divide one int by another and Java gives you an int, discarding anything after the decimal point. It does not round; it truncates.

1 / 2 is **0**. Not 0.5, not 1. Zero.

This looks like pedantry until you see that a past paper's dry-run question is built entirely on it, and the answer is 0 when every student's instinct says otherwise. F.9 traces exactly that question.

F.9 · THE DRY RUN

Being the computer, on paper

Your lecturer's own word for this is "dry run", and the exam asks for it by name, for marks, on both papers we have. It is also the only way to find a logic error. Learn it now and use it for the rest of the book.

TEACH · THE METHOD

A dry run means executing the program by hand, one statement at a time, writing down what every variable holds after every step. No skipping, no "obviously this loops five times". The machine does not skip, so neither do you: the whole point is to catch the step where your assumption and the code part company.

Draw a table. One column per variable, plus one for anything printed. One row per step. Then walk the code.

CODE · THE ONE FROM THE 2024/25 PAPER

PAST PAPER · Q1 C

```
1 public static int sumSeries(int i) {
2     int sum = 0;
3     for (int j = 1; j <= i; j++) {
4         sum += j / (j + 1);
5     }
6     return sum;
7 }
```

DRY RUN · SUMSERIES(3), STEP BY STEP

Cover the table. Trace it yourself first, then compare. Remember F.8: j and $j+1$ are both `int`, so $j / (j + 1)$ is **int division**.

$i = 3$

STEP	J	J <= i ?	J / (J + 1)	SUM AFTER
start	-	-	-	0
pass 1	1	1 <= 3, yes	1 / 2 = 0	0 + 0 = 0
pass 2	2	2 <= 3, yes	2 / 3 = 0	0 + 0 = 0
pass 3	3	3 <= 3, yes	3 / 4 = 0	0 + 0 = 0
test	4	4 <= 3, no: stop	-	0
return	4	-	-	returns 0

Every term is zero, so the sum is zero, and the same holds for 4, 5 and 7. The question hands you four values hoping you will grind through the arithmetic and miss the one thing that matters: these are ints, and $1/2$ is 0.

CODE · THE SAME METHOD, ACTUALLY RUN

VERIFIED ON THE JVM

```

1 public class SumSeries {
2     public static int sumSeries(int i) {
3         int sum = 0;
4         for (int j = 1; j <= i; j++) {
5             sum += j / (j + 1);
6         }
7         return sum;
8     }
9
10    public static void main(String[] args) {
11        System.out.println(sumSeries(3));
12        System.out.println(sumSeries(4));
13        System.out.println(sumSeries(5));
14        System.out.println(sumSeries(7));
15    }
16 }

```

OUTPUT

```

0
0
0
0

```

● MUST-MEMORISE · HOW TO DRY RUN

1. A column per variable, a column for output, a row per step.
2. Write the value after **every** statement, even when it does not change.
3. Evaluate the right-hand side fully before writing into the left.
4. Check the loop's condition **every** pass, including the failing one.
5. Never skip a pass because you can "see" the pattern.

MEMORY HOOK: you are not reading the code, you are **being** the computer. The computer never skips a row, and it never sees a pattern.

Trace this on paper before reading on. Values start `nums = {2, 4, 6, 8}`, and `x = nums[0]`. Square brackets pick one item out of a list; F.10 explains them properly in a moment.

```
int x = nums[0];
for (int i = 1; i < nums.length; i++) {
    x = x - nums[i];
}
System.out.println(x);
```

`NUMS = {2, 4, 6, 8}`

STEP	I	NUMS[I]	X = X - NUMS[I]	X AFTER
start	-	-	<code>x = nums[0]</code>	2
pass 1	1	4	<code>2 - 4</code>	-2
pass 2	2	6	<code>-2 - 6</code>	-8
pass 3	3	8	<code>-8 - 8</code>	-16
test	4	-	<code>4 < 4</code> is false: stop	-16
print	4	-	-	-16

Now redo it: cover the table and trace it again with `nums = {5, 1, 1}`. You should get 3. If you got 7, you started `i` at 0 and subtracted `nums[0]` from itself.

F.10 · ARRAYS, EARLY AND HONEST

A row of numbered boxes

TEACH · WHY THIS IS HERE AND NOT IN MODULE 8

Your course teaches arrays in Module 8, near the end. But every recent exam puts array problems in Section A, and arrays are the natural thing to loop over when you meet loops in Module 4. Waiting until Module 8 would mean using them before explaining them, which this book does not do.

So here is an honest introduction, enough to read any listing in this book. **Module 8 is still where arrays are taught in full:** searching, copying, two dimensions, the standard algorithms. This is the floor, not the whole house.

TEACH · AN ARRAY

An **array** is a fixed row of boxes, all of the same type, sharing one name and told apart by a number called an **index**.

```
int[] nums = {2, 4, 6, 8};
```

makes four `int` boxes in a row. `int[]` means "array of `int`". You reach one box with square brackets: `nums[0]` is the first, `nums[1]` the second.

`nums.length` is how many boxes there are: 4. It is a property, not a method, so there are **no brackets after it**.

● MUST-MEMORISE · INDEXES START AT ZERO

An array of length **n** has indexes **0 to n-1**. There is no `nums[n]`. The last item is always `nums[nums.length - 1]`.

Reaching outside that range does not give you rubbish or zero: the JVM stops the program with `ArrayIndexOutOfBoundsException`. Every past paper plants this, usually as `i <= arr.length` in a loop.

MEMORY HOOK: an index is not "which one" but "how far from the start". The first box is zero steps from the start. So `{2, 4, 6, 8}` has a box 3 and no box 4.

CODE · INDEXES, LENGTH, AND THE EDGE

```

1 public class ArrayBasics {
2     public static void main(String[] args) {
3         int[] nums = {2, 4, 6, 8};
4         System.out.println("length      : " + nums.length);
5         System.out.println("first  [0] : " + nums[0]);
6         System.out.println("last   [3] : " + nums[nums.length - 1]);
7
8         // walk every box: i stops BEFORE length
9         for (int i = 0; i < nums.length; i++) {
10             System.out.print(nums[i] + " ");
11         }
12         System.out.println();
13     }
14 }

```

OUTPUT

```

length      : 4
first  [0] : 2
last   [3] : 8
2 4 6 8

```

CODE · ONE CHARACTER WRONG

CRASHES ON PURPOSE

```

1 public class OffByOne {
2     public static void main(String[] args) {
3         int[] nums = {2, 4, 6, 8};
4         try {
5             for (int i = 0; i <= nums.length; i++) { // <= is the bug
6                 System.out.println("reading nums[" + i + "] ...");
7                 System.out.println(" got " + nums[i]);
8             }
9         } catch (ArrayIndexOutOfBoundsException e) {
10             System.out.println("CRASH: " + e);
11         }
12     }
13 }

```

OUTPUT

```

reading nums[0] ...
 got 2
reading nums[1] ...
 got 4
reading nums[2] ...
 got 6
reading nums[3] ...
 got 8
reading nums[4] ...
CRASH: java.lang.ArrayIndexOutOfBoundsException: Index 4 out of bounds for length 4

```

The four real boxes print fine, then index 4 stops the program dead. Note it printed "reading nums[4]" before crashing: the crash happens at the moment of the read, not before. (try and catch are here only to show you the crash without the program dying; Module 10 teaches them.)

RECALL AND CHECK

`double[] prices = {15.5, 42.0, 8.75};` What is `prices.length`, what is the largest legal index, and what does `prices[3]` do?

ANSWER

Length is 3. The largest legal index is 2, because indexes run 0 to $n-1$. `prices[3]` compiles perfectly and then throws `ArrayIndexOutOfBoundsException` at run time. It is a run-time error, not a compile-time one: the compiler does not know what is in `i` until the program runs.

F.11 · JOPTIONPANE, EARLY AND HONEST

Pop-up boxes, and a promise about what they are

TEACH · WHY THIS IS HERE TOO

The most recent paper demands `JOptionPane` for input and output in five of its six questions, sometimes saying "JOptionPane only". You cannot pass that paper without it. It is also a method call on a class from a library, and this book will not pretend that means nothing.

So: here is the pattern, and an honest account of what each word is. The full explanation of classes, objects and libraries is Module 6, and this box will make complete sense then. Until then you may use this as a shape you copy exactly. That is not cheating; it is the order the exam forces on us.

SYNTAX · THE TWO CALLS YOU NEED

```
import javax.swing.JOptionPane;           <- at the very top of the file

JOptionPane.showMessageDialog(null, "some text");  <- show a message

String s = JOptionPane.showInputDialog("Enter N:"); <- ask; you always get TEXT
int n = Integer.parseInt(s);               <- turn that text into a number
```

TEACH · WHAT THOSE WORDS ACTUALLY ARE

WORD	HONESTLY, WHAT IT IS
<code>import</code>	Tells the compiler where to find a class written by someone else. Without it, Java does not know the name <code>JOptionPane</code> .
<code>javax.swing</code>	A package : a named folder of ready-made classes that came with your JDK. Swing is Java's window toolkit.
<code>JOptionPane</code>	A class, exactly like your <code>Hello</code> class, but written by Java's authors.
<code>showMessageDialog</code>	A method belonging to that class. It is static, in the sense of F.6, which is why you can call it without making an object first.
<code>null</code>	"No parent window." <code>null</code> means "nothing here". You will meet it properly in Module 6.
<code>Integer.parseInt</code>	A method that takes text and hands back the <code>int</code> it represents. "25" is text; 25 is a number; they are not the same thing.

● MUST-MEMORISE · INPUT ALWAYS ARRIVES AS TEXT

showInputDialog hands you a String, **always**, even when the user typed 25. To do arithmetic you must convert:

```
int n = Integer.parseInt(JOptionPane.showInputDialog("Enter N:"));
double d = Double.parseDouble(JOptionPane.showInputDialog("Enter price:"));
```

MEMORY HOOK: the dialog is a form, and everything written on a form is handwriting, even the numbers. **parse** is you reading the handwriting and deciding it is a number.

⚠ TRAP · "25" + 1 IS NOT 26

Forget to parse, and + does something perfectly legal and completely wrong. With text on either side, + means **join**, not add: "25" + 1 is the text "251". It compiles, it runs, it prints nonsense. A logic error, in the sense of F.7, and no tool will warn you.

CODE · THE TRAP, RUN FOR REAL

```
1 public class JoinVsAdd {
2     public static void main(String[] args) {
3         String typed = "25";           // what a dialog hands you
4
5         System.out.println(typed + 1); // join: WRONG
6         System.out.println(Integer.parseInt(typed) + 1); // add: right
7     }
8 }
```

OUTPUT

```
251
26
```

TEACH · SYSTEM.EXIT(0), AND WHY THE EXAM PAPERS END WITH IT

Look at the end of almost every JOptionPane program on both papers:

```
JOptionPane.showMessageDialog(null, "Sum: " + sum);
System.exit(0);
```

System.exit(0) **ends the whole program immediately**, right there, whatever was going to happen next. The number is the **exit status** that the program hands back to whatever started it, and the convention is older than Java: **0 means it finished normally**, and anything else means it failed. That is why the line is System.exit(0) and never System.exit(1).

Why it is there. A plain main ends when its last line runs. A program that has opened a dialog is different: showing one starts extra **threads** to run the window, and those threads are not finished just because main is. System.exit(0) is the author saying "we are done, stop everything", instead of waiting for the window machinery to decide.

Now the honest part, because you will be told otherwise. You will hear that without System.exit(0) a Swing program **hangs forever**. We tested it on the JDK this book is built against: after touching Swing, two non-daemon threads (AWT-Shutdown and AWT-EventQueue-0) are indeed still alive when main returns, and the program **exited anyway**, cleanly. Modern Java shuts that machinery down once no window is left showing. So the line is **insurance, not a rescue**.

Write it anyway, for the reason that matters in an exam: the paper's own code has it, so it is what the marker expects to see, and it costs one line. But if someone tells you your dialog program will hang without it, that is a claim about a Java old enough to be worth checking.

MEMORY HOOK: **exit(0) means "stop now, and nothing went wrong."** Zero is success, every other number is a complaint.

TEACH · A NOTE ON PRACTISING THIS

JOptionPane opens a real window, so it cannot show its output on a printed page and the listings in this book use `System.out.println` instead. They are the same program with a different mouth.

When the exam says "JOptionPane only", it means the marks are for the dialogs, so write the dialogs. When you are practising at home and want to see what your loop did, `System.out.println` is faster. Know both, and read the question.

FILL IN THE GAP · FOUNDATIONS, ALL OF IT

Book shut. Pen.

1. The compiler turns `.java` into _____, which contains _____, which is run by the _____.
2. WORA is true because the _____ is the same on every platform while the _____ differs.
3. Biggest to smallest: _____ > _____ > _____.
4. `7 / 2` where both are `int` gives _____.
5. An array of length 6 has indexes _____ to _____; `a[6]` throws _____.
6. `showInputDialog` always returns a _____, so you must call _____ before doing arithmetic.
7. An error that compiles, runs, finishes and is wrong is a _____ error, and only a _____ will find it.

ANSWERS

1. `.class`, bytecode, JVM · 2. bytecode, JVM · 3. JDK, JRE, JVM · 4. 3 · 5. 0, 5, `ArrayIndexOutOfBoundsException` · 6. `String`, `Integer.parseInt` (or `Double.parseDouble`) · 7. logic, dry run

PART II

Module One Introduction to Computer Programming

What programming is, why the world runs on it, and the five ways of thinking about a problem that the industry calls paradigms. This is the module that gives you the vocabulary: class, object, attribute, method. The words you will use for the rest of the course, and the rest of your career.



Programming, and why it matters

By the end of this unit you should be able to explain computer programming and its significance, identify the major programming paradigms and the languages associated with each, and identify classes, objects, attributes and methods in a written Java program.

Foundations showed you what a machine does. This unit is about the words: the vocabulary your lecturer, your exam and your future colleagues all use. Most of the marks here are definitions, and definitions are the cheapest marks on any paper.

TEACH · COMPUTER PROGRAMMING

Computer programming is the process of writing instructions that tell a computer how to perform tasks. Those instructions are written as code, in a specialised language such as Java, Python or C++.

Learn that sentence in that form. It is your course's own definition, and "define computer programming" is worth marks in the exam almost every year.

TEACH · WHY PROGRAMMING MATTERS

Your course gives four reasons, and an exam question asking for "three major benefits" wants three of these:

BENEFIT	WHAT IT MEANS
Task automation	Repetitive work is done by the machine, saving time and removing human error.
Software development	Websites, apps and enterprise systems all exist because someone wrote them.
Problem solving	It trains logical thinking, which transfers to fields far outside computing.
Career growth	Diverse, well-paid work in software engineering, data science and beyond.

Programming is called the backbone of software development because it drives innovation in artificial intelligence, web development, data analysis and automation. That phrasing is the expected answer to a past in-text question.

RECALL AND CHECK

Define computer programming in one sentence, and give three benefits of learning it.

ANSWER

Computer programming is the process of writing instructions that enable a computer to perform specific tasks. Three benefits: task automation, which saves time and reduces errors; the creation of software such as websites and mobile apps; and the development of logical thinking and problem-solving skills.

Five ways to think about a problem

TEACH · A PARADIGM

A **programming paradigm** is a fundamental style or approach to writing computer programs. It is not a language. It is a way of thinking about what a program is, and languages are built to support one or more of them.

Your course names five. The whole set turns on one distinction, so learn that distinction and the five arrange themselves.

● MUST-MEMORISE · THE ONE DISTINCTION UNDERNEATH ALL FIVE

Imperative paradigms say **HOW**: step by step, do this, then this.

Declarative paradigms say **WHAT**: describe the result you want and let the language work out the steps.

Everything else is detail. Imperative and object-oriented are HOW styles; declarative, logic-based and functional lean WHAT.

MEMORY HOOK: a taxi. Imperative is giving the driver turn-by-turn directions. Declarative is saying "the airport, please".

TEACH · THE FIVE, WITH THEIR LANGUAGES

PARADIGM	THE IDEA	LANGUAGES
Imperative	Step by step: tell the computer <i>how</i> to achieve the task. Built on variables, loops and conditionals.	C, and Java's method bodies
Functional	Focuses on <i>what</i> to compute. Functions are first-class citizens; pure functions; avoid changing variables (immutability).	Haskell, Lisp, JavaScript in functional style
Declarative	Express the logic of a computation without describing its control flow.	SQL
Logic-based	A declarative style built on formal logic: write facts and rules, and the engine infers answers by deduction.	Prolog
Object-oriented	Organise code as objects, which are instances of classes, each holding its own data and behaviour.	Java, C++, Python in OOP style

Java is object-oriented, and that is the answer if you are asked which paradigm this course sits in. Note that Java is not *purely* OOP: the inside of any method you write is plain imperative code, step after step. Most real languages blend paradigms.

TEACH · THE SAME TASK, THREE WAYS

To make the difference concrete, here is "give me the total of the amounts over 20", written three ways. You are not expected to write SQL or Prolog in this course; read them only to feel the shift from HOW to WHAT.

```
imperative (Java)  total = 0;
                   for (int i = 0; i < a.length; i++)
                       if (a[i] > 20) total += a[i];          <- you write every step

declarative (SQL)  SELECT SUM(amount) FROM sales WHERE amount > 20;
                                                           <- no loop, no counter

logic (Prolog)    big(X) :- sale(X), X > 20.                 <- facts and rules only
```

The imperative version names a counter, a loop and an order. The other two never mention any of it. That is the whole distinction, and it is what an exam answer should say.

⚠ TRAP · A PARADIGM IS NOT A LANGUAGE

"Java is a paradigm" is wrong, and so is "OOP is a language". Java is a *language* that supports the object-oriented *paradigm*. Python supports several. The paradigm is the style of thinking; the language is the grammar you express it in. Examiners notice this one.

FILL IN THE GAP · THE FIVE PARADIGMS

1. A fundamental style or approach to writing programs is a _____.
2. _____ programming says HOW, step by step, using variables, loops and conditionals.
3. _____ programming says WHAT rather than how; SQL is the example.
4. In functional programming, functions are _____ citizens and variables are not changed, a property called _____.
5. Logic-based programming, whose language is _____, expresses computation as _____ and rules.
6. Java's paradigm is _____.

ANSWERS

1. paradigm · 2. Imperative · 3. Declarative · 4. first-class, immutability · 5. Prolog, facts · 6. object-oriented

UNIT 1.3 · CLASSES, OBJECTS, ATTRIBUTES AND METHODS

The four words the rest of the course is built on

Module 6 teaches object-oriented programming properly. This unit gives you the four words and lets you find them in a program, which is exactly what your learning outcome asks and no more.

TEACH · THE MAIN IDEA OF OOP

Object-oriented programming organises code using **objects**, which represent real-world things. Each object carries both:

- ▶ **data**, called its attributes or state: what it *is*;
- ▶ **methods**, its behaviour: what it can *do*.

The main idea is to model software as a collection of interacting objects, each responsible for its own data and its own logic.

● MUST-MEMORISE · CLASS VERSUS OBJECT

A **class** is a blueprint. It describes what something will be like. It is written once, and on its own it holds no data.

An **object** is an **instance** of a class: one actual thing built from that blueprint, with its own values. One class, many objects.

MEMORY HOOK: the class is the architect's drawing; the objects are the houses built from it. Nobody lives in the drawing, and every house has its own furniture.

```

1 // The CLASS: a blueprint for dogs. No dog exists yet.
2 class Dog {
3     String name;        // ATTRIBUTE (data / state)
4     int age;            // ATTRIBUTE
5
6     void bark() {        // METHOD (behaviour)
7         System.out.println(name + " says: Woof!");
8     }
9 }
10
11 public class DogDemo {
12     public static void main(String[] args) {
13         Dog a = new Dog(); // OBJECT 1: an instance of Dog
14         a.name = "Rex";
15         a.age = 3;
16
17         Dog b = new Dog(); // OBJECT 2: same blueprint, own values
18         b.name = "Bella";
19         b.age = 5;
20
21         a.bark();
22         b.bark();
23         System.out.println(a.name + " is " + a.age + ", "
24                             + b.name + " is " + b.age);
25     }
26 }

```

OUTPUT

```

Rex says: Woof!
Bella says: Woof!
Rex is 3, Bella is 5

```

One bark() method was written, and it printed a different name each time, because it belongs to whichever object it was called on. That sentence is the whole of OOP in miniature, and Module 6 unpacks it.

TEACH · FINDING THE FOUR IN ANY PROGRAM

Your learning outcome is to *identify* these in written Java. There is a mechanical way to do it:

LOOK FOR	YOU HAVE FOUND	IN THE LISTING ABOVE
class Name {	a class (blueprint)	Dog, DogDemo
a variable declared straight inside the class	an attribute (state)	name, age
returnType name(...) {	a method (behaviour)	bark(), main()
new Name()	an object being created	a, b

The word new is the giveaway: **new is the moment an object comes into existence**. Before line 15 there were no dogs, only the idea of a dog.

⚠ TRAP · THE ATTRIBUTE IS NOT INSIDE A METHOD

name and age are declared directly inside the class, not inside bark(). That is what makes them attributes: they belong to the object and live as long as it does.

A variable declared inside a method is a **local variable**. It is born when the method starts and dies when it ends, and it is not an attribute. Asked to identify attributes, students routinely list local variables and lose the mark.

RECALL AND CHECK

In one sentence each: what is a class, what is an object, what is an attribute, what is a method?

ANSWER

A class is a blueprint describing what something will be like. An object is an instance of a class: one actual thing built from that blueprint, with its own values. An attribute is a piece of data an object holds, its state. A method is a piece of behaviour, something the object can do.

WORKED EXAMPLE · READING AN UNFAMILIAR CLASS

EXAM SKILL

An exam gives you a class you have never seen and asks you to identify its parts. Work outside in, and never read the method bodies first.

- 1 Find every class keyword. Those are your blueprints.
A file can hold several. Only one may be public, and it names the file (F.6).
- 2 Inside each class, list variables declared before any method opens. Those are the attributes.
Indentation is a lie: it is not what makes them attributes. Position relative to the braces is.
- 3 List everything shaped `type name(parameters) {`. Those are the methods.
- 4 Search for `new`. Every occurrence is an object being born.

Now redo it: cover the Dog listing and write out its two classes, two attributes, two methods and two objects from memory. Then do the same for the Hello program in F.5. (It has one class, no attributes, one method, and no objects at all: perfectly legal, and worth understanding why.)

PART III

Module Two

Java Setup

Java itself: where it came from, what "write once, run anywhere" actually means, and the machinery that makes it true. Then the practical part, done properly and in full: installing the kit, telling your computer where it lives, and proving it works.



What Java is, and what makes it Java

By the end of this unit you should be able to explain the fundamental concepts of Java programming, install and configure the JDK on any operating system, and set up the environment variables needed to run Java applications.

TEACH · JAVA, IN ONE PARAGRAPH

Java is a versatile, object-oriented programming language used to build scalable applications across multiple platforms. It launched in **1995** and has remained a first choice for web, desktop, enterprise and mobile development ever since.

Its defining property is "**Write Once, Run Anywhere**" (WORA), powered by the Java Virtual Machine, which gives it cross-platform compatibility.

TEACH · YOU HAVE ALREADY MET THE MACHINERY

F.3 built the compile-and-run story from zero: source becomes bytecode becomes a running program, and the JVM is what makes the same bytecode work everywhere. F.4 got a JDK onto your machine and your first program running.

This unit is not a repeat. It is the same material as your course examines it: the formal definitions, the distinctions between the three J-words, and the question forms that actually appear on the paper. If F.3 and F.4 are hazy, go back now; the rest of this unit assumes them.

● MUST-MEMORISE · JAVA'S FOUR HEADLINE PROPERTIES

Platform independent: compiled to bytecode, run by a JVM, so one compilation runs anywhere (WORA).

Object-oriented: code is organised as classes and objects.

Versatile: web, desktop, enterprise and mobile.

Scalable: suits systems that must grow.

MEMORY HOOK: Java is a **POVS** language: Platform independent, Object-oriented, Versatile, Scalable. Then attach one sentence to each letter, because a bare list earns half the marks a sentence does.

RECALL AND CHECK

What does WORA stand for, what makes it possible, and what is the role of the JVM?

ANSWER

WORA stands for "Write Once, Run Anywhere": Java's ability to run the same code on multiple platforms. It is possible because Java compiles to bytecode rather than to machine code, and bytecode is identical on every platform. The JVM's role is to read that bytecode and carry it out on its own particular platform, which is what makes the same compiled file run everywhere.

JDK, JRE, JVM, and getting them working

TEACH · THE JDK

To write Java you download and install the **Java Development Kit**. The JDK is a software development environment used for developing Java applications: it provides the tools needed to **write, compile and run** them.

That three-verb phrase is your course's own wording, and it is the shape of the expected answer to "what is the JDK?".

TEACH · WHICH OF THE THREE DO YOU NEED?

The three J-words nest, as F.4 set out: **JDK > JRE > JVM**. The exam likes the distinction, so hold it precisely:

NAME	IS	LETS YOU	WHO NEEDS IT
JVM	the virtual machine that executes bytecode	execute bytecode	everyone, and it comes inside the others
JRE	JVM + the standard class library	run Java programs	a user running someone else's program
JDK	JRE + development tools, above all javac	write, compile and run	you

Install the JDK. A JRE cannot compile, and a language you cannot compile is a language you cannot learn.

TEACH · INSTALLING AND CONFIGURING

The full procedure, with every step and every trap, is **F.4**: download the JDK for your operating system, install it, set `JAVA_HOME` to the JDK folder, add `%JAVA_HOME%\bin` to `PATH`, open a *new* terminal, and verify.

It is in Foundations rather than here for one reason: you needed a working compiler before Module 1, not after it. Nothing about it is different when your course reaches this unit.

The verification command is the one your course gives:

```
TERMINAL
C:\> java --version
openjdk 17.0.16 2025-07-15
OpenJDK Runtime Environment Temurin-17.0.16+8 (build 17.0.16+8)
OpenJDK 64-Bit Server VM Temurin-17.0.16+8 (build 17.0.16+8, mixed mode, sharing)
```

Your version numbers will differ, and that is fine: any JDK 17 or later runs everything in this book. What matters is that the command answers at all. Silence or "not recognised" means `PATH`, and F.4.1 fixes it.

⚠ TRAP · JAVA --VERSION IS NOT PROOF YOU CAN COMPILE

Your course's verification step only runs `java --version`. That command answers happily on a machine with a JRE and no compiler, and then `javac Hello.java` fails and you cannot see why.

Always check **both**. If `java` answers and `javac` does not, you have a JRE, not a JDK.

```
TERMINAL
C:\> javac --version
javac 17.0.16
```

● MUST-MEMORISE · THE TWO ENVIRONMENT VARIABLES

`JAVA_HOME` = the JDK folder itself.

`C:\Program Files\Java\jdk-17`

`PATH` gets the **bin** inside it.

`%JAVA_HOME%\bin`

`PATH` is the list of folders your terminal searches when you type a command. That is why the `bin` goes there: `bin` is the folder the commands actually live in.

MEMORY HOOK: HOME is the house, PATH is the front door. You address a letter to the house; you walk in through the door.

FILL IN THE GAP · JAVA AND ITS ENVIRONMENT

1. Java was launched in _____ and its defining property is _____, powered by the _____.
2. The JDK provides the tools needed to _____, _____ and _____ Java applications.
3. A user who only wants to run a Java program needs the _____; a developer needs the _____.
4. JAVA_HOME is set to the _____, while PATH receives _____.
5. After changing PATH you must _____ before the change takes effect.

ANSWERS

1. 1995, WORA / Write Once Run Anywhere, JVM · 2. write, compile, run · 3. JRE, JDK · 4. JDK folder, %JAVA_HOME%\bin · 5. open a new terminal (an open one read PATH when it started)

WORKED EXAMPLE · DIAGNOSING A BROKEN SETUP

EXAM SKILL

"I installed Java but javac says it is not recognised." Work it in this order, and stop at the first step that fails.

- 1 Does java --version answer?
If yes, Java is installed and PATH is partly right, so jump to step 3. If no, PATH is wrong or nothing is installed.
- 2 Does the JDK folder exist in File Explorer, with a bin holding javac.exe?
If there is no javac.exe, you installed a JRE. Reinstall the JDK. This is the single most common cause.
- 3 Is this terminal older than the change you made?
Close it, open a new one, try again. Second most common cause, and it costs people hours.
- 4 Does echo %JAVA_HOME% print the JDK folder, with no \bin on the end and no quotes?
- 5 Does echo %PATH% contain that folder plus \bin?

Now redo it: from memory, name the two most common causes in order. Then say why java can work while javac does not, in one sentence.

PART IV

Module Three

Java Fundamentals

The raw material. Types and the exact sizes of the boxes they make, the rules for naming them, and every operator the language gives you for combining them. This is the module the rest of the course stands on, and it is where the most marks quietly live.



UNIT 3.1 · THE FOUR BUILDING BLOCKS

Variables, data types, operators, control structures

By the end of this unit you should be able to explain Java's data types and their memory sizes, declare and use variables with correct naming conventions, identify the various operators, identify the forms of control structure, write and compile basic programs, and handle user input and output.

Your course says programming rests on four building blocks. Every program you write for the rest of your life is made of these and nothing else.

● MUST-MEMORISE · THE FOUR BUILDING BLOCKS

Variables hold data. **Data types** say what kind of data. **Operators** combine and compare it. **Control structures** decide what runs, and how often.

MEMORY HOOK: a kitchen. Variables are the containers, data types say what may go in each, operators are the knives and the mixer, control structures are the recipe deciding order and repetition.

UNIT 3.2 · VARIABLES AND THE RULES FOR NAMING THEM

Declaring a variable, and what you may call it

TEACH · DECLARATION

F.8 introduced a variable as a named box holding one value. Formally: **a variable is a container that stores data**, and **a variable in Java must be declared before it is used**. You declare it by writing its data type and then its name.

```
int number;
```

int is the data type, number is the name. That line makes the box but puts nothing in it. To make it and fill it in one go:

```
dataType variableName = value;
```

which your course calls **declaration and initialisation**. Learn that one-line syntax exactly: "what is the syntax to declare and initialise a variable" is a stated in-text question, so it is a candidate for the two-mark define slot in Section A.

CODE · FOUR DECLARATIONS

FROM THE SLIDES

```
1 public class Declare {
2     public static void main(String[] args) {
3         int age = 25;           // Integer variable
4         String name = "John";   // String variable
5         boolean isActive = true; // Boolean variable
6         double price = 19.99;   // Double variable
7
8         System.out.println(name + " is " + age + ", active: " + isActive
9                               + ", price: " + price);
10    }
11 }
```

OUTPUT

```
John is 25, active: true, price: 19.99
```

● MUST-MEMORISE · THE SIX NAMING RULES

1. Must **start** with a letter, an underscore `_`, or a dollar sign `$`.
2. Cannot start with a **digit**.
3. May contain letters, digits, `_` and `$`.
4. Are **case-sensitive**: `age`, `AGE` and `Age` are three different variables.
5. Cannot contain **spaces** or special characters like `!`, `@`, `-`.
6. Cannot be a **keyword**: `int`, `class`, `float`, `if`, `public`, `else`, `private` are all illegal as names.

MEMORY HOOK: rules 1, 2, 3 and 5 all say the same thing once you see it: **a name is letters, digits, _ and \$, and it may not start with a digit**. Learn that one sentence, then add case-sensitivity and no-keywords. Six rules, two facts.

TEACH · VALID AND INVALID, SIDE BY SIDE

VALID	INVALID	WHY IT IS REJECTED
<code>person, name2</code>	<code>1variable, 2name</code>	starts with a digit (rule 2)
<code>_myName</code>	<code>123LondonStreet</code>	starts with a digit (rule 2)
<code>\$fileName</code>	<code>my-variable</code>	hyphen is a special character (rule 5)
<code>totalPrice</code>	<code>my@name</code>	at-sign is a special character (rule 5)
<code>Age, age, AGE</code>	<code>int, class</code>	keyword (rule 6)

Note the third column of the first two rows: `my-variable` is rejected not because hyphens are ugly but because Java has already given `-` a job. It would read `my-variable` as `my` minus `variable`.

⚠ TRAP · CASE-SENSITIVITY IS NOT A FORMALITY

Rule 4 sounds like pedantry and is the source of real bugs. Declare `int count = 0;` and later write `Count++` and Java does not correct you; it says `cannot find symbol`, because `Count` is a different name that was never declared.

This is a planted error on the 2024/25 paper: a loop body writes `Count` where the variable is `count`. Two of that question's three marks are for spotting it.

RECALL AND CHECK

Which of these are illegal variable names, and why? `2fast`, `_total`, `grand-total`, `double`, `$price`, `my name`

ANSWER

Illegal: `2fast` starts with a digit; `grand-total` contains a hyphen, a special character; `double` is a keyword; `my name` contains a space. Legal: `_total` and `$price`, because underscore and dollar are both allowed starters.

Primitive and non-primitive, and the sizes you must know

TEACH · WHAT A DATA TYPE IS FOR

Data types specify the type of data a variable can hold, and the type **determines the operations that can be performed** on it. That second half is the part students skip and the exam rewards: the type is not just a label on the box, it decides what you are allowed to do.

+ on two ints adds. + on two Strings joins. Same symbol, different operation, chosen entirely by the types. F.11's "25" + 1 trap is precisely this.

● MUST-MEMORISE · PRIMITIVE VERSUS NON-PRIMITIVE

Primitive types are the most basic type of data in Java. They represent **single values** and are built into the language. There are eight: byte, short, int, long, float, double, char, boolean.

Non-primitive types are also called **reference types**, because they **refer to objects in memory**. They can store objects and can be user-defined. The four your course names: **Classes, Interfaces, Arrays, Strings**.

MEMORY HOOK: a primitive variable *is* the value; a reference variable is a *street address* where the value lives. The box holds the number itself, or it holds directions to it.

TEACH · THE EIGHT PRIMITIVES, WITH SIZES

Memory sizes are a stated learning outcome, so this table is examinable in full. It is reproduced from your course, **with three errors corrected**; the next box shows exactly what was wrong and why it matters.

TYPE	SIZE	DEFAULT	DESCRIPTION AND RANGE
byte	1 byte	0	8-bit signed integer. Range -128 to 127
short	2 bytes	0	16-bit signed integer. Range -32,768 to 32,767
int	4 bytes	0	32-bit signed integer. Range -2^{31} to $2^{31}-1$, that is -2,147,483,648 to 2,147,483,647
long	8 bytes	0L	64-bit signed integer. Range -2⁶³ to $2^{63}-1$
float	4 bytes	0.0f	32-bit floating-point. Precision up to 7 digits
double	8 bytes	0.0d	64-bit floating-point. Precision up to 15 digits
char	2 bytes	' '	a single 16-bit Unicode character, in single quotes: 'A', '1'
boolean	see below	false	stores true or false

The pattern is worth more than the table: sizes double down the integer family, 1, 2, 4, 8 bytes, and each doubling squares the range.

⚠ TRAP · THREE ERRORS IN YOUR COURSE'S OWN TABLE

Reproduced honestly, because you will sit an exam that may repeat them.

ROW	WHAT THE SLIDE SAYS	WHAT IS TRUE
int	"-2,147,483,648 to 2,147,483,648 "	the top is 2,147,483,647 . The formula beside it, $2^{31}-1$, is right; the number written out is one too big.
long	"Range is 2^{63} to $2^{63}-1$ "	a minus sign is missing: -2^{63} to $2^{63}-1$. As printed it reads "from about 9 quintillion to about 9 quintillion", which is not a range.
boolean	"1 bite "	a typo, and the honest answer is subtler: Java does not define the size of a boolean . It is left to the JVM, and typically occupies a whole byte. Write "not specified by the language, typically 1 byte" and you cannot be wrong.

The int one is worth savouring. The slide's claimed maximum, 2,147,483,648, is the one value that is **exactly too big to fit**. Ask Java for it and it wraps around to the most negative number there is.

CODE · ASKING THE JVM WHO IS RIGHT

VERIFIED ON THE JVM

```

1 public class Limits {
2     public static void main(String[] args) {
3         System.out.println("int min = " + Integer.MIN_VALUE);
4         System.out.println("int max = " + Integer.MAX_VALUE);
5         System.out.println("long min = " + Long.MIN_VALUE);
6         System.out.println("long max = " + Long.MAX_VALUE);
7
8         // the slide's claimed int max, one past the real one:
9         System.out.println("max + 1 = " + (Integer.MAX_VALUE + 1));
10    }
11 }

```

OUTPUT

```

int min = -2147483648
int max = 2147483647
long min = -9223372036854775808
long max = 9223372036854775807
max + 1 = -2147483648

```

Look at the last line. Add one to the largest int and you do not get an error, and you do not get 2,147,483,648. You get the *smallest* int. The counter runs off the end of its 32 bits and wraps to the far end, silently. This is called **integer overflow**, and it is a logic error in the sense of F.7: nothing warns you.

```

1 public class EightPrimitives {
2     public static void main(String[] args) {
3         byte    b = 100;
4         short   s = 32000;
5         int     i = 2000000;
6         long    big = 9000000000L;    // the L is not optional
7         float   rate = 3.14f;        // the f is not optional
8         double  d = 3.14159;
9         char    c = 'J';              // single quotes, exactly one character
10        boolean ok = true;
11
12        System.out.println("byte    " + b);
13        System.out.println("short   " + s);
14        System.out.println("int     " + i);
15        System.out.println("long    " + big);
16        System.out.println("float   " + rate);
17        System.out.println("double  " + d);
18        System.out.println("char    " + c);
19        System.out.println("boolean " + ok);
20    }
21 }

```

OUTPUT

```

byte    100
short   32000
int     2000000
long    9000000000
float   3.14
double  3.14159
char    J
boolean true

```

A table of types tells you a long holds nineteen digits. It cannot tell you that **writing one down needs an L**. That is what a listing is for, and it is why this box exists next to the table above.

● MUST-MEMORISE · A NUMBER LITERAL IS AN INT, AND A DECIMAL LITERAL IS A DOUBLE, UNTIL YOU SAY OTHERWISE

```

long big = 9000000000L;  the L is not optional.
float rate = 3.14f;    the f is not optional.

```

Every whole number you type is an int the moment you type it, **before** anything looks at what you are assigning it to. So 9000000000 is an int that will not fit in an int, and it is rejected where it stands. The L makes it a long literal instead.

Every decimal you type is a double in the same way. So 3.14 is a double, and putting a double into a float loses precision, which Java never does silently. The f makes it a float literal.

MEMORY HOOK: the literal is typed before it is assigned. Java reads what you wrote first and asks where it is going second, which is why the variable's type cannot rescue the number.

```

1 public class SuffixBroken {
2     public static void main(String[] args) {
3         long big = 90000000000;
4         float rate = 3.14;
5         System.out.println(big + " " + rate);
6     }
7 }

```

WHAT JAVAC SAYS

```

SuffixBroken.java:3: error: integer number too large
    long big = 90000000000;
                ^
1 error

```

One error, and there are two bugs. The float line is just as wrong, and the compiler has not mentioned it: "integer number too large" happens while it is still reading the characters, so it never gets as far as checking types. Add the L and recompile, and the second one appears: incompatible types: possible lossy conversion from double to float. Both were run to check.

This is worth meeting now, in Module 3, because it is how the compiler always behaves. **The number of errors it prints is not the number of bugs you have**, and the exam's debug questions are built on exactly that gap.

TEACH · THE FOUR NON-PRIMITIVE TYPES

TYPE	YOUR COURSE'S DEFINITION	MET IN
Class	a template used to create objects; it defines the properties (variables) and behaviour (methods) an object can have	Module 1, in full in Module 6
Interface	an abstract type defining a set of methods that a class must implement	Module 6
Array	a data structure that can hold multiple values of the same type	F.10, in full in Module 8
String	an object, used to store text	Module 7

This is why String is capitalised while int is not, the loose end left in F.8: **the eight primitives are keywords built into the language; every non-primitive is a class**, and Java's convention capitalises class names. The capital letter is a clue about what a thing is.

TEACH · THE WRAPPER CLASSES, AND THE CONSTANTS YOU SHOULD STOP MEMORISING

Every primitive has a **class** that shadows it: int has Integer, long has Long, double has Double, char has Character, boolean has Boolean. They are called the **wrapper classes**, they live in java.lang so they need no import, and each carries the useful things that a primitive, not being an object, cannot carry itself.

Two of those things you have already used, and two more are worth knowing now.

```

1 public class Wrappers {
2     public static void main(String[] args) {
3         System.out.println("Integer.MAX_VALUE = " + Integer.MAX_VALUE);
4         System.out.println("Integer.MIN_VALUE = " + Integer.MIN_VALUE);
5         System.out.println("Long.MAX_VALUE = " + Long.MAX_VALUE);
6         System.out.println("Long.MIN_VALUE = " + Long.MIN_VALUE);
7
8         System.out.println("Integer.parseInt(\"25\") + 1 = " + (Integer.parseInt("25") + 1));
9         System.out.println("Double.parseDouble(\"3.5\") * 2 = " + (Double.parseDouble("3.5") * 2));
10
11        System.out.println("Integer.toBinaryString(13) = " + Integer.toBinaryString(13));
12
13        // the overflow the slides got wrong, stated by the constant itself
14        System.out.println("MAX_VALUE + 1 = " + (Integer.MAX_VALUE + 1));
15    }
16 }

```

OUTPUT

```

Integer.MAX_VALUE = 2147483647
Integer.MIN_VALUE = -2147483648
Long.MAX_VALUE = 9223372036854775807
Long.MIN_VALUE = -9223372036854775808
Integer.parseInt("25") + 1 = 26
Double.parseDouble("3.5") * 2 = 7.0
Integer.toBinaryString(13) = 1101
MAX_VALUE + 1 = -2147483648

```

● MUST-MEMORISE · FOUR WRAPPER MEMBERS YOU WILL ACTUALLY USE

MEMBER	WHAT IT IS FOR
Integer.MAX_VALUE Integer.MIN_VALUE Long.MAX_VALUE, and so on	Constants , so no brackets. The largest and smallest value the type can hold. You never have to remember the digits, and you must never type them out.
Integer.parseInt(s)	Text to int. F.11: every dialog hands you a String.
Double.parseDouble(s)	Text to double.
Integer.toBinaryString(n)	The number written in binary , as text. 13 gives "1101".

MEMORY HOOK: the primitive holds the value; the wrapper knows about the type. How big it can get, how to read one out of text, how to write one in binary: that knowledge lives on the class.

⚠ TRAP · THE CONSTANT IS ALSO THE PROOF YOUR SLIDES ARE WRONG

Read the last line of that output. `Integer.MAX_VALUE + 1` printed **-2147483648**.

Now read it beside the error three boxes above, where the course's own table gives `int`'s maximum as **2,147,483,648**. That number is not a near miss and it is not a typo for something harmless: **it is precisely the value you get by overflowing the real maximum**. Add one to the biggest `int` and it wraps around to the most negative one, and the digits that come out are the digits the slide printed, with a minus sign the slide dropped.

So the table did not merely mistype the limit. It printed the symptom of exceeding the limit and labelled it the limit.

This is the whole argument for `Integer.MAX_VALUE`. Type the digits and you can be wrong, as your own course was, in print, in a table whose job was those digits. Write the constant and you cannot be.

FILL IN THE GAP · TYPES

1. There are _____ primitive types. A non-primitive type is also called a _____ type because it _____.
2. `int` occupies _____ bytes; `double` occupies _____ bytes; `char` occupies _____ bytes.
3. The default value of a `boolean` is _____; of a `long` it is _____.
4. A `char` literal is written in _____ quotes; a `String` literal in _____ quotes.
5. The largest `int` is _____; adding one to it gives _____.

ANSWERS

1. eight, reference, refers to an object in memory · 2. 4, 8, 2 · 3. `false`, `0L` · 4. `single`, `double` · 5. 2,147,483,647, -2,147,483,648 (it overflows)

UNIT 3.4 · CONSTANTS

final: a box that will not change

TEACH · CONSTANTS

A **constant** is a data container whose value cannot change once it has been assigned. Constants improve readability and prevent accidental modification. In Java you declare one with the `final` keyword.

```
final dataType CONSTANT_NAME = value;
```

By convention constant names are written in **capitals with underscores**. That is a convention, not a rule: the compiler accepts `final double pi`, and every Java programmer alive will read it as a mistake.

CODE · FINAL, AND WHAT HAPPENS IF YOU FIGHT IT

```
1 public class FinalDemo {
2     public static void main(String[] args) {
3         final double PI = 3.14159;
4         final int MAX_USERS = 100;
5         System.out.println(PI + " " + MAX_USERS);
6
7         PI = 3.14;           // the compiler refuses this
8     }
9 }
```

WHAT THE COMPILER SAYS

```
FinalDemo.java:7: error: cannot assign a value to final variable PI
    PI = 3.14;
    ^
1 error
```

Note *when* this is caught: at compile time, not run time. The program never runs at all. That is the value of `final`, in the sense of F.7: it converts a possible logic error into a compile-time error, which is the cheapest kind.

Your course's own example: total price of some items including 15% tax, with the rate as a constant.

```
1 public class TaxInclusivePricing {
2     public static void main(String[] args) {
3         final double TAX_RATE = 0.15;        // 15%, fixed
4         int quantity = 3;
5         double basePrice = 250.0;
6
7         double total = quantity * basePrice * (1 + TAX_RATE);
8         System.out.println("Total: " + total);
9     }
10 }
```

OUTPUT

```
Total: 862.4999999999999
```

Look at that output. Three items at 250.00 plus 15% is exactly 862.50, and Java printed 862.4999999999999. Nothing is broken and this is not a bug in your code: it is what `double` is, and Unit 3.6 explains it. It is also why real money is never stored in a `double`. Your course's slide does not mention this; the machine does.

Now redo it: write the same program with `TAX_RATE` as an ordinary variable rather than `final`, and say in one sentence what you have lost.

RECALL AND CHECK

Why declare `final double INTEREST_RATE = 0.05`; rather than just writing `0.05` wherever it is needed?

ANSWER

Three reasons. Readability: `INTEREST_RATE` says what the number means and `0.05` does not. Safety: `final` makes accidental modification a compile-time error rather than a silent logic error. And change: the rate lives in one place, so a new rate is a one-line edit rather than a hunt for every `0.05` in the file, some of which might mean something else entirely.

Every operator Java gives you

TEACH · OPERATORS

Operators are symbols that perform operations on variables, constants or values. Your course names eight families. "Identify various operators" is a learning outcome, so know the list and one example of each.

FAMILY	SYMBOLS	DOES
1. Arithmetic	+ - * / %	basic mathematics
2. Relational	== != > >= < <=	compares two values, gives true or false
3. Logical	&& ! ^	combines boolean values
4. Assignment	= += -= *= /=	puts a value into a variable
5. Unary	++ -- + - !	acts on a single operand
6. Ternary	? :	shorthand for if-else
7. instanceof	instanceof	is this object an instance of that class?
8. Bitwise	& ^ ~ << >>	works on the individual bits of a number

Families 1 to 5 are the working set and are covered in full below. Family 6 is here. Family 7 needs objects, so it waits for Module 6. Family 8 is named by your course and not used by it, and it has never appeared on a paper we have; know that it exists and manipulates individual bits.

SYNTAX · THE TERNARY OPERATOR

```
condition ? value1 : value2
```

read it as: if (condition) give me value1, otherwise give me value2

It takes **three** operands, which is what "ternary" means, and it is the only operator in Java that does.

CODE · TERNARY, AND THE IF-ELSE IT REPLACES

```

1 public class Ternary {
2     public static void main(String[] args) {
3         int n = 7;
4
5         // the ternary way, in one line
6         String a = (n % 2 == 0) ? "even" : "odd";
7
8         // exactly the same thing, the long way
9         String b;
10        if (n % 2 == 0) { b = "even"; } else { b = "odd"; }
11
12        System.out.println(n + " is " + a + " / " + b);
13    }
14 }
```

OUTPUT

```
7 is odd / odd
```

You have already met this operator without being told its name: the 2025/26 paper's Armstrong question ends with (sum == n ? "ARMSTRONG" : "NOT ARMSTRONG"). If you cannot read that line you cannot answer a ten-mark question.

Five symbols, and four ways they surprise you

TEACH · THE FIVE

Your course's own table, with $a = 7$, $b = 2$, $x = 18.0$, $y = 9.0$:

OP	NAME	EXAMPLE	RESULT	NOTES
+	Addition	$a + b$	9	int, long, float, double
-	Subtraction	$a - b$	5	all numeric types
*	Multiplication	$a * b$	14	all numeric types
/	Division	$a / b = 7/2$	3	int / int truncates. $x / y = 2.0$
%	Modulus	$a \% b = 7\%2$	1	the remainder after division

% is worth dwelling on because this course leans on it constantly. $a \% b$ is what is left over: 7 divided by 2 is 3 remainder 1, so $7 \% 2$ is 1. Every "is it even", "is it a multiple of 4", "get the last digit" question in your past papers is %.

● MUST-MEMORISE · THE THREE THINGS % BUYS YOU

$n \% 2 == 0$ means **n is even**
 $n \% k == 0$ means **n divides exactly by k**
 $n \% 10$ gives **the last digit of n**, and $n / 10$ removes it

That last line is an entire exam technique. The Armstrong, palindrome, digit-sum and Harshad questions on your papers are all built from $\% 10$ and $/ 10$ in a loop, and nothing else.

MEMORY HOOK: % peels the last digit off, / pushes it away. Together they walk a number's digits from right to left.

WORKED EXAMPLE · WALKING A NUMBER'S DIGITS

EXAM TECHNIQUE

This tiny loop is the engine of four separate past-paper questions. Learn it as a shape.

```

1 public class Digits {
2     public static void main(String[] args) {
3         int n = 4821;
4         int temp = n, sum = 0, reversed = 0;
5
6         while (temp > 0) {
7             int d = temp % 10;           // peel the last digit off
8             sum += d;                     // use it
9             reversed = reversed * 10 + d; // build the reverse
10            temp /= 10;                   // push it away
11        }
12        System.out.println("n          = " + n);
13        System.out.println("digitsum = " + sum);
14        System.out.println("reversed = " + reversed);
15    }
16 }
```

OUTPUT

```

n          = 4821
digitsum = 15
reversed = 1284
```

DRY RUN · DIGITS, FOR N = 4821

Cover it and trace first. The 2025/26 paper states that for $N = 4821$ the reversed value is 1284, so this is that question's engine, checked.

$N = 4821$

PASS	TEMP	D = TEMP%10	SUM	REVERSED	TEMP /= 10
start	4821	-	0	0	-
1	4821	1	0+1 = 1	0*10+1 = 1	482
2	482	2	1+2 = 3	1*10+2 = 12	48
3	48	8	3+8 = 11	12*10+8 = 128	4
4	4	4	11+4 = 15	128*10+4 = 1284	0
test	0	-	15	1284	0 > 0 false: stop

Notice reversed = reversed * 10 + d: multiplying by 10 shifts what you have left, and adding d drops the new digit into the gap. That one line reverses any number, and it is worth memorising as a phrase.

3.6.1 The pitfalls

TEACH · YOUR COURSE NAMES THREE

Division by zero, modulus with negative numbers, and string concatenation. Its example program is below, and it is worth more than your course claims.

CODE · THE SLIDE'S PROGRAM, EXACTLY AS PRINTED

FROM THE SLIDES

```
1 public class ArithmeticOperatorsPitfalls {
2     public static void main(String[] args) {
3         System.out.println(10/0);
4         System.out.println(10.0/0);
5         System.out.println(-7%3);
6         System.out.println(7%-3);
7     }
8 }
```

Your course says: line 3 throws an `ArithmeticException`; line 4 prints `Infinity`; line 5 prints `-1`; line 6 prints `1`. Every one of those statements about the **expressions** is true. The claim about the **program** is not.

⚠ TRAP · LINES 4, 5 AND 6 NEVER RUN

An uncaught exception **stops the program dead**. Line 3 throws, so the JVM abandons main there and lines 4, 5 and 6 are never reached. Run this program exactly as printed and it produces **no output at all**, only a crash report:

WHAT ACTUALLY HAPPENS

```
Exception in thread "main" java.lang.ArithmeticException: / by zero
    at ArithmeticOperatorsPitfalls.main(ArithmeticOperatorsPitfalls.java:3)
```

So if an exam hands you this program and asks "state the output", the answer is **not** "Infinity, -1, 1". It is: the program throws `ArithmeticException: / by zero` at line 3 and terminates, printing none of the remaining lines.

This matters beyond one slide. It is the difference between reading code line by line as four independent facts, and dry running it as one program with a flow that can stop.

```

1 public class Pitfalls {
2     public static void main(String[] args) {
3         try {
4             System.out.println(10/0);           // int / int -> throws
5         } catch (ArithmeticException e) {
6             System.out.println("line 3 -> " + e.getMessage());
7         }
8         System.out.println("line 4 -> " + (10.0/0)); // double -> Infinity
9         System.out.println("line 5 -> " + (-7%3)); // sign follows the LEFT
10        System.out.println("line 6 -> " + (7%-3)); // sign follows the LEFT
11    }
12 }

```

OUTPUT

```

line 3 -> / by zero
line 4 -> Infinity
line 5 -> -1
line 6 -> 1

```

● MUST-MEMORISE · THE THREE ARITHMETIC PITFALLS

1. Division by zero depends on the type.

10/0 with ints **throws** `ArithmeticException` and stops the program.

10.0/0 with a double gives **Infinity** and carries on. (And 0.0/0.0 gives NaN, "not a number".)

2. Modulus keeps the sign of the LEFT operand.

-7 % 3 is **-1**. 7 % -3 is **1**.

3. + joins when either side is text.

"25" + 1 is "251", not 26.

MEMORY HOOK: for modulus, **the remainder inherits its sign from the number being divided, not the divider**. Ask "what is left of -7?" and the answer is negative.

⚠ TRAP · INTEGERS THROW, DOUBLES DO NOT

This asymmetry catches everyone. There is no int that can represent infinity, so Java has no choice but to throw. double has a genuine Infinity value built into it, so it returns one and the program keeps running with a poisoned number in it, which is arguably worse: a crash is loud, an Infinity quietly contaminates every later calculation.

TEACH · WHEN THE FIVE OPERATORS ARE NOT ENOUGH: THE MATH CLASS

Java gives you + - * / % and nothing else. There is no operator for a square root, none for a power, and none for "the bigger of these two". Those live as **methods on a class called Math**, which comes with the JDK.

You do not import it. Math lives in `java.lang`, the one package Java imports into every file for you (String and System are there too, which is why they have never needed an import either). And you never make a Math object: every one of its methods is static, so you call them on the class itself, exactly as F.6 described and Unit 5.5 will explain properly.

```

1 public class MathTools {
2     public static void main(String[] args) {
3         System.out.println("Math.PI           = " + Math.PI);
4         System.out.println("Math.sqrt(9)      = " + Math.sqrt(9));
5         System.out.println("Math.pow(2, 10)   = " + Math.pow(2, 10));
6         System.out.println("Math.abs(-7)     = " + Math.abs(-7));
7         System.out.println("Math.max(3, 9)   = " + Math.max(3, 9));
8         System.out.println("Math.min(3, 9)   = " + Math.min(3, 9));
9         System.out.println("Math.round(2.4) = " + Math.round(2.4));
10        System.out.println("Math.round(2.5) = " + Math.round(2.5));
11        System.out.println("Math.round(-2.5) = " + Math.round(-2.5));
12    }
13 }

```

OUTPUT

```

Math.PI           = 3.141592653589793
Math.sqrt(9)      = 3.0
Math.pow(2, 10)   = 1024.0
Math.abs(-7)     = 7
Math.max(3, 9)   = 9
Math.min(3, 9)   = 3
Math.round(2.4)  = 2
Math.round(2.5)  = 3
Math.round(-2.5) = -2

```

● MUST-MEMORISE · THE MATH MEMBERS WORTH KNOWING BY HEART

CALL	GIVES BACK	WHAT IT IS FOR
Math.PI	double	Not a method: a constant , so no brackets. 3.141592653589793.
Math.sqrt(x)	double	Square root.
Math.pow(a, b)	double	a to the power b.
Math.abs(x)	same type as x	Distance from zero, so never negative.
Math.max(a, b) Math.min(a, b)	same type	The larger, and the smaller, of two.
Math.round(x)	long (from a double)	Nearest whole number.

MEMORY HOOK: Math hands back doubles. sqrt, pow and PI are all double, whatever you feed them.

1. Math.pow returns a double, always. It printed 1024.0, not 1024. So this does not compile:

```
int cube = Math.pow(d, 3);    incompatible types: possible lossy conversion from double to int
```

You must cast: `int cube = (int) Math.pow(d, 3);`.

And now a myth, because you will be told it. You will hear that `Math.pow(5, 3)` can come back as 124.999999999, so casting to `int` truncates it to 124. **In Java, that is not true**, and this book does not repeat claims it has not run. We checked every base from 2 to 1000 and every power from 2 to 12 whose exact answer fits in a double: `Math.pow` returned the **exact** value every single time, zero exceptions. `Math.pow(5, 3) == 125.0` is true.

What is true is smaller and duller: a double can only hold whole numbers exactly up to about 9×10^{15} (Unit 3.3). Past that it rounds, because there are no bits left, and no method can rescue you from it. That is the double's limit, not a flaw in `Math.pow`.

So why does the Armstrong answer on the 2025/26 paper multiply in a loop rather than call `Math.pow`? Not for accuracy. Because it wants an `int`, and `pow` hands back a double: the loop skips the cast entirely, and the paper's own code does it that way.

Do keep one thing from the myth, because the **cast** really does behave that way: `(int)` **truncates**, it does not round (Unit 3.3). `(int) 124.999999999` is 124. If a value ever does arrive slightly under, the cast will not save you: that is what `Math.round` is for.

2. Math.round rounds half up, and "up" means toward positive infinity, not away from zero. So 2.5 gives 3 and -2.5 gives -2, not -3. The rule is `Math.round(x) = Math.floor(x + 0.5)`, and adding 0.5 to -2.5 gives exactly -2.0. Both halves round the same direction on the number line, which is not what "round half up" sounds like it means.

Note also `Math.round(2.4)` gave 2, an **int-looking** value that is really a long. Assigning it to an `int` needs a cast too.

UNIT 3.7 · RELATIONAL OPERATORS

Comparing, and the one comparison you must never make

TEACH · THE SIX

Relational operators compare two values and return a boolean, true or false. With `a = 5`, `b = 3`:

OP	NAME	EXAMPLE	RETURNS	NOTES
<code>==</code>	Equal to	<code>a == b</code>	false	compares values for primitives, references for objects
<code>!=</code>	Not equal to	<code>a != b</code>	true	the opposite of <code>==</code>
<code>></code>	Greater than	<code>a > b</code>	true	numbers
<code><</code>	Less than	<code>a < b</code>	false	numbers
<code>>=</code>	Greater or equal	<code>a >= b</code>	true	numbers
<code><=</code>	Less or equal	<code>a <= b</code>	false	numbers

Hold on to that note against `==`: **values for primitives, references for objects**. It is quiet now and it detonates in Module 7, where comparing two Strings with `==` is the single most famous bug in Java.

```

1 public class Compare {
2     public static void main(String[] args) {
3         int y = 5, x = 5;
4         double a = 0.1 + 0.2;
5         double b = 0.3;
6         System.out.println(y == x);           // true
7         System.out.println(a == b);           // false. Look below.
8         System.out.println(a);                // why it is false
9
10        int num = 10;
11        double decimal = 10.0;
12        char letter = 'A';
13        System.out.println(num == decimal);    // true: compares values, not types
14        System.out.println(letter > num);      // true: 'A' is 65
15    }
16 }

```

OUTPUT

```

true
false
0.30000000000000004
true
true

```

● MUST-MEMORISE · NEVER COMPARE DOUBLES WITH ==

$0.1 + 0.2 == 0.3$ is **false**, because $0.1 + 0.2$ is actually 0.30000000000000004 .

A double stores a number in binary, and 0.1 has no exact binary form, exactly as $1/3$ has no exact decimal form. The tiny error is real and it survives the addition.

To compare two doubles, ask whether they are **close enough**:

```
if (Math.abs(a - b) < 0.0000001) { ... }
```

MEMORY HOOK: writing $1/3$ in decimal you must stop somewhere: 0.3333. Binary has to stop somewhere too, and 0.1 is binary's $1/3$. **Doubles are approximations wearing a convincing disguise.**

⚠ TRAP · == COMPARES VALUES, NOT TYPES

`num == decimal`, comparing `int` 10 with `double` 10.0, is **true**. Java widens the `int` to a `double` and compares the values. The types differ; the values do not; `==` only cares about the values.

And `letter > num`, comparing `'A'` with 10, is **true**, because a `char` is a number: `'A'` is 65. This is why `char` sits in the integer family of primitives rather than off on its own.

Combining conditions, and the shortcut Java takes

TEACH · THE FOUR

Logical operators combine boolean values. With `a = true` and `b = false`:

OP	NAME	RETURNS TRUE WHEN	EXAMPLE
<code>&&</code>	Logical AND	both operands are true	<code>a && b = false</code>
<code> </code>	Logical OR	at least one operand is true	<code>a b = true</code>
<code>!</code>	Logical NOT	reverses the value	<code>!a = false</code>
<code>^</code>	Logical XOR	exactly one operand is true	<code>a ^ b = true</code>

XOR is the one people forget: exclusive OR, true when the two sides *differ*. `true ^ true` is false, where `true || true` is true.

CODE · ALL FOUR, WITH EVERY COMBINATION

```

1 public class Logic {
2     public static void main(String[] args) {
3         boolean[] vals = {true, false};
4         System.out.println(" p      q      p&&q    p||q    p^q    !p");
5         for (boolean p : vals) {
6             for (boolean q : vals) {
7                 System.out.printf(" %-6b %-6b %-6b %-6b %-6b %-6b%n",
8                                     p, q, (p && q), (p || q), (p ^ q), (!p));
9             }
10        }
11    }
12 }

```

OUTPUT

```

p      q      p&&q    p||q    p^q    !p
true   true    true    true    false  false
true   false   false    true    true   false
false  true     false    true    true   true
false  false   false    false   false  true

```

That is a complete truth table, and it was printed by the machine rather than typed by me. Read the `p^q` column against `p||q`: they differ in exactly one row, the one where both are true.

● MUST-MEMORISE · SHORT-CIRCUIT EVALUATION

`&&` stops as soon as it meets a **false**: if the left side is false the whole thing is false, so **the right side is never evaluated**.

`||` stops as soon as it meets a **true**, for the mirror reason.

This is not an optimisation you can ignore. It is a tool: it is what lets you write `if (i < a.length && a[i] > 0)` safely. The left guard runs first, and when it is false the array is never touched, so no exception. Swap the two around and the same line crashes.

MEMORY HOOK: `&&` is a pessimist and `||` is an optimist. The pessimist stops at the first bad news; the optimist stops at the first good news. Neither reads any further.

CODE · SHORT-CIRCUITING, MADE VISIBLE

```

1 public class ShortCircuit {
2     // prints when it is called, so we can see IF it is called
3     static boolean loud(String tag, boolean v) {
4         System.out.println("    (evaluated " + tag + ")");
5         return v;
6     }
7
8     public static void main(String[] args) {
9         System.out.println("false && loud:");
10        boolean r1 = false && loud("right", true);
11        System.out.println("    result " + r1);
12
13        System.out.println("true || loud:");
14        boolean r2 = true || loud("right", false);
15        System.out.println("    result " + r2);
16
17        System.out.println("true && loud:");
18        boolean r3 = true && loud("right", true);
19        System.out.println("    result " + r3);
20    }
21 }

```

OUTPUT

```

false && loud:
    result false
true || loud:
    result true
true && loud:
    (evaluated right)
    result true

```

The proof is in what is **missing**. In the first two cases "(evaluated right)" never prints, because the right-hand side was never run. In the third it does, because true && ? still has to look.

FILL IN THE GAP · OPERATORS

1. $7 / 2$ is _____ but $7.0 / 2$ is _____.
2. $10 / 0$ _____ while $10.0 / 0$ gives _____.
3. $-7 \% 3$ is _____ because the remainder takes the sign of the _____ operand.
4. $0.1 + 0.2 == 0.3$ is _____ because the sum is really _____.
5. && does not evaluate its right side when the left is _____. This is called _____.
6. To test that n is even, write n _____ $2 == 0$. To get the last digit of n , write _____.
7. The ternary operator takes _____ operands and its syntax is _____.

ANSWERS

1. 3, 3.5 · 2. throws ArithmeticException, Infinity · 3. -1, left · 4. false, 0.30000000000000004 · 5. false, short-circuit evaluation · 6. %, $n \% 10$ · 7. three, condition ? value1 : value2

UNIT 3.9 · OPERATOR PRECEDENCE

Which operator goes first

TEACH · PRECEDENCE

When several operators meet in one expression, **precedence** decides which acts first. You already know this from arithmetic: $2 + 3 * 4$ is 14, not 20, because multiplication binds tighter than addition. Java works the same way and extends it to every operator it has.

"Why is operator precedence important?" and "how can parentheses change the evaluation of expressions?" are both stated in-text questions, so both are candidates for the define slot.

● MUST-MEMORISE · PRECEDENCE, HIGHEST FIRST

1. () parentheses
2. ++ -- ! unary
3. * / % multiplicative
4. + - additive
5. < <= > >= relational
6. == != equality
7. && logical AND
8. || logical OR
9. ?: ternary
10. = += -= *= /= assignment (last, and right to left)

Two facts do most of the work: **arithmetic beats comparison, comparison beats logic, and assignment is always last.** That is why `x = a + b > c && d` parses the way you would hope: add, then compare, then AND, then assign.

MEMORY HOOK: the order is the order you would do it on paper. **Work out the numbers, then compare them, then combine the answers, then write the result down.** Assignment is writing it down, so it is last.

CODE · PRECEDENCE, AND PARENTHESES OVERRIDING IT

```
1 public class Precedence {
2     public static void main(String[] args) {
3         System.out.println(2 + 3 * 4);        // * first
4         System.out.println((2 + 3) * 4);      // parentheses win
5
6         int a = 5, b = 3, c = 2;
7         System.out.println(a - b - c);        // left to right: (5-3)-2
8         System.out.println(a - (b - c));      // forced the other way
9
10        // arithmetic, then relational, then logical
11        System.out.println(a + b > c && b < a);
12        System.out.println(10 / 2 * 5);       // same rank: left to right
13    }
14 }
```

OUTPUT

```
14
20
0
4
true
25
```

The last line is the one that catches people. / and * have the **same** precedence, so Java goes left to right: $(10 / 2) * 5 = 25$, not $10 / (2 * 5) = 1$. Equal precedence is settled by **associativity**, and for arithmetic that means left first.

⚠ TRAP · PARENTHESES COST NOTHING

You will not be asked to recite the precedence table under exam pressure, and you should not rely on it under any pressure. **Write the parentheses.** $(a + b) > (c * d)$ takes two seconds longer to type, is impossible to misread, and cannot be wrong. Every professional Java programmer does this, and the reason is not ignorance of the table.

Where it does matter is *reading*: an exam hands you an expression without parentheses and asks what it evaluates to. Then you need the table.

Three ways to get data from the user

TEACH · THE THREE

Input handling lets users provide data from the keyboard, files or networks. Your course names three ways:

WAY	LIVES IN	USE IT WHEN
Scanner	java.util	almost always. The most common, and the one to reach for.
BufferedReader with InputStreamReader	java.io	you want speed, or you are already reading files (Module 10).
Console	java.io	you need a password typed without it appearing on screen.

And there is a fourth this course actually examines most: **JOptionPane**, from F.11, which asks in a pop-up window. The 2025/26 paper demands it by name. Know all four; reach for Scanner when the question does not say.

CODE · SCANNER

FROM THE SLIDES

```

1 import java.util.Scanner;
2 public class ScannerInput {
3     public static void main(String[] args) {
4         Scanner input = new Scanner(System.in);
5
6         System.out.print("Enter your name: ");
7         String name = input.nextLine();
8         System.out.print("Welcome " + name);
9
10        input.close();
11    }
12 }
```

OUTPUT, WITH "JOHN DOE" TYPED

Enter your name: Welcome John Doe

Your course shows this as two lines, "Enter your name: John Doe" then "Welcome John Doe". Both are right, and the difference is worth understanding. When a *person* types, the terminal echoes their keystrokes and their Enter moves the cursor down, so "John Doe" and the line break appear on screen without the program printing them. Feed the same program its input from a file and that echo does not happen, so you get the single line above. **The program printed exactly the same characters both times.**

TEACH · READING SCANNER'S FOUR LINES

LINE	WHAT IT IS DOING
<code>import java.util.Scanner;</code>	tells the compiler where Scanner lives. Without it, "cannot find symbol".
<code>new Scanner(System.in)</code>	makes a Scanner object reading from <code>System.in</code> , which is the keyboard. <code>new</code> creates an object, exactly as in Module 1.
<code>input.nextLine()</code>	waits for the user to press Enter, then hands back everything typed, as a <code>String</code> .
<code>input.close()</code>	releases the resource. Good manners, and Module 10 explains why it matters.

● MUST-MEMORISE · SCANNER'S READ METHODS

`nextLine()` reads a whole line, returns `String`
`next()` reads one word, returns `String`
`nextInt()` reads a whole number, returns `int`
`nextDouble()` reads a decimal, returns `double`
`nextBoolean()` returns `boolean`

Note the difference from `JOptionPane` in F.11: a dialog always hands you text and you must parse it. `Scanner` will do the conversion for you if you ask the right method. `nextInt()` needs no `Integer.parseInt()`.

MEMORY HOOK: the method name says what you get back. `nextInt` gives an `int`. The only one that surprises is `nextLine` versus `next`: **line takes everything, next takes one word.**

⚠ TRAP · NEXTINT() THEN NEXTLINE() EATS YOUR INPUT

The most notorious `Scanner` bug, and it looks like the computer ignoring you. `nextInt()` takes the digits and **leaves the Enter key sitting in the buffer**. The next `nextLine()` finds that leftover newline, says "ah, an empty line", and returns immediately without letting you type.

The fix is an extra `input.nextLine();` after the `nextInt()`, purely to consume the stray newline. It looks like a pointless line of code and it is load-bearing.

CODE · THE TRAP, AND THE FIX, BOTH RUN

```
1 import java.util.Scanner;
2 public class ScannerTrap {
3     public static void main(String[] args) {
4         Scanner in = new Scanner(System.in);
5
6         // BROKEN: nextInt leaves the newline behind
7         int age = in.nextInt();
8         String city = in.nextLine(); // grabs the leftover newline
9         System.out.println("broken -> age=" + age + " city=[" + city + "]");
10
11         in.nextLine(); // clear the rest of that line
12
13         // FIXED: consume the newline before reading the line
14         int age2 = in.nextInt();
15         in.nextLine(); // <- the load-bearing line
16         String city2 = in.nextLine();
17         System.out.println("fixed -> age=" + age2 + " city=[" + city2 + "]");
18         in.close();
19     }
20 }
```

OUTPUT, GIVEN 25 / LAGOS / 30 / ABUJA

```
broken -> age=25 city=[]
fixed -> age=30 city=[Abuja]
```

Look at `city=[]`. The program never waited, and "Lagos" was never read. Nothing crashed and nothing warned: a logic error, in the sense of F.7.

```

1 import java.io.*;
2 public class BufferedReaderInput {
3     public static void main(String[] args) throws IOException {
4         BufferedReader br =
5             new BufferedReader(new InputStreamReader(System.in));
6
7         System.out.print("Enter an integer: ");
8         int n = Integer.parseInt(br.readLine());
9
10        System.out.println("You entered: " + n);
11        br.close();
12    }
13 }

```

OUTPUT, WITH 42 TYPED

Enter an integer: You entered: 42

Two things to notice. `readLine()` always returns a `String`, so `Integer.parseInt` is compulsory, exactly as with `JOptionPane` in F.11. And `throws IOException` on `main`: reading can fail, and Java forces you to acknowledge it. Module 10 explains what that phrase really means; for now, without it this will not compile.

TEACH · CONSOLE, AND WHY IT MAY HAND YOU NOTHING

The third way, for passwords. `System.console()` gives a `Console` object whose `readPassword` reads without echoing the characters to the screen.

```

import java.io.Console;

public class ConsoleExample {
    public static void main(String[] args) {
        Console console = System.console();
        if (console == null) {                                <- this check matters
            System.err.println("Console not available!");
            return;
        }
        String username = console.readLine("Enter username: ");
        char[] passwordChars = console.readPassword("Enter password: ");
        String password = new String(passwordChars);
        System.out.println("Username: " + username);
    }
}

```

This is the one listing in this book that is not run for you, and the reason is the lesson: **`System.console()` returns null unless the program is attached to a real terminal**. Run it from an IDE, or with its input piped from a file, and there is no console, so the program prints "Console not available!" and stops. That is why your course's slide checks for null first, and why the check is not optional politeness.

Notice also that `readPassword` returns `char[]`, not `String`: a security decision, so the password can be erased from memory afterwards. The slide's own comment, "don't do this in real apps", is about printing it out.

print, println, printf

● MUST-MEMORISE · THE THREE OUTPUT METHODS

System.out.print(x) prints, cursor **stays** on the line.

System.out.println(x) prints, cursor **moves** to the next line.

System.out.printf(fmt, ...) prints with **formatting**, and does **not** move to a new line unless you ask with %n.

"What are the differences between print, println and printf" is a stated in-text question, which makes it a strong candidate for the 2 or 3 mark slot. Answer with all three, including the %n detail.

MEMORY HOOK: ln means line-end. The \n is not decoration; it is the newline, and it is the only difference between the first two.

CODE · PRINTF AND THE FORMAT STRING

FROM THE SLIDES

```
1 public class PrintFormatOne {
2     public static void main(String[] args) {
3         double pi = 3.14159;
4         System.out.printf("The value of Pi is %.2f", pi);
5         // Prints pi with two decimal places
6     }
7 }
```

OUTPUT

The value of Pi is 3.14

TEACH · HOW PRINTF WORKS

The first argument is a **format string**: ordinary text with **format specifiers** in it. Each specifier starts with % and is a slot. The arguments after the format string fill the slots, left to right.

```
System.out.printf("%s scored %d, average %.1f%n", name, mark, avg);
```

Three slots, three arguments, in order. The letter after the % says what kind of value goes there, and getting it wrong is a run-time crash, not a compile error.

TEACH · THE FORMAT SPECIFIERS

Your course's table, **with two corrections**, each flagged and explained in the trap below. Every Example column here was produced by running it.

FORMAT	DESCRIPTION	EXAMPLE OUTPUT
%s	String	John
%d	Integer	100
%f	Floating-point, six decimals by default	3.141590
%.2f	2 decimal places	3.14
%c	Character	A
%x or %X	Hexadecimal, lower or upper case	ff or FF
%e / %E	Scientific notation, lower or upper	1.234568e+06
%05d	Pad with zeros to width 5	00005
%+d	Force a sign	+10
%-5d	Left justified, width 5	10
%5d	Right justified, minimum width 5	5
%n	A new line	(moves the cursor)

⚠ TRAP · TWO ERRORS IN THE SPECIFIER TABLE

THE SLIDE SAYS	RUNNING IT GIVES	WHY
%f of 3.14159 is 3.14159	3.141590	bare %f always prints six decimal places, padding with a zero. To get 3.14159 you must ask: %.5f.
"%x or %X"	X% throws UnknownFormatConversionException	the % comes first , always. It is %X. As printed, the program crashes at run time.

The %f one is the dangerous one, because it is silent. Ask for %f expecting your number back and you get a spurious trailing zero in your output, and an exam asking for exact output will not accept it.

```
1 public class Specifiers {
2     public static void main(String[] args) {
3         System.out.printf("%s    -> [%s]\n", "John");
4         System.out.printf("%d    -> [%d]\n", 100);
5         System.out.printf("%f    -> [%f]\n", 3.14159);
6         System.out.printf("%.2f  -> [%.2f]\n", 3.14159);
7         System.out.printf("%c    -> [%c]\n", 'A');
8         System.out.printf("%x    -> [%x]   %X -> [%X]\n", 255, 255);
9         System.out.printf("%e    -> [%e]\n", 1234567.891);
10        System.out.printf("%05d   -> [%05d]\n", 5);
11        System.out.printf("%+d    -> [%+d]\n", 10);
12        System.out.printf("%-5d   -> [%-5d]\n", 10);
13        System.out.printf("%5d    -> [%5d]\n", 5);
14    }
15 }
```

OUTPUT

```
%s    -> [John]
%d    -> [100]
%f    -> [3.141590]
%.2f  -> [3.14]
%c    -> [A]
%x    -> [ff]   %X -> [FF]
%e    -> [1.234568e+06]
%05d   -> [00005]
%+d    -> [+10]
%-5d   -> [10   ]
%5d    -> [    5]
```

The square brackets are there so you can see the padding. Note % in the format string: that is how you print a literal percent sign, since a lone % would start a specifier.

A past paper asks for student records "in a formatted table" using `printf` "for proper column alignment". This is the technique.

- 1 Decide a width for each column, wide enough for the longest entry plus breathing room.
Name needs 6 for "Lilian", so take 8.
- 2 Left justify text with `%-8s`, right justify numbers with `%5d`.
This is what makes a table readable: text lines up on the left, digits line up on the right.
- 3 Give decimals a fixed number of places with `%.2f`, so the points align in a column.
- 4 End every line with `%n`.

```

1 public class Table {
2     public static void main(String[] args) {
3         String[] names = {"John", "Lilian", "Ken"};
4         int[] ages      = {20, 21, 19};
5         double[] gpas   = {3.75, 3.95, 3.65};
6
7         System.out.printf("%-8s %5s %6s%n", "Name", "Age", "GPA");
8         for (int i = 0; i < names.length; i++) {
9             System.out.printf("%-8s %5d %6.2f%n", names[i], ages[i], gpas[i]);
10        }
11    }
12 }

```

OUTPUT

Name	Age	GPA
John	20	3.75
Lilian	21	3.95
Ken	19	3.65

Now redo it: on blank paper, write the two `printf` lines from memory. Then change the GPA column to three decimal places and predict the output before you run it.

FILL IN THE GAP · INPUT AND OUTPUT

1. Scanner lives in the _____ package and is created with `new Scanner(_____)`.
2. To read a whole line use _____; to read a single word use _____; to read a whole number use _____.
3. After `nextInt()`, a following `nextLine()` returns _____ because _____.
4. `br.readLine()` returns a _____, so to get a number you must call _____.
5. `printf("%f", 3.14159)` prints _____, not 3.14159, because bare `%f` gives _____ decimal places.
6. To print a literal percent sign in a format string, write _____. To start a new line, write _____.
7. `System.console()` returns _____ when the program is not attached to a real terminal.

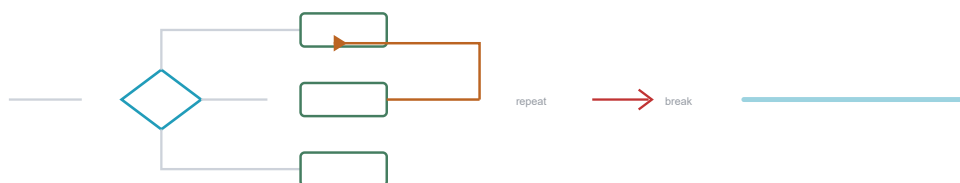
ANSWERS

1. `java.util`, `System.in` · 2. `nextLine()`, `next()`, `nextInt()` · 3. an empty string, `nextInt` left the newline in the buffer · 4. `String`, `Integer.parseInt` · 5. 3.141590, six · 6. `%%`, `%n` · 7. `null`

PART V

Module Four Java Control Structures

Until now your programs have run top to bottom and stopped. This module is where they start to think: choosing between paths, repeating work, and jumping out early. If one module decides your grade, it is this one.



Three ways to stop running in a straight line

By the end of this unit you should be able to define control structures and explain their purpose, identify and describe their types, and use conditional statements (if, if-else, if-else-if, switch and the ternary operator) to manage program flow.

TEACH • CONTROL STRUCTURES

Control structures are special instructions that manage the flow of execution in a program. They let it make decisions, repeat actions, or choose between paths based on conditions.

By default, code executes **linearly, from top to bottom**. Every program you have written so far did exactly that. A control structure is how you break that line.

● MUST-MEMORISE • THE THREE CATEGORIES

- 1. Selection** (decision making): executes different code based on conditions. if, if-else, switch.
- 2. Iteration** (loops): repeats a block of code. for, while, do-while.
- 3. Jump** (branching): transfers control elsewhere. break, continue, return.

"What are control structures in Java, and why are they important?" is a stated in-text question. Answer with the definition, then the three categories, then why: they let a program make decisions, repeat actions, and control which parts run.

MEMORY HOOK: choose, repeat, escape. Selection chooses, iteration repeats, jumping escapes. Everything a program does with its own flow is one of those three.

Choosing

SYNTAX • THE THREE SHAPES

```

if (condition) {                <- runs only when condition is true
    // code
}

if (condition) {                <- one block or the other, never both
    // runs if true
} else {
    // runs if false
}

if (condition1) {               <- the else-if ladder: FIRST match wins
    // runs if condition1
} else if (condition2) {       <- only tested if condition1 was false
    // runs if condition2
} else {
    // runs if nothing above matched
}

```

TEACH • THE CONDITION IS A BOOLEAN, AND ONLY A BOOLEAN

Whatever is inside the brackets must evaluate to true or false. That is why Unit 3.7's relational operators exist: they turn numbers into the booleans that if eats.

If you have come from C or Python, note that Java will **not** accept a number here. `if (5)` does not compile. This is a mercy: it makes the classic `if (x = 5)` typo, assignment instead of comparison, a compile error rather than a silent bug.

```
1 public class IfElseStatementTwo {
2     public static void main(String[] args) {
3         int score = 75;
4
5         if (score >= 90) {
6             System.out.println("Your grade is A");
7         } else if (score >= 80) {
8             System.out.println("Your grade is B");
9         } else if (score >= 70) {
10            System.out.println("Your grade is C");
11        } else if (score >= 60) {
12            System.out.println("Your grade is D");
13        } else {
14            System.out.println("Your grade is F");
15        }
16    }
17 }
```

OUTPUT

Your grade is C

● MUST-MEMORISE · A LADDER STOPS AT THE FIRST TRUE

In an else if ladder, **the first condition that is true wins, and nothing below it is even tested.** Exactly one branch runs.

This is why the ladder above needs no upper bounds. `score >= 70` would be true for 95 as well, but 95 never reaches that rung: it matched `>= 90` three lines earlier and the ladder stopped.

It follows that **order is everything**. Write the same ladder with `score >= 60` first and every score above 60 gets a D.

MEMORY HOOK: a ladder is climbed from the top and you get off at the first rung that holds you. **Put the hardest condition first.**

⚠ TRAP · A LADDER IS NOT THE SAME AS SEPARATE IFs

Replace every `else if` above with a plain `if` and the program still compiles, still runs, and prints **two** grades for a score of 75: C and D. Each `if` is now an independent question, so all of them get asked, and 75 answers yes to two of them.

This is not a small stylistic point. Your 2025/26 paper's Section C explicitly demands **nested** if-else "not independent/separate if checks", and says so twice, in two different questions. Unit 4.4 is about exactly this.

CODE · THE SAME LADDER, BROKEN INTO SEPARATE IFS

```
1 public class SeparateIfs {
2     public static void main(String[] args) {
3         int score = 75;
4         // every 'else' removed. Each if is now its own question.
5
6         if (score >= 90) { System.out.println("A"); }
7         if (score >= 80) { System.out.println("B"); }
8         if (score >= 70) { System.out.println("C"); }
9         if (score >= 60) { System.out.println("D"); }
10        else { System.out.println("F"); }
11    }
12 }
```

OUTPUT

C
D

Two grades for one student, and the else now belongs only to the last if, so F is correctly skipped but for the wrong reason. Nothing crashed. Nothing warned. A logic error, in the sense of F.7, and it is exactly what the exam is guarding against when it insists on nesting.

UNIT 4.3 · THE SWITCH STATEMENT

Many cases, one expression, and the fall-through trap

TEACH · SWITCH

The switch statement streamlines multiple if-else checks by evaluating an expression once and executing the block that matches the corresponding case value.

That is the difference from a ladder, and it is the answer to the stated in-text question "how does switch differ from an if-else ladder?": a ladder evaluates a **new condition at every rung**; a switch evaluates **one expression once** and jumps straight to the matching case.

The price is that a switch can only compare for **equality** against **constant** values. You cannot write `case score >= 90:`. Ranges need a ladder; exact values suit a switch.

SYNTAX · SWITCH

```
switch (expression) {
    case value1:
        // code
        break;
    case value2:
        // code
        break;
    default:
        // code
        break;
}
```

<- evaluated ONCE

<- jumps out of the whole switch

<- when nothing matched

```
1 public class Switch1 {
2     public static void main(String[] args) {
3         int category = 1;
4         switch (category) {
5             case 1:
6                 System.out.println("Pre-teen");
7                 break;
8             case 2:
9                 System.out.println("Teenager");
10                break;
11             case 3:
12                 System.out.println("Adult");
13                 break;
14             default:
15                 System.out.println("Senior Adult!");
16                 break;
17         }
18     }
19 }
```

OUTPUT

Pre-teen

```
1 public class SwitchStringExample {
2     public static void main(String[] args) {
3         String level = "intermediate";
4         switch (level) {
5             case "beginner":
6                 System.out.println("Welcome, beginner!");
7                 break;
8             case "intermediate":
9                 System.out.println("You're doing well!");
10                break;
11             case "advanced":
12                 System.out.println("You're almost an expert!");
13                 break;
14             default:
15                 System.out.println("Unknown level.");
16         }
17     }
18 }
```

OUTPUT

You're doing well!

A switch works on int (and the smaller integer types), char, String and enums. Note the last break is missing here and it does not matter: default is last, so there is nothing to fall into. Write it anyway, out of habit, because the day you add a case below it the habit saves you.

● MUST-MEMORISE · FALL-THROUGH

`break` terminates a case, jumping immediately to the end of the switch block.

Without a break, the program continues executing the subsequent case(s), a behaviour known as **fall-through**, until it meets a break or reaches the end of the switch.

Read that carefully, because it is stranger than it looks. Fall-through does not re-test the next case. Once a case matches, execution simply **keeps going downward through the code**, ignoring the case labels entirely, until something stops it.

MEMORY HOOK: case is a **door in**, not a box. You enter at the matching door and keep walking. `break` is the way out. No break, and you walk through every room after yours.

CODE · FALL-THROUGH, SEEN

RUN THIS ONE TWICE

```
1 public class FallThrough {
2     static void withBreaks(int c) {
3         System.out.print("  with breaks, c=" + c + " -> ");
4         switch (c) {
5             case 1: System.out.print("one "); break;
6             case 2: System.out.print("two "); break;
7             case 3: System.out.print("three "); break;
8         }
9         System.out.println();
10    }
11
12    static void noBreaks(int c) {           // every break removed
13        System.out.print("  no breaks,   c=" + c + " -> ");
14        switch (c) {
15            case 1: System.out.print("one ");
16            case 2: System.out.print("two ");
17            case 3: System.out.print("three ");
18        }
19        System.out.println();
20    }
21
22    public static void main(String[] args) {
23        withBreaks(1); withBreaks(2); withBreaks(3);
24        noBreaks(1);   noBreaks(2);   noBreaks(3);
25    }
26 }
```

OUTPUT

```
with breaks, c=1 -> one
with breaks, c=2 -> two
with breaks, c=3 -> three
no breaks,   c=1 -> one two three
no breaks,   c=2 -> two three
no breaks,   c=3 -> three
```

Look at `c=1` without breaks: it printed "one two three". It did not test case 2 and decide 1 equals 2. It entered at case 1 and **kept walking**. And `c=3` printed only "three", because there was nothing below to fall into. The damage depends entirely on how far down the door is.

TEACH · FALL-THROUGH ON PURPOSE

Fall-through is a trap, not a bug: it is deliberate, and occasionally it is exactly what you want. Stacking cases with no code between them is the standard way to say "any of these":

```
switch (day) {
    case 6:
    case 7:                <- 6 falls straight into 7
        System.out.println("Weekend");
        break;
    default:
        System.out.println("Weekday");
}
```

Case 6 has no body, so it falls into case 7, and both print "Weekend". This is idiomatic and every Java programmer reads it instantly.

RECALL AND CHECK

What is the purpose of the switch statement, and how does it differ from an if-else ladder? Give one thing a ladder can do that a switch cannot.

ANSWER

A switch streamlines multiple if-else checks by evaluating one expression once and executing the block matching that value. A ladder evaluates a separate condition at each rung; a switch evaluates a single expression and jumps to the matching case. A ladder can test **ranges** and any boolean condition (score >= 90, a && b); a switch can only compare the expression for equality against constant values.

UNIT 4.4 · NESTED IF · THE ONE THE EXAM DEMANDS

Conditions inside conditions

TEACH · NESTED IF

Nested if statements occur when an if or else block contains another if or else statement inside it. This lets you test conditions **hierarchically**: check complex conditions that depend on multiple criteria, and act differently on each combination.

The word that matters is **hierarchically**. The inner question is only asked when the outer one has already been answered, which means the inner question can assume the outer is true.

CODE · POSITIVE, THEN EVEN OR ODD

FROM THE SLIDES

```
1 public class NestedIf {
2     public static void main(String[] args) {
3         int number = 10;
4
5         if (number > 0) {                // outer: is it positive?
6             if (number % 2 == 0) {        // inner: only asked if positive
7                 System.out.println(number + " is positive and even");
8             } else {
9                 System.out.println(number + " is positive and odd");
10            }
11        } else {
12            System.out.println(number + " is not positive");
13        }
14    }
15 }
```

OUTPUT

10 is positive and even

● MUST-MEMORISE · NESTED IS NOT THE SAME AS COMBINED

Nested: the inner condition is asked **only inside** the outer branch. Each branch can do something different, including doing nothing.

Combined with &&: one flat question. You get one answer, true or false, and you lose the ability to distinguish *which* part failed.

Your 2025/26 paper insists on nesting in Section C, twice, and it explains why in its own words: "the nesting must reflect that days attending is only checked within each tier branch, not as a separate top-level check". Marks are awarded for the **structure**, not just the answer.

MEMORY HOOK: nesting is a **conversation**: "are you Gold? Then how many days?" Combining is a **form**: "tick if Gold and three days." The conversation can respond differently to each answer.

The paper's rules: Gold with 3 days 25%; Gold with 1 to 2 days 15%; Silver with 3 days 15%; Silver with 1 to 2 days 5%; Bronze any days 0%; and any tier that is not "Gold" or "Silver" must be treated as Bronze. It demands **nested** if-else, not a switch and not separate ifs.

- 1 Branch on the thing the hierarchy is *about*: the tier. That is the outer question.

The paper tells you this outright: days are checked within each tier branch.

- 2 Inside each tier branch, ask about days.

Now "3 days" can mean 25% under Gold and 15% under Silver, with no repetition of the tier test.

- 3 Make the final else the Bronze case.

This is how "anything that is not Gold or Silver is Bronze" falls out for free, rather than being a fourth test.

```

1 public class Discount {
2     static double discountFor(String tier, int days) {
3         if (tier.equals("Gold")) {           // outer: the tier
4             if (days == 3) {                 // inner: only for Gold
5                 return 25.0;
6             } else {
7                 return 15.0;
8             }
9         } else if (tier.equals("Silver")) {
10            if (days == 3) {                 // inner: only for Silver
11                return 15.0;
12            } else {
13                return 5.0;
14            }
15        } else {                             // not Gold, not Silver
16            return 0.0;                       // -> Bronze
17        }
18    }
19
20    public static void main(String[] args) {
21        String[] tiers = {"Gold", "Gold", "Silver", "Silver", "Bronze", "platinum"};
22        int[] days = {3, 2, 3, 1, 3, 3};
23        for (int i = 0; i < tiers.length; i++) {
24            System.out.printf("%-9s %d day(s) -> %.0f%%\n",
25                              tiers[i], days[i], discountFor(tiers[i], days[i]));
26        }
27    }
28 }

```

OUTPUT

```

Gold      3 day(s) -> 25%
Gold      2 day(s) -> 15%
Silver    3 day(s) -> 15%
Silver    1 day(s) -> 5%
Bronze    3 day(s) -> 0%
platinum  3 day(s) -> 0%

```

The last row is the one that earns the mark: "platinum" is not Gold and not Silver, so it fell to the final else and was treated as Bronze, exactly as the paper demands, without a single line written to handle it. **That is what good nesting buys you.** (`tier.equals(...)` rather than `==` is Module 7's rule, and it is not optional: it is the most famous bug in Java.)

Now redo it: cover the listing and write `discountFor` from the rules alone. Then check: does your version handle "platinum" without mentioning it?

```

1 public class LeapYear {
2     static boolean isLeap(int year) {
3         if (year % 4 == 0) {           // divisible by 4?
4             if (year % 100 == 0) {    // ... and by 100?
5                 return year % 400 == 0; // then it must also be /400
6             }
7             return true;              // /4 but not /100: leap
8         }
9         return false;                // not /4: never leap
10    }
11
12    public static void main(String[] args) {
13        int[] years = {2024, 2023, 1900, 2000};
14        for (int y : years) {
15            System.out.println(y + " is " + (isLeap(y) ? "" : "not ") + "a leap year");
16        }
17    }
18 }

```

OUTPUT

```

2024 is a leap year
2023 is not a leap year
1900 is not a leap year
2000 is a leap year

```

Your course's slide checks only 2024. The interesting years are 1900 and 2000: both divisible by 4 and by 100, and only 2000 is also divisible by 400. Those two rows are what the nesting is *for*, and a version that gets 2024 right can still get 1900 wrong. **Always test the case the example avoids.**

UNIT 4.5 · THE THREE LOOPS

for, while, do-while

By the end of this unit you should be able to explain the purpose of loops, compare and contrast *for*, *while* and *do-while*, and describe the branching statements *break*, *continue* and *return*.

TEACH · LOOPS

Loops are used to execute a block of code repeatedly based on a condition. Java provides three, and they differ only in *when* they ask the question.

● MUST-MEMORISE · WHICH LOOP, AND WHEN

for: use when the **number of iterations is known**. Counting.

while: use when the number of iterations is **not known in advance** and depends on a condition. It may run **zero** times.

do-while: like while, but the body **always runs at least once**, because the condition is tested at the bottom.

MEMORY HOOK: for counts, while waits, do-while insists. The only structural difference between while and do-while is whether the question is asked at the top of the loop or the bottom, and that decides whether zero is a possible number of runs.

SYNTAX · THE THREE, SIDE BY SIDE

```
for (initialization; condition; update) {    <- all three parts, one line
    // code
}

initialization;                             <- outside
while (condition) {                         <- tested BEFORE each pass
    // code
    update;                                <- inside, and YOU must remember it
}

initialization;
do {
    // code                                <- runs at least once, always
    update;
} while (condition);                        <- tested AFTER each pass. Note the ;
```

The semicolon after while (condition) in do-while is compulsory, and it is the only place in Java where a while is followed by one. Forgetting it is a compile error; adding one after an ordinary while is a disaster, and Unit 4.7 shows why.

CODE · THE SAME JOB, ALL THREE WAYS

```
1 public class ThreeLoops {
2     public static void main(String[] args) {
3         System.out.print("for:      ");
4         for (int i = 1; i <= 5; i++) {
5             System.out.print(i + " ");
6         }
7
8         System.out.print("\nwhile:   ");
9         int j = 1;
10        while (j <= 5) {
11            System.out.print(j + " ");
12            j++;
13        }
14
15        System.out.print("\ndo-while: ");
16        int k = 1;
17        do {
18            System.out.print(k + " ");
19            k++;
20        } while (k <= 5);
21        System.out.println();
22    }
23 }
```

OUTPUT

```
for:      1 2 3 4 5
while:    1 2 3 4 5
do-while: 1 2 3 4 5
```

Identical output, three times. For a job where the count is known, all three work and **the for loop is the right one**, because it puts the counter, the test and the step on a single line where you can see all three at once. Note how the while and do-while versions scatter those same three parts across four lines: that is the cost, and it is also exactly what Unit 4.6 makes you undo and redo.

```

1 public class ZeroTimes {
2     public static void main(String[] args) {
3         // a condition that is false from the very start
4         int i = 10;
5
6         System.out.println("while, i = 10, condition i < 5:");
7         while (i < 5) {
8             System.out.println("    ran");
9             i++;
10        }
11        System.out.println("    (body ran 0 times)");
12
13        int k = 10;
14        System.out.println("do-while, k = 10, same condition:");
15        do {
16            System.out.println("    ran anyway, k = " + k);
17            k++;
18        } while (k < 5);
19    }
20 }

```

OUTPUT

```

while, i = 10, condition i < 5:
    (body ran 0 times)
do-while, k = 10, same condition:
    ran anyway, k = 10

```

Identical condition, identical starting value, different result. The while asked first and never entered; the do-while ran the body and asked afterwards. This is the entire difference between them, and it is why do-while is the right choice for "keep asking until the input is valid": you must ask **once** before you can have anything to validate.

TEACH · THE FOR LOOP'S THREE PARTS

PART	IN FOR (INT I = 0; I < 5; I++)	WHEN IT HAPPENS
initialization	int i = 0	once, before anything else
condition	i < 5	before every pass, including the first. False means stop.
update	i++	after every pass, before the condition is retested

The order matters and people guess it wrong: **init, test, body, update, test, body, update, ...** The update runs *after* the body, not before, which is why i is 0 on the first pass and not 1.

UNIT 4.6 · CONVERTING BETWEEN LOOPS · WORTH 10 MARKS

Turning a for into a while, and back

TEACH · WHY THIS UNIT EXISTS

Both papers ask for this, by name, for real marks. The 2025/26 Section A says "Retain other things but change the For Loop to While loop and the While loops to For loops", inside a ten-mark question. The 2024/25 paper asks you to demonstrate all three loop types.

It is worth marks precisely because it is mechanical. Learn it as a recipe and it is free marks; improvise it under pressure and you will drop the update or the scope.

● MUST-MEMORISE · THE CONVERSION RECIPE

for -> **while**, in three moves:

1. **initialization** goes **above** the loop;
2. **condition** goes in the while ();
3. **update** goes at the **bottom of the body**.

while -> **for**: the same three moves, backwards. Find the variable being counted, its test, and where it changes, and fold all three into the for header.

MEMORY HOOK: a for loop is a while loop with its three parts **collected onto one line**. Converting is unpacking or repacking, nothing more.

WORKED EXAMPLE · FOR TO WHILE, STEP BY STEP

Take the innermost loop from the 2025/26 Armstrong question:

```
for (int p = 0; p < digits; p++) power *= d;
```

- 1 Lift the **initialization** out, above the loop: `int p = 0;`
- 2 Move the **condition** into the while: `while (p < digits)`
- 3 Push the **update** to the bottom of the body: `p++;`
This is the step people forget, and forgetting it does not give a wrong answer. It gives an infinite loop.
- 4 Add braces, because the body now has two statements where it had one.
The original needed none: a one-statement body needs no braces. Two statements without braces is a bug.

Which gives, with everything else retained:

```
int p = 0;
while (p < digits) {
    power *= d;
    p++;
}
```

```

1 public class Convert {
2     static int withFor(int d, int digits) {
3         int power = 1;
4         for (int p = 0; p < digits; p++) power *= d;
5         return power;
6     }
7
8     static int withWhile(int d, int digits) {
9         int power = 1;
10        int p = 0;                // 1. init lifted out
11        while (p < digits) {      // 2. condition
12            power *= d;
13            p++;                  // 3. update at the bottom
14        }
15        return power;
16    }
17
18    public static void main(String[] args) {
19        boolean allMatch = true;
20        for (int d = 0; d <= 9; d++) {
21            for (int n = 0; n <= 4; n++) {
22                if (withFor(d, n) != withWhile(d, n)) allMatch = false;
23            }
24        }
25        System.out.println("3^0 = " + withFor(3, 0) + " / " + withWhile(3, 0));
26        System.out.println("3^3 = " + withFor(3, 3) + " / " + withWhile(3, 3));
27        System.out.println("identical for every d 0-9, digits 0-4: " + allMatch);
28    }
29 }

```

OUTPUT

```

3^0 = 1 / 1
3^3 = 27 / 27
identical for every d 0-9, digits 0-4: true

```

Note the last line: rather than eyeballing two versions and trusting them, the program checked all fifty combinations and reported. **That is how you verify a conversion in an exam too**, on paper, with one or two values: convert, then trace both and compare.

⚠ TRAP · THE TWO WAYS A CONVERSION GOES WRONG

1. The dropped update. Leave out `p++` and the condition never changes, so the loop runs forever. In an exam this costs you the marks; on a machine it hangs.

2. The scope change. `for (int p = 0; ...)` declares `p` *inside* the loop, so it ceases to exist afterwards. Lift it out and `p` now survives the loop. Usually harmless, occasionally a compile error ("variable `p` is already defined") if you convert two loops in the same method and both lift out an `i`.

Both are avoided by following the recipe rather than rewriting from scratch.

FILL IN THE GAP · CONVERTING

Convert this to a while loop, in pen, before reading on.

```
int sum = 0;
for (int i = 1; i <= 10; i++) {
    sum += i;
}
```

1. The initialization _____ goes _____ the loop.
2. The condition _____ goes inside while ().
3. The update _____ goes at the _____ of the body.
4. Forgetting step 3 produces _____.

ANSWERS

1. int i = 1;, above · 2. i <= 10 · 3. i++;, bottom · 4. an infinite loop

The whole thing: int sum = 0; int i = 1; while (i <= 10) { sum += i; i++; }, and the sum is 55.

UNIT 4.7 · BRANCHING · BREAK, CONTINUE, RETURN

Escaping

● MUST-MEMORISE · THE THREE BRANCHING STATEMENTS

break: exit the loop (or switch) **immediately**, even if its condition is still true.

continue: skip **the rest of this pass** and jump straight to the next iteration. The loop carries on.

return: exit the whole **method** immediately, optionally handing back a value.

MEMORY HOOK: break leaves the building, continue skips to the next floor, return quits the job.

Notice the scale: break and continue act on the loop, return acts on the method containing it.

```

1 public class BreakContinue {
2     public static void main(String[] args) {
3         System.out.println("break at i == 5:");
4         for (int i = 1; i <= 8; i++) {
5             if (i == 5) break;           // leave the loop entirely
6             System.out.println(" i = " + i);
7         }
8
9         System.out.println("continue at i == 3:");
10        for (int i = 1; i <= 4; i++) {
11            if (i == 3) continue;        // skip just this pass
12            System.out.println(" i = " + i);
13        }
14    }
15 }

```

OUTPUT

```

break at i == 5:
 i = 1
 i = 2
 i = 3
 i = 4
continue at i == 3:
 i = 1
 i = 2
 i = 4

```

Read the two outputs against each other. break stopped at 4 and 5 through 8 never happened. continue missed out 3 and **carried on to 4**. That is the whole distinction, and both papers plant a bug that turns on it.

⚠ TRAP · CONTINUE IN A FOR LOOP STILL RUNS THE UPDATE

continue jumps to the **next iteration**, which in a for loop means the update `i++` still happens. Fine.

In a while loop it means the update is **skipped**, because the update is a statement in the body and you just jumped over the rest of the body. So this runs forever:

```

int i = 0;
while (i < 5) {
    if (i == 3) continue;    <- jumps back to the test with i STILL 3
    System.out.println(i);
    i++;                    <- never reached when i is 3
}

```

This is the one listing in this book that is not executed for the obvious reason. It matters because Unit 4.6 tells you to convert for loops into while loops, and a continue inside one is exactly where a careless conversion turns a working program into a hang.

Look at this, from the 2025/26 Section A:

```
if (arr[j] % 4 == 0) {
    continue;           <- skips the multiples of 4
}
sum += arr[j] - j;      <- so this only ever sees NON-multiples
```

The question says the program should "find the first number that's a multiple of 4" and add it. The code continues **when** it finds a multiple of 4, so it skips precisely the numbers it was supposed to collect and processes precisely the ones it should have ignored. The logic is exactly inverted.

The other paper plants the mirror image: `if (prices[i] > 20) continue;` where the spec says skip anything **under** 20.

The lesson: `continue` means "skip this one". So the condition in front of it must describe the items you want to **throw away**, not the ones you want. Read every `continue` as the word "unless" and the bug becomes visible.

```
1 public class ReturnDemo {
2     static void greet(String name) {
3         if (name == null || name.isEmpty()) {
4             System.out.println("Invalid name.");
5             return;           // leave greet() right now
6         }
7         System.out.println("Hello, " + name + "!");
8     }
9
10    static int square(int n) {
11        return n * n;          // return WITH a value
12    }
13
14    public static void main(String[] args) {
15        greet("Jumoke");
16        greet("");
17        System.out.println("square(5) = " + square(5));
18    }
19 }
```

OUTPUT

```
Hello, Jumoke!
Invalid name.
square(5) = 25
```

Two shapes of `return`. `greet` is `void`, so its `return;` carries nothing and exists purely to leave early. `square` is `int`, so its `return` must carry a value. Module 5 is entirely about this.

The guard uses one library method worth naming. `isEmpty()` is a `String` method that answers `true` when the string holds **no characters at all**. So `""` is empty, and `" "` is **not**: a space is a character. It is the same question as `length() == 0`, written so it reads like English.

And the **order** of the two tests is not a matter of taste. `||` is **short-circuit** (Unit 3.8): if `name == null` is already true, the right-hand side is never evaluated at all. Swap them to `name.isEmpty() || name == null` and a null name calls `isEmpty()` on nothing, which throws `NullPointerException` before the null check it was supposed to be protected by ever runs. **Check for null first, always.** It compiles either way.

FILL IN THE GAP · CONTROL STRUCTURES

1. The three categories of control structure are _____, _____ and _____.
2. In an else-if ladder, _____ condition that is true wins, and the rest are _____.
3. Without break, a switch keeps executing subsequent cases. This is called _____.
4. A while loop may run _____ times; a do-while always runs at least _____ time(s), because _____.
5. Converting for to while: init goes _____, condition goes _____, update goes _____. Forgetting the update gives _____.
6. break exits the _____; continue skips _____; return exits the _____.
7. A continue guarded by if (x) processes the items where x is _____.

ANSWERS

1. selection/decision, iteration/loops, jump/branching · 2. the first, never tested · 3. fall-through · 4. zero, one, the condition is tested after the body · 5. above the loop, in the while (), at the bottom of the body, an infinite loop · 6. loop (or switch), the rest of this pass, method · 7. false

PART VI

Module Five Methods

Writing something once and using it everywhere. How to hand data in, how to get an answer back, and the one question about what really crosses that boundary that almost every Java course, including this one, answers wrongly.



Write it once, use it everywhere

By the end of this unit you should be able to identify the key parts of a method, define void and non-void methods and call them, compare formal parameters with actual arguments, differentiate static from instance methods, explain parameter passing, and use method overloading.

TEACH · A METHOD

A method is a reusable block of code that performs a specific task. You have been writing one since F.5, called `main`, and calling others (`println`, `parseInt`) all along. This module is about writing your own.

Your course gives three reasons they exist, and an in-text question asks for them:

BENEFIT	MEANS
Modularising code	breaking code into smaller, manageable parts
Improving readability	logical separation of tasks
Reducing redundancy	avoiding repetition: write once, reuse

SYNTAX · THE ANATOMY OF A METHOD

```

public static int add (int a, int b) {
    |         |         |         |         |
    |         |         |         |         +--- PARAMETER LIST: what it needs
    |         |         |         +----- NAME
    |         +----- RETURN TYPE: what it hands back
    +----- static: belongs to the class (5.5)
    +----- ACCESS MODIFIER: who may call it

    return a + b;          <- the body, and the value going back
}

```

● MUST-MEMORISE · THE METHOD SIGNATURE

A method's **signature** is its **name** plus its **parameter list** (the number, types and order of its parameters).

The **return type** is **NOT** part of the signature. Neither is the access modifier, nor the parameter names.

This is not trivia. Your 2025/26 paper asks you to "state precisely the role that the method signature plays in enabling the compiler to resolve an overloaded call at compile time", and your 2024/25 paper asks you to define method signature outright. It is worth marks on both papers we have.

MEMORY HOOK: the signature is **what you must write at the call site**. When you call `add(2, 3)` you write the name and the arguments; you never write the return type. So the compiler cannot use what you never wrote. **Signature = name + parameters.**

```
1 public class SigProof {
2     static int value() { return 1; }
3     static double value() { return 1.0; } // same name, no parameters
4
5     public static void main(String[] args) {
6         System.out.println(value());
7     }
8 }
```

WHAT THE COMPILER SAYS

```
SigProof.java:3: error: method value() is already defined in class SigProof
    static double value() { return 1.0; }
                ^
1 error
```

Two methods differing **only** in return type are, to the compiler, **the same method defined twice**. It refuses before you can even call one. And that is the reason: at `value()` there is nothing in what you typed to tell the compiler which you meant. **This listing is the proof of the box above**, and it is the "state precisely" the exam is asking for.

UNIT 5.2 · VOID AND NON-VOID METHODS

Methods that do, and methods that answer

● MUST-MEMORISE · VOID VERSUS NON-VOID

Void methods perform actions but **do not return any value**. Their return type is the keyword `void`.

Non-void (returning) methods return a value. Their return type is the type of that value, and every path through them **must** return one.

MEMORY HOOK: `void` is a **command** ("print this"), non-void is a **question** ("what is the square of 5?"). A command needs no answer; a question is not allowed to go unanswered.

```

1 public class Methods {
2
3     // VOID: does something, answers nothing
4     static void sayHello(String name) {
5         System.out.println("Hello " + name);
6     }
7
8     // VOID: prints a sum, but does not hand it back
9     static void printSum(int a, int b) {
10        System.out.println("The sum of " + a + " and " + b + " is " + (a + b));
11    }
12
13    // NON-VOID: answers, and prints nothing
14    static int square(int n) {
15        return n * n;
16    }
17
18    // NON-VOID: builds a String and hands it back
19    static String getGreeting(String name) {
20        return "Hello, " + name + "!";
21    }
22
23    public static void main(String[] args) {
24        sayHello("James"); // call: nothing comes back
25        printSum(5, 10);
26        System.out.println("Square: " + square(5)); // call: use the answer
27        System.out.println(getGreeting("Oghenetega"));
28    }
29 }

```

OUTPUT

```

Hello James
The sum of 5 and 10 is 15
Square: 25
Hello, Oghenetega!

```

⚠ TRAP · PRINTING IS NOT RETURNING

The single most common beginner confusion in this module. `printSum` and `square` both "give you a sum", and they are completely different things.

`printSum(5, 10)` puts text on the screen and evaluates to **nothing**. You cannot write `int x = printSum(5, 10);` it does not compile, because there is no value to put in `x`.

`square(5)` puts **nothing** on the screen and evaluates to 25, which you can store, add to, or print. It is *useful to other code*; `printSum` is only useful to a human watching.

Rule of thumb: a method that computes should return, not print. Printing is a decision about presentation, and it belongs to whoever called you. Your exam questions ask for methods that "return the count" and then display it in main: that separation is deliberate and it is marked.

```

1 public class MissingReturn {
2     static String grade(int score) {
3         if (score >= 50) {
4             return "pass";
5         }
6         // what about scores below 50? No return here.
7     }
8
9     public static void main(String[] args) {
10        System.out.println(grade(70));
11    }
12 }

```

WHAT THE COMPILER SAYS

```

MissingReturn.java:7: error: missing return statement
    }
    ^
1 error

```

The compiler is not asking whether the missing path *ever happens*. It requires that **every** route out of a non-void method carries a value. This is the same mercy as F.7's compile-time errors: a question that can go unanswered becomes a refusal to build, rather than a mystery at run time.

UNIT 5.3 · PARAMETERS AND ARGUMENTS

Two words for two different things

● MUST-MEMORISE · FORMAL PARAMETER VS ACTUAL ARGUMENT

A **formal parameter** is the variable in the method's **declaration**: `static int square(int n)`. It is a name for a value that does not exist yet.

An **actual argument** is the value you pass at the **call**: `square(5)`. It is a real value, right now.

Your learning outcome says "compare formal parameters with actual arguments", so this distinction is examinable in its own right.

MEMORY HOOK: the parameter is the empty seat; the argument is the passenger. The seat is described when the bus is built; the passenger arrives at departure.

TEACH · PARAMETERIZED AND NON-PARAMETERIZED

Parameterized methods take one or more parameters to do their work. **Non-parameterized methods take none**: their brackets are empty.

A non-parameterized method is not useless: it either works on data it can already see, or it fetches something from the outside world. Your course's examples are the date and time, and a random number, neither of which needs you to supply anything.

```

1 public class Params {
2
3     // PARAMETERIZED: needs a name and an age to do anything
4     static String getAgeInfo(int age, String name) {
5         return name + " is " + age + " years old.";
6     }
7
8     static String getFullName(String first, String last) {
9         return first + " " + last;
10    }
11
12    // NON-PARAMETERIZED: empty brackets, needs nothing from you
13    static String banner() {
14        return "=== report ===";
15    }
16
17    public static void main(String[] args) {
18        System.out.println(banner());
19        // 45 and "Blake" are the ACTUAL ARGUMENTS.
20        // age and name are the FORMAL PARAMETERS.
21        System.out.println(getAgeInfo(45, "Blake"));
22        System.out.println("Full Name: " + getFullName("John", "Doe"));
23    }
24 }

```

OUTPUT

```

=== report ===
Blake is 45 years old.
Full Name: John Doe

```

⚠ TRAP · ARGUMENTS GO IN BY POSITION, NOT BY NAME

`getAgeInfo(int age, String name)` is called as `getAgeInfo(45, "Blake")`. Note the order: **age first**, because that is the order the parameters were declared in, even though the sentence it builds reads "Blake is 45".

Java matches arguments to parameters **by position only**. The names are for you, not for the compiler. Swap them to `getAgeInfo("Blake", 45)` and it does not compile, which is lucky: if both parameters were `String`, it would compile and quietly print "45 is Blake years old."

UNIT 5.4 · PARAMETER PASSING · READ THIS TWICE

What actually crosses the boundary

This unit corrects your course, and the correction is the single most important idea in the module. It is also the most misunderstood topic in all of Java, so if you get this right you are ahead of most graduates.

TEACH · WHAT YOUR COURSE SAYS

"Parameter passing refers to the mechanism of transferring data between methods. The two types are:

- ▶ **Pass-by-value** (primitive types): a copy of the value is passed. Changes inside the method do not affect the original variable.
- ▶ **Pass-by-reference** (objects and arrays): changes inside the method affect the original."

The first bullet is right. The second is **wrong**, and your course's own slides contain the evidence that it is wrong.

Java is pass-by-value. Always. There is no pass-by-reference in Java.

For a primitive, the value copied is **the number**.

For an object or array, the value copied is **the reference**: the address, in the sense of Unit 3.3. So the method gets its own copy of the address, pointing at the **same** object.

That is why you can **change what the address points at** (and the caller sees it), but you cannot **change which address the caller holds**.

MEMORY HOOK: you photocopy your house key and hand over the copy. They can go in and **rearrange the furniture**, and you will see it. They cannot **make your key open a different house**. The key was copied; the house was not.

CODE · THE PROOF, IN ONE PROGRAM

VERIFIED ON THE JVM

```

1  public class PassByValue {
2
3      // a primitive: the NUMBER is copied
4      static void modifyValue(int n) {
5          n = 25;
6          System.out.println("  inside method: " + n);
7      }
8
9      // an array: the REFERENCE is copied. Same array underneath.
10     static void mutate(int[] a) {
11         a[0] = 99;           // rearranging the furniture
12     }
13
14     // an array: pointing the COPY of the reference somewhere new
15     static void reassign(int[] a) {
16         a = new int[]{7, 7, 7}; // making the copied key open a new house
17         a[0] = 55;           // caller cannot possibly see this
18     }
19
20     public static void main(String[] args) {
21         int myNumber = 15;
22         System.out.println("primitive before: " + myNumber);
23         modifyValue(myNumber);
24         System.out.println("primitive after : " + myNumber);
25
26         int[] nums = {1, 2, 3};
27         mutate(nums);
28         System.out.println("after mutate   : " + nums[0] + "   <- changed");
29
30         int[] nums2 = {1, 2, 3};
31         reassign(nums2);
32         System.out.println("after reassign : " + nums2[0] + "   <- UNCHANGED");
33     }
34 }

```

OUTPUT

```

primitive before: 15
  inside method: 25
primitive after : 15
after mutate   : 99   <- changed
after reassign : 1    <- UNCHANGED

```

Look at the last line, because it settles the argument. If Java really passed arrays by reference, then `reassign` would have replaced the caller's array and `nums2[0]` would be 7. It is 1. The method could reach through its copy of the reference and change the array (`mutate`), but it could not change which array the caller was holding. That is pass-by-value of a reference, and it is not pass-by-reference.

⚠ TRAP · YOUR COURSE STATES THE EVIDENCE AND MISSES THE CONCLUSION

Read your own slide again, on the pass-by-reference array example. It says, correctly: "Because arrays are passed by reference, this change affects the original array. **However, reassigning the array inside the method** [does not affect the original]."

That "however" is the whole thing. It is not an exception to pass-by-reference; it is **proof that this was never pass-by-reference**. In a language that genuinely has it, reassignment propagates. The slide observed the exact behaviour that disproves its own label, and shrugged.

What to write in the exam. If asked to describe parameter passing, the safe and correct answer covers both: "Java passes all arguments by value. For primitives the value is the data itself, so the method's changes are invisible to the caller. For objects and arrays the value is a copy of the reference, so the method can modify the object the caller sees, but cannot make the caller's variable refer to a different object. Some texts call the second case pass-by-reference; strictly it is passing a reference by value." That is right, it earns the mark, and it shows you know why.

RECALL AND CHECK

What happens when you pass a primitive type to a method? And what one experiment proves Java is not pass-by-reference for arrays?

ANSWER

Only a copy of the value is used, so any change inside the method does not affect the original variable. The experiment: inside a method, **reassign** the array parameter to a brand-new array and change that. If Java were pass-by-reference, the caller's variable would now point at the new array. It does not: the caller still sees its original. Only the copy of the reference was redirected.

UNIT 5.5 · STATIC AND INSTANCE METHODS

Belonging to the class, or to an object

● MUST-MEMORISE · STATIC VERSUS INSTANCE

Static methods belong to the class, not to instances. They can be called **without creating an object**, and they can only directly access other static methods and variables. Used for utility and helper methods.

Instance (non-static) methods belong to an object. You must create an object with `new` before you can call one, and they can see that object's own data.

This is why `main` is static, the loose end from F.6: when your program starts, no object exists yet, so the JVM needs a method it can call without one.

MEMORY HOOK: `Math.sqrt(9)` needs no object: square roots do not belong to any particular thing. `rex.bark()` needs `Rex`: barking belongs to a dog. **Static is about the idea; instance is about the thing.**

TEACH · EVERY METHOD YOU HAVE WRITTEN SO FAR IS STATIC

Your course notes this and it is worth pausing on. Every method in this book until now has carried static, because every one has been called from `main` without an object in sight.

Module 6 is where objects arrive properly and instance methods become the normal case. The `Dog` example back in Module 1 already showed one: `bark()` had no static, which is exactly why it needed a `Dog` to be called on, and why it could print `name`.

```

1 class Person {
2     String name;                // instance data: one per object
3
4     void greet() {                // INSTANCE method: no 'static'
5         System.out.println("Hello, " + name + "!");
6     }
7 }
8
9 class Rectangle {
10     int length, width;
11
12     void setDimensions(int l, int w) { length = l; width = w; }
13     int getArea() { return length * width; }
14 }
15
16 public class StaticVsInstance {
17
18     static int square(int n) { return n * n; }    // STATIC: a pure utility
19
20     public static void main(String[] args) {
21         System.out.println(square(5));           // no object needed
22
23         Person p = new Person();                 // instance method NEEDS an object
24         p.name = "Alice";
25         p.greet();
26
27         Rectangle r = new Rectangle();
28         r.setDimensions(5, 3);
29         System.out.println("Area: " + r.getArea());
30     }
31 }

```

OUTPUT

```

25
Hello, Alice!
Area: 15

```

```

1 public class StaticTrap {
2     int counter = 0;                // instance data: belongs to an object
3
4     void bump() { counter++; }       // instance method
5
6     public static void main(String[] args) {
7         bump();                     // which object's counter would this bump?
8     }
9 }

```

WHAT THE COMPILER SAYS

```

StaticTrap.java:7: error: non-static method bump() cannot be referenced from a static context
    bump();
    ^
1 error

```

"Non-static method cannot be referenced from a static context" is the error you will meet most often this year, and its meaning is a fair question: `bump()` increments **some object's** counter, and `main` is not running on any object, so there is no answer to "whose?". The fix is to make one: `new StaticTrap().bump();`, or to make the method static if it genuinely needs no object.

One name, several methods

● MUST-MEMORISE • OVERLOADING

Method overloading is a feature that allows a class to have more than one method with the same name but with different parameter lists.

The compiler picks which one you meant, **at compile time**, using:

1. the **method signature** (name + parameter list), and
2. the **arguments passed** at the call.

This is your 2025/26 Section A, part A, worth two marks, and it wants the "state precisely" part: the compiler matches the types, number and order of the arguments you wrote against each candidate's parameter list, and since the return type is not part of the signature, it plays no role in the decision.

MEMORY HOOK: overloading is **one word with several meanings, disambiguated by context**, exactly like "run" in English. "Run a race" and "run a program" are different acts; you know which from what follows the verb. The compiler knows from what is in the brackets.

CODE • OVERLOADING, FROM THE SLIDES

FROM THE SLIDES

```

1 public class Plus {
2
3     static void print(String name) { System.out.println("I am " + name); }
4     static void print(int age)     { System.out.println("I am " + age + " years old"); }
5
6     static int add(int a, int b)      { return a + b; }
7     static int add(int a, int b, int c) { return a + b + c; }
8     static double add(double a, double b) { return a + b; }
9
10    public static void main(String[] args) {
11        print("John");           // String -> the first print
12        print(18);               // int    -> the second print
13
14        System.out.println(add(7, 8));      // two ints
15        System.out.println(add(2, 3, 4));   // three ints
16        System.out.println(add(1.5, 2.5)); // two doubles
17    }
18 }

```

OUTPUT

```

I am John
I am 18 years old
15
9
4.0

```

Three add methods, and the compiler chose correctly three times without being told. Note the last output is 4.0, not 4: the double version ran, and a double prints with its decimal point. That detail is exactly the sort of thing a "state the output" question turns on.

TEACH · WHAT COUNTS AS A DIFFERENT PARAMETER LIST

DIFFERENT BY	OVERLOADS?	EXAMPLE
number of parameters	yes	<code>add(int,int)</code> vs <code>add(int,int,int)</code>
types of parameters	yes	<code>add(int,int)</code> vs <code>add(double,double)</code>
order of types	yes	<code>f(int,String)</code> vs <code>f(String,int)</code>
return type only	no	compile error: already defined (5.1)
parameter names only	no	<code>f(int a)</code> vs <code>f(int b)</code> are the same signature

UNIT 5.7 · TYPE PROMOTION

When there is no exact match

● MUST-MEMORISE · THE PROMOTION ORDER

When an exact type match is not found, Java automatically promotes smaller types to larger ones:

byte → short → int → long → float → double

char → int

"Can Java promote data types automatically in method overloading?" is a stated in-text question. Yes: if no exact match exists, it promotes step by step along that chain until it finds a method that fits.

MEMORY HOOK: promotion only ever goes **uphill**, towards the bigger box, because a bigger box can always hold a smaller value without losing anything. Java will never quietly go downhill and lose your data: that needs a cast, and you have to ask.

```

1 public class Promotion {
2
3     static void show(int x)    { System.out.println("  show(int) got " + x); }
4     static void only(double d) { System.out.println("  only(double) got " + d); }
5
6     public static void main(String[] args) {
7         // there is no show(char). 'A' is promoted char -> int.
8         System.out.println("show('A'):");
9         show('A');
10
11        // there is no only(byte). Promoted byte -> short -> int
12        // -> long -> float -> double, step by step, until it fits.
13        byte b = 5;
14        System.out.println("only(byte 5):");
15        only(b);
16
17        // and the plain arithmetic version of the same rule
18        char letter = 'A';
19        System.out.println("letter + 0 = " + (letter + 0));
20    }
21 }

```

OUTPUT

```

show('A'):
  show(int) got 65
only(byte 5):
  only(double) got 5.0
letter + 0 = 65

```

"65" is the answer to your course's own example: 'A' was promoted to its Unicode value, because there was no method taking a char. This is the same fact as Unit 3.7's `letter > num`, seen from the other side: **a char really is a number**, and Java will use that whenever it needs to.

⚠ TRAP · AN EXACT MATCH ALWAYS BEATS A PROMOTION

Add a `show(char)` to the listing above and `show('A')` prints A, not 65. Java promotes **only when it has to**: it looks for an exact match first, and only widens if none exists.

This is why adding a harmless-looking overload can silently change what your existing calls do. It is also a favourite exam trick: given several overloads, being asked which one runs.

FILL IN THE GAP · METHODS

1. A method signature is its _____ plus its _____. The _____ is **not** part of it.
2. Two methods differing only in return type produce _____.
3. The variable in the declaration is the _____; the value at the call is the _____.
4. Java passes arguments by _____, always. For an array, the thing copied is the _____, which is why _____ is visible to the caller but _____ is not.
5. A _____ method can be called without creating an object; that is why `main` is one.
6. The promotion order is `byte` → _____ → `int` → _____ → `float` → _____, and `char` → _____.
7. Given both `show(int)` and `show(char)`, `show('A')` calls _____, because _____.

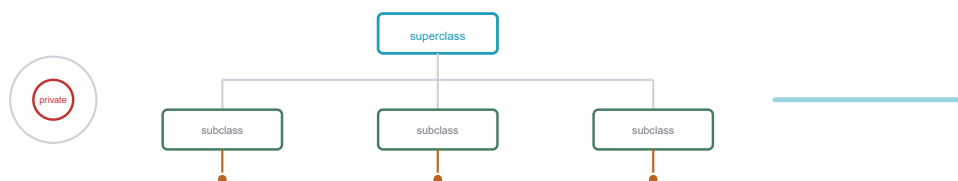
ANSWERS

1. name, parameter list, return type · 2. a compile error ("already defined") · 3. formal parameter, actual argument · 4. value, reference, mutating the array, reassigning it · 5. static · 6. short, long, double, int · 7. `show(char)`, an exact match always beats a promotion

PART VII

Module Six Object-Oriented Programming

The paradigm the language is named for and built around. Classes and the objects made from them, the four pillars, and the two ideas that turn a pile of classes into a design: inheriting behaviour, and letting the same call mean different things. The largest module, and the one Section B rests on.



Organising software around things, not steps

By the end of this unit you should be able to define OOP and describe its purpose, and explain its key principles: classes, objects, encapsulation and abstraction.

TEACH • OBJECT-ORIENTED PROGRAMMING

OOP is a paradigm that organises software design around objects (instances of classes) rather than functions and logic, to model real-world entities.

Compare that with Module 1's imperative paradigm, which organises a program around *steps*. Both write the same instructions in the end. The difference is what you put at the centre of your thinking: the verbs, or the nouns.

● MUST-MEMORISE • THE FOUR PILLARS

Your course lists six key concepts, and says **the last four are the pillars of OOP**:

i. Class ii. Object

iii. Encapsulation iv. Abstraction v. Inheritance vi. Polymorphism

Encapsulation: bundle data and methods together, and restrict direct access.

Abstraction: hide the internal detail, show only the essentials.

Inheritance: a class acquires the attributes and behaviour of another.

Polymorphism: different objects respond to the same method call in different ways.

MEMORY HOOK: A PIE: Abstraction, Polymorphism, Inheritance, Encapsulation. Four letters, four pillars, and you will not blank on the list under pressure.

TEACH • WHY OOP

REASON	MEANS
Modularity and reusability	code is organised into reusable, self-contained components, easier to maintain and scale
Real-world modelling	objects map onto the things the software is actually about
Maintainability	a change to one object's internals need not touch anything else

The blueprint and the building

TEACH • CLASS AND OBJECT, FORMALLY

Module 1 gave you these informally, and the definitions have not changed:

A class is a blueprint or template for creating objects. It defines the structure: the attributes (variables) and the behaviour (methods) an object will have.

An object is a specific instance of a class. Once a class is defined, you create objects from it.

SYNTAX · DECLARING A CLASS, CREATING AN OBJECT

```
accessModifier class ClassName {           <- the blueprint
    // attributes (fields)
    // methods
}

ClassName objectName = new ClassName();    <- one object, built from it
|           |           |
|           |           +--- 'new' builds it and calls the constructor
|           +----- the reference variable (Unit 3.3)
+----- its type
```

CODE · A CLASS, AND TWO OBJECTS FROM IT

FROM THE SLIDES

```
1  class Student {
2      String name;           // attribute
3      int age;               // attribute
4
5      void displayInfo() {    // behaviour
6          System.out.println("Name: " + name + ", Age: " + age);
7      }
8  }
9
10 public class StudentDemo {
11     public static void main(String[] args) {
12         Student s1 = new Student();    // object 1
13         s1.name = "Ada";
14         s1.age = 20;
15
16         Student s2 = new Student();    // object 2: its own data
17         s2.name = "Ken";
18         s2.age = 19;
19
20         s1.displayInfo();
21         s2.displayInfo();
22
23         s1.name = "Adaeze";            // changing one
24         s1.displayInfo();
25         s2.displayInfo();              // ... does not touch the other
26     }
27 }
```

OUTPUT

```
Name: Ada, Age: 20
Name: Ken, Age: 19
Name: Adaeze, Age: 20
Name: Ken, Age: 19
```

One class, two objects, each with its own name and age. Changing s1 left s2 untouched. That independence is the whole point of an instance.

Setting an object up as it is born

● MUST-MEMORISE · A CONSTRUCTOR

A constructor is a special method used to initialise objects of a class. It is called automatically when an object is created.

Its key features:

1. Its **name must be exactly the class name**.
2. It has **no return type**. Not even void.
3. It is called automatically by new.

MEMORY HOOK: a constructor has no return type because it is not a question, it is **the act of making the thing**. Writing `void Car()` does not make a constructor; it makes an ordinary method that happens to be named `Car`, and your object is silently built by the default constructor instead. That is a real bug and hard to see.

TEACH · DEFAULT AND PARAMETERIZED

A default constructor takes no parameters. If you do not define any constructor at all, **Java automatically provides one**, which sets every field to its type's default value (Unit 3.3: 0, false, and null for objects).

A parameterized constructor accepts one or more arguments, letting you initialise an object with specific values at the moment of creation rather than assigning them afterwards.

```

1 // 1. NO constructor written: Java supplies a default one
2 class Book {
3     String title;
4     int pages;
5 }
6
7 // 2. A no-argument constructor, written by hand
8 class Car {
9     String brand;
10    int year;
11
12    Car() {                // name = class, no return type
13        brand = "Unknown";
14        year = 2000;
15    }
16 }
17
18 // 3. A parameterized constructor
19 class Bike {
20     String brand;
21     int year;
22
23     Bike(String brand, int year) {
24         this.brand = brand;    // 'this' = the object being built
25         this.year = year;
26     }
27 }
28
29 public class Constructors {
30     public static void main(String[] args) {
31         Book b = new Book();
32         System.out.println("Book : " + b.title + ", " + b.pages);
33
34         Car c = new Car();
35         System.out.println("Car : " + c.brand + ", " + c.year);
36
37         Bike k = new Bike("Trek", 2024);
38         System.out.println("Bike : " + k.brand + ", " + k.year);
39     }
40 }

```

OUTPUT

```

Book : null, 0
Car : Unknown, 2000
Bike : Trek, 2024

```

The Book line is the one to study: null and 0. Nobody wrote those. Java's supplied default constructor filled each field with its type's default, and null for a String is exactly the value that will throw a NullPointerException the moment anyone uses it. This is why parameterized constructors exist.

● MUST-MEMORISE · THIS

this refers to **the object the method is currently running on**.

this.brand = brand; means "the **field** called brand belonging to this object = the **parameter** called brand".

It is needed exactly when a parameter has the same name as a field, which is the normal case in a constructor, because both want the same obvious name. Without this, brand = brand assigns the parameter to itself and the field stays null. It compiles. It runs. It does nothing.

MEMORY HOOK: the nearer name always wins, and the parameter is nearer. this. is how you say "no, I mean **my** one".

⚠ TRAP · WRITING A CONSTRUCTOR REMOVES THE FREE ONE

Java supplies a default constructor **only if you write no constructor at all**. Add a parameterized one and the free no-argument version **vanishes**.

So a class with only `Bike(String, int)` cannot be built with `new Bike()`: it does not compile. If you want both, write both. This bites hardest in inheritance, in Unit 6.6, where a superclass with only a parameterized constructor forces every subclass to call it explicitly.

UNIT 6.4 · ENCAPSULATION

Private data, public doors

● MUST-MEMORISE · ENCAPSULATION

Encapsulation is the mechanism of bundling data (fields) and methods (behaviour) into a single unit (a class), while restricting direct access to some components.

You achieve it in two moves:

1. declare the fields **private**;
2. provide public **getter** and **setter** methods for controlled access.

"Define encapsulation as an OOP principle" is **2025/26 Section A, Question Two, part A, for two marks**. Learn the sentence in the card, verbatim.

MEMORY HOOK: a **cash machine**. The money and the ledger are sealed inside; you interact only through the slot and the keypad. The bank does not trust you with a door into the safe, and the slot can refuse you.

TEACH · WHY BOTHER

REASON	MEANS
Data hiding	prevents unauthorised access or modification
Validation	setters can check input, for example preventing a negative balance
Flexibility	the internals can change without breaking anyone who uses the class

Validation is the one that makes it concrete. A public `double balance;` can be set to -5000 by anybody, from anywhere, and the class is powerless. A private field with a setter means every change must pass through code *you* wrote, so the object can refuse to enter an impossible state.

```
1 class BankAccount {
2     private String accountNumber;    // PRIVATE: nobody outside touches these
3     private double balance;
4
5     BankAccount(String accountNumber, double balance) {
6         this.accountNumber = accountNumber;
7         this.balance = balance;
8     }
9
10    public double getBalance() {        // GETTER: read, but not write
11        return balance;
12    }
13
14    public void deposit(double amount) { // SETTER, with VALIDATION
15        if (amount <= 0) {
16            System.out.println(" rejected: deposit must be positive");
17            return;
18        }
19        balance += amount;
20    }
21
22    public void withdraw(double amount) {
23        if (amount > balance) {
24            System.out.println(" rejected: insufficient funds");
25            return;
26        }
27        balance -= amount;
28    }
29 }
30
31 public class BankDemo {
32     public static void main(String[] args) {
33         BankAccount a = new BankAccount("ACC-001", 1000.0);
34         System.out.println("start    : " + a.getBalance());
35
36         a.deposit(500);
37         System.out.println("deposit  : " + a.getBalance());
38
39         a.deposit(-9999);           // the object refuses
40         System.out.println("after    : " + a.getBalance());
41
42         a.withdraw(10000);          // and refuses again
43         System.out.println("final    : " + a.getBalance());
44     }
45 }
```

OUTPUT

```
start    : 1000.0
deposit  : 1500.0
    rejected: deposit must be positive
after    : 1500.0
    rejected: insufficient funds
final    : 1500.0
```

The account defended itself twice. Had balance been public, `a.balance = -9999`; would have worked and no code in the class could have stopped it.

```

1  class Account {
2      private double balance = 1000.0;
3      public double getBalance() { return balance; }
4  }
5
6  public class Breakin {
7      public static void main(String[] args) {
8          Account a = new Account();
9          a.balance = -9999;    // straight at the field
10     }
11 }

```

WHAT THE COMPILER SAYS

```

Breakin.java:9: error: balance has private access in Account
    a.balance = -9999;
      ^
1 error

```

Note **when** it is stopped: compile time, in the sense of F.7. `private` is not a polite request or a run-time check. The program cannot be built.

UNIT 6.5 · ABSTRACTION

Showing what, hiding how

● MUST-MEMORISE · ABSTRACTION, AND HOW IT DIFFERS FROM ENCAPSULATION

Abstraction is the process of hiding internal implementation details and showing only the essential features of an object.

The pair everyone confuses:

Encapsulation hides DATA, using private fields and public methods. It is about **protection**.

Abstraction hides COMPLEXITY, using simple methods, abstract classes and interfaces. It is about **simplicity**.

MEMORY HOOK: a car. **Encapsulation** is the locked bonnet: you cannot reach the engine. **Abstraction** is the steering wheel: you do not need to. One says "you may not", the other says "you need not".

```

1 class Car {
2     // the messy detail: private, and none of the driver's business
3     private void turnOnEngine() { System.out.println(" engine on"); }
4     private void checkFuel() { System.out.println(" fuel checked"); }
5     private void startIgnition() { System.out.println(" ignition started"); }
6
7     public void start() {           // the whole interface the driver needs
8         checkFuel();
9         turnOnEngine();
10        startIgnition();
11        System.out.println("Car started.");
12    }
13 }
14
15 public class CarStartSystem {
16     public static void main(String[] args) {
17         new Car().start();           // one word. That is the point.
18     }
19 }

```

OUTPUT

```

fuel checked
engine on
ignition started
Car started.

```

The caller wrote `start()` and nothing else. Three private steps ran, in an order the caller never had to know, and could not have got wrong. "How does abstraction simplify code for users?" is a stated in-text question, and this listing is the answer.

TEACH · ABSTRACT CLASSES AND INTERFACES

Two stronger tools, both about promising *what* without saying *how*.

An abstract class cannot be instantiated directly. It provides partial implementation of some methods and leaves others abstract, for subclasses to fill in.

An interface defines a set of methods that any implementing class must implement, without specifying how they work.

● MUST-MEMORISE · INTERFACE VERSUS ABSTRACT CLASS

FEATURE	INTERFACE	ABSTRACT CLASS
Purpose	defines a contract for the implementing class	a contract plus some common behaviour
Inheritance	a class may implement many	a class may extend only one
Keyword	implements	extends
Fields	constants only	ordinary fields allowed
Constructor	none	has one (called via super)

"What is the difference between an abstract class and an interface?" and "when should you use an interface instead of an abstract class?" are both stated in-text questions.

MEMORY HOOK: an abstract class is **what a thing is**; an interface is **what a thing can do**. A Dog *is* an Animal (only one of those). A Dog *can* Swim, and Run, and Fetch (as many as you like). Hence one extends and many implements.

```
1 // ABSTRACT CLASS: partial. Cannot be built on its own.
2 abstract class Vehicle {
3     abstract void start();           // no body: subclasses MUST supply one
4
5     void fuelUp() {                  // concrete: shared by all vehicles
6         System.out.println("  refuelling...");
7     }
8 }
9
10 // INTERFACE: a pure contract. No bodies at all.
11 interface Movable {
12     void move();
13 }
14
15 class Car extends Vehicle implements Movable {
16     void start() { System.out.println("Car starts with a key"); }
17     public void move() { System.out.println("Car drives on roads"); }
18 }
19
20 class Boat extends Vehicle implements Movable {
21     void start() { System.out.println("Boat starts with a button"); }
22     public void move() { System.out.println("Boat sails on water"); }
23 }
24
25 public class AbstractDemo {
26     public static void main(String[] args) {
27         Vehicle[] fleet = { new Car(), new Boat() };
28         for (Vehicle v : fleet) {
29             v.start();               // same call, different behaviour
30             v.fuelUp();               // inherited, identical for both
31         }
32
33         Movable[] things = { new Car(), new Boat() };
34         for (Movable m : things) m.move();
35     }
36 }
```

OUTPUT

```
Car starts with a key
  refuelling...
Boat starts with a button
  refuelling...
Car drives on roads
Boat sails on water
```

Car extends **one** class and implements an interface at the same time. Note `fuelUp()` printed identically for both: that is the "some common behaviour" an interface could not have given you. And the loop already shows polymorphism, which is Unit 6.7.

```

1 abstract class Shape {
2     abstract double area();
3 }
4
5 public class NoAbstract {
6     public static void main(String[] args) {
7         Shape s = new Shape(); // what would its area() do?
8     }
9 }

```

WHAT THE COMPILER SAYS

```

NoAbstract.java:7: error: Shape is abstract; cannot be instantiated
    Shape s = new Shape();
                ^
1 error

```

And the reason is in the comment: `area()` has no body, so a bare `Shape` object would have a method with nothing to run. "Abstract" means **incomplete on purpose**, and Java will not build something incomplete.

UNIT 6.6 · INHERITANCE

Getting behaviour for free

By the end of this unit you should be able to explain and implement inheritance, apply polymorphism, differentiate overriding from overloading, and understand access modifiers.

● MUST-MEMORISE · INHERITANCE

Inheritance allows a class (the subclass, derived class or child class) to inherit attributes and behaviour from another class (the superclass, base class or parent class).

The keyword is `extends`:

```
class Dog extends Animal { ... }
```

Benefits: **code reuse**, **extensibility**, and **hierarchical design**.

MEMORY HOOK: the **is-a** test. A Dog *is an* Animal, so Dog `extends` Animal. A Car *is not a* Engine, it *has* one, so that is a field, not inheritance. If "is-a" sounds wrong out loud, do not use `extends`.

⚠ TRAP · THE KEYWORD IS EXTENDS, WITH AN S

Your course's slide titles it "The **extend** keyword". It is `extends`. Java has no keyword `extend`, and `class Dog extend Animal` does not compile.

Worth knowing because a debug question could plant exactly this, and because a wrong keyword in a written exam answer is a wrong answer.

TEACH · THE THREE TYPES OF INHERITANCE

TYPE	SHAPE	MEANING
Single	Animal → Dog	a subclass inherits from one superclass
Multilevel	Animal → Dog → Puppy	a class derives from a derived class
Hierarchical	Animal → Lion, and Animal → Cat	several subclasses inherit from one class

You may notice one missing: **multiple inheritance**, where a class has two superclasses. **Java does not allow it for classes**, which is exactly why a class may only extend one thing but may implement many interfaces.

```
1 class Animal {
2     public void eat() { System.out.println("Animal eats food"); }
3 }
4
5 // SINGLE: Dog from Animal
6 class Dog extends Animal {
7     public void bark() { System.out.println("Dog barks"); }
8 }
9
10 // MULTILEVEL: Puppy from Dog, and so from Animal too
11 class Puppy extends Dog {
12     public void weep() { System.out.println("Puppy weeps"); }
13 }
14
15 // HIERARCHICAL: Lion and Cat both from Animal
16 class Lion extends Animal {
17     public void roar() { System.out.println("Lion roars"); }
18 }
19 class Cat extends Animal {
20     public void meow() { System.out.println("Cat meows"); }
21 }
22
23 public class Inherit {
24     public static void main(String[] args) {
25         Dog d = new Dog();
26         d.eat();      // inherited: never written in Dog
27         d.bark();
28
29         Puppy p = new Puppy();
30         p.eat();      // from Animal, two levels up
31         p.bark();     // from Dog, one level up
32         p.weep();     // its own
33
34         new Lion().roar();
35         new Cat().meow();
36     }
37 }
```

OUTPUT

```
Animal eats food
Dog barks
Animal eats food
Dog barks
Puppy weeps
Lion roars
Cat meows
```

Puppy has three methods and only one was written in it. **That is code reuse, and it is what inheritance buys.**

● MUST-MEMORISE · THE FOUR ACCESS MODIFIERS

MODIFIER	VISIBLE
public	everywhere
protected	same package, and subclasses even in other packages
(none, "default")	same package only
private	same class only. Reachable from outside only indirectly , through a public or protected method.

A subclass can only access what is visible to it. So default access works only while parent and child share a package: separate them and it becomes a compile-time error, which your course notes.

MEMORY HOOK: in order, the doors open wider: **private < default < protected < public**. Only protected is unusual, and its job is exactly "family only": it exists so children can inherit what strangers cannot see.

CODE · PRIVATE, REACHED INDIRECTLY

FROM THE SLIDES

```

1  class Animal {
2      public void eat()      { System.out.println("Animal eats food"); }
3      protected void sleep() { System.out.println("Animal sleeps"); }
4      void move()           { System.out.println("Animal moves"); } // default
5
6      private void breathe() { System.out.println("Animal breathes"); }
7
8      public void showBreathing() { // the public door to a private room
9          breathe();
10     }
11 }
12
13 class Cat extends Animal {
14     public void rest() {
15         sleep();           // protected: a subclass may
16         System.out.println("Cat rests");
17     }
18 }
19
20 public class AccessDemo {
21     public static void main(String[] args) {
22         Cat c = new Cat();
23         c.eat();
24         c.rest();
25         c.move();
26         c.showBreathing(); // breathe() runs, indirectly
27     }
28 }

```

OUTPUT

```

Animal eats food
Animal sleeps
Cat rests
Animal moves
Animal breathes

```

breathe() is private, so Cat cannot call it and neither can main. Yet "Animal breathes" printed, because showBreathing() is public and lives *inside* Animal, where private is visible. That is Unit 6.4's cash machine again: not a locked box, a controlled door.

super is a reference to the **immediate superclass**. Two uses:

1. **super(args)**: call the superclass constructor. Must be the **first statement** in the subclass constructor.
2. **super.method()**: call the superclass's version of a method you have overridden.

If you do not call it explicitly, **Java automatically invokes the superclass's no-argument constructor**. And if the superclass has no no-argument constructor, the subclass **must** call a parameterized one with `super(arguments)`, or it will not compile. That is Unit 6.3's trap arriving with consequences.

MEMORY HOOK: you cannot build the first floor before the ground floor. The superclass part of the object must exist before the subclass part, which is why `super(...)` must come first.

CODE · BOTH USES OF SUPER

FROM THE SLIDES

```

1  class Vehicle {
2      String brand;
3
4      Vehicle(String brand) {          // ONLY a parameterized constructor
5          this.brand = brand;
6          System.out.println("Vehicle constructor: " + brand);
7      }
8
9      void start() { System.out.println("Vehicle is starting"); }
10 }
11
12 class Car extends Vehicle {
13
14     Car(String brand) {
15         super(brand);                // COMPULSORY, and must be first
16         System.out.println("Car constructor");
17     }
18
19     void start() {                    // overrides Vehicle.start()
20         System.out.println("Car is starting");
21         super.start();                // ... and still calls the original
22     }
23 }
24
25 public class SuperDemo {
26     public static void main(String[] args) {
27         new Car("Toyota").start();
28     }
29 }

```

OUTPUT

```

Vehicle constructor: Toyota
Car constructor
Car is starting
Vehicle is starting

```

Read the order. The **Vehicle** constructor ran **before** the **Car** constructor, even though you wrote `new Car(...)`: ground floor first. Then `start()` printed the **Car** line and the **Vehicle** line, because `super.start()` reached back to the version it had replaced.

SYNTAX · @OVERRIDE

Written on the line directly above a method that is meant to override one:

```
@Override  
public void withdraw(double amount) { ... }
```

It changes **nothing** about how the program runs. It is a note to the **compiler**, and it asks one question: *does this method really override something?* If the answer is no, the code does not compile.

Java does not require it. Overriding works without it, and the exam will not fail you for leaving it out. Write it anyway. The next box shows why.

⚠ TRAP · THE OVERRIDE THAT SILENTLY WAS NOT ONE

An overriding method must have the **same name and the same parameter list** as the one it replaces. Change the parameter type by one word and you have not overridden anything: you have written an **overload** (Unit 5.6), a second, unrelated method that merely shares a name. Nothing warns you.

Here `Account.withdraw` takes a `double` and `Saving.withdraw` takes an `int`:

```
1 class Account {
2     protected double balance = 5000;
3
4     public void withdraw(double amount) {
5         balance -= amount;
6         System.out.println("Account.withdraw ran. Balance is now " + balance);
7     }
8 }
9
10 class Saving extends Account {
11
12     // MEANT to override Account.withdraw. Look hard at the parameter type.
13     public void withdraw(int amount) {
14         System.out.println("Saving.withdraw ran: the 1000 rule was applied");
15     }
16 }
17
18 public class SilentOverload {
19     public static void main(String[] args) {
20         Account acc = new Saving();
21         acc.withdraw(600);
22         System.out.println("Did the saving rule run? Read the line above.");
23     }
24 }
```

OUTPUT

```
Account.withdraw ran. Balance is now 4400.0
Did the saving rule run? Read the line above.
```

The saving rule never ran. The money left the account with no rule applied, and the program printed no error, because as far as Java is concerned nothing went wrong. This is the worst kind of bug: a silent one.

Why it happens. The variable is declared `Account acc`, so at **compile time** the compiler sees only the methods `Account` has, and the only `withdraw` there takes a `double`. So it compiles the call as `withdraw(double)`, widening the literal `600` to `600.0` (Unit 3.5). At **run time**, Java looks for an override of `withdraw(double)` in `Saving`. There is not one: `Saving` has `withdraw(int)`, which is a different method. So the parent's version runs.

Overloading is chosen at compile time, from the type the variable is DECLARED as. Overriding is chosen at run time, from the type the object ACTUALLY is. Confuse the two and you get exactly this: a method that looks overridden and is not.

Now add `@Override` to the same broken method. The compiler is obliged to check the claim, and the claim is false:

```
1 class Account {
2     protected double balance = 5000;
3
4     public void withdraw(double amount) {
5         balance -= amount;
6     }
7 }
8
9 class Saving extends Account {
10
11     @Override
12     public void withdraw(int amount) {
13         System.out.println("the 1000 rule was applied");
14     }
15 }
```

```

14     }
15 }
16
17 public class OverrideCaught {
18     public static void main(String[] args) {
19         new Saving().withdraw(600);
20     }
21 }

```

COMPILER OUTPUT

error: method does not override or implement a method from a supertype

The silent runtime bug became a loud compile-time error, which is the trade you always want. One word turned a bug you would have hunted for an hour into a message that names the problem.

MEMORY HOOK: `@Override` is a claim the compiler checks. Make the claim, and Java will tell you when you are wrong.

UNIT 6.7 · POLYMORPHISM

One call, many behaviours

● MUST-MEMORISE · POLYMORPHISM AND ITS TWO TYPES

Polymorphism is the ability of different objects to respond to the same method in different ways. It lets objects be treated as instances of their parent class rather than their actual class.

Compile-time (static) polymorphism is achieved through **method overloading**. Which method runs is decided by the compiler.

Runtime (dynamic) polymorphism is achieved through **method overriding**. Which method runs is decided by the JVM, from the object's actual type, while the program runs.

MEMORY HOOK: the names say when the decision is made. **Overloading is decided by reading; overriding is decided by running.** The compiler can see which `add()` you wrote; only the running program knows whether that `Animal` variable is holding a `Dog`.

● MUST-MEMORISE · OVERLOADING VERSUS OVERRIDING

25/26 Q2 B · 3 MARKS

	OVERLOADING	OVERRIDING
1. Where	same class	subclass, redefining the parent's method
2. Parameters	must be different	must be identical
3. Decided	at compile time	at run time
4. Return type	may differ freely	must be the same (or a subtype)
5. Purpose	same idea, different inputs	same input, different behaviour
6. Inheritance	not needed	impossible without it

"State three differences between Method overloading and Method overriding" is **2025/26 Section A, Question Two, part B, for three marks**. Give rows 1, 2 and 3: they are the crispest, and each is a genuinely different axis.

MEMORY HOOK: **overLOAD** adds more (more versions, different parameters, in one class). **overRIDE** replaces (same signature, new behaviour, in a child). Load means adding; ride means riding over the top.

```

1 class Appliance {
2     void operate() { System.out.println("Appliance is operating"); }
3 }
4
5 class WashingMachine extends Appliance {
6     void operate() { System.out.println("Washing clothes"); } // OVERRIDE
7 }
8
9 class Microwave extends Appliance {
10    void operate() { System.out.println("Heating food"); } // OVERRIDE
11 }
12
13 public class Poly {
14
15     // COMPILE-TIME polymorphism: overloading
16     static int add(int a, int b) { return a + b; }
17     static int add(int a, int b, int c) { return a + b + c; }
18     static double add(double a, double b) { return a + b; }
19
20     public static void main(String[] args) {
21         System.out.println("compile-time (overloading):");
22         System.out.println("  " + add(1, 2) + " " + add(1, 2, 3) + " " + add(1.5, 2.5));
23
24         System.out.println("runtime (overriding):");
25         Appliance myAppliance; // declared type: Appliance
26
27         myAppliance = new WashingMachine(); // actual object decides
28         System.out.print(" "); myAppliance.operate();
29
30         myAppliance = new Microwave();
31         System.out.print(" "); myAppliance.operate();
32
33         myAppliance = new Appliance();
34         System.out.print(" "); myAppliance.operate();
35     }
36 }

```

OUTPUT

```

compile-time (overloading):
  3 6 4.0
runtime (overriding):
  Washing clothes
  Heating food
  Appliance is operating

```

This is the heart of Module 6. The variable `myAppliance` is declared `Appliance` and never changes its declared type. The line `myAppliance.operate()` is written once. It printed three different things, because each time the JVM looked at **what the object actually was**, not what the variable said it was. The compiler could not have known: only the running program can.

TEACH · WHY THE EXAM KEEPS ASKING FOR THIS SHAPE

Your 2025/26 Question Four ends: "Demonstrates polymorphism by storing each object in a **BankAccount reference** and calling the withdrawal method through it, confirming that the correct overridden version runs."

That sentence is describing exactly the listing above. The marks are for three things: the **declared type is the parent**, the **object is the child**, and calling through the parent reference **still runs the child's method**. If you write `SavingAccount s = new SavingAccount();` you have not demonstrated polymorphism, even though the right method runs: the point is lost, and so is the mark.

Write `BankAccount acc = new SavingAccount();`. That single line is the whole demonstration.

RECALL AND CHECK

Given `Appliance a = new Microwave();`, which `operate()` runs, and which type decides: the declared type or the actual object?

ANSWER

Microwave's. The **actual object** decides, at run time. The declared type (`Appliance`) decides only **which methods you are allowed to call** at compile time; among those, the object decides which version actually runs. That split is the whole of runtime polymorphism.

FILL IN THE GAP · OOP

1. The four pillars are _____, _____, _____ and _____.
2. Encapsulation is achieved by making fields _____ and providing public _____ and _____ methods.
3. Encapsulation hides _____; abstraction hides _____.
4. A constructor's name must be _____ and it has _____.
5. If you write no constructor, Java gives you a _____. If you write a parameterized one, the free one _____.
6. A class may extends _____ class(es) and implements _____ interface(s).
7. Overloading happens in _____ and is resolved at _____ time; overriding happens in _____ and is resolved at _____ time.
8. To demonstrate polymorphism you must store the object in a _____ reference.

ANSWERS

1. encapsulation, abstraction, inheritance, polymorphism (A PIE) · 2. private, getter, setter · 3. data, complexity · 4. exactly the class name, no return type · 5. default constructor, disappears · 6. one, many · 7. the same class, compile, a subclass, run · 8. superclass (parent)

PART VIII

Module Seven Strings and String Processing

Text: the type you will use more than any other, and the one that breaks the rules you have just learned. It looks like a primitive and is an object. It cannot be changed. And comparing two of them with the operator you have used all year is the most famous mistake in Java, made worse because it usually works.



An object that behaves like a value

By the end of this unit you should be able to explain strings, perform concatenation, substring extraction and searching, differentiate *String*, *StringBuilder* and *StringBuffer*, and use the mutable classes for efficient manipulation.

● MUST-MEMORISE • STRING

A *String* is an object representing a sequence of characters (char values) enclosed in double quotes.

It is a **final class** in the `java.lang` package. Final means it **cannot be subclassed**.

"What is a *String*, and how can you create one?" is a stated in-text question, so this definition is a candidate for the two-mark slot.

MEMORY HOOK: a *String* is a **necklace of chars**. The beads are `char` values (Unit 3.3), each at a numbered position, and the clasp is welded shut: you can read any bead, and you can never swap one.

TEACH • TWO WAYS TO CREATE ONE, AND THEY ARE NOT THE SAME

1. *String* literals, stored in the **String Pool**:

```
String s1 = "Hello";
String s2 = "Hello";    // REUSES the same object from the String Pool
```

2. The new keyword, which always builds a fresh object:

```
String s3 = new String("Hello");
String s4 = new String("Hello");    // a SECOND, separate object
```

The **String Pool** is a store Java keeps of every literal it has seen. Ask for the literal "Hello" twice and Java hands you the **same object** both times, because a *String* can never change, so sharing it is safe. Say `new` and you have explicitly demanded a new object, so Java makes one, pool or no pool.

This sounds like an implementation detail. It is the reason for Unit 7.3, which is the most important unit in this module.

● MUST-MEMORISE • IMMUTABILITY

Strings are immutable: once created, their value cannot be changed. Any operation that appears to modify a string actually creates a new *String* object.

"Why are *Strings* immutable, and what is the implication?" is a stated in-text question. The implication: it improves **security** and **performance** (sharing via the pool is safe), but it is **inefficient for frequent changes**, because every change allocates a new object. That inefficiency is why *StringBuilder* exists (Unit 7.5).

MEMORY HOOK: a *String* is **printed**, not written in pencil. "Editing" it means printing a new page. The old page still exists, unchanged, until nobody is looking at it.

```

1 public class Immutable {
2     public static void main(String[] args) {
3         String s = "Java";
4
5         s.concat(" Programming"); // result thrown away!
6         System.out.println("without assignment: " + s);
7
8         String t = s.concat(" Programming"); // result kept
9         System.out.println("with assignment : " + t);
10    }
11 }

```

OUTPUT

```

without assignment: Java
with assignment : Java Programming

```

`s.concat(...)` did not fail, and it did not change `s`. It **built a new String and returned it**, and because nobody caught the return value, that new String was discarded instantly. This is the practical meaning of immutable: **every String method returns a new String and changes nothing**. If you do not assign the result, nothing happened.

UNIT 7.2 · THE STRING METHODS

Everything you need, in one place

TEACH · THE REFERENCE CARD

This is the complete set this course uses. Every Result column below was produced by running it, on `str = "Hello, World!"`.

METHOD	RESULT	DOES
<code>length()</code>	13	how many characters. A method , with brackets.
<code>charAt(0)</code>	H	the char at an index. Indexes start at 0 .
<code>indexOf('o')</code>	4	index of the first occurrence, or -1 if absent
<code>indexOf("World")</code>	7	works for a whole substring too
<code>contains("World")</code>	true	is it in there at all?
<code>startsWith("Hello")</code>	true	does it begin with this?
<code>endsWith("!")</code>	true	does it end with this?
<code>toUpperCase()</code>	HELLO, WORLD!	a new String, upper cased
<code>concat(" x")</code>	a new String	the same as +
<code>equals(other)</code>	see 7.3	same characters? Use this, not ==
<code>equalsIgnoreCase(other)</code>	see 7.3	same, ignoring case
<code>compareTo(other)</code>	see 7.3	lexicographic order
<code>substring(3)</code>	see below	from index 3 to the end
<code>substring(3, 8)</code>	see below	from 3 up to but not including 8
<code>split(" ")</code>	a String[]	break it up on a separator

● MUST-MEMORISE · SUBSTRING: START INCLUSIVE, END EXCLUSIVE

`substring(start)` gives everything from start to the end.

`substring(start, end)` gives characters from start **up to but NOT including** end.

So its length is exactly $\text{end} - \text{start}$.

Your course states this plainly ("start is inclusive while end is exclusive") and it is still the thing people get wrong under pressure.

MEMORY HOOK: the same rule as a for loop: `for (int i = start; i < end; i++)` also stops *before* end. Java is consistent: **an end bound is always a fence you stop at, not a post you include.**

CODE · EVERY METHOD, RUN

VERIFIED ON THE JVM

```
1 public class StringMethods {
2     public static void main(String[] args) {
3         String str = "Hello, World!";
4         System.out.println("length()           : " + str.length());
5         System.out.println("charAt(0)          : " + str.charAt(0));
6         System.out.println("indexOf('o')       : " + str.indexOf('o'));
7         System.out.println("indexOf(\"World\")   : " + str.indexOf("World"));
8         System.out.println("contains(\"World\") : " + str.contains("World"));
9         System.out.println("startsWith(\"Hello\") : " + str.startsWith("Hello"));
10        System.out.println("endsWith(\"!\")      : " + str.endsWith("!"));
11        System.out.println("toUpperCase()    : " + str.toUpperCase());
12
13        String cs = "Computer Science";
14        System.out.println("substring(3)       : " + cs.substring(3));
15        System.out.println("substring(3, 8)    : " + cs.substring(3, 8));
16
17        // concatenation: the + operator, or concat()
18        String firstName = "John", lastName = "Doe";
19        System.out.println("+ operator      : " + firstName + " " + lastName);
20        System.out.println("concat()         : " + "Hello".concat(" World"));
21    }
22 }
```

OUTPUT

```
length()           : 13
charAt(0)          : H
indexOf('o')       : 4
indexOf("World")   : 7
contains("World") : true
startsWith("Hello"): true
endsWith("!")      : true
toUpperCase()     : HELLO, WORLD!
substring(3)       : puter Science
substring(3, 8)    : puter
+ operator         : John Doe
concat()           : Hello World
```

Check `substring(3, 8)` against the rule: indexes 3, 4, 5, 6, 7, stopping before 8. "Computer" is C0 o1 m2 p3 u4 t5 e6 r7, so you get "puter", five characters, which is $8 - 3 = 5$. The r at index 7 is included; index 8, the space, is not.

⚠ TRAP · LENGTH VS LENGTH()

An array has `arr.length`, a **property**, with no brackets (F.10). A String has `str.length()`, a **method**, with brackets. A String is an object with methods; an array's length is a plain field.

Get it backwards and it does not compile, so it costs you seconds on a machine and marks on paper. Both papers hand you array and String code in the same question.

Why comparing text with == destroys programs

This unit is the reason Unit 3.7 told you to remember that == compares values for primitives and **references** for objects. Here is the bill.

● MUST-MEMORISE • NEVER COMPARE STRINGS WITH ==

== on two Strings asks "are these the same object?"

.equals() asks "do these have the same characters?"

You almost always mean the second. **Always use .equals() for text.**

MEMORY HOOK: == asks "are you living at the same address?"; .equals() asks "do you look the same?". Identical twins in different houses are equal but not ==.

CODE • THE WHOLE TRAP, IN FIVE LINES OF OUTPUT

VERIFIED ON THE JVM

```
1 public class EqualsTrap {
2     public static void main(String[] args) {
3         String s1 = "Hello";           // literal -> String Pool
4         String s2 = "Hello";           // literal -> the SAME pooled object
5         String s3 = new String("Hello"); // new -> a fresh object
6         String s4 = new String("Hello"); // new -> another fresh object
7
8         System.out.println("s1 == s2      : " + (s1 == s2));
9         System.out.println("s3 == s4      : " + (s3 == s4));
10        System.out.println("s1 == s3      : " + (s1 == s3));
11        System.out.println("s1.equals(s3) : " + s1.equals(s3));
12    }
13 }
```

OUTPUT

```
s1 == s2      : true
s3 == s4      : false
s1 == s3      : false
s1.equals(s3) : true
```

Every one of those four strings contains "Hello". Read the outputs again.

⚠ TRAP • THE REASON THIS BUG IS SO VICIOUS

Look at the first line: **s1 == s2 is true.**

That is what makes this the most famous bug in Java. If == were simply always wrong for Strings, everyone would learn it on day one. Instead it **works perfectly** whenever both sides are literals, because the pool hands you the same object twice. So you write ==, you test it with literals, it passes, you ship it.

Then a real String arrives, from JOptionPane.showInputDialog, from Scanner, from a file, built by concatenation, and it is a **new object**, not a pooled literal. Now == is false even though the text matches, your if never fires, and nothing crashes to tell you.

Every user input in your exam is one of those. Question Five's tier is typed by a user. if (tier == "Gold") would be false for a user who typed Gold. tier.equals("Gold") is true. That is the difference between a working program and a zero.

```

1 public class Comparing {
2     public static void main(String[] args) {
3         String a = "Java", b = "java";
4
5         System.out.println("equals          : " + a.equals(b));
6         System.out.println("equalsIgnoreCase : " + a.equalsIgnoreCase(b));
7         System.out.println("compareTo       : " + a.compareTo(b));
8         System.out.println("'J' is " + (int) 'J' + ", 'j' is " + (int) 'j');
9     }
10 }

```

OUTPUT

```

equals          : false
equalsIgnoreCase : true
compareTo       : -32
'J' is 74, 'j' is 106

```

Ignore the number for a second and read the sign. **compareTo** returns **negative** if this comes before the other, **zero** if equal, **positive** if after. So -32 means "Java" sorts before "java". And the size is not magic: 'J' is 74 and 'j' is 106, so $74 - 106 = -32$. It is comparing the char codes of Unit 3.3, and capitals come first.

UNIT 7.4 · STRINGBUILDER AND STRINGBUFFER

When immutability becomes the problem

TEACH · WHY THEY EXIST

The **String** class is **immutable**. Every modification creates a new **String** object, which can be **inefficient** when dealing with frequent or large-scale string operations. To overcome this, Java provides two **mutable** alternatives: **StringBuilder** and **StringBuffer**.

Concretely: building a 1000-character string by adding one character at a time with **+** creates a thousand **String** objects, each a copy of the last plus one. A **StringBuilder** edits one buffer in place.

● MUST-MEMORISE · THE THREE COMPARED

FEATURE	STRING	STRINGBUILDER	STRINGBUFFER
Mutability	Immutable	Mutable	Mutable
Thread-safe	No	No	Yes (synchronized)
Performance	Slow with many modifications	Fast (single-threaded)	Slower (synchronisation costs)
Use when	content does not change	frequent changes, one thread	frequent changes, many threads

"When should you use **StringBuilder** instead of **String**?" is a stated in-text question. The difference between the two mutable ones is **only** thread-safety, and its cost is speed.

MEMORY HOOK: Builder is Buffer without the locks. Buffer is safe for many threads and pays for it; Builder assumes you are alone and is faster. Alone is the normal case, so **StringBuilder is the default choice**.

```

1 public class Builder {
2     public static void main(String[] args) {
3         StringBuilder sb = new StringBuilder("Java");
4         System.out.println("start      [" + sb + "]);
5
6         sb.append(" Programming");
7         System.out.println("append    [" + sb + "]);
8
9         sb.insert(5, "Language ");
10        System.out.println("insert(5,..) [" + sb + "]);
11
12        sb.replace(5, 13, "Core ");
13        System.out.println("replace(5,13) [" + sb + "]);
14
15        sb.delete(5, 10);
16        System.out.println("delete(5,10) [" + sb + "]);
17
18        sb.reverse();
19        System.out.println("reverse   [" + sb + "]);
20    }
21 }

```

OUTPUT

```

start      [Java]
append     [Java Programming]
insert(5,..) [Java Language Programming]
replace(5,13) [Java Core  Programming]
delete(5,10) [Java  Programming]
reverse     [gnimmargorP  avaJ]

```

Your course gives only the final answer, Result: gnimmargorP avaJ, which is correct but teaches nothing. The six lines above are the same program printing after every step, and now the double space is explicable: `replace(5, 13, "Core ")` swapped the eight characters "Language" for the five "Core ", leaving the original space at index 13 next to the new one. It survives every later step and ends up in the middle of the reversed text.

The one call not yet named: `insert(index, text)` pushes text in so that it **starts at** index, sliding everything from that position rightwards to make room. Nothing is overwritten, so the builder grows by exactly the length of what you inserted. Contrast it with `replace(5, 13, "Core ")` on the line above, which **removes** the characters between the two indexes before putting the new ones in, and so can shrink the builder.

Note also that **none of these returned anything you had to keep**. `sb.append(...)` changed `sb` itself. That is the whole difference from Unit 7.1's `concat`, which changed nothing and returned everything.

⚠ TRAP · YOUR EXAM MAY FORBID STRINGBUILDER

2025/26 Question Three asks you to build a comma-separated list and says: "Build this string using **plain String concatenation only (no StringBuilder, no array of booleans returned).**"

So the efficient tool is banned, on purpose. The examiner is testing that you can do it with `+` and a loop, and probably that you handle the trailing comma. Read the constraint and obey it: using `StringBuilder` there scores zero regardless of correctness.

Which is the real lesson of this module: **know both, and let the question choose.**

The classic difficulty is the separator: three items need two commas, not three, and a stray trailing comma looks like a bug to a marker.

- 1 Start with an empty String, not null.

null + "x" gives the text "nullx". An empty String gives "x".

- 2 Before appending an item, append a comma **only if the string is not already empty**.

This is the trick. It puts separators between items rather than after them, and needs no special case for the last one.

- 3 Append the item.

```

1 public class JoinNoBuilder {
2     static String join(int[] vals) {
3         String out = ""; // empty, never null
4         for (int i = 0; i < vals.length; i++) {
5             if (!out.isEmpty()) {
6                 out = out + ", "; // separator BEFORE, not after
7             }
8             out = out + vals[i];
9         }
10        return out;
11    }
12
13    public static void main(String[] args) {
14        System.out.println("three : [" + join(new int[]{20, 40, 60}) + "]");
15        System.out.println("one   : [" + join(new int[]{20}) + "]");
16        System.out.println("none  : [" + join(new int[]{}) + "]");
17    }
18 }

```

OUTPUT

```

three : [20, 40, 60]
one   : [20]
none  : []

```

No trailing comma, and the one-item and empty cases came out right without a single line written for them. Those are the edge cases a marker checks first.

Now redo it: on blank paper, write join from memory. Then say why starting with String out = null; would print null20, 40, 60.

FILL IN THE GAP · STRINGS

1. A String is an _____ representing a sequence of _____, and it is a _____ class, so it cannot be _____.
2. Strings are _____: any operation that seems to modify one actually _____.
3. "Hello" written as a literal is stored in the _____, so two identical literals are _____ object. Using new always creates _____.
4. == on Strings asks _____; .equals() asks _____.
5. s1 == s2 for two identical literals is _____, which is why the bug is so _____.
6. "Computer".substring(3, 8) gives _____, because start is _____ and end is _____.
7. An array uses _____; a String uses _____.
8. StringBuilder differs from StringBuffer only in _____, at the cost of _____.

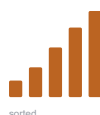
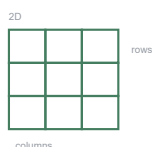
ANSWERS

1. object, chars, final, subclassed · 2. immutable, creates a new String · 3. String Pool, the same, a new object · 4. are these the same object, do these have the same characters · 5. true, vicious (it works until the text comes from a user) · 6. puter, inclusive, exclusive · 7. length, length() · 8. thread-safety (Buffer is synchronized), speed

PART IX

Module Eight Arrays and Array Manipulation

You met arrays in Foundations because the course could not wait until here. Now they get their proper treatment: a second dimension, the built-in toolkit, searching and sorting, and the parallel-array technique the exam demands when it bans every collection class in the language.



What F.10 promised to finish

By the end of this unit you should be able to define arrays and explain their uses, declare and manipulate arrays of different dimensions, use array methods for sorting and searching, and carry out array operations.

TEACH · WHERE YOU ALREADY ARE

F.10 taught you an array as a fixed row of numbered boxes, indexes 0 to n-1, arr.length with no brackets, and `ArrayIndexOutOfBoundsException` when you overshoot. All of that still holds and none of it is repeated here.

This module adds what F.10 promised: the formal definition, a second dimension, the built-in toolkit, searching, sorting, and the techniques your exam actually asks for.

● MUST-MEMORISE · AN ARRAY

An array is a data structure used to store multiple values of the same data type in a single variable. Each element is identified by an **index**.

A **1D array** (linear array) is a linear collection of elements of the same type, stored in **contiguous memory locations** and accessed by index.

Two words earn their place. **Same data type**: an `int[]` holds only ints, which is what makes it different from a list of anything. **Contiguous**: the boxes sit next to each other in memory, which is why reaching `arr[9000]` is exactly as fast as `arr[0]`. Java does not walk to it; it computes where it is.

MEMORY HOOK: an array is a **street of identical houses, numbered from 0**. Same type of house, all in a row, and you find number 40 by arithmetic, not by walking past 39 doors.

SYNTAX · THE THREE WAYS TO MAKE ONE

```
// 1. declare, then allocate, then assign (three steps)
int[] numbers;           <- DECLARATION: a name, no array yet
numbers = new int[3];    <- ALLOCATION: three boxes, all 0
numbers[0] = 10;         <- ASSIGNMENT
```

```
// 2. declare and allocate together
int[] numbers = new int[3];
```

```
// 3. declare, allocate and fill in one line (the usual way)
int[] numbers = {10, 45, 30}; <- size taken from the list
```

The size is fixed at allocation and can never change. That is the defining limitation of an array, and the reason `ArrayList` exists. Your exam bans `ArrayList`, so this limitation is permanent for you.

```

1  import java.util.Arrays;
2
3  public class Declare8 {
4      public static void main(String[] args) {
5          // 1. three steps
6          int[] a;
7          a = new int[3];
8          System.out.println("fresh int[3] : " + Arrays.toString(a));
9          a[0] = 10; a[1] = 45; a[2] = 30;
10         System.out.println("after filling : " + Arrays.toString(a));
11
12         // 2 and 3
13         int[] b = new int[3];
14         int[] c = {10, 45, 30};
15         System.out.println("b           : " + Arrays.toString(b));
16         System.out.println("c           : " + Arrays.toString(c));
17
18         // defaults for other types (Unit 3.3)
19         System.out.println("new double[2] : " + Arrays.toString(new double[2]));
20         System.out.println("new boolean[2]: " + Arrays.toString(new boolean[2]));
21         System.out.println("new String[2] : " + Arrays.toString(new String[2]));
22     }
23 }

```

OUTPUT

```

fresh int[3] : [0, 0, 0]
after filling : [10, 45, 30]
b           : [0, 0, 0]
c           : [10, 45, 30]
new double[2] : [0.0, 0.0]
new boolean[2]: [false, false]
new String[2] : [null, null]

```

new int[3] is not empty: it is **three zeros**. Java fills every new array with its type's default from Unit 3.3, which is why the String[] is full of null. That last row is a NullPointerException waiting for anyone who forgets to fill it.

UNIT 8.2 · TWO-DIMENSIONAL ARRAYS

An array of arrays

● MUST-MEMORISE · 2D ARRAYS

A 2D array is an array of arrays. Picture a table with rows and columns. Each element needs **two indices**: one for the row, one for the column.

```
int[][] matrix = new int[3][2]; 3 rows, 2 columns
```

matrix[r][c]: **row first, column second**. matrix.length is the number of **rows**; matrix[0].length is the number of **columns**.

"What is the syntax to declare and allocate a 2D array of size 3x2?" is a stated in-text question, and the answer is the line in the card.

MEMORY HOOK: RC, like a spreadsheet: Row then Column. And matrix.length counts rows because a 2D array literally is an array of rows: each row is itself an array, so it has its own .length.

```

1 public class TwoD {
2     public static void main(String[] args) {
3         // declare + allocate: 3 rows, 2 columns
4         int[][] numbers = new int[3][2];
5         numbers[0][0] = 1; numbers[0][1] = 2;
6         numbers[1][0] = 3; numbers[1][1] = 4;
7         numbers[2][0] = 5; numbers[2][1] = 6;
8
9         // or all at once
10        int[][] grid = {{1, 2}, {3, 4}, {5, 6}};
11
12        System.out.println("rows    : " + grid.length);
13        System.out.println("columns : " + grid[0].length);
14
15        // a nested loop: outer walks rows, inner walks columns
16        for (int r = 0; r < grid.length; r++) {
17            for (int c = 0; c < grid[r].length; c++) {
18                System.out.print(grid[r][c] + " ");
19            }
20            System.out.println();          // end of a row
21        }
22    }
23 }

```

OUTPUT

```

rows    : 3
columns : 2
1 2
3 4
5 6

```

The nested loop is the shape to learn: **outer loop for rows, inner loop for columns, and `println()` at the end of each row** to break the line. Note `grid[r].length` rather than `grid[0].length` in the inner condition: rows in Java need not all be the same length, so asking each row for its own is the habit that never breaks.

UNIT 8.3 · THE ARRAYS TOOLKIT

java.util.Arrays, and what it gives you

TEACH · OPERATIONS AND METHODS

Your course draws a distinction worth keeping: an **array operation** is a task you want done (sort, search, copy, compare), and an **array method** is a predefined method that does it for you. The methods live in the **Arrays** class, in `java.util`, so they need an import.

METHOD	OPERATION	NOTE
<code>Arrays.sort(a)</code>	sort	ascending, in place : it changes a
<code>Arrays.sort(a, from, to)</code>	sort a range	from inclusive, to exclusive, as always
<code>Arrays.toString(a)</code>	display	gives <code>[10, 20, 30]</code> . Essential: see the trap below.
<code>Arrays.equals(a, b)</code>	compare	compares contents . This is the <code>.equals</code> of Module 7 again.
<code>Arrays.copyOf(a, n)</code>	copy	a new array of length n, padded or truncated
<code>Arrays.fill(a, v)</code>	fill	every element set to v
<code>System.arraycopy(src, sp, dst, dp, n)</code>	copy	note the spelling: all lower case after "array"

⚠ TRAP · PRINTING AN ARRAY DIRECTLY PRINTS GIBBERISH

`System.out.println(arr)` does not print the contents. It prints something like `[I@4617c264`: the type and a hash code, because an array is an object (Unit 3.3) and nobody taught it how to describe itself.

Always `Arrays.toString(arr)`. "How do you get a string representation of an array?" is a stated in-text question, and the answer is that method.

The same reasoning explains `Arrays.equals`: `arr1 == arr2` compares references, exactly as with Strings, so it is false for two different arrays holding identical contents.

CODE · THE TOOLKIT, RUN

VERIFIED ON THE JVM

```
1 import java.util.Arrays;
2
3 public class Toolkit {
4     public static void main(String[] args) {
5         int[] a = {5, 2, 9, 1, 5, 6};
6         Arrays.sort(a); // in place: a itself changes
7         System.out.println("sorted : " + Arrays.toString(a));
8
9         int[] b = {12, 7, 1, 99, 3};
10        Arrays.sort(b, 0, 3); // only indexes 0,1,2
11        System.out.println("first 3 sorted: " + Arrays.toString(b));
12
13        int[] x = {10, 20, 30};
14        int[] y = {10, 20, 30};
15        System.out.println("x == y : " + (x == y));
16        System.out.println("Arrays.equals : " + Arrays.equals(x, y));
17
18        System.out.println("copyOf(x, 5) : " + Arrays.toString(Arrays.copyOf(x, 5)));
19        System.out.println("copyOf(x, 2) : " + Arrays.toString(Arrays.copyOf(x, 2)));
20
21        int[] dst = new int[3];
22        System.arraycopy(x, 0, dst, 0, x.length);
23        System.out.println("arraycopy : " + Arrays.toString(dst));
24    }
25 }
```

OUTPUT

```
sorted : [1, 2, 5, 5, 6, 9]
first 3 sorted: [1, 7, 12, 99, 3]
x == y : false
Arrays.equals : true
copyOf(x, 5) : [10, 20, 30, 0, 0]
copyOf(x, 2) : [10, 20]
arraycopy : [10, 20, 30]
```

Three things worth staring at. The range sort touched only the first three and left 99 and 3 exactly where they were. `x == y` is **false** for two arrays with identical contents, for precisely the reason Module 7 gave. And `copyOf(x, 5)` padded with zeros while `copyOf(x, 2)` silently threw the 30 away: it never complains, it just does what you asked.

```
1 public class LinearSearch {
2
3     // returns the INDEX, or -1 if not there. -1 is the Java convention.
4     static int search(int[] arr, int target) {
5         for (int i = 0; i < arr.length; i++) {
6             if (arr[i] == target) {
7                 return i;                // found: leave at once (Unit 4.7)
8             }
9         }
10        return -1;                        // checked every one: not here
11    }
12
13    public static void main(String[] args) {
14        int[] data = {4, 8, 15, 16, 23, 42};
15        System.out.println("search 15 -> index " + search(data, 15));
16        System.out.println("search 4  -> index " + search(data, 4));
17        System.out.println("search 99 -> index " + search(data, 99));
18    }
19 }
```

OUTPUT

```
search 15 -> index 2
search 4  -> index 0
search 99 -> index -1
```

A linear search checks each element sequentially until it finds the target. Returning -1 for "absent" is a convention worth adopting: it cannot be confused with a real index, because indexes start at 0. It is the same convention `String.indexOf` uses (Module 7), and that consistency is not an accident.

The four patterns worth real marks

Every array question on both papers is one of these four, or a blend. Learn them as shapes and you are writ-

WORKED EXAMPLE · 1. ACCUMULATE: SUM, COUNT, MAX

PAST PAPER · 24/25 Q2 D, Q5 B

One pass, one running total. The whole family looks the same.

- 1 Declare the accumulator **before** the loop.
Inside it, it would be reborn every pass.
- 2 Seed it correctly. Sum and count start at 0. **Max starts at arr[0], never at 0.**
Seed max at 0 and an array of negatives returns 0, which is not in the array. This is a real, silent bug.
- 3 Update it once per element.
- 4 Use it after the loop.

```

1 public class Accumulate {
2
3     static int countNegatives(int[] arr) {      // 24/25 Q5 B, verbatim
4         int count = 0;
5         for (int i = 0; i < arr.length; i++) {
6             if (arr[i] < 0) count++;
7         }
8         return count;
9     }
10
11    static int max(int[] arr) {
12        int best = arr[0];                      // seed from the data
13        for (int i = 1; i < arr.length; i++) { // start at 1: 0 is already in
14            if (arr[i] > best) best = arr[i];
15        }
16        return best;
17    }
18
19    static int sumOdd(int[] arr) {
20        int sum = 0;
21        for (int v : arr) {                      // for-each: read only
22            if (v % 2 != 0) sum += v;
23        }
24        return sum;
25    }
26
27    public static void main(String[] args) {
28        int[] data = {3, -4, 7, -1, 8, 5};
29        System.out.println("negatives : " + countNegatives(data));
30        System.out.println("max       : " + max(data));
31        System.out.println("sum odd   : " + sumOdd(data));
32
33        int[] allNeg = {-9, -3, -17};           // the seed trap
34        System.out.println("max of all-negatives : " + max(allNeg));
35    }
36 }

```

OUTPUT

```

negatives : 2
max       : 8
sum odd   : 14
max of all-negatives : -3

```

The last line is the point. max seeded from arr[0] returns -3, correctly. Seed it at 0 and it returns 0, a number that is not in the array at all, and nothing warns you.

And check sum odd before you accept it: the odd values are 3, 7, -1 and 5, totalling **14**. Forget that -1 is odd and you get 15.

That is not a hypothetical slip. The test `v % 2 == 1` looks like the obvious way to ask "is this odd", and it returns **15** on this data, because `-1 % 2` is -1, not 1: Unit 3.6's rule that the remainder keeps the sign of the left operand,

arriving where nobody was looking for it. So `% 2 == 1` silently skips **every negative odd number** in any array you ever write.

Always test odd with `v % 2 != 0`. It is right for negatives and positives alike, and it is why the listing above uses it.

Now redo it: write max on paper starting `int best = 0`; and trace it on `{-9, -3, -17}` to watch it fail. That is how a trap becomes memorable.

TEACH · THE FOR-EACH LOOP

`for (int v : arr)` reads "for each int v in arr". It hands you every element in turn, with no index and no chance of an off-by-one.

Its limits, which decide when you may use it: **no index** (so you cannot compare with `arr[i-1]`), and **writing to v changes nothing**, because v is a copy, exactly as Unit 5.4 said. Use it to read; use a counted loop to write or to look sideways.

WORKED EXAMPLE · 2. COMPARE WITH THE NEIGHBOUR

PAST PAPER · 25/26 Q3 A

The paper: "a consecutive duplicate is any position i ($i \geq 1$) where `saleAmounts[i] == saleAmounts[i-1]`", over `{20, 20, 35, 40, 40, 40, 15, 60, 60, 25}`, in **one pass**. The hint says it must not flag 35, 15 or 25.

- 1 **Start at 1, not 0.** Index 0 has no predecessor, so `arr[-1]` would throw.
The question tells you this by writing $i \geq 1$. Read the gift.
- 2 Compare `arr[i]` with `arr[i-1]`.
- 3 Count the matches.

```
1 public class Duplicates {
2
3     static int countConsecutive(int[] a) {
4         int count = 0;
5         for (int i = 1; i < a.length; i++) { // i starts at 1
6             if (a[i] == a[i - 1]) count++;
7         }
8         return count;
9     }
10
11     public static void main(String[] args) {
12         int[] sales = {20, 20, 35, 40, 40, 40, 15, 60, 60, 25};
13         System.out.println("consecutive duplicates: " + countConsecutive(sales));
14     }
15 }
```

OUTPUT

consecutive duplicates: 4

DRY RUN · COUNTCONSECUTIVE, EVERY PASS

Cover it and trace first. Note {40, 40, 40} contributes **two**, not one: positions 4 and 5 both match their predecessor.
{20, 20, 35, 40, 40, 40, 15, 60, 60, 25}

I	A[I]	A[I-1]	EQUAL?	COUNT
1	20	20	yes	1
2	35	20	no	1
3	40	35	no	1
4	40	40	yes	2
5	40	40	yes	3
6	15	40	no	3
7	60	15	no	3
8	60	60	yes	4
9	25	60	no	4
stop	i = 10, not < 10			returns 4

WORKED EXAMPLE · 3. LOOK BOTH WAYS, AND THE EDGES

PAST PAPER · 25/26 Q3 A II

The harder half: flag every amount that equals **its predecessor or its successor**. The paper is explicit about the edges: "Index 0 checks only its successor; the last index checks only its predecessor."

That sentence is the whole difficulty. Ask a[i-1] at index 0 or a[i+1] at the last index and you throw. The tool is Unit 3.8's **short-circuit**: guard the index first, and && never evaluates the lookup.

```

1 public class Flagged {
2
3     static String flagged(int[] a) {
4         String out = ""; // plain concatenation only
5         for (int i = 0; i < a.length; i++) {
6             boolean prev = (i > 0) && a[i] == a[i - 1];
7             boolean next = (i < a.length - 1) && a[i] == a[i + 1];
8             if (prev || next) {
9                 if (!out.isEmpty()) out = out + ", ";
10                out = out + a[i];
11            }
12        }
13        return out;
14    }
15
16    public static void main(String[] args) {
17        int[] sales = {20, 20, 35, 40, 40, 40, 15, 60, 60, 25};
18        System.out.println("flagged: " + flagged(sales));
19    }
20 }
```

OUTPUT

```
flagged: 20, 20, 40, 40, 40, 60, 60
```

Check it against the paper's own hint: **35, 15 and 25 are absent**, exactly as required. And look at the two guards. At i = 0, (i > 0) is false, so && stops and a[-1] is **never evaluated**. Swap the two sides of that && and the program throws on its first line. That is short-circuiting doing real work, exactly as Unit 3.8 promised.

Now redo it: write the two guard lines from memory, then say what happens if you write a[i] == a[i-1] && i > 0 instead.

The paper: fill "two **parallel arrays** of length V", the hours parked and the permit type, and it bans every collection class. Parallel arrays are the answer to "I need two facts about each thing, and I may not use an object or a list".

The idea: two arrays, same length, where **index i means the same thing in both**. hours[3] and permits[3] describe the same vehicle.

```

1 public class Parallel {
2
3     static double fee(int hours, String permit) {
4         if (permit.equals("Staff")) {           // nested, per the paper
5             if (hours <= 2) return 0.0;         // first 2 hours free
6             double f = (hours - 2) * 1.00;
7             return f > 10.00 ? 10.00 : f;       // capped
8         } else {                               // anything not Staff = Visitor
9             double f = (hours <= 3) ? 6.00 : 6.00 + (hours - 3) * 2.50;
10            return f > 25.00 ? 25.00 : f;
11        }
12    }
13
14    public static void main(String[] args) {
15        int[]    hours    = {1, 5, 12, 2, 20};    // parallel:
16        String[] permits = {"Staff", "Staff", "Staff", "Visitor", "guest"};
17
18        double total = 0;
19        for (int i = 0; i < hours.length; i++) { // ONE index, BOTH arrays
20            double f = fee(hours[i], permits[i]);
21            total += f;
22            System.out.printf("%-8s %2dh -> %5.2f\n", permits[i], hours[i], f);
23        }
24        System.out.printf("total %.2f\n", total);
25    }
26 }

```

OUTPUT

```

Staff    1h ->  0.00
Staff    5h ->  3.00
Staff   12h -> 10.00
Visitor  2h ->  6.00
guest   20h -> 25.00
total 44.00

```

Read the rows against the paper's rules. Staff for 1 hour is free. Staff for 5 pays for 3 of them. Staff for 12 would pay 10.00 and is capped at exactly 10.00. And "guest" is not exactly "Staff", so it was treated as a Visitor and capped at 25.00, which is the rule the paper states and the row a marker checks.

The discipline parallel arrays demand: the two arrays must stay the same length and in the same order forever. Sort one without the other and every vehicle silently gets someone else's permit. That fragility is exactly why objects (Module 6) exist, and why the exam banning them is an artificial constraint you must simply obey.

FILL IN THE GAP · ARRAYS

1. An array stores multiple values of the _____ in a single variable, in _____ memory locations.
2. `new int[3]` contains _____; `new String[2]` contains _____.
3. For `int[][] m = new int[3][2]`, `m.length` is _____ and `m[0].length` is _____.
4. `System.out.println(arr)` prints _____, so you must use _____ instead.
5. Two arrays with identical contents: `a == b` is _____ but `Arrays.equals(a, b)` is _____.
6. A linear search returns _____ when the target is absent, because _____.
7. When finding a maximum, seed the accumulator with _____, never _____, because _____.
8. To check `a[i-1]` safely at every index, guard with _____ `&& a[i] == a[i-1]`, which is safe because of _____.

ANSWERS

1. same data type, contiguous · 2. `[0, 0, 0]`, `[null, null]` · 3. 3 (rows), 2 (columns) · 4. a type and hash code like `[I@4617c264, Arrays.toString(arr)]` · 5. false, true · 6. -1, no real index is negative · 7. `arr[0]`, 0, an all-negative array would wrongly return 0 · 8. `i > 0`, short-circuit evaluation

PART X

Module Nine Recursion

A method that calls itself. It sounds like a paradox and it is simply a loop wearing different clothes, and once you see the two halves that make it work, and the one that stops it running forever, it becomes the most natural way to say some things.



A method that calls itself

By the end of this unit you should be able to explain recursion, write recursive methods, identify base cases and recursive cases, and recognise problems where recursion is advantageous.

TEACH · RECURSION

Recursion is a programming technique in which a method calls itself in order to solve a problem.

The problem is broken into **smaller sub-problems of the same type**, solved independently until they reach a **base case**, which stops the recursion.

The phrase that does the work is **of the same type**. Recursion is only available when a smaller version of your problem is *the same problem*. Finding 5! means finding 4! and multiplying by 5: the sub-problem is another factorial, so factorial can call itself.

MUST-MEMORISE · THE TWO CASES

The base case is a condition that **stops** the recursion. Without one, the method calls itself indefinitely and the program dies with a **StackOverflowError**.

The recursive case is where the method calls itself, on a **smaller** input, moving towards the base case.

Every recursive method has both, and every recursive method that works has a recursive case that genuinely **gets closer** to the base case. Those are the only two things to check, and they are what an exam wants named.

MEMORY HOOK: a staircase. The recursive case is "take one step down, then solve the rest". The base case is the ground: the step where you stop. No ground and you fall forever; steps that do not go downward and you never reach it.

CODE · FACTORIAL, THE CLASSIC

FROM THE SLIDES

```

1 public class Factorial {
2
3     static int factorial(int n) {
4         if (n == 0 || n == 1) {           // BASE CASE: stop here
5             return 1;
6         }
7         return n * factorial(n - 1);     // RECURSIVE CASE: smaller n
8     }
9
10    public static void main(String[] args) {
11        System.out.println("factorial(6) = " + factorial(6));
12        System.out.println("factorial(3) = " + factorial(3));
13        System.out.println("factorial(0) = " + factorial(0));
14    }
15 }
```

OUTPUT

```

factorial(6) = 720
factorial(3) = 6
factorial(0) = 1
```

Note $n - 1$: that is what makes it **smaller**. Write `factorial(n)` there and you have an infinite recursion, because the input never moves towards the base case.

DRY RUN · FACTORIAL(3), DOWN AND BACK UP

A recursion trace has two halves, and beginners only ever draw the first. **Going down**, each call waits for the one below it. **Coming back up**, each answer is multiplied on the way out. Nothing is multiplied until the bottom is reached.

FACTORIAL(3)

DEPTH	CALL	WHAT IT SAYS	WAITING FOR
1	factorial(3)	3 is not 0 or 1, so return 3 * factorial(2)	factorial(2)
2	factorial(2)	2 is not 0 or 1, so return 2 * factorial(1)	factorial(1)
3	factorial(1)	base case: return 1	nothing. Turn around.

NOW THE ANSWERS COME BACK UP

DEPTH	WAS WAITING ON	NOW BECOMES	RETURNS
3	-	1	1
2	2 * factorial(1)	2 * 1	2
1	3 * factorial(2)	3 * 2	6

Each recursive call adds a new frame to the call stack until the base case is reached. Then the values are returned back up. The stack is real memory, and that is why an infinite recursion produces a `StackOverflowError` rather than simply hanging: it runs out of room to remember who is waiting for whom.

CODE · THE STACK, MADE VISIBLE

WATCH THE INDENTATION

```

1 public class StackTrace {
2
3     static int fact(int n, String pad) {
4         System.out.println(pad + "-> fact(" + n + ") called");
5         if (n <= 1) {
6             System.out.println(pad + "    base case, returning 1");
7             return 1;
8         }
9         int smaller = fact(n - 1, pad + "    ");
10        int result = n * smaller;
11        System.out.println(pad + "<- fact(" + n + ") returns " + n + " * "
12                           + smaller + " = " + result);
13        return result;
14    }
15
16    public static void main(String[] args) {
17        fact(4, "");
18    }
19 }
```

OUTPUT

```

-> fact(4) called
  -> fact(3) called
    -> fact(2) called
      -> fact(1) called
        base case, returning 1
      <- fact(2) returns 2 * 1 = 2
    <- fact(3) returns 3 * 2 = 6
  <- fact(4) returns 4 * 6 = 24
```

The shape of the output is the shape of the call stack. Every arrow going right is a call going deeper, and nothing returns until the innermost one hits the base case. Then the answers come back out in reverse order. That V shape is what recursion looks like, and if you can draw it you can trace any recursive question.

⚠ TRAP · NO BASE CASE, OR ONE YOU NEVER REACH

Two ways to recurse forever, and both end in `StackOverflowError`:

```
static int bad1(int n) {  
    return n * bad1(n - 1);    <- no base case at all  
}  
  
static int bad2(int n) {  
    if (n == 0) return 1;      <- a base case...  
    return n * bad2(n);        <- ...but n never changes, so never reached  
}
```

`bad2` is the nastier one, because it *looks* right: it has a base case, so a careless reader ticks the box. The check is not "is there a base case" but "**does every recursive call get closer to it**".

And note `bad1(-1)`: even the correct factorial recurses forever on a negative, because `n == 0 || n == 1` is never true going down from -1. A base case must catch **every** way of arriving at the bottom.

UNIT 9.2 · THE STANDARD RECURSIONS

The four your course gives, and the pattern under them

● MUST-MEMORISE · THE RECIPE FOR WRITING ANY RECURSIVE METHOD

1. **Ask when it is trivially over.** That is your base case.
2. **Assume the method already works for a smaller input.** Do not think about how.
3. **Say how to build your answer from that smaller answer.**
4. **Check the smaller input actually moves towards the base case.**

Step 2 is the leap everyone struggles with, and it is the whole trick. Writing `factorial`, you do **not** trace what `factorial(n-1)` does. You **trust** it, and only ask "given `(n-1)!`, how do I get `n!`?". The answer is "multiply by `n`", and you are done.

MEMORY HOOK: trust the smaller you. Recursion feels impossible while you try to hold every level in your head at once. It becomes easy the moment you delegate one level down and refuse to look.

```

1 public class Recursions {
2
3     // 1. FIBONACCI:  $f(n) = f(n-1) + f(n-2)$ . TWO base cases.
4     static int fib(int n) {
5         if (n == 0) return 0;
6         if (n == 1) return 1;
7         return fib(n - 1) + fib(n - 2);
8     }
9
10    // 2. EXPONENTIATION:  $power(b, e) = b * power(b, e-1)$ 
11    static int power(int base, int exp) {
12        if (exp == 0) return 1; // anything^0 is 1
13        return base * power(base, exp - 1);
14    }
15
16    // 3. SUM OF NATURALS:  $sum(n) = n + sum(n-1)$ 
17    static int sum(int n) {
18        if (n == 1) return 1;
19        return n + sum(n - 1);
20    }
21
22    // 4. SUM OF DIGITS:  $sumDigits(n) = (n \% 10) + sumDigits(n / 10)$ 
23    static int sumDigits(int n) {
24        if (n == 0) return 0;
25        return (n \% 10) + sumDigits(n / 10);
26    }
27
28    public static void main(String[] args) {
29        System.out.print("fibonacci: ");
30        for (int i = 0; i < 8; i++) System.out.print(fib(i) + " ");
31        System.out.println();
32
33        System.out.println("3 raised to power 10: " + power(3, 10));
34        System.out.println("Sum of first 5 numbers: " + sum(5));
35        System.out.println("Sum of digits of 12345 is: " + sumDigits(12345));
36    }
37 }

```

OUTPUT

```

fibonacci: 0 1 1 2 3 5 8 13
3 raised to power 10: 59049
Sum of first 5 numbers: 15
Sum of digits of 12345 is: 15

```

Look at sumDigits against Unit 3.6. It is the **same** `% 10` and `/ 10` pair that walked a number's digits in a loop, said recursively instead: peel the last digit, add it to the answer for the rest. The loop and the recursion are the same idea, which is exactly the point of Unit 9.3.

Note also that fib needs **two** base cases, because its recursive case reaches **two** steps back. A recursion needs enough base cases to catch every way of hitting the bottom.

UNIT 9.3 · RECURSION VERSUS ITERATION

When it helps, and when it hurts

TEACH · THEY ARE EQUIVALENT

Anything you can write with a loop you can write recursively, and the other way round. Your learning outcome asks you to "recognise problems where recursion is advantageous", which means the choice is about **clarity and cost**, not about possibility.

CODE · THE SAME JOB, BOTH WAYS

```

1 public class BothWays {
2
3     static int factRecursive(int n) {
4         if (n <= 1) return 1;
5         return n * factRecursive(n - 1);
6     }
7
8     static int factIterative(int n) {
9         int result = 1;
10        for (int i = 2; i <= n; i++) result *= i;
11        return result;
12    }
13
14    public static void main(String[] args) {
15        boolean same = true;
16        for (int n = 0; n <= 12; n++) {
17            if (factRecursive(n) != factIterative(n)) same = false;
18        }
19        System.out.println("recursive(6) = " + factRecursive(6));
20        System.out.println("iterative(6) = " + factIterative(6));
21        System.out.println("identical for n = 0 to 12: " + same);
22    }
23 }

```

OUTPUT

```

recursive(6) = 720
iterative(6) = 720
identical for n = 0 to 12: true

```

● MUST-MEMORISE · WHICH TO CHOOSE

	RECURSION	ITERATION (A LOOP)
Memory	a stack frame per call	constant
Risk	StackOverflowError	an infinite loop, but no crash
Speed	slower (call overhead)	faster
Reads well for	trees, graphs, divide and conquer, anything defined in terms of itself	counting, walking an array

Use recursion when the problem is naturally recursive: your course names traversing data structures such as trees and graphs. Use a loop for anything you would describe with the word "repeat".

MEMORY HOOK: recursion trades memory for clarity. If the clarity is not there, you paid for nothing.

⚠ TRAP · NAIVE FIBONACCI IS CATASTROPHICALLY SLOW

The fib in Unit 9.2 is the standard example and it is a terrible program. fib(n) calls itself **twice** at every level, so the number of calls roughly doubles with each step: fib(40) makes over 300 million calls to compute forty numbers.

It recomputes the same values endlessly: fib(5) computes fib(3) twice, fib(2) three times, and so on down. A loop does it in forty steps.

So Fibonacci is the classic recursion *teaching* example and a classic case of when **not** to use recursion in real code. Both facts are worth knowing, and an exam asking "when is recursion advantageous" is asking you to know the difference.

```

1 public class FibCost {
2     static int calls = 0;
3
4     static int fib(int n) {
5         calls++; // count every single call
6         if (n == 0) return 0;
7         if (n == 1) return 1;
8         return fib(n - 1) + fib(n - 2);
9     }
10
11    static int fibLoop(int n) {
12        int a = 0, b = 1;
13        for (int i = 0; i < n; i++) { int t = a + b; a = b; b = t; }
14        return a;
15    }
16
17    public static void main(String[] args) {
18        for (int n : new int[]{5, 10, 20, 30}) {
19            calls = 0;
20            int v = fib(n);
21            System.out.printf("fib(%2d) = %,-8d recursive calls: %,9d    loop steps: %d\n",
22                             n, v, calls, n);
23        }
24        System.out.println("loop agrees: " + (fib(20) == fibLoop(20)));
25    }
26 }

```

OUTPUT

```

fib( 5) = 5          recursive calls:      15    loop steps: 5
fib(10) = 55         recursive calls:     177    loop steps: 10
fib(20) = 6,765      recursive calls:    21,891   loop steps: 20
fib(30) = 832,040    recursive calls: 2,692,537   loop steps: 30
loop agrees: true

```

Read the last row. To find the 30th Fibonacci number, the recursion made **2.7 million calls**. The loop took **30 steps** and got the same answer. The two columns grow at completely different rates, and that gap is the entire argument for knowing when *not* to recurse.

FILL IN THE GAP · RECURSION

1. Recursion is a technique where a method _____ to solve _____ of the same type.
2. The _____ case stops the recursion; the _____ case calls itself on a _____ input.
3. Without a base case, the program dies with a _____, because each call adds a _____.
4. Having a base case is not enough: every recursive call must _____.
5. fib needs _____ base cases, because its recursive case reaches _____ steps back.
6. sumDigits(n) = _____ + sumDigits(_____), and its base case is _____.
7. Recursion costs _____ and buys _____. Prefer it for _____.

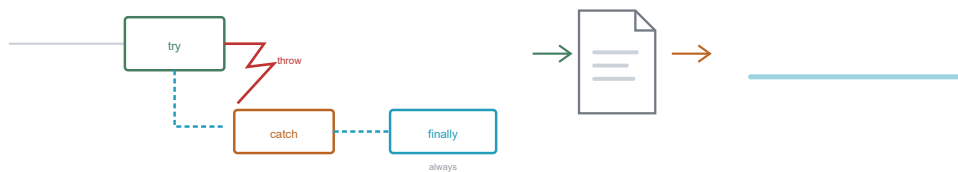
ANSWERS

1. calls itself, smaller sub-problems · 2. base, recursive, smaller · 3. StackOverflowError, frame to the call stack · 4. get closer to the base case · 5. two, two · 6. n % 10, n / 10, n == 0 · 7. memory (a stack frame per call), clarity, naturally recursive structures like trees and graphs

PART XI

Module Ten Exception Handling and File I/O

Everything so far assumed the world behaves. This module is about what happens when it does not: the file is missing, the user types letters into a number, the disk is full. Java makes you say what you will do about it, and Section B pays thirteen marks for saying it well.



What happens when the world misbehaves

By the end of this unit you should be able to explain exception handling and its importance, identify types of exceptions and errors, implement try, catch and finally, use throw and throws, write custom exceptions, and handle file input and output.

● MUST-MEMORISE • AN EXCEPTION

An exception is an event that disrupts the normal flow of a program. It is an **object** representing an error condition.

Handle them because: they **prevent program crashes**, they let you give a **user-friendly error message**, and they let the program **recover and carry on**.

"An object representing an error condition" is worth pausing on. When something goes wrong, Java *constructs an object* describing it and hands it to you. That is why you can call `e.getMessage()` on it: it is a thing, with fields, in the sense of Module 6.

MEMORY HOOK: an exception is a **note thrown out of a burning building**. The method that hit trouble cannot carry on, so it writes down what went wrong and throws it to whoever will catch it. If nobody catches it, the building comes down: F.7's run-time error.

● MUST-MEMORISE • CHECKED VERSUS UNCHECKED

25/26 Q1 B • 3 MARKS

	CHECKED	UNCHECKED
1. When	checked at compile time	occur at run time
2. Obligation	must be caught or declared with throws, or it will not compile	no obligation at all
3. Cause	things outside your control: the file is gone, the network died	programming mistakes: a null, a bad index, /0
4. Examples	IOException, FileNotFoundException, SQLException, ClassNotFoundException	NullPointerException, ArrayIndexOutOfBoundsException, ArithmeticException, NumberFormatException

"State three differences between checked exceptions and unchecked exceptions" is **2025/26 Section A, Question One, part B, for three marks**. Give rows 1, 2 and 3, with an example each.

MEMORY HOOK: **checked = the compiler checks you have a plan**. It is about things that can go wrong even when your code is perfect, so Java forces you to think about them. Unchecked ones are your own bugs, and the compiler does not make you plan for your own mistakes.

TEACH · EXCEPTION VERSUS ERROR

	EXCEPTION	ERROR
Is	an event disrupting normal flow, which can be caught and handled	a serious problem beyond your control
Subclass of	<code>java.lang.Exception</code>	<code>java.lang.Error</code>
Checked?	may be checked or unchecked	always unchecked
Catch it?	yes, with try-catch	generally not: it signals something serious
Recover?	possible to recover and continue	generally not
Examples	<code>IOException</code> , <code>ArithmeticException</code>	<code>StackOverflowError</code> , <code>OutOfMemoryError</code>

You have already met an Error: Module 9's `StackOverflowError`. Note it is an **Error**, not an Exception, and that its name ends in Error is the clue. You *could* catch it, and you should not: the right response to running out of stack is to fix the recursion.

UNIT 10.2 · TRY, CATCH, FINALLY

Saying what you will do about it

SYNTAX · THE THREE BLOCKS

```
try {
    // code that MIGHT throw
} catch (IOException e) {
    // runs ONLY if that exception was thrown
} finally {
    // runs ALWAYS: exception or not, caught or not
}
```

● MUST-MEMORISE · THE FIVE KEYWORDS

try: defines a block that might throw.

catch: catches and handles a specific exception from that block.

finally: always executes afterwards, for cleanup such as closing files.

throw: **actually throws** an exception, right now. One word, inside a method.

throws: **declares** that a method might throw, in its signature. It throws nothing itself; it warns the caller.

"What are the main keywords used in Java for handling exceptions, and how do they differ?" is a stated in-text question, and the answer is that card.

MEMORY HOOK: **throw is an act; throws is a warning.** The **s** is the difference between doing it and declaring you might. "I throw a ball" versus "this job throws balls at you".

```
1 public class TryCatch {
2
3     static void attempt(int a, int b) {
4         try {
5             System.out.println(" result: " + (a / b));
6         } catch (ArithmeticException e) {
7             System.out.println(" Error: Division by zero!");
8         } finally {
9             System.out.println(" This block always runs.");
10        }
11    }
12
13    public static void main(String[] args) {
14        System.out.println("10 / 2:");
15        attempt(10, 2);           // no exception
16        System.out.println("10 / 0:");
17        attempt(10, 0);          // exception, caught
18        System.out.println("done. The program did NOT crash.");
19    }
20 }
```

OUTPUT

```
10 / 2:
  result: 5
  This block always runs.
10 / 0:
  Error: Division by zero!
  This block always runs.
done. The program did NOT crash.
```

Compare this with Unit 3.6, where the identical 10/0 killed the program dead and lines 4 to 6 never ran. That is the entire value of try-catch: the same fault, and the program carries on. And note finally printed **both** times, on the happy path and the failing one. That is what "always" means.

```

1 public class MultiCatch {
2
3     static void attempt(int which) {
4         try {
5             int[] arr = {1, 2, 3};
6             if (which == 1) System.out.println(arr[5]);           // bad index
7             if (which == 2) System.out.println(10 / 0);          // bad maths
8             if (which == 3) System.out.println(Integer.parseInt("abc")); // bad text
9         } catch (ArrayIndexOutOfBoundsException e) {
10            System.out.println(" Array index out of bounds!");
11        } catch (ArithmeticException e) {
12            System.out.println(" Arithmetic problem: " + e.getMessage());
13        } catch (Exception e) {
14            System.out.println(" Something else: " + e.getClass().getSimpleName());
15        }
16    }
17
18    public static void main(String[] args) {
19        attempt(1); attempt(2); attempt(3);
20    }
21 }

```

OUTPUT

```

Array index out of bounds!
Arithmetic problem: / by zero
Something else: NumberFormatException

```

Only the matching catch block runs. The third call was not an index or arithmetic problem, so it fell through to catch (Exception e), which catches everything because every exception is a subclass of Exception: Module 6's inheritance doing real work.

That third line printed the exception's name because of `e.getClass().getSimpleName()`. Any object can be asked `getClass()` for the class it belongs to, and a class can be asked `getSimpleName()` for its own name with the package left off: **NumberFormatException** rather than `java.lang.NumberFormatException`. It is how you report *what* went wrong when your catch is broad enough that you do not already know. `getMessage()` answers a different question, the detail (`/ by zero`), which is why a catch this wide is usually worth printing both.

⚠ TRAP · ORDER YOUR CATCH BLOCKS NARROWEST FIRST

Put catch (Exception e) **first** and the code will not compile: every later catch is unreachable, because the general one already caught everything. Java tells you so.

The rule follows from inheritance: **catch the specific before the general**, always. A catch for a parent class shadows every child below it.

```
1 public class ThrowThrows {
2
3     // 'throw' ACTS: it creates and hurls an exception
4     static void registerUser(int age) {
5         if (age < 18) {
6             throw new IllegalArgumentException("Age must be 18");
7         }
8         System.out.println("    registered, age " + age);
9     }
10
11     // 'throws' DECLARES: this method might throw, caller beware
12     static int divide(int a, int b) throws ArithmeticException {
13         if (b == 0) {
14             throw new ArithmeticException("Cannot divide by zero.");
15         }
16         return a / b;
17     }
18
19     public static void main(String[] args) {
20         registerUser(25);
21
22         try {
23             registerUser(15);
24         } catch (IllegalArgumentException e) {
25             System.out.println("    caught: " + e.getMessage());
26         }
27
28         try {
29             divide(10, 0);
30         } catch (ArithmeticException e) {
31             System.out.println("    Error: " + e.getMessage());
32         }
33     }
34 }
```

OUTPUT

```
registered, age 25
caught: Age must be 18
Error: Cannot divide by zero.
```

Your course's slide shows `registerUser(15)` **uncaught**, and prints the crash: `Exception in thread "main" java.lang.IllegalArgumentException: Age must be 18`. That is what happens with no try-catch, and it is worth seeing. Here it is caught, so the program survives and prints a civil message. Same throw, two fates, and the difference is entirely whether anyone was listening.

```

1 // checked: extends Exception. Callers MUST handle it.
2 class InvalidAgeException extends Exception {
3     InvalidAgeException(String message) {
4         super(message); // Module 6: hand it up to Exception
5     }
6 }
7
8 // unchecked: extends RuntimeException. Callers need not.
9 class InvalidScoreException extends RuntimeException {
10     InvalidScoreException(String message) { super(message); }
11 }
12
13 public class Custom {
14
15     static void checkAge(int age) throws InvalidAgeException {
16         if (age < 18) throw new InvalidAgeException("Age must be 18");
17         System.out.println(" age " + age + " accepted");
18     }
19
20     static void checkScore(int score) { // no 'throws' needed
21         if (score < 0 || score > 100) {
22             throw new InvalidScoreException("Score out of range: " + score);
23         }
24         System.out.println(" score " + score + " accepted");
25     }
26
27     public static void main(String[] args) {
28         try {
29             checkAge(25);
30             checkAge(15);
31         } catch (InvalidAgeException e) {
32             System.out.println(" caught checked: " + e.getMessage());
33         }
34
35         checkScore(75);
36         try {
37             checkScore(150);
38         } catch (InvalidScoreException e) {
39             System.out.println(" caught unchecked: " + e.getMessage());
40         }
41     }
42 }

```

OUTPUT

```

age 25 accepted
caught checked: Age must be 18
score 75 accepted
caught unchecked: Score out of range: 150

```

A custom exception is a user-defined exception class that extends either **Exception** (for checked) or **RuntimeException** (for unchecked). Which parent you pick is the decision: it decides whether every caller is forced to handle it. Note `checkAge` needs `throws` in its signature and `checkScore` does not: that is Unit 10.1's table, enforced by the compiler.

Reading and writing files

TEACH · THE EIGHT CLASSES, AND WHICH THE EXAM ALLOWS

File I/O refers to reading data from and writing data to files. Your course names four ways each direction:

WRITE	READ	NOTE
FileWriter	FileReader	the basic pair: one character at a time
BufferedWriter	BufferedReader	wraps the basic one for efficiency, and adds readLine()
PrintWriter	Scanner	print/println/printf onto a file
Files.write()	Files.readAllLines()	modern NIO: one line does everything

Read your exam's constraint. 2025/26 Question Three says "using **only** java.io.FileWriter, java.io.FileReader, java.io.BufferedReader, java.io.BufferedWriter, and java.io.IOException". That bans PrintWriter, Scanner and all of Files. The 2024/25 paper does the opposite and asks about PrintWriter and Files.write() by name. Know all four pairs.

● MUST-MEMORISE · THE FILE I/O SHAPE

Every file operation is the same four steps:

1. **open** it (constructing the writer/reader opens the file)
2. **use** it
3. **close** it, in a finally or with try-with-resources
4. **handle** IOException, because you have no choice: it is checked.

MEMORY HOOK: a file is a **library book**. Take it out, read it, **and give it back**, even if you were interrupted. That last clause is why finally exists.

● MUST-MEMORISE · TRY-WITH-RESOURCES

A try-with-resources block declares the resource in brackets after try, and closes it automatically, whatever happens:

```
try (FileWriter w = new FileWriter("f.txt")) { ... }
```

No finally, no close(), no leak. It is the modern way and it is what the 2025/26 paper means by "either in a finally block or via try-with-resources".

"Define try-with-resources block" is asked outright on the 2024/25 paper.

MEMORY HOOK: the resource in the brackets is **on loan for the duration of the block**. When the block ends, by success or by explosion, Java gives it back for you.

```
1 import java.io.*;
2 import java.nio.file.*;
3 import java.util.*;
4
5 public class Writing {
6     public static void main(String[] args) throws IOException {
7
8         // 1. FileWriter: the basic one
9         try (FileWriter w = new FileWriter("test.txt")) {
10             w.write("first line\n");
11             w.write("second line\n");
12         }
13         System.out.println("test.txt -> " + Files.readAllLines(Paths.get("test.txt")));
14
15         // 2. PrintWriter: print / println / printf onto a file
16         try (PrintWriter pw = new PrintWriter("output.txt")) {
17             pw.print("This is on the ");
18             pw.println("same line");
19             pw.printf("The number again is %d\n", 221);
20         }
21         System.out.println("output.txt -> " + Files.readAllLines(Paths.get("output.txt")));
22
23         // 3. BufferedWriter: wraps a FileWriter, buffers for efficiency
24         try (BufferedWriter bw = new BufferedWriter(new FileWriter("buffer.txt"))) {
25             bw.write("buffered text");
26             bw.newLine();
27         }
28         System.out.println("buffer.txt -> " + Files.readAllLines(Paths.get("buffer.txt")));
29
30         // 4. NIO Files.write: creates the file if it does not exist
31         Files.write(Paths.get("nio.txt"), Arrays.asList("Hello World"));
32         System.out.println("nio.txt -> " + Files.readAllLines(Paths.get("nio.txt")));
33     }
34 }
```

OUTPUT

```
test.txt -> [first line, second line]
output.txt -> [This is on the same line, The number again is 221]
buffer.txt -> [buffered text]
nio.txt -> [Hello World]
```

Note output.txt: two calls, print then println, produced **one** line, exactly as F.6 said. This is your 2024/25 Question Three, part C, and the question is asking whether you know that print does not break the line.

"What is the output of the data.txt file?"

```
1 import java.io.*;
2 import java.nio.file.*;
3
4 public class DataTxt {
5     public static void main(String[] args) throws IOException {
6         PrintWriter pw = new PrintWriter("data.txt");
7         pw.print("Age: ");
8         pw.println(25);
9         pw.printf("Score: %.2f", 89.456);
10        pw.close();
11
12        // now show exactly what landed in the file
13        System.out.println("--- data.txt ---");
14        System.out.print(new String(Files.readAllBytes(Paths.get("data.txt"))));
15        System.out.println();
16        System.out.println("--- end ---");
17    }
18 }
```

OUTPUT

```
--- data.txt ---
Age: 25
Score: 89.46
--- end ---
```

Three things earn the marks, and each is a separate fact:

- 1 print("Age: ") does **not** break the line, so 25 lands beside it.
- 2 println(25) **does** break the line, so Score starts on a new one.
- 3 printf("%.2f", 89.456) gives **89.46**, not 89.456 and not 89.45: %.2f **rounds** to two places.
Unit 3.11's table again. Bare %f would have given 89.456000.

Now redo it: cover the output and write the exact contents of data.txt on paper, including where the line breaks fall. Then say what changes if pw.close() is deleted. (The answer is worth knowing: possibly nothing at all reaches the file, because the buffer is never flushed.)

```

1  import java.io.*;
2  import java.nio.file.*;
3  import java.util.*;
4
5  public class Reading {
6      public static void main(String[] args) throws IOException {
7          // make a file to read
8          Files.write(Paths.get("data.txt"), Arrays.asList("line one", "line two"));
9
10         // 1. FileReader: one CHARACTER at a time, until read() returns -1
11         System.out.print("FileReader      : ");
12         try (FileReader fr = new FileReader("data.txt")) {
13             int ch;
14             while ((ch = fr.read()) != -1) { // -1 means end of file
15                 System.out.print((char) ch); // cast int -> char
16             }
17         }
18
19         // 2. BufferedReader: one LINE at a time, until readLine() returns null
20         System.out.print("BufferedReader : ");
21         try (BufferedReader br = new BufferedReader(new FileReader("data.txt"))) {
22             String line;
23             while ((line = br.readLine()) != null) { // null means end
24                 System.out.print "[" + line + "];"
25             }
26         }
27         System.out.println();
28
29         // 3. Scanner
30         System.out.print("Scanner      : ");
31         try (Scanner sc = new Scanner(new File("data.txt"))) {
32             while (sc.hasNextLine()) System.out.print "[" + sc.nextLine() + "];"
33         }
34         System.out.println();
35
36         // 4. Files.readAllLines: the whole file, one line of code
37         System.out.println("readAllLines : " + Files.readAllLines(Paths.get("data.txt")));
38     }
39 }

```

OUTPUT

```

FileReader      : line one
line two
BufferedReader : [line one][line two]
Scanner      : [line one][line two]
readAllLines   : [line one, line two]

```

Two sentinel values to memorise, and they are different: `read()` returns `-1` at the end of the file, and `readLine()` returns `null`. Both loops use the same trick, `while ((x = read()) != sentinel)`, which assigns and tests in one breath. That idiom appears in every file-reading program you will ever write.

`Scanner` is the exception, and that is why it reads differently. It offers `hasNextLine()`, which answers `true` when there is another line waiting and `false` at the end of the file, so you **ask before you take** rather than taking and comparing against a sentinel. Its partner `nextLine()` then hands you that line. The pair is easier to read, and that is the whole of its advantage here: `while (sc.hasNextLine())` says what it means, where `while ((line = in.readLine()) != null)` has to be learned once and recognised forever.

The four names in those two listings that are not readers or writers at all:

CALL	WHAT IT DOES
<code>Paths.get("data.txt")</code>	Turns a file name into a <code>Path</code> , the object the NIO methods take instead of a <code>String</code> .
<code>Files.write(path, lines)</code>	Writes every line to the file and closes it, in one call. Creates it if absent; replaces it if present.
<code>Files.readAllBytes(path)</code>	Reads the whole file into a byte array in one call. <code>new String(...)</code> around it turns those bytes into text.
<code>Arrays.asList("a", "b")</code>	Makes a fixed list out of loose values, because <code>Files.write</code> wants a collection of lines, not one <code>String</code> .

Learn these because the paper asks for them by name. The 24/25 paper, Question Three part A, is: "Define the following: `PrintWriter`, **`Files.write()` method** and try-with-resources block." Three marks for three sentences, and one of them is on this table.

What they are for. They are the whole file operation in one line, with no loop and no `close()`, because the method opens, works and closes before it returns. That is their appeal and also their limit: they hold the entire file in memory at once, so they suit a settings file and not a five-gigabyte log. The reader-and-writer way further up the page is the one that streams.

MEMORY HOOK: `Paths.get` names it, `Files` does it. Everything in `java.nio.file` takes a `Path`, and `Paths.get` is how a `String` becomes one.

They are **not** allowed in the 25/26 file question, which names its five permitted classes and none of these are among them. Knowing an API and being allowed to use it are different things: read the restriction before you reach for the shortcut.

⚠ TRAP · `FILEREADER.READ()` RETURNS AN INT, NOT A CHAR

It has to. A `char` has no spare value to mean "the file is over", because every possible `char` is a legitimate character. So `read()` widens to `int` and uses **-1**, a value no character has, as the end-of-file marker. That is why the loop declares `int ch` and casts `(char) ch` to print it.

Declare `char ch` and the comparison `!= -1` is never true, and you loop forever on a file that has ended.

The paper wants: write three students to `input.txt` with `FileWriter` in a try-catch; read `output.txt` with `FileReader` and handle it not existing; a method that counts the lines in a named file and **handles its own exceptions internally**; every resource closed; every catch printing a clear, specific message. Only the five named `java.io` classes.

```

1  import java.io.BufferedReader;
2  import java.io.BufferedWriter;
3  import java.io.FileReader;
4  import java.io.FileWriter;
5  import java.io.IOException;
6
7  public class StudentFiles {
8
9      // (i) write at least three students, one per line
10     static void writeStudents() {
11         try (BufferedWriter w = new BufferedWriter(new FileWriter("input.txt"))) {
12             w.write("CS001, Ada Obi, 4.51"); w.newLine();
13             w.write("CS002, Ken Bello, 3.88"); w.newLine();
14             w.write("CS003, Lilian Uche, 4.05"); w.newLine();
15             System.out.println("wrote 3 students to input.txt");
16         } catch (IOException e) {
17             System.out.println("Could not write input.txt: " + e.getMessage());
18         }
19     }
20
21     // (ii) read output.txt, handling the case where it does not exist
22     static void readStudents(String fileName) {
23         try (BufferedReader r = new BufferedReader(new FileReader(fileName))) {
24             String line;
25             while ((line = r.readLine()) != null) {
26                 System.out.println(" " + line);
27             }
28         } catch (IOException e) {
29             System.out.println("Could not read " + fileName + ": " + e.getMessage());
30         }
31     }
32
33     // (iii) count lines, handling its OWN exceptions internally
34     static int countLines(String fileName) {
35         int count = 0;
36         try (BufferedReader r = new BufferedReader(new FileReader(fileName))) {
37             while (r.readLine() != null) count++;
38         } catch (IOException e) {
39             System.out.println("Could not count " + fileName + ": " + e.getMessage());
40             return 0; // handled internally: no throws clause
41         }
42         return count;
43     }
44
45     public static void main(String[] args) {
46         writeStudents();
47
48         System.out.println("reading input.txt:");
49         readStudents("input.txt");
50
51         System.out.println("reading output.txt (does not exist):");
52         readStudents("output.txt");
53
54         System.out.println("input.txt has " + countLines("input.txt") + " records");
55         System.out.println("output.txt has " + countLines("output.txt") + " records");
56     }
57 }

```

OUTPUT

```
wrote 3 students to input.txt
reading input.txt:
    CS001, Ada Obi, 4.51
    CS002, Ken Bello, 3.88
    CS003, Lilian Uche, 4.05
reading output.txt (does not exist):
Could not read output.txt: output.txt (The system cannot find the file specified)
input.txt has 3 records
Could not count output.txt: output.txt (The system cannot find the file specified)
output.txt has 0 records
```

The missing file is the whole question. The paper asks you to read `output.txt`, a file it never asked you to create, and says "including the case where `output.txt` does not exist". That is deliberate: it wants to see your catch block actually fire. Ours did, printed a clear specific message naming the file, and the program carried on to count the records. Nothing crashed.

Every resource is in a `try-with-resources`, so all three are closed automatically even on the failing path, which satisfies "properly closed (either in a `finally` block or via `try-with-resources`)". And `countLines` catches its own `IOException` and returns 0 rather than declaring `throws`, which is what "must handle its own exceptions internally" demands.

Now redo it: write `countLines` from memory. Then say why it does **not** have `throws IOException` in its signature, and what the question would look like if it did.

FILL IN THE GAP · EXCEPTIONS AND FILES

1. An exception is an _____ that disrupts normal flow; it is an _____ representing an error condition.
2. Checked exceptions are checked at _____ time and _____ be caught or declared. Unchecked ones occur at _____ time.
3. `IOException` is _____; `NullPointerException` is _____.
4. `throw` _____; `throws` _____.
5. The _____ block always runs, and is used for _____.
6. Catch blocks must be ordered _____ first, or the code _____.
7. A custom exception extends _____ to be checked, or _____ to be unchecked.
8. `read()` signals end of file with _____; `readLine()` signals it with _____.
9. `StackOverflowError` is an _____, not an exception, and you should _____ catch it.

ANSWERS

1. event, object · 2. compile, must, run · 3. checked, unchecked · 4. actually throws an exception now, declares that a method might throw · 5. finally, cleanup such as closing files · 6. narrowest / most specific, does not compile (unreachable catch) · 7. `Exception`, `RuntimeException` · 8. `-1`, `null` · 9. Error, not (it signals something you should fix, not handle)

PART XII

Solved Past Papers

Every question on both papers, worked in full. Not the four you would attempt in the hall: all of them, because a question you skip is still a topic you can be examined on next year. Every program here was compiled and run, and where a paper prints its own answer, our solution reproduces it exactly.



The 2025/26 paper, and its rigid template

Three hours, 70 marks, and the single most useful thing you can know before you open it.

● MUST-MEMORISE · THE SHAPE OF THE PAPER

Attempt one question from each Section.

Section A: 20 marks, Q1 **or** Q2

Section B: 25 marks, Q3 **or** Q4

Section C: 25 marks, Q5 **or** Q6

And: **use comments in Section B and Section C** to distinguish what task each block of code performs. That is an instruction, not advice.

MEMORY HOOK: 20 + 25 + 25. Section A is the cheapest and the most predictable, so it is where you bank marks fastest.

TEACH · SECTION A IS A TEMPLATE, AND BOTH QUESTIONS USE IT

Q1 and Q2 are the same question with different nouns. Once you see this, you can prepare for Section A precisely:

PART	MARKS	ALWAYS ASKS	Q1 / Q2 IN 2025/26
A	2	Define one term	method overloading / encapsulation
B	3	State three differences between two things	checked vs unchecked / overloading vs overriding
C	5	Find the errors , given the correct output	a loop over an array with planted bugs
D	10	State the output , then convert the loops	Armstrong numbers / perfect numbers

Part D is worth **half of Section A** and it is two mechanical tasks: dry run (F.9) and loop conversion (Unit 4.6). Neither requires an idea. Both require practice.

● MUST-MEMORISE · THE FIVE PLANTED ERRORS, BOTH YEARS

Part C plants the **same five kinds** of error in both Q1 and Q2:

1. **Wrong array type:** `int[]` holding decimal literals.
2. **Wrong accumulator type:** `int sum` for a decimal total.
3. **Off by one:** `i <= arr.length`.
4. **Inverted continue:** skipping exactly what it should keep.
5. **Wrong stop count:** the wrong number in the final `if`.

Check for those five first, in that order, and you will find them in about a minute. They are the same five in both questions and in both years.

MEMORY HOOK: TTOIS: Type, Total, Off-by-one, Inverted, Stop. Read the snippet five times, once per letter, rather than once hoping to spot everything.

Question One, worked in full

Q1 A · DEFINE METHOD OVERLOADING, AND THE ROLE OF THE SIGNATURE

2 MARKS

The question. "Define method overloading in Java, and state precisely the role that the method signature plays in enabling the compiler to resolve an overloaded call at compile time."

Model answer.

Method overloading is a feature that allows a class to have more than one method with the **same name** but **different parameter lists** (differing in the number, types, or order of parameters).

The **method signature** is a method's **name together with its parameter list**. Its role in resolution is this: at a call such as `add(2, 3)`, the compiler compares the number, types and order of the **arguments written at the call site** against the parameter list of every candidate with that name, and selects the one whose signature matches (widening the argument types if no exact match exists). Because the resolution uses only what appears at the call site, and the **return type is not part of the signature**, the return type plays **no role**: two methods differing only by return type are rejected by the compiler as a duplicate definition. The choice is therefore made entirely at **compile time**, which is why overloading is called compile-time (static) polymorphism.

Where the marks are: one for the definition, one for "signature = name + parameter list" and the fact that the return type is excluded. Saying "the compiler picks the right one" without saying *how* is half an answer. See Units 5.1 and 5.6.

Q1 B · THREE DIFFERENCES: CHECKED VS UNCHECKED

3 MARKS

Model answer. Three differences, one mark each:

1. **When they are detected.** Checked exceptions are checked by the compiler at **compile time**; unchecked exceptions occur at **run time** and the compiler does not examine them.
2. **Whether you must handle them.** A checked exception **must** be either caught in a try-catch or declared with throws, or the code **will not compile**. An unchecked exception carries no such obligation.
3. **What causes them.** Checked exceptions represent conditions outside the program's control (a missing file, a failed network call), whereas unchecked exceptions represent **programming errors** (dereferencing null, a bad index, dividing by zero).

Examples (add them; they cost one line and prove the point): checked: `IOException`, `FileNotFoundException`, `ClassNotFoundException`. Unchecked: `NullPointerException`, `ArrayIndexOutOfBoundsException`, `ArithmeticException`.

Where the marks are: three *distinct* axes. Writing "one is checked and one is not" three different ways scores one mark. See Unit 10.1.

Q1 C · FIND THE ERRORS

5 MARKS

The question. The code should loop through the array, and for each starting index *i* find the first number from *i* onward that is a multiple of 4; add (**that number - 1**) to sum, count it as a match, stop after 3 matches, then display the sum and match count. **The correct output is Sum: 141.0, Matches: 3.**

Read the output first. "141.0" tells you the sum is a double before you have read a line of code. That single character finds two of the five errors.

```

1 import javax.swing.JOptionPane;
2 public class Snippet {
3     public static void main(String[] args) {
4         int[] arr = {24.0, 40.0, 66.0, 80.0, 98.0};           // ERROR 1 and 2
5         int sum = 0;                                       // ERROR 2
6         int matches = 0;
7         boolean stop = false;
8         for (int i = 0; i <= arr.length && !stop; i++) {    // ERROR 3
9             for (int j = i; j < arr.length; j++) {
10                 if (arr[j] % 4 == 0) {                     // ERROR 4
11                     continue;
12                 }
13                 sum += arr[j] - j;                         // ERROR 5
14                 matches++;
15                 if (matches == 3) {
16                     stop = true;
17                 }
18                 break;
19             }
20         }
21         JOptionPane.showMessageDialog(null, "Sum: " + sum);
22         JOptionPane.showMessageDialog(null, "Matches: " + matches);
23         System.exit(0);
24     }
25 }

```

WORKED EXAMPLE · THE FIVE ERRORS, AND WHY EACH IS ONE

- ❶ **Line 4: `int[]` cannot hold 24.0.** Those are double literals. This is a **compile** error: "incompatible types: possible lossy conversion from double to int". Fix: `double[] arr`.
The output confirms it: 141.0 is a double.
- ❷ **Line 5: `int sum` cannot total doubles,** and could never print "141.0". Fix: `double sum = 0`;
Unit 3.3: the type decides what you may do and how it prints.
- ❸ **Line 8: `i <= arr.length` is off by one.** Indexes run 0 to length-1 (F.10). Fix: `i < arr.length`.
Here it would not actually throw, because the inner loop never runs when `i = 5`, but it is still wrong and it is one of the five marks.
- ❹ **Lines 10 to 12: the `continue` is inverted.** The spec says find numbers that **are** multiples of 4. This skips them and processes everything else: precisely backwards. Fix: `if (arr[j] % 4 != 0) continue`;
Unit 4.7: read every `continue` as "unless". This is the error worth the most attention, because it is the one that is pure logic.
- ❺ **Line 13: `arr[j] - j` subtracts the index.** The spec says add **(that number - 1)**. Fix: `sum += arr[j] - 1`;
A subtle one: with `j` it would give the right answer for `j = 1` only, so a spot check could miss it.

```

1 // JOptionPane replaced by println so the output can be printed here;
2 // in the exam, keep the dialogs exactly as the paper has them.
3 public class Q1C {
4     public static void main(String[] args) {
5         double[] arr = {24.0, 40.0, 66.0, 80.0, 98.0}; // fix 1
6         double sum = 0; // fix 2
7         int matches = 0;
8         boolean stop = false;
9         for (int i = 0; i < arr.length && !stop; i++) { // fix 3
10             for (int j = i; j < arr.length; j++) {
11                 if (arr[j] % 4 != 0) { // fix 4
12                     continue;
13                 }
14                 sum += arr[j] - 1; // fix 5
15                 matches++;
16                 if (matches == 3) {
17                     stop = true;
18                 }
19                 break;
20             }
21         }
22         System.out.println("Sum: " + sum);
23         System.out.println("Matches: " + matches);
24     }
25 }

```

OUTPUT

Sum: 141.0
Matches: 3

That is the proof. The paper printed Sum: 141.0 and Matches: 3, and the corrected program produces exactly that. If your five fixes do not reproduce the stated output, one of them is wrong, and the paper has handed you the means to check your own answer.

DRY RUN · WHY THE ANSWER IS 141

ARR = {24.0, 40.0, 66.0, 80.0, 98.0}

I	FIRST MULTIPLE OF 4 FROM I ONWARD	ADD (THAT - 1)	MATCHES	SUM
0	arr[0] = 24	24 - 1 = 23	1	23.0
1	arr[1] = 40	40 - 1 = 39	2	62.0
2	66 is not, arr[3] = 80 is	80 - 1 = 79	3	141.0
-	matches == 3, so stop = true			141.0

Note $i = 2$ skipped 66 and found 80, which is exactly what the inner loop is for: "the first multiple of 4 from i onward". And the outer loop never reached $i = 3$, because stop ended it.

The question. An Armstrong number equals the sum of its own digits each raised to the power of the number of digits. State the five outputs for the ArmstrongChecker code, for {370, 371, 372, 407, 408}.

The answer. Compute each digit cubed (all five are 3-digit numbers, so the power is 3):

N	SUM OF DIGITS ³	= N?	OUTPUT
370	27 + 343 + 0 = 370	yes	370 -> ARMSTRONG
371	27 + 343 + 1 = 371	yes	371 -> ARMSTRONG
372	27 + 343 + 8 = 378	no	372 -> NOT ARMSTRONG
407	64 + 0 + 343 = 407	yes	407 -> ARMSTRONG
408	64 + 0 + 512 = 576	no	408 -> NOT ARMSTRONG

The arithmetic: $3^3 = 27$, $7^3 = 343$, $0^3 = 0$, $1^3 = 1$, $2^3 = 8$, $4^3 = 64$, $8^3 = 512$. Three of the five are Armstrong numbers, which is worth noticing: the examiner chose 370, 371 and 407 because they are the three three-digit Armstrong numbers next to each other, and 372 and 408 as near misses.

```

1 public class ArmstrongChecker {
2     public static void main(String[] args) {
3         int[] numbers = {370, 371, 372, 407, 408};
4         for (int idx = 0; idx < numbers.length; idx++) {
5             int n = numbers[idx], digits = 0, temp = n, sum = 0;
6             while (temp > 0) { digits++; temp /= 10; }
7             temp = n;
8             while (temp > 0) {
9                 int d = temp % 10, power = 1;
10                for (int p = 0; p < digits; p++) power *= d;
11                sum += power;
12                temp /= 10;
13            }
14            System.out.println(n + " -> " + (sum == n ? "ARMSTRONG" : "NOT ARMSTRONG"));
15        }
16    }
17 }

```

OUTPUT

```

370 -> ARMSTRONG
371 -> ARMSTRONG
372 -> NOT ARMSTRONG
407 -> ARMSTRONG
408 -> NOT ARMSTRONG

```

The question. "Retain other things but change the For Loop to While loop and the While loops to For loops."

Read it precisely. The original has **three** loops:

LOOP	CURRENTLY	BECOMES
the outer one over idx	for	while
counting the digits	while	for
summing the powers	while	for
the inner p loop	for	while

"The For Loop" (singular) is the outer one; "the While loops" (plural) are the two digit loops. The inner-most p loop is also a for, so convert it too: the safest reading is that **every** for becomes a while and every while becomes a for. Apply Unit 4.6's recipe to each.

```

1 public class ArmstrongConverted {
2     public static void main(String[] args) {
3         int[] numbers = {370, 371, 372, 407, 408};
4
5         // FOR -> WHILE: init out, condition in, update at the bottom
6         int idx = 0;
7         while (idx < numbers.length) {
8             int n = numbers[idx], digits = 0, temp = n, sum = 0;
9
10            // WHILE -> FOR: no init needed, temp is the counter
11            for (; temp > 0; temp /= 10) {
12                digits++;
13            }
14
15            temp = n;
16
17            // WHILE -> FOR
18            for (; temp > 0; temp /= 10) {
19                int d = temp % 10, power = 1;
20
21                // FOR -> WHILE
22                int p = 0;
23                while (p < digits) {
24                    power *= d;
25                    p++;
26                }
27                sum += power;
28            }
29
30            System.out.println(n + " -> " + (sum == n ? "ARMSTRONG" : "NOT ARMSTRONG"));
31            idx++; // the update, at the bottom
32        }
33    }
34 }

```

OUTPUT

```

370 -> ARMSTRONG
371 -> ARMSTRONG
372 -> NOT ARMSTRONG
407 -> ARMSTRONG
408 -> NOT ARMSTRONG

```

Identical output to the original. That is the test of a conversion: same program, different clothes.

Two points a marker looks for. The `for (; temp > 0; temp /= 10)` has an **empty initialisation**, which is legal and is the honest conversion: the original `while` had nothing to initialise, because `temp` already existed. Do not invent a variable to fill the slot. And `idx++` sits at the **bottom** of the `while` body, which is Unit 4.6's step 3 and the one people forget: leave it out and the program runs forever.

Question Two, worked in full

Q2 A · DEFINE ENCAPSULATION

2 MARKS

Model answer.

Encapsulation is the OOP principle of **bundling data (fields) and the methods that operate on them into a single unit (a class), while restricting direct access to some of its components.**

In Java it is achieved by declaring the fields private and exposing controlled access through public get-ter and setter methods. This gives **data hiding** (unauthorised code cannot read or modify the state), **validation** (a setter can reject invalid input, for example preventing a negative balance), and **flexibility** (the internal representation can change without breaking any code that uses the class).

Where the marks are: one for the bundling-plus- restricting definition, one for how it is achieved (private fields, public getters and setters). Add one benefit if you have room. See Unit 6.4.

Q2 B · THREE DIFFERENCES: OVERLOADING VS OVERRIDING

3 MARKS

Model answer. Three differences, one mark each:

1. **Where they occur.** Overloading happens **within a single class**: several methods share a name. Overriding happens **between a superclass and a subclass**: the subclass replaces the inherited method.
2. **The parameter list.** Overloaded methods must have **different** parameter lists. An overriding method must have **exactly the same** name and parameter list as the one it overrides.
3. **When the method is chosen.** Overloading is resolved at **compile time** by the compiler (static polymorphism); overriding is resolved at **run time** by the JVM, from the object's actual type (dynamic polymorphism).

Where the marks are: three separate axes. If you have room, add that overriding **requires inheritance** and overloading does not. See Unit 6.7.

Q2 C · FIND THE ERRORS

5 MARKS

The question. Loop through an array of prices, **skip anything under 20**, apply a 10% discount to prices 20 and above, add the discounted price to total, count it, and **stop once 4 items have been discounted**. Then display the total and count. **Correct output: Total: 202.725, Count: 4.**

The same five errors as Q1, in the same order. This is the payoff of the template.

WORKED EXAMPLE · THE FIVE ERRORS

- 1 **Line 4: `int[] prices = {15.5, ...}`.** Decimal literals in an int array: a compile error. Fix: `double[] prices`.
The output "202.725" says double before you read anything else.
- 2 **Line 5: `int total = 0`;** cannot hold 202.725. Fix: `double total = 0`;
- 3 **Line 8: `i <= prices.length`** is off by one. Fix: `i < prices.length`.
*Unlike Q1, this one **would** throw `ArrayIndexOutOfBoundsException` at `i = 6`, because line 9 reads `prices[i]` straight away. Same error, worse consequence.*
- 4 **Line 9: `if (prices[i] > 20) continue;`** is inverted. The spec says skip anything **under** 20. This skips everything **over** 20, i.e. exactly the items to be discounted. Fix: `if (prices[i] < 20) continue;`
- 5 **Line 15: `if (count == 5)`.** The spec says stop once **4** have been discounted. Fix: `if (count == 4)`.
Only four of the six prices are 20 or above anyway, so with `== 5` the flag never fires. The count would still be 4, and the total would still be right, which makes this the easiest error to miss: it does not change the answer, it just means the code does not do what it was asked.

```

1 public class Q2C {
2     public static void main(String[] args) {
3         double[] prices = {15.5, 42.0, 8.75, 60.0, 33.25, 90.0}; // fix 1
4         double total = 0; // fix 2
5         int count = 0;
6         boolean done = false;
7         for (int i = 0; i < prices.length && !done; i++) { // fix 3
8             if (prices[i] < 20) { // fix 4
9                 continue;
10            }
11            double discount = prices[i] * 0.1;
12            total += prices[i] - discount;
13            count++;
14            if (count == 4) { // fix 5
15                done = true;
16            }
17        }
18        System.out.println("Total: " + total);
19        System.out.println("Count: " + count);
20    }
21 }

```

OUTPUT

Total: 202.725
Count: 4

Matches the paper exactly.

DRY RUN · WHY THE ANSWER IS 202.725

PRICES = {15.5, 42.0, 8.75, 60.0, 33.25, 90.0}, 10% OFF EACH

I	PRICE	UNDER 20?	DISCOUNTED (90%)	COUNT	TOTAL
0	15.5	yes: skip	-	0	0
1	42.0	no	37.8	1	37.8
2	8.75	yes: skip	-	1	37.8
3	60.0	no	54.0	2	91.8
4	33.25	no	29.925	3	121.725
5	90.0	no	81.0	4	202.725

count reaches 4, done becomes true, and the loop ends having consumed the whole array anyway. Total 202.725, Count 4.

The question. NumberClassifier counts, for 1 to 30, how many numbers are perfect (proper divisors sum to the number), abundant (sum to more) and deficient (sum to less). State the output.

The answer: Perfect: 2, Abundant: 5, Deficient: 23

The reasoning, which is worth writing out because "no need to show workings" does not mean "do not check":

CLASS	COUNT	WHICH
Perfect	2	6 (1+2+3 = 6) and 28 (1+2+4+7+14 = 28)
Abundant	5	12 (16), 18 (21), 20 (22), 24 (36), 30 (42)
Deficient	23	everything else, including every prime, and 1

Two checks that catch mistakes. $2 + 5 + 23 = 30$, so every number is classified exactly once. And **1** is deficient: its proper divisors sum to 0, because 1 has no proper divisor, and the loop for (div = 1; div < num; div++) never runs for num = 1. Forget that and you get 22, not 23.

```

1 public class NumberClassifier {
2     public static void main(String[] args) {
3         int perfectCount = 0, abundantCount = 0, deficientCount = 0;
4         for (int num = 1; num <= 30; num++) {
5             int sum = 0;
6             for (int div = 1; div < num; div++) {
7                 if (num % div == 0) sum += div;
8             }
9             if (sum == num) perfectCount++;
10            else if (sum > num) abundantCount++;
11            else deficientCount++;
12        }
13        System.out.println("Perfect: " + perfectCount + ", Abundant: " + abundantCount
14            + ", Deficient: " + deficientCount);
15        System.out.println("check: " + (perfectCount + abundantCount + deficientCount)
16            + " numbers classified");
17    }
18 }

```

OUTPUT

```

Perfect: 2, Abundant: 5, Deficient: 23
check: 30 numbers classified

```

```

1 public class NumberClassifierWhile {
2     public static void main(String[] args) {
3         int perfectCount = 0, abundantCount = 0, deficientCount = 0;
4
5         int num = 1;                                // 1. init above
6         while (num <= 30) {                          // 2. condition
7             int sum = 0;
8
9             int div = 1;                            // the inner one too
10            while (div < num) {
11                if (num % div == 0) sum += div;
12                div++;                               // 3. update at the bottom
13            }
14
15            if (sum == num) perfectCount++;
16            else if (sum > num) abundantCount++;
17            else deficientCount++;
18
19            num++;                                    // 3. update at the bottom
20        }
21        System.out.println("Perfect: " + perfectCount + ", Abundant: " + abundantCount
22            + ", Deficient: " + deficientCount);
23    }
24 }

```

OUTPUT

Perfect: 2, Abundant: 5, Deficient: 23

Identical output, which is the whole test. Note `int sum = 0;` stays **inside** the outer loop: it must be reset for every number, and moving it out is the classic way to wreck this conversion. The recipe only moves the loop's **own** three parts; everything else stays exactly where it was, which is what "retain other things" means.

S.4 · 2025/26 · QUESTION THREE · 25 MARKS

Question Three, worked in full

An array question that hides a second question inside it, and a file question that is testing whether you read the words on the page.

Q3 A · CONSECUTIVE DUPLICATES IN A SALES LOG

12 MARKS

The question. A store records daily sales in an integer array `saleAmounts[]`. A "consecutive duplicate" is any position `i` (`i >= 1`) where `saleAmounts[i] == saleAmounts[i-1]`. Given:

INDEX	0	1	2	3	4	5	6	7	8	9
AMOUNT	20	20	35	40	40	40	15	60	60	25

Write a program that (i) counts, in **one pass**, the indices `i >= 1` where the amount equals the one before it; (ii) returns a single `String` listing, in order and comma separated, every amount that equals its **predecessor or its successor**, built with **plain String concatenation only**; (iii) displays both in **one** `JOptionPane` dialog. The paper adds a hint: *it must not flag 35, 15, or 25.*

● MUST-MEMORISE · PARTS I AND II COUNT DIFFERENT THINGS

Part i counts positions. Part ii lists values. They are not the same question asked twice, and they do not give the same number.

Part i asks how many **boundaries** have a twin across them. Part ii asks which **elements** are touching a twin at all.

Look at the three 40s. They contain **two** boundaries (index 3 to 4, and 4 to 5) but **three** elements. So part i counts 2 there and part ii lists 3. Read the two parts as one question and you will answer one of them wrong.

MEMORY HOOK: fences and posts. Part i counts the fences between posts, part ii counts the posts. A run of n twins has n posts and $n-1$ fences, always.

DRY RUN · BOTH ANSWERS, BY HAND, BEFORE WRITING ANY CODE

SALEAMOUNTS = {20, 20, 35, 40, 40, 40, 15, 60, 60, 25}

I	AMOUNT	SAME AS PREVIOUS?	SAME AS NEXT?	PART I COUNTS?	PART II FLAGS?
0	20	no predecessor	yes (20)	-	yes
1	20	yes (20)	no (35)	yes	yes
2	35	no (20)	no (40)	no	no
3	40	no (35)	yes (40)	no	yes
4	40	yes (40)	yes (40)	yes	yes
5	40	yes (40)	no (15)	yes	yes
6	15	no (40)	no (60)	no	no
7	60	no (15)	yes (60)	no	yes
8	60	yes (60)	no (25)	yes	yes
9	25	no (60)	no successor	no	no
Answers				4	20, 20, 40, 40, 40, 60, 60

Check it against the hint. The paper told you 35, 15 and 25 must not be flagged, and they are the only three that are not. The examiner has handed you a way to test your own trace before you write a line of code, which is exactly what the hint is for.

And check the two answers against each other. The runs are 20-20 (2 long), 40-40-40 (3 long) and 60-60 (2 long). Part ii flags $2 + 3 + 2 = 7$ values. Part i counts $1 + 2 + 1 = 4$ positions. They differ by exactly **3**, the number of runs, because each run loses one to the fences-and-posts rule. If your two answers do not differ by the number of runs, one of them is wrong.

```

1 public class SalesDuplicates {
2
3     // Part i: ONE pass. Count every index i >= 1 whose amount equals the one before.
4     static int countConsecutiveDuplicates(int[] sales) {
5         int count = 0;
6         for (int i = 1; i < sales.length; i++) {
7             if (sales[i] == sales[i - 1]) {
8                 count++;
9             }
10        }
11        return count;
12    }
13
14    // Part ii: every amount equal to its predecessor OR its successor, in order,
15    // comma separated, built with plain String concatenation only.
16    static String flaggedAmounts(int[] sales) {
17        String result = "";
18        for (int i = 0; i < sales.length; i++) {
19            boolean sameAsPrevious = (i > 0) && (sales[i] == sales[i - 1]);
20            boolean sameAsNext = (i < sales.length - 1) && (sales[i] == sales[i + 1]);
21            if (sameAsPrevious || sameAsNext) {
22                if (result.equals("")) {
23                    result = "" + sales[i];
24                } else {
25                    result = result + ", " + sales[i];
26                }
27            }
28        }
29        return result;
30    }
31
32    // Part iii: one dialog showing both answers.
33    public static void main(String[] args) {
34        int[] saleAmounts = {20, 20, 35, 40, 40, 40, 15, 60, 60, 25};
35
36        int duplicates = countConsecutiveDuplicates(saleAmounts);
37        String flagged = flaggedAmounts(saleAmounts);
38
39        String message = "Consecutive duplicate count: " + duplicates
40            + "\nFlagged amounts: " + flagged;
41        System.out.println(message);
42    }
43 }

```

OUTPUT

```

Consecutive duplicate count: 4
Flagged amounts: 20, 20, 40, 40, 60, 60

```

Exactly the hand trace, including the three amounts the paper told us not to flag. The dialog is a `println` here so its text can be printed in this book; in the hall, write `JOptionPane.showMessageDialog(null, message);` and import `javax.swing.JOptionPane` (F.11).

⚠ TRAP · THREE WAYS TO LOSE MARKS ON PART A

1. Starting part i's loop at 0. The question says $i \geq 1$, and for a reason: `sales[0 - 1]` is `sales[-1]`, which throws `ArrayIndexOutOfBoundsException` (Unit 8.1). Index 0 has no predecessor, so the loop starts at 1. This is where "one pass" is easiest to get right and easiest to throw away.

2. Forgetting the boundary guards in part ii. Index 0 has no predecessor and the last index has no successor. The guards $(i > 0)$ and $(i < \text{sales.length} - 1)$ are what stop the reads from running off either end. They work because `&&` is **short-circuit** (Unit 3.6): when $i > 0$ is false, Java never evaluates `sales[i - 1]`, so it never throws. Swap the two sides of the `&&` and the program crashes on the first element.

3. The leading comma. Concatenating `", " + sales[i]` every time gives `, 20, 20, 40...` with a comma stuck on the front. The test `if (result.equals(""))` writes the first value bare and every later value with its separator in front. Compare Strings with `.equals()`, never with `==` (Unit 7.3).

The question bans `StringBuilder` deliberately: it wants to see that you can handle the separator yourself. Use `StringBuilder` anyway and you have answered a question the paper did not ask.

S.4 · 2025/26 · QUESTION THREE B

The file question, and the trick in its wording

Q3 B · STUDENT RECORDS WITH BASIC FILE I/O

13 MARKS

The question. Using **only** `FileWriter`, `FileReader`, `BufferedReader`, `BufferedWriter` and `IOException` (**or its subclasses**), write a program that:

1. creates `input.txt` and writes at least three students (ID, name, CGPA) into it with `FileWriter`, one per line, wrapped in try-catch;
2. reads student details from `output.txt` with `FileReader` and prints each line exactly as it appears, handling any exception **including the case where `output.txt` does not exist**;
3. has a method that takes a file name, counts the lines, and returns the count, **handling its own exceptions internally**;
4. closes every resource, and prints a clear, specific message in every catch block.

● MUST-MEMORISE · IT WRITES `INPUT.TXT` AND READS `OUTPUT.TXT`

Part 1 writes **`input.txt`**. Part 2 reads **`output.txt`**. The program never creates `output.txt`. That is not a misprint for you to tidy up. It is the question.

Read part 2 again: "handle any exception that may occur while reading, **including the case where `output.txt` does not exist**." The examiner has told you, in the question itself, that the file will be missing. The marks are for the **catch block running**, not for the file being read.

So do not "fix" it by reading `input.txt` instead. Write exactly what the paper asks, and let the catch block do its job. That is the answer.

MEMORY HOOK: the missing file is the exam. When a question names a file it never created, the catch block is the thing being marked.

TEACH · THE FIVE CLASSES, AND WHY THE LIST IS EXACTLY THESE FIVE

The restriction looks arbitrary. It is not: the five are exactly what you need and nothing more (Unit 10.3).

CLASS	DIRECTION	WHAT IT IS FOR
FileWriter	out	Writes characters to a file. On its own it writes one character at a time.
BufferedWriter	out	Wraps a <code>FileWriter</code> and collects characters, writing them in blocks. Also gives you <code>newLine()</code> .
FileReader	in	Reads characters from a file. On its own it reads one character at a time: it has no <code>readLine()</code> .
BufferedReader	in	Wraps a <code>FileReader</code> and gives you <code>readLine()</code> , which is the whole reason it is on the list.
IOException	-	The checked exception every one of the above can throw. You must catch it or declare it (Unit 10.1).

The pairing is the point. The question says "read using `FileReader`" and "one student per line". A `FileReader` alone cannot give you a line. So you wrap it: `new BufferedReader(new FileReader(fileName))`. That is why both appear on the list, and it is why the list is not padding.

And read "or its subclasses" carefully. Those three words are permission. `FileNotFoundException` is a subclass of `IOException`, so you are explicitly allowed to catch it separately, which is exactly how you answer part 2's "including the case where `output.txt` does not exist" with a message that says what actually happened.

⚠ TRAP · CATCH THE CHILD BEFORE THE PARENT, OR IT WILL NOT COMPILE

When you catch both, the **subclass must come first**. Java tries catch blocks top to bottom and takes the first that fits. Put `IOException` first and it swallows everything, leaving the `FileNotFoundException` block unreachable. Java does not let that pass:

```
1 import java.io.BufferedReader;
2 import java.io.FileNotFoundException;
3 import java.io.FileReader;
4 import java.io.IOException;
5
6 public class CatchOrder {
7     public static void main(String[] args) {
8         try (BufferedReader in = new BufferedReader(new FileReader("any.txt"))) {
9             System.out.println(in.readLine());
10        } catch (IOException e) {
11            System.out.println("the parent is caught first");
12        } catch (FileNotFoundException e) {
13            System.out.println("this line can never run");
14        }
15    }
16 }
```

COMPILER OUTPUT

```
error: exception FileNotFoundException has already been caught
```

This is one of Java's friendlier errors: it refuses to compile code containing a block that could never run. Reverse the two catch blocks and it compiles. **Specific first, general last**, every time.

```

1  import java.io.BufferedReader;
2  import java.io.BufferedWriter;
3  import java.io.FileNotFoundException;
4  import java.io.FileReader;
5  import java.io.FileWriter;
6  import java.io.IOException;
7
8  public class StudentRecords {
9
10     // Part i: create input.txt and write three student records, one per line.
11     static void writeStudents(String fileName) {
12         try (BufferedWriter out = new BufferedWriter(new FileWriter(fileName))) {
13             out.write("STU001, Ada Bello, 4.51");
14             out.newLine();
15             out.write("STU002, Femi Okonkwo, 3.78");
16             out.newLine();
17             out.write("STU003, Ngozi Eze, 4.92");
18             out.newLine();
19             System.out.println("Wrote 3 student records to " + fileName);
20         } catch (IOException e) {
21             System.out.println("Could not write to " + fileName + ": " + e.getMessage());
22         }
23     }
24
25     // Part ii: read a file and print every line exactly as it appears.
26     static void printStudents(String fileName) {
27         try (BufferedReader in = new BufferedReader(new FileReader(fileName))) {
28             String line = in.readLine();
29             while (line != null) {
30                 System.out.println(line);
31                 line = in.readLine();
32             }
33         } catch (FileNotFoundException e) {
34             System.out.println("Cannot read " + fileName + ": the file does not exist.");
35         } catch (IOException e) {
36             System.out.println("Error while reading " + fileName + ": " + e.getMessage());
37         }
38     }
39
40     // Part iii: count the records in a file. Handles its own exceptions; never throws.
41     static int countRecords(String fileName) {
42         int lines = 0;
43         try (BufferedReader in = new BufferedReader(new FileReader(fileName))) {
44             while (in.readLine() != null) {
45                 lines++;
46             }
47         } catch (FileNotFoundException e) {
48             System.out.println("Cannot count " + fileName + ": the file does not exist.");
49         } catch (IOException e) {
50             System.out.println("Error while counting " + fileName + ": " + e.getMessage());
51             return -1;
52         }
53         return lines;
54     }
55
56     public static void main(String[] args) {
57         writeStudents("input.txt");
58
59         System.out.println("--- part ii: reading output.txt, exactly as the paper asks ---");
60         printStudents("output.txt");
61
62         System.out.println("--- the same method on the file we did write ---");
63     }

```

```

64     printStudents("input.txt");
65
66     System.out.println("--- part iii: counting records ---");
67     System.out.println("input.txt holds " + countRecords("input.txt") + " records");
68     System.out.println("output.txt holds " + countRecords("output.txt") + " records");
69 }
70 }

```

OUTPUT

```

Wrote 3 student records to input.txt
--- part ii: reading output.txt, exactly as the paper asks ---
Cannot read output.txt: the file does not exist.
--- the same method on the file we did write ---
STU001, Ada Bello, 4.51
STU002, Femi Okonkwo, 3.78
STU003, Ngozi Eze, 4.92
--- part iii: counting records ---
input.txt holds 3 records
Cannot count output.txt: the file does not exist.
output.txt holds -1 records

```

Line 3 of that output is the 13 marks. The program did exactly what the paper asked, the file was not there, and the catch block said so in words a human can read.

TEACH · FOUR DECISIONS IN THAT PROGRAM WORTH DEFENDING

- 1. try-with-resources answers part 4 on its own.** Writing `try (BufferedWriter out = ...)` closes the writer automatically, whether the block ends normally or throws (Unit 10.2). The paper allows a `finally` block instead, and it is worth knowing why the bracket form is better: a `close()` inside `finally` can itself throw, and if it does while an earlier exception is already travelling, the second exception **replaces** the first and the real cause disappears. If you do use `finally`, the `close` must sit inside its own try-catch, and it must test for null, because the resource may have failed to open at all.
- 2. Wrapping means closing one thing, not two.** `out.close()` on the `BufferedWriter` closes the `FileWriter` inside it. Closing the wrapper closes what it wraps, so one bracket handles both.
- 3. countRecords returns -1, not 0, when it fails.** The method must "handle its own exceptions internally", so it cannot throw, so it must return something. Return 0 and the caller cannot tell a missing file from an empty one. **-1 is impossible as a line count**, so it can only mean failure. A value chosen because it cannot be mistaken for a real answer is called a **sentinel**, and picking a good one is a real skill.
- 4. Read the last two lines of the output in the right order.** "Cannot count output.txt" prints **before** "output.txt holds -1 records", even though the second is the `println` that contains the first as an argument. Java **evaluates the arguments before it calls the method**: `countRecords` runs to completion, printing its own message, and only then does the outer `println` receive -1 and print. Argument first, call second, always.

Question Four, worked in full

The bank account system: 25 marks for the topic this course has been building towards since Module 6.

Q4 · INHERITANCE AND POLYMORPHISM

25 MARKS

The question. A bank wants a system representing different types of accounts using inheritance and polymorphism.

Part 1, BankAccount (9 marks). **Private** fields for account number, holder name and balance; a constructor initialising all three; a method that adds to the balance; a method that deducts if funds suffice and otherwise rejects with a message; a method that displays the details.

Part 2, subclasses (10 marks). SavingAccount overrides the withdrawal so it is allowed only if the balance **after** the withdrawal would still be ≥ 1000 . CurrentAccount overrides it so every **successful** withdrawal adds a 50 reward afterwards.

Part 3, main (6 marks). One of each; deposit into each; at least one successful and one unsuccessful or reward-triggering withdrawal on each; details before and after; and **demonstrate polymorphism by storing each object in a BankAccount reference** and calling the withdrawal through it, confirming the correct overridden version runs.

● MUST-MEMORISE · THE QUESTION CONTAINS A CONTRADICTION, AND RESOLVING IT IS THE EXAM

Part 1 says the balance must be **private**.

Part 2 says the subclasses must **change the balance**.

A private field is invisible to a subclass. So the subclass cannot touch it.

This is not the examiner being careless. It is the whole of Units 6.4 and 6.6 arriving in one question: **encapsulation says the field is private, and inheritance says the child must still be able to work with it**. The way out is the only way out, and it is the answer being marked: **the subclass goes through the parent's public methods**, using `getBalance()` to read and `super.withdraw(amount)` or `deposit(amount)` to change.

MEMORY HOOK: the child knocks, it does not climb in through the window. `getBalance()` to look, `super.withdraw()` to act.

⚠ TRAP · A SUBCLASS CANNOT SEE A PRIVATE FIELD, AND IT WILL SAY SO

The tempting move is to write the arithmetic straight into the subclass. Java stops you at the compiler:

```
1 class Account {
2     private double balance = 5000;
3 }
4
5 class Saving extends Account {
6     public boolean withdraw(double amount) {
7         balance = balance - amount;
8         return true;
9     }
10 }
11
12 public class SubclassReach {
13     public static void main(String[] args) {
14         new Saving().withdraw(100);
15     }
16 }
```

COMPILER OUTPUT

```
error: balance has private access in Account
```

Private means the declaring class only, and a subclass is a different class. Inheritance gives Saving the field (every Saving object really does contain a balance) but not **access** to it. The field is there and untouchable, which is exactly what private is for (Unit 6.4).

Would protected fix it? It would compile, and it is the wrong answer **here**, because part 1 said private. Marks are for doing what the question said. Beyond the exam, going through the parent's methods is better anyway: every rule the parent enforces on a withdrawal keeps being enforced. Make the field protected and each subclass has to remember those rules for itself, which is how a bank ends up with an account that can be overdrawn down one code path and not another. See the access table in Unit 6.6.

TEACH · WHY WITHDRAW RETURNS BOOLEAN

The paper does not say what withdraw returns, and the obvious choice is void. Then read part 2 ii again: a CurrentAccount adds the reward on **every successful withdrawal**. Not on every attempt. So the override has to **know whether the withdrawal succeeded**, and a void method tells it nothing.

Making withdraw return boolean makes the override three honest lines:

```
boolean succeeded = super.withdraw(amount);
if (succeeded) { deposit(50); }
return succeeded;
```

The alternative, comparing the balance before and after, works but is worse: it re-derives an answer the parent already knew and threw away. **When a method knows something the caller needs, it should return it.** Notice that this decision is forced by a single word in part 2 ii, which is what "read the question" means in practice.

```

1 // ===== Part 1: the superclass =====
2 class BankAccount {
3     private String accountNumber;
4     private String holderName;
5     private double balance;
6
7     // Part 1 i: a constructor that initialises all three fields.
8     public BankAccount(String accountNumber, String holderName, double balance) {
9         this.accountNumber = accountNumber;
10        this.holderName = holderName;
11        this.balance = balance;
12    }
13
14    // The private fields are reachable only through these doors (Unit 6.4).
15    public String getAccountNumber() { return accountNumber; }
16    public String getHolderName()    { return holderName; }
17    public double getBalance()       { return balance; }
18
19    // Part 1 ii: add a given amount to the balance.
20    public void deposit(double amount) {
21        balance = balance + amount;
22        System.out.println("Deposit " + amount + " -> " + accountNumber
23            + " balance is now " + balance);
24    }
25
26    // Part 1 iii: deduct if funds are sufficient, otherwise reject with a message.
27    // Returns whether it succeeded, which is what CurrentAccount needs to know.
28    public boolean withdraw(double amount) {
29        if (amount <= balance) {
30            balance = balance - amount;
31            System.out.println("Withdraw " + amount + " -> " + accountNumber
32                + " balance is now " + balance);
33            return true;
34        }
35        System.out.println("REJECTED: " + accountNumber + " holds only " + balance
36            + ", so " + amount + " cannot be withdrawn");
37        return false;
38    }
39
40    // Part 1 iv: display the account's current details.
41    public void displayDetails() {
42        System.out.println("Account " + accountNumber + " | " + holderName
43            + " | balance " + balance);
44    }
45 }
46
47 // ===== Part 2 i: a saving account must never drop below 1000 =====
48 class SavingAccount extends BankAccount {
49     public SavingAccount(String accountNumber, String holderName, double balance) {
50         super(accountNumber, holderName, balance);
51     }
52
53     @Override
54     public boolean withdraw(double amount) {
55         if (getBalance() - amount < 1000) {
56             System.out.println("REJECTED: a saving account must retain 1000. Withdrawing "
57                 + amount + " from " + getAccountNumber() + " would leave "
58                 + (getBalance() - amount));
59             return false;
60         }
61         return super.withdraw(amount);
62     }
63 }

```

```

65 // ===== Part 2 ii: every successful withdrawal earns a 50 reward =====
66 class CurrentAccount extends BankAccount {
67     public CurrentAccount(String accountNumber, String holderName, double balance) {
68         super(accountNumber, holderName, balance);
69     }
70
71     @Override
72     public boolean withdraw(double amount) {
73         boolean succeeded = super.withdraw(amount);
74         if (succeeded) {
75             System.out.println("Reward earned on " + getAccountNumber() + ":");
76             deposit(50);
77         }
78         return succeeded;
79     }
80 }
81
82 // ===== Part 3: the main method =====
83 public class BankSystem {
84     public static void main(String[] args) {
85         // Part 3 i: at least one of each.
86         SavingAccount saving = new SavingAccount("SA-1001", "Ada Bello", 5000.0);
87         CurrentAccount current = new CurrentAccount("CA-2002", "Femi Okonkwo", 3000.0);
88
89         System.out.println("=== iv: details BEFORE the transactions ===");
90         saving.displayDetails();
91         current.displayDetails();
92
93         // Part 3 ii: deposit into each.
94         System.out.println("=== ii: deposit into each ===");
95         saving.deposit(2000.0);
96         current.deposit(1000.0);
97
98         // Part 3 iii: a successful and an unsuccessful withdrawal on each.
99         System.out.println("=== iii: a successful and an unsuccessful withdrawal on each ===");
100        saving.withdraw(4000.0);
101        saving.withdraw(2500.0);
102        current.withdraw(1000.0);
103        current.withdraw(999999.0);
104
105        System.out.println("=== iv: details AFTER the transactions ===");
106        saving.displayDetails();
107        current.displayDetails();
108
109        // Part 3 v: polymorphism. Same reference type, same call, same amount.
110        System.out.println("=== v: polymorphism, same reference type, same call, same amount ===");
111        BankAccount acc = new SavingAccount("SA-3003", "Ngozi Eze", 1500.0);
112        acc.withdraw(600.0);
113        acc.displayDetails();
114
115        acc = new CurrentAccount("CA-4004", "Tunde Ade", 1500.0);
116        acc.withdraw(600.0);
117        acc.displayDetails();
118    }
119 }

```

OUTPUT

```

=== iv: details BEFORE the transactions ===
Account SA-1001 | Ada Bello | balance 5000.0
Account CA-2002 | Femi Okonkwo | balance 3000.0
=== ii: deposit into each ===
Deposit 2000.0 -> SA-1001 balance is now 7000.0
Deposit 1000.0 -> CA-2002 balance is now 4000.0
=== iii: a successful and an unsuccessful withdrawal on each ===
Withdraw 4000.0 -> SA-1001 balance is now 3000.0
REJECTED: a saving account must retain 1000. Withdrawing 2500.0 from SA-1001 would leave 500.0

```

```

Withdraw 1000.0 -> CA-2002 balance is now 3000.0
Reward earned on CA-2002:
Deposit 50.0 -> CA-2002 balance is now 3050.0
REJECTED: CA-2002 holds only 3050.0, so 999999.0 cannot be withdrawn
=== iv: details AFTER the transactions ===
Account SA-1001 | Ada Bello | balance 3000.0
Account CA-2002 | Femi Okonkwo | balance 3050.0
=== v: polymorphism, same reference type, same call, same amount ===
REJECTED: a saving account must retain 1000. Withdrawing 600.0 from SA-3003 would leave 900.0
Account SA-3003 | Ngozi Eze | balance 1500.0
Withdraw 600.0 -> CA-4004 balance is now 900.0
Reward earned on CA-4004:
Deposit 50.0 -> CA-4004 balance is now 950.0
Account CA-4004 | Tunde Ade | balance 950.0

```

All four classes live in one file because only one may be public and that one must match the file name: BankSystem.java. In the hall, write them as four classes exactly as they are here, or as four separate files. Both are correct.

DRY RUN · EVERY BALANCE, AND WHY EACH RULE FIRED

CALL	BALANCE BEFORE	WHICH RULE DECIDED	ALLOWED?	BALANCE AFTER
SA-1001, a SavingAccount, opens at 5000				
deposit(2000)	5000.0	no rule: deposits always land	yes	7000.0
withdraw(4000)	7000.0	7000 - 4000 = 3000, and 3000 >= 1000	yes	3000.0
withdraw(2500)	3000.0	3000 - 2500 = 500, and 500 < 1000	no	3000.0
CA-2002, a CurrentAccount, opens at 3000				
deposit(1000)	3000.0	no rule	yes	4000.0
withdraw(1000)	4000.0	funds suffice, so the reward fires	yes	3000.0, then +50 = 3050.0
withdraw(999999)	3050.0	funds do not suffice, so no reward	no	3050.0

The last row is the one to study. The reward sits inside if (succeeded), so a rejected withdrawal earns nothing. Had withdraw returned void, that if could not have been written, and CA-2002 would have been paid 50 for a transaction that never happened.

● MUST-MEMORISE · HOW TO PROVE POLYMORPHISM IN SIX LINES

Part 3 v asks you to **confirm the correct overridden version runs**. A proof needs a control: change **one** thing and show the behaviour changes.

So: same reference type, same method, same amount, same starting balance. Only the object differs.

```
BankAccount acc = new SavingAccount("SA-3003", "Ngozi Eze", 1500.0);
acc.withdraw(600.0);           // rejected: would leave 900, under 1000

acc = new CurrentAccount("CA-4004", "Tunde Ade", 1500.0);
acc.withdraw(600.0);           // allowed, and pays a 50 reward: 950
```

Both variables are declared BankAccount. Both calls are acc.withdraw(600.0), character for character. Both accounts hold 1500.0. One is refused and one is paid a bonus.

That is the proof, and here is the sentence that earns the mark: the compiler only knew acc was a BankAccount, so the choice of which withdraw to run could not have been made at compile time. It was made at **run time**, from the object's **actual** class. That is dynamic method dispatch, and it is what "polymorphism" names (Unit 6.7).

MEMORY HOOK: the reference decides what you may call. The object decides what actually runs.

Compare this with the TRAP at the end of Unit 6.6, where the same setup silently ran the **parent's** method because the override was really an overload. Same two lines of code, opposite outcomes, and the difference was one parameter type. That is why @Override sits on every override here.

S.6 · 2025/26 · QUESTION FIVE · 25 MARKS

Question Five, worked in full

Two programs that are really one skill: read the question so literally that the structure it demands writes itself.

● MUST-MEMORISE · SECTION C DICTATES YOUR STRUCTURE, AND MARKS IT

Section C does not merely ask for a working program. It tells you **which construct to use**, and using a different one loses the mark even when the output is right.

"Use a **switch**" · "use **nested** if-else (not a switch, and not independent/separate if checks)" · "using a **nested control statement**" · "no **StringBuilder**" · "no **collection classes**" · "**JOptionPane only**" · "each task must be its own **method**".

And from the front page: **use comments in Section B and Section C** to say what each block does. That is an instruction, and it is free marks.

MEMORY HOOK: underline every noun that names a construct before you write anything. Those nouns are the marking scheme.

The question. Using JOptionPane only, price tickets for a 3-day conference and produce a sales summary, with four methods:

1. takes a day number, uses a **switch** to return a base price for Day 1, 2 or 3 (**pick any three distinct prices and state them clearly**); any other day **throws IllegalArgumentException**;
2. takes a tier and days attending, and uses **nested if-else** to return a discount: Gold 3 days 25%, Gold 1-2 days 15%, Silver 3 days 15%, Silver 1-2 days 5%, Bronze 0%, and **any tier that is not "Gold" or "Silver" is Bronze**. The nesting must reflect that days are only checked **within** each tier branch;
3. takes a base total and a discount percentage, applies it, and returns **50.0** if the result would fall below 50.0 (a price floor);
4. `main()`: ask for N attendees; loop N times asking days and tier; sum the day prices into a per-attendee base total; get the discount; get the final price; accumulate total revenue and a count of Gold, Silver and Bronze; then show **one** summary dialog.

● MUST-MEMORISE · CHOOSE PRICES THAT LET THE FLOOR FIRE

The question lets you **pick the three prices**. That freedom is a trap with a right answer.

Pick 15000, 12000 and 9000 and part 3's price floor **can never fire**. The largest possible discount is 25%, so nothing ever falls near 50.0. You will have written a floor you cannot demonstrate and the examiner cannot see work.

Pick **30.0, 25.0 and 20.0** and the floor becomes live: one Gold day is 30, less 15% is 25.50, which is below 50.0, so the method returns 50.0. The 50.0 floor is itself the hint about the scale the examiner had in mind.

MEMORY HOOK: when a question lets you choose the numbers, choose numbers that make every rule visible. A rule that never fires is a rule you cannot be shown to have written correctly.

TEACH · WHAT "NESTED, NOT SEPARATE" ACTUALLY MEANS HERE

Part 2 is unusually explicit: nested if-else, **not** a switch, **not** independent if checks, and "the nesting must reflect that days attending is only checked **within** each tier branch, not as a separate top-level check."

So the shape is forced. The tier decides the branch; the days are examined **inside** it:

WHAT THE QUESTION DEMANDS

```
if (Gold) { if (days == 3) 25 else 15 }
else if (Silver) { if (days == 3) 15 else 5 }
else { 0 }
```

WHAT LOSES THE MARK

```
if (Gold && days == 3) 25;
if (Gold && days < 3) 15;
if (Silver && days == 3) 15;
```

Right answers, wrong structure: days is tested at the top level, four separate times. This is what "not independent checks" forbids.

Why the final else is the whole Bronze rule. "Any tier string that is not Gold or Silver must be treated as Bronze" is not an extra check you add: it is what a bare else already means. Write `else if (tier.equals("Bronze"))` and you have broken the rule, because "Platinum" would then match nothing and fall through with no discount assigned. The bare else catches Bronze, Platinum, "gold" in lower case, and an empty string, which is exactly what was asked (Unit 4.4).

The same else must also do the **counting**. If "Platinum" is treated as Bronze for the discount, it is a Bronze attendee in the summary too. Count it anywhere else and your dialog contradicts your own pricing.

```

1  import java.util.Scanner;
2
3  public class ConferenceTickets {
4
5      // Part i: a switch returning the base price for a day. Any other day is illegal.
6      static double basePriceForDay(int day) {
7          switch (day) {
8              case 1: return 30.0;
9              case 2: return 25.0;
10             case 3: return 20.0;
11             default: throw new IllegalArgumentException(
12                 "Day must be 1, 2 or 3 but was " + day);
13         }
14     }
15
16     // Part ii: NESTED if-else. Days are only examined inside a tier branch.
17     static double discountPercent(String tier, int daysAttending) {
18         if (tier.equals("Gold")) {
19             if (daysAttending == 3) {
20                 return 25.0;
21             } else {
22                 return 15.0;
23             }
24         } else if (tier.equals("Silver")) {
25             if (daysAttending == 3) {
26                 return 15.0;
27             } else {
28                 return 5.0;
29             }
30         } else {
31             return 0.0;
32         }
33     }
34
35     // Part iii: apply the discount, with a price floor of 50.0.
36     static double applyDiscount(double baseTotal, double percent) {
37         double discounted = baseTotal - (baseTotal * percent / 100.0);
38         if (discounted < 50.0) {
39             return 50.0;
40         }
41         return discounted;
42     }
43
44     public static void main(String[] args) {
45         Scanner in = new Scanner(System.in);
46
47         int attendees = Integer.parseInt(in.nextLine());
48
49         double totalRevenue = 0;
50         int goldCount = 0, silverCount = 0, bronzeCount = 0;
51
52         for (int a = 1; a <= attendees; a++) {
53             int days = Integer.parseInt(in.nextLine());
54             String tier = in.nextLine();
55
56             double baseTotal = 0;
57             for (int d = 1; d <= days; d++) {
58                 baseTotal += basePriceForDay(d);
59             }
60
61             double percent = discountPercent(tier, days);
62             double finalPrice = applyDiscount(baseTotal, percent);
63             totalRevenue += finalPrice;
64
65             if (tier.equals("Gold")) {

```

```

66         goldCount++;
67     } else if (tier.equals("Silver")) {
68         silverCount++;
69     } else {
70         bronzeCount++;
71     }
72
73     System.out.println("Attendee " + a + ": " + tier + ", " + days + " day(s), base "
74         + baseTotal + ", discount " + percent + "%, pays " + finalPrice);
75 }
76
77 System.out.println("SUMMARY");
78 System.out.println("Total revenue: " + totalRevenue);
79 System.out.println("Gold: " + goldCount + ", Silver: " + silverCount
80     + ", Bronze: " + bronzeCount);
81 }
82 }

```

OUTPUT

```

Attendee 1: Gold, 3 day(s), base 75.0, discount 25.0%, pays 56.25
Attendee 2: Gold, 1 day(s), base 30.0, discount 15.0%, pays 50.0
Attendee 3: Silver, 2 day(s), base 55.0, discount 5.0%, pays 52.25
Attendee 4: Bronze, 1 day(s), base 30.0, discount 0.0%, pays 50.0
Attendee 5: Platinum, 3 day(s), base 75.0, discount 0.0%, pays 75.0
SUMMARY
Total revenue: 283.5
Gold: 2, Silver: 1, Bronze: 2

```

Read attendees 2 and 4: both would have paid less than 50 and both paid exactly 50.0, so **the floor is visibly working**. Read attendee 5: "Platinum" got a 0% discount **and** was counted as Bronze, so the two Bronze rules agree. Input is a Scanner here so the run can be printed; in the hall, every prompt is `JOptionPane.showInputDialog` and the summary is one `showMessageDialog` (F.11).

⚠ TRAP · THE SWITCH MUST RETURN, OR FALL THROUGH

Part 1's method must give back a price for 1, 2 and 3 and **throw** otherwise. Writing `return` inside each case is what makes that safe: a return leaves the method immediately, so no `break` is needed and there is no fall-through to worry about (Unit 4.3).

Use `double price;` with assignments instead and you must not forget the breaks, and the compiler will additionally complain that `price` "might not have been initialized" unless every path assigns it. The default: `throw` is what convinces the compiler every path is covered: an exception is a legitimate way to leave a method that promised to return a double, because it does not return at all.

Note also that `throw` needs no `throws` clause here. `IllegalArgumentException` is **unchecked** (Unit 10.1), which is Q1 B's distinction being examined a second time on the same paper.

The cold-storage log, and the streak that ends at midnight

Q5 B · SUSTAINED BREACHES OF A SAFE LIMIT

13 MARKS

The question. A warehouse records its temperature once an hour for 24 hours. A reading is a **breach** if it is **above 8 degrees C**. Using JOptionPane only:

1. prompt 24 times, one dialog per hour. Each input must parse as a number **between -20 and 50 inclusive**; if invalid, re-prompt for the **same** hour using a nested loop, until valid. Store all 24 in a 1-D array;
2. scan the array **once**, using only loops and conditionals (**no extra arrays**), to find the **number of qualifying streaks** (runs of **3 or more** consecutive breach-hours) and the **length of the longest** breach streak;
3. using nested if-else, classify: **CRITICAL** if the longest streak is ≥ 6 , **WARNING** if it is 3 to 5, **SAFE** if no streak of 3 or more occurred;
4. show one dialog with the streak count, the longest length, and the classification.

● MUST-MEMORISE · COUNT THE RUN WHEN IT TURNS 3, AND MEASURE IT AS YOU GO

The whole of part 2 is three variables and one pass:

currentRun: breach hours in a row right now, reset to 0 on any non-breach.

longestStreak: updated **every breach hour**, not at the end of a run.

qualifyingStreaks: incremented when `currentRun == 3`, **exactly**.

Why == 3 and not ≥ 3 . A run passes through 3 exactly once, on its third hour. So `== 3` counts each qualifying run **once**, however long it becomes. Use `$\geq 3$` and a 6-hour run is counted four times, giving 4 streaks where there is one.

MEMORY HOOK: a run is counted at the moment it qualifies, not while it continues. The threshold is a door you pass through once, not a room you stay in.

⚠ TRAP · THE STREAK THAT IS STILL RUNNING WHEN THE DAY ENDS

The instinct is to close each run in the else, where the non-breach hour is: "the run just ended, so now measure it."

```
if (readings[hour] > 8) {
    currentRun++;
} else {
    if (currentRun > longestStreak) longestStreak = currentRun; // only on a NON-breach
    if (currentRun >= 3) qualifyingStreaks++;
    currentRun = 0;
}
```

It works, until the day ends mid-streak. If hours 18 to 23 are all breaches, the loop finishes with `currentRun = 6` and **never reaches the else**. The longest streak of the day is never measured and never counted. The program reports SAFE on a day the freezer was over temperature for six straight hours.

The fix is not to bolt a copy of the else-block on after the loop. That is the usual patch and it duplicates the logic in two places, so the next edit changes one and forgets the other. Instead, **update on every breach hour**:

Measure `longestStreak` **as the run grows**, and count the run **the instant it turns 3**. Then a run that is still going when the array ends has already been measured and already been counted, and the end of the array needs no special case at all.

This is worth more than one exam question. **A boundary you never treat specially cannot be got wrong at the boundary.**

```

1  import java.util.Scanner;
2
3  public class ColdStorage {
4
5      public static void main(String[] args) {
6          Scanner in = new Scanner(System.in);
7          double[] readings = new double[24];
8
9          // Part i: 24 prompts, each re-prompted until the value is valid.
10         for (int hour = 0; hour < 24; hour++) {
11             boolean valid = false;
12             while (!valid) {
13                 String typed = in.nextLine();
14                 try {
15                     double t = Double.parseDouble(typed);
16                     if (t >= -20 && t <= 50) {
17                         readings[hour] = t;
18                         valid = true;
19                     } else {
20                         System.out.println("  hour " + hour + ": " + t
21                             + " is outside -20 to 50. Asking again.");
22                     }
23                 } catch (NumberFormatException e) {
24                     System.out.println("  hour " + hour + ": \"" + typed
25                         + "\" is not a number. Asking again.");
26                 }
27             }
28         }
29
30         // Part ii: ONE scan, no extra arrays.
31         int currentRun = 0, longestStreak = 0, qualifyingStreaks = 0;
32         for (int hour = 0; hour < 24; hour++) {
33             if (readings[hour] > 8) {
34                 currentRun++;
35                 if (currentRun > longestStreak) {
36                     longestStreak = currentRun;
37                 }
38                 if (currentRun == 3) {
39                     qualifyingStreaks++;
40                 }
41             } else {
42                 currentRun = 0;
43             }
44         }
45
46         // Part iii: classify.
47         String classification;
48         if (longestStreak >= 6) {
49             classification = "CRITICAL";
50         } else if (longestStreak >= 3) {
51             classification = "WARNING";
52         } else {
53             classification = "SAFE";
54         }
55
56         // Part iv: one dialog.
57         String message = "Qualifying streaks (3+ hours): " + qualifyingStreaks
58             + "\nLongest breach streak: " + longestStreak + " hour(s)"
59             + "\nClassification: " + classification;
60         System.out.println(message);
61     }
62 }

```

OUTPUT

```
hour 0: 99.0 is outside -20 to 50. Asking again.
hour 0: "abc" is not a number. Asking again.
Qualifying streaks (3+ hours): 3
Longest breach streak: 6 hour(s)
Classification: CRITICAL
```

The run fed hour 0 a 99, then the text "abc", then a valid 5, so both halves of "must parse as a number between -20 and 50" are shown rejecting input and re-prompting **the same hour**.

DRY RUN · THE 24 HOURS, AND WHERE THE THREE STREAKS ARE

BREACH = ABOVE 8. THE BAR SHOWS BREACH HOURS.

HOURS	READINGS	BREACH RUN	COUNTED?
0 to 3	5, 9, 10, 4	2 in a row	no: under 3
4 to 7	9, 9, 9, 2	3 in a row	yes, streak 1
8 to 13	12, 11, 9, 9, 10, 3	5 in a row	yes, streak 2
14 to 17	1, 2, 3, 4	none	no
18 to 23	9, 9, 9, 9, 9, 9	6 in a row, to the last hour	yes, streak 3
Answers		longest = 6	3 streaks, CRITICAL

The first run of 2 is the control: it proves the counter is not simply counting runs. The last run is the trap above, live: it reaches hour 23 and stops because the day stopped, not because the temperature did. It is both the longest streak and the one that decides the classification, and the else-branch version would have missed it entirely and printed WARNING.

S.7 · 2025/26 · QUESTION SIX · 25 MARKS

Question Six, worked in full

Q6 A · CLASSIFY A NUMBER BY ITS DIGIT PATTERN

12 MARKS

The question. Using JOptionPane only, classify one positive integer N. **Constraints:** everything inside main(), using loops and nested/chained if-else and/or switch. **No arrays and no user-defined helper methods.** Everything inline.

- in **one loop**, built only on % and / (no converting N to a String or array), compute **both** the digit sum of N and N with its digits **reversed**;
- N is a palindrome if it equals its reversed value;
- inline, decide whether the **digit sum** is prime, testing divisibility only up to its **square root**;
- classify, strictly in this priority order: **PALINDROME-PRIME-SUM** (palindrome and sum prime), **PALINDROME** (palindrome, sum not prime), **HARSHAD** (not a palindrome, but N divides exactly by its digit sum), **STANDARD** otherwise;
- show N, its digit sum, its reversed value and its classification in a **single** dialog.

● MUST-MEMORISE · THE DIGIT LOOP, AND THE COPY THAT SAVES IT

Peeling digits off a number is three lines you should be able to write without thinking:

```
int digit = temp % 10; takes the last digit
reversed = reversed * 10 + digit; pushes it onto the right of the answer
temp = temp / 10; drops the last digit
Loop while (temp > 0). Both answers fall out of one pass.
```

The line the marks hang on is `int temp = n`; The loop destroys whatever it walks: `temp` ends at 0. Walk `n` itself and by step 2 there is nothing left to compare the reversed value against, and nothing left to divide in step 4. **Copy it, consume the copy.**

`reversed * 10 + digit` is worth a moment. Take 4821: the loop pulls 1, then 2, then 8, then 4. Start at 0, and each step shifts what you have one place left and drops the new digit into the units: 1, then 12, then 128, then 1284. The digits come off backwards, which is exactly why pushing them on in the order they arrive reverses the number.

MEMORY HOOK: % takes the last digit, / removes it. Every digit problem in this course is those two lines and a loop.

TEACH · THE SQUARE ROOT TEST, WITHOUT MATH.SQRT

Part 3 says test divisibility only up to the square root of the digit sum. The clean way needs no square root at all:

```
boolean sumIsPrime = (digitSum >= 2);
for (int f = 2; f * f <= digitSum; f++) {
    if (digitSum % f == 0) { sumIsPrime = false; break; }
}
```

`f * f <= digitSum` is "f is at most the square root of digitSum", squared on both sides. It stays in whole numbers, so there is no `Math.sqrt`, no double, and no floating-point comparison to get wrong at the boundary (Unit 3.6).

Why stopping at the square root is correct, not just faster. If `digitSum = a * b`, then `a` and `b` cannot both be above the square root, or their product would exceed `digitSum`. So every factor pairs with one at or below the square root. Find none there and there are none at all.

And the initialiser is the whole edge case. `digitSum >= 2` is what makes **1 not prime** and **0 not prime**. The loop cannot help you: for `digitSum = 1` it never runs, so whatever you started with survives. Start at true and the program declares 1 prime, and then `N = 1` (a palindrome with digit sum 1) is misclassified. See the table below, last row.

```

1  import java.util.Scanner;
2
3  public class PalindromeClassifier {
4      public static void main(String[] args) {
5          Scanner in = new Scanner(System.in);
6
7          int n = Integer.parseInt(in.nextLine());
8
9          // Step i: ONE loop, using only % and /, computing BOTH the digit sum and
10         // the reversed value. temp is consumed; n must survive for step ii.
11         int temp = n;
12         int digitSum = 0;
13         int reversed = 0;
14         while (temp > 0) {
15             int digit = temp % 10;
16             digitSum = digitSum + digit;
17             reversed = reversed * 10 + digit;
18             temp = temp / 10;
19         }
20
21         // Step ii: palindrome is a comparison against the ORIGINAL n.
22         boolean isPalindrome = (n == reversed);
23
24         // Step iii: is digitSum prime? Test factors only up to its square root.
25         boolean sumIsPrime = (digitSum >= 2);
26         for (int f = 2; f * f <= digitSum; f++) {
27             if (digitSum % f == 0) {
28                 sumIsPrime = false;
29                 break;
30             }
31         }
32
33         // Step iv: classify, in the paper's priority order.
34         String classification;
35         if (isPalindrome) {
36             if (sumIsPrime) {
37                 classification = "PALINDROME-PRIME-SUM";
38             } else {
39                 classification = "PALINDROME";
40             }
41         } else if (n % digitSum == 0) {
42             classification = "HARSHAD";
43         } else {
44             classification = "STANDARD";
45         }
46
47         // Step v: one dialog with all four facts.
48         String message = "N: " + n
49             + "\nDigit sum: " + digitSum
50             + "\nReversed: " + reversed
51             + "\nClassification: " + classification;
52         System.out.println(message);
53     }
54 }

```

OUTPUT

```

N: 4884
Digit sum: 24
Reversed: 4884
Classification: PALINDROME

```

4884 is the paper's own example. It is a palindrome, and $24 = 2 \times 12$ is not prime, so it is PALINDROME and not PALINDROME-PRIME-SUM. That is the priority order doing its job.

This is **not** the exam answer: the question forbids helper methods and demands a single N. It is the identical block of statements with a loop around it, so that every row below is real output rather than someone's arithmetic.

```

1  import java.util.Scanner;
2
3  // The SAME statements as the answer, with a loop around them, so that every row
4  // of the table in this book is real output rather than someone's arithmetic.
5  public class ClassifierBench {
6      public static void main(String[] args) {
7          Scanner in = new Scanner(System.in);
8          System.out.println("    N | sum | reversed | classification");
9          while (in.hasNextLine()) {
10             String line = in.nextLine();
11             if (line.isEmpty()) {
12                 continue;
13             }
14             int n = Integer.parseInt(line);
15
16             int temp = n, digitSum = 0, reversed = 0;
17             while (temp > 0) {
18                 int digit = temp % 10;
19                 digitSum = digitSum + digit;
20                 reversed = reversed * 10 + digit;
21                 temp = temp / 10;
22             }
23
24             boolean isPalindrome = (n == reversed);
25
26             boolean sumIsPrime = (digitSum >= 2);
27             for (int f = 2; f * f <= digitSum; f++) {
28                 if (digitSum % f == 0) {
29                     sumIsPrime = false;
30                     break;
31                 }
32             }
33
34             String classification;
35             if (isPalindrome) {
36                 classification = sumIsPrime ? "PALINDROME-PRIME-SUM" : "PALINDROME";
37             } else if (n % digitSum == 0) {
38                 classification = "HARSHAD";
39             } else {
40                 classification = "STANDARD";
41             }
42
43             System.out.printf("%6d | %3d | %8d | %s\n", n, digitSum, reversed, classification);
44         }
45     }
46 }

```

OUTPUT

N	sum	reversed	classification
4884	24	4884	PALINDROME
7	7	7	PALINDROME-PRIME-SUM
4821	15	1284	STANDARD
21	3	12	HARSHAD
131	5	131	PALINDROME-PRIME-SUM
1	1	1	PALINDROME

Six rows, and every branch of part 4 is exercised. **4884** and **4821** are the paper's own two examples and both come out as the paper says. **7** is its one-digit palindrome. **4821**: the digit sum is 15 and $4821 = 15 \times 321 + 6$, so it is not a Harshad number either, and it falls to STANDARD. **1** is the row worth staring at: a palindrome whose digit sum is 1,

and 1 is **not** prime, so PALINDROME. Get the initialiser wrong and this row alone would read PALINDROME-PRIME-SUM.

⚠ TRAP · "PRIORITY ORDER" MEANS ELSE-IF, NOT FOUR IFs

Part 4 says **checked strictly in this priority order** and **exactly one** category. Both phrases point at the same structure: one chain, where reaching a test means every test above it already failed.

Write four independent ifs and a palindrome with a prime digit sum matches the first, then falls into the others too, and your single dialog reports whichever assignment landed last. The chain is what makes "exactly one" true, and it is also what makes HARSHAD's rule readable: **HARSHAD never has to say "and is not a palindrome"**, because it is only ever reached when the palindrome test failed. The order does that work, which is precisely why the question fixes it (Unit 4.2).

Watch `n % digitSum` in the HARSHAD test. It is safe here only because `N` is a **positive** integer, so `digitSum` is at least 1 and never 0. Take that guarantee away and it throws `ArithmeticException` (Unit 3.6): a division by zero hiding in a remainder.

S.7 · 2025/26 · QUESTION SIX B

The parking garage, and three counts from one method

Q6 B · PARKING FEES FOR A BATCH OF VEHICLES

13 MARKS

The question. Using `JOptionPane` only, process vehicles exiting on one day and total the fees. **Clear separation of concerns: each task must be its own method. No collection classes (e.g. `ArrayList`) anywhere.**

1. prompt for the number of vehicles `V`, then fill two **parallel arrays** of length `V`: hours parked (int per vehicle; a value `<= 0` is invalid, so use a **nested control statement** to re-prompt until valid) and permit type (String per vehicle, "Staff" or "Visitor");
2. a method taking one vehicle's hours and permit, computing its fee with **nested if-else**: **Staff**, first 2 hours free, each additional hour 1.00, capped at 10.00/day; **Visitor**, no free period, first 3 hours a flat 6.00, each hour beyond 3 costs 2.50, capped at 25.00/day; and **any permit that is not exactly "Staff" is treated as Visitor**;
3. a method taking the hours array, tallying **SHORT-STAY** (1-2), **STANDARD** (3-6) and **EXTENDED** (>6), returning the three counts **without any collection class**;
4. `main()`: call (ii) for every vehicle across both arrays to accumulate revenue, call (iii) once, and display **one** dialog with the total and the three counts.

● MUST-MEMORISE · THREE COUNTS, ONE RETURN, NO COLLECTIONS

Part 3 asks for **three** numbers back. A Java method returns **one** value. And `ArrayList` is banned. So return an `int[]` of length 3.

An array is not a collection class. That distinction is the question: "collection classes (e.g. `ArrayList`)" means `java.util`'s List/Set/Map family. An array is a built-in Java type, not a class you import, which is why F.10 and Module 8 exist and why the ban costs you nothing.

Document the slots in a comment (`[0]` short, `[1]` standard, `[2]` extended) and use them in that order. The alternative the ban is steering you away from is three separate fields or three methods walking the same array three times.

MEMORY HOOK: one return value, several numbers, no collections: that is an array.

TEACH · PARALLEL ARRAYS, AND THE INVARIANT THAT KEEPS THEM HONEST

Two arrays are **parallel** when index i in one and index i in the other describe the same thing. `hoursParked[3]` and `permitType[3]` are one car.

The whole idea rests on one invariant: **they are the same length and they are always written together**. The loop in part 1 fills both in the same iteration, which is what keeps them aligned. Fill them in two separate loops, or continue past one but not the other, and vehicle 3's hours are paired with vehicle 4's permit. Nothing throws. Every fee is simply wrong.

In Module 6's terms this is what a class is for: a `Vehicle` object holding hours and permit together cannot fall out of step, because there is no "step" to fall out of. The paper asks for parallel arrays, so write parallel arrays; but knowing what they are the poor relation *of* is what Q4 was testing.

```

1  import java.util.Scanner;
2
3  public class ParkingGarage {
4
5      // Part ii: one vehicle's fee, by NESTED if-else. The permit decides the branch;
6      // the hours are only examined inside it.
7      static double feeFor(int hours, String permit) {
8          double fee;
9          if (permit.equals("Staff")) {
10             if (hours <= 2) {
11                 fee = 0.0;
12             } else {
13                 fee = (hours - 2) * 1.00;
14                 if (fee > 10.00) {
15                     fee = 10.00;
16                 }
17             }
18         } else {
19             if (hours <= 3) {
20                 fee = 6.00;
21             } else {
22                 fee = 6.00 + (hours - 3) * 2.50;
23                 if (fee > 25.00) {
24                     fee = 25.00;
25                 }
26             }
27         }
28         return fee;
29     }
30
31     // Part iii: tally the three categories. Three counts, one return value, and no
32     // collection class: an int[] of length 3. An array is not a collection.
33     // [0] SHORT-STAY (1-2), [1] STANDARD (3-6), [2] EXTENDED (>6)
34     static int[] tally(int[] hoursParked) {
35         int[] counts = new int[3];
36         for (int i = 0; i < hoursParked.length; i++) {
37             if (hoursParked[i] <= 2) {
38                 counts[0]++;
39             } else if (hoursParked[i] <= 6) {
40                 counts[1]++;
41             } else {
42                 counts[2]++;
43             }
44         }
45         return counts;
46     }
47
48     public static void main(String[] args) {
49         Scanner in = new Scanner(System.in);
50
51         int vehicles = Integer.parseInt(in.nextLine());
52
53         // Part i: two PARALLEL arrays. Index i is one vehicle in both.
54         int[] hoursParked = new int[vehicles];
55         String[] permitType = new String[vehicles];
56
57         for (int i = 0; i < vehicles; i++) {
58             int hours = Integer.parseInt(in.nextLine());
59             while (hours <= 0) {
60                 System.out.println("  vehicle " + i + ": " + hours
61                     + " is not a valid number of hours. Asking again.");
62                 hours = Integer.parseInt(in.nextLine());
63             }

```

```

64         hoursParked[i] = hours;
65         permitType[i] = in.nextLine();
66     }
67
68     // Part iv: accumulate revenue across both arrays, then tally once.
69     double totalRevenue = 0;
70     for (int i = 0; i < vehicles; i++) {
71         double fee = feeFor(hoursParked[i], permitType[i]);
72         totalRevenue += fee;
73         System.out.println("Vehicle " + i + ": " + hoursParked[i] + "h, "
74             + permitType[i] + " -> " + fee);
75     }
76
77     int[] counts = tally(hoursParked);
78
79     String message = "Total revenue: " + totalRevenue
80         + "\nSHORT-STAY (1-2 hrs): " + counts[0]
81         + "\nSTANDARD (3-6 hrs): " + counts[1]
82         + "\nEXTENDED (>6 hrs): " + counts[2];
83     System.out.println(message);
84 }
85 }

```

OUTPUT

```

vehicle 0: 0 is not a valid number of hours. Asking again.
Vehicle 0: 2h, Staff -> 0.0
Vehicle 1: 5h, Staff -> 3.0
Vehicle 2: 15h, Staff -> 10.0
Vehicle 3: 1h, Visitor -> 6.0
Vehicle 4: 5h, Visitor -> 11.0
Vehicle 5: 12h, Guest -> 25.0
Total revenue: 55.0
SHORT-STAY (1-2 hrs): 2
STANDARD (3-6 hrs): 2
EXTENDED (>6 hrs): 2

```

DRY RUN · EVERY RULE IN THE FEE TABLE, FIRED ONCE

HOURS	PERMIT	WHICH RULE, AND THE ARITHMETIC	FEE
2	Staff	first 2 hours free, and 2 is not beyond 2	0.00
5	Staff	$(5 - 2) \times 1.00 = 3.00$, under the 10.00 cap	3.00
15	Staff	$(15 - 2) \times 1.00 = 13.00$, over the cap	10.00
1	Visitor	flat 6.00 for the first 3 hours, even for 1	6.00
5	Visitor	$6.00 + (5 - 3) \times 2.50 = 11.00$	11.00
12	Guest	not exactly "Staff", so Visitor: $6.00 + 9 \times 2.50 = 28.50$, over the cap	25.00
Total revenue			55.00

Every rule in the question fires exactly once in that table, which is how the test data was chosen: both free periods, both per-hour rates, **both** caps, and the not-Staff-means-Visitor rule. When you invent your own test data, invent it like this: one row per rule, so a single run proves the lot.

Two rows repay a second look. **1 hour as a Visitor costs 6.00**: "first 3 hours cost a flat 6.00" is a flat fee, not a rate, so an hour and three hours cost the same, and a Visitor pays more than a Staff member who stayed fifteen. That is not a bug, it is the pricing the question specifies. And **Guest at 12 hours** is the only row testing the fallback rule; delete it from your test data and a program that crashes on unknown permits would look perfectly healthy.

● MUST-MEMORISE · A CAP IS A CEILING YOU APPLY AFTER, NOT A RULE YOU FOLD IN

Compute the fee, **then** clamp it:

```
fee = 6.00 + (hours - 3) * 2.50;  
if (fee > 25.00) { fee = 25.00; }
```

The temptation is to work out the hour at which the cap bites (for Visitors, 10.6 hours) and branch on **that**. It gives the same answers and it is a worse answer: you have hidden a magic number that no line of the question contains, and changing the rate to 3.00 silently breaks it, because the crossover moves and your constant does not.

This is the same shape as Q5 A's price floor, which is a cap from below. Both read as one line placed after the sum, and both then say plainly what the question said: **capped at a maximum of 25.00/day**.

MEMORY HOOK: compute, then clamp. Never solve for where the clamp starts.

S.8 · 2024/25 · HOW THIS PAPER WORKS

The 2024/25 paper, and why you are reading it

A different shape, the same four skills, and a title that changed while the subject did not.

● MUST-MEMORISE · THE FORMAT CHANGED; THE SKILLS DID NOT

2024/25: six questions at 17.5 marks, **attempt any four**. No sections.

2025/26: three sections, **one question from each**, 20 + 25 + 25.

Both are **70 marks over 3 hours**. Both test exactly four things: **define, debug, dry run, write a program**.

The course was even retitled between the two: "Object-Oriented Programming (Java)" became "Computer Programming I (Java)". Same code, COS221, same lecturer, same syllabus.

So do not learn a paper. Learn the four skills. A format can change again between the writing of this book and the day you sit down; the four things being tested have not changed in two years and will not.

MEMORY HOOK: define, debug, dry run, write. Every question on both papers is one of those four, or a bundle of all four.

TEACH · THE 24/25 TEMPLATE, QUESTION BY QUESTION

Each of the six questions is the same four parts, in the same order, for the same marks. Recognising this is worth as much as any single answer:

PART	MARKS	WHAT IT ALWAYS IS
A	3	Define or describe three terms. One mark each, one sentence each.
B	3	Debug: list and correct the errors in a snippet.
C	4	Dry run: state the output, sometimes per iteration.
D	7.5	Write a program to a specification.

Q5 and Q6 shuffle the order slightly (Q5 B asks you to write a method; Q6 A asks for loop snippets), but the marks are identical and so is the intent.

This book answers all six. The paper says attempt four, and in the hall you will. But a question you skip is a topic you can be examined on next year, and the 25/26 paper proves it: it took this paper's file I/O, its inheritance and its digit-loops and asked them again in a different costume.

Question One, worked in full

Q1 A · DEFINE METHOD SIGNATURE, OVERLOADING, AND VOID METHOD

3 MARKS

Model answer. One mark each, and each is one sentence.

1. **Method signature.** A method's **name together with its parameter list** (the number, types and order of its parameters). The return type is **not** part of the signature. It is what the compiler uses to tell one method from another.
2. **Method overloading.** Declaring more than one method with the **same name** but **different parameter lists** in the same class, so the compiler chooses between them at compile time by matching the arguments at the call site against each signature.
3. **Void method.** A method declared void, meaning it **returns no value**. It is called as a statement on its own, cannot be used in an expression, and may use a bare return; to exit early.

Where the marks are: the examiner asked for signature and overloading in the same breath deliberately, because the second is defined by the first. Say "signature = name + parameter list, return type excluded" and the overloading definition writes itself. Units 5.1, 5.2 and 5.6.

Q1 B · DEBUG SQUARECALC

3 MARKS

The code as printed:

```

1 public class SquareCalcBroken {
2     public static void main(String[] ) {
3         int num = 5;
4         double result = square(num)
5         System.out.println("Square is: " + result);
6     }
7
8     public static int square(int) {
9         return n * n;

```

COMPILER OUTPUT

```
error: <identifier> expected
```

The errors, listed. The question says **list and correct**, so list them: a correction with no list is half an answer.

1. **Line 2: main(String[]) has no parameter name.** A parameter needs a type **and** a name. Fix: main(String[] args).
2. **Line 4: no semicolon** after double result = square(num).
3. **Line 8: square(int) has no parameter name.** Fix: square(int n).
4. **Line 9: n does not exist.** It is the symptom of error 3: name the parameter and n exists. Worth saying out loud in your answer, because it shows you understand the cause rather than listing every red squiggle.
5. **The braces are missing** for both square and the class.

Three marks, five errors. Do not stop at three: the marker is looking for the **set**, and errors 3 and 4 are one fault with two faces.

```

1 public class SquareCalc {
2     public static void main(String[] args) {
3         int num = 5;
4         double result = square(num);
5         System.out.println("Square is: " + result);
6     }
7
8     public static int square(int n) {
9         return n * n;
10    }
11 }

```

OUTPUT

Square is: 25.0

Look hard at that output: 25.0, not 25. square returns an int, and 25 is stored into a double result. Java **widens** the int to a double on the way in (Unit 5.7), so what gets printed is a double, and a double always prints its point. Nothing is wrong; but if you wrote "Square is: 25" you have missed a mark that costs nothing to get right.

Q1 C · DRY RUN SUMSERIES WITH 3, 4, 5, 7

4 MARKS

The paper gives you this and four values to trace:

```

public static int sumSeries(int i) {
    int sum = 0;
    for (int j = 1; j <= i; j++) {
        sum += j / (j + 1);
    }
    return sum;
}

```

The answer is 0. For all four values.

● MUST-MEMORISE · FOUR VALUES, ONE ANSWER, AND THAT IS THE QUESTION

j and j + 1 are both int, so j / (j + 1) is **integer division**.

j is always **smaller** than j + 1. A smaller int divided by a larger int is **always 0**. Every term is 0. The sum is 0.

1/2 is 0. 2/3 is 0. 3/4 is 0. Not 0.5, 0.67, 0.75: **0**, because integer division throws the remainder away (Unit 3.6). It does not round; it truncates.

Why the examiner gave you four values. Being handed 3, 4, 5 and 7 shouts "four different answers". It is bait. The four values are there so that you either notice the types once and answer all four in one line, or trace fractions by hand for ten minutes and write four confident wrong numbers.

MEMORY HOOK: int / int is an int. Before you trace any division, look at the two types. One double anywhere on top and the answer changes completely: 1.0 / 2 is 0.5.

```

1 public class SumSeriesTrace {
2
3     // exactly as the paper prints it
4     public static int sumSeries(int i) {
5         int sum = 0;
6         for (int j = 1; j <= i; j++) {
7             sum += j / (j + 1);
8         }
9         return sum;
10    }
11
12    public static void main(String[] args) {
13        System.out.println("sumSeries(3) = " + sumSeries(3));
14        System.out.println("sumSeries(4) = " + sumSeries(4));
15        System.out.println("sumSeries(5) = " + sumSeries(5));
16        System.out.println("sumSeries(7) = " + sumSeries(7));
17        System.out.println("and the terms themselves: 1/2 = " + (1 / 2)
18            + ", 2/3 = " + (2 / 3) + ", 3/4 = " + (3 / 4));
19        System.out.println("with a double on top:    1.0/2 = " + (1.0 / 2));
20    }
21 }

```

OUTPUT

```

sumSeries(3) = 0
sumSeries(4) = 0
sumSeries(5) = 0
sumSeries(7) = 0
and the terms themselves: 1/2 = 0, 2/3 = 0, 3/4 = 0
with a double on top:    1.0/2 = 0.5

```

Four inputs, one answer. The last two lines are the whole explanation: every term really is 0, and the single change that would rescue the method is a double on top.

DRY RUN · HOW TO WRITE THIS UP FOR 4 MARKS

Do not just write "0" four times. Show the trace for one value and generalise; that is what the marks are for.

SUMSERIES(3): THE LOOP RUNS FOR $j = 1, 2, 3$

J	J / (J + 1)	IN REAL ARITHMETIC	AS INT DIVISION	SUM
1	1 / 2	0.5	0	0
2	2 / 3	0.667	0	0
3	3 / 4	0.75	0	0

Then one sentence: **"for any i, every term $j/(j+1)$ has $j < j+1$, so integer division gives 0 every time; therefore sumSeries returns 0 for 3, 4, 5 and 7 alike."** That sentence is the answer to all four values, and it is worth more than four traced tables.

Q1 D · OVERLOAD AREA FOR CIRCLE, RECTANGLE AND TRIANGLE

7.5 MARKS

The question. Write a program that *overloads* a method named area to compute (i) the area of a circle, (ii) the area of a rectangle, (iii) the area of a triangle, and call them from main.

Circle needs one number. Rectangle needs two. Triangle needs two.

So rectangle is `area(double, double)`, and triangle (base and height) is **also** `area(double, double)`.

That is the same signature. It will not compile.

Overloading needs different **parameter lists**, and "base, height" is not different from "length, width": the parameter *names* are not part of the signature, only the **types** (Q1 A, which you just answered).

Write both and javac says:

error: method `area(double,double)` is already defined in class

The resolution: give the triangle a **three-number** formula, so its signature is `area(double, double, double)` and distinct. Heron's formula takes the three sides:

$$s = (a + b + c) / 2 \quad \text{and} \quad \text{area} = \sqrt{s(s-a)(s-b)(s-c)}$$

MEMORY HOOK: overloads differ by types, not by names. If two of your methods take the same types in the same order, you have one method and a compiler error.

This is the single most valuable thing in Question One, and the examiner may not have noticed it. Say it explicitly in your answer: "base and height would collide with length and width, so the triangle is done by Heron's formula on three sides, giving a distinct signature." That sentence proves you understand overloading, which is what the question is for.

CODE · Q1 D, COMPLETE

VERIFIED ON THE JVM

```

1 public class AreaOverload {
2
3     // i. circle: one number, so the signature is area(double)
4     public static double area(double radius) {
5         return Math.PI * radius * radius;
6     }
7
8     // ii. rectangle: two numbers, so the signature is area(double, double)
9     public static double area(double length, double width) {
10        return length * width;
11    }
12
13    // iii. triangle: base and height would ALSO be area(double, double), which is
14    // the rectangle's signature and will not compile. Three sides gives a distinct
15    // signature and a genuine formula: Heron's.
16    public static double area(double sideA, double sideB, double sideC) {
17        double s = (sideA + sideB + sideC) / 2;
18        return Math.sqrt(s * (s - sideA) * (s - sideB) * (s - sideC));
19    }
20
21    public static void main(String[] args) {
22        System.out.println("Circle, radius 7      : " + area(7.0));
23        System.out.println("Rectangle, 5 by 3      : " + area(5.0, 3.0));
24        System.out.println("Triangle, sides 3 4 5 : " + area(3.0, 4.0, 5.0));
25    }
26 }

```

OUTPUT

```

Circle, radius 7      : 153.93804002589985
Rectangle, 5 by 3      : 15.0
Triangle, sides 3 4 5 : 6.0

```

Check the triangle without trusting Heron. 3, 4, 5 is a right triangle, so its area must be half of 3×4 , which is 6. Heron gave exactly 6.0. When you use a formula you cannot see through, feed it a case whose answer you already know. `Math.PI` and `Math.sqrt` are Unit 3.6.

Question Two, worked in full

Q2 A · DEFINE DYNAMIC ARRAY INITIALISATION, LENGTH, AND THE OUT-OF-BOUNDS EXCEPTION

3 MARKS

Model answer.

1. **Dynamic array initialisation.** Creating the array with `new` and a size decided **while the program runs**, rather than from a literal list written in the source: `int[] a = new int[n];` where `n` is only known at run time. Every element starts at its default (0 for `int`, `null` for objects), and the values are filled in afterwards. Contrast static initialisation, `int[] a = {2, 4, 6, 8};`, where the size and the values are both fixed in the source.
2. **The length property.** `a.length` gives the number of elements. It is a **field, not a method**, so there are no brackets: writing `a.length()` is a compile error. It is **final**: an array's length is fixed the moment it is created and can never change. A `String`'s length IS a method, `s.length()`. The examiner knows that catches people; Unit 7.2.
3. **The out-of-bounds exception.** `ArrayIndexOutOfBoundsException`, thrown at **run time** when you use an index below 0 or at or above length. Valid indexes are **0 to length - 1**, so `a[a.length]` always throws. It is **unchecked**, so the compiler will not warn you about it.

Where the marks are: "length is a field, not a method" and "indexes are 0 to length-1" are the two facts every other part of this question depends on. F.10 and Unit 8.1.

Q2 B · DEBUG COPYARRAY

3 MARKS

The code as printed:

```

1 public class CopyArrayBroken {
2     public static void main(String[] args) {
3         int[] a = {2, 4, 6, 8};
4         int b;
5         System.arraycopy(a, 0, b, 0, a.length);
6         for(int i = 0; i <= b.length; i) {
7             System.out.println(b[i]);
8         }
9     }
10 }
```

COMPILER OUTPUT

```
error: not a statement
```

The errors, listed.

1. **int b; is not an array.** It is a single `int`, and it is never given a value. `System.arraycopy` needs a **destination array that already exists and has room**: it copies into an array, it does not create one. Fix: `int[] b = new int[a.length];` This is Q2 A part 1 being examined: dynamic initialisation, sized from `a.length`.
2. **System.arraycopy has a capital C.** The method is `System.arraycopy`, all lower case. Java is case sensitive, so this is not a typo the compiler forgives: it is a method that does not exist.
3. **i <= b.length is off by one.** The last index is `length - 1`, so this reads `b[4]` on an array of 4 and throws `ArrayIndexOutOfBoundsException`. Fix: `i < b.length`. Q2 A part 3, being examined.
4. **i is not an increment.** The third slot of the `for` needs `i++`. As written it is not even a statement, which is the error `javac` reports first. Without it the loop would never end.

Notice what this question is. Its three errors are its own part A, in order: dynamic initialisation, the length property, the out-of-bounds exception. Part A is not a warm-up, it is the answer key to part B.

CODE · Q2 B CORRECTED

VERIFIED ON THE JVM

```

1 public class CopyArray {
2     public static void main(String[] args) {
3         int[] a = {2, 4, 6, 8};
4         int[] b = new int[a.length];
5         System.arraycopy(a, 0, b, 0, a.length);
6         for (int i = 0; i < b.length; i++) {
7             System.out.println(b[i]);
8         }
9     }
10 }

```

OUTPUT

```

2
4
6
8

```

System.arraycopy(a, 0, b, 0, a.length) reads: from a starting at 0, into b starting at 0, a.length elements. Source, source position, destination, destination position, count. Unit 8.3.

Q2 C · DRY RUN THE SUBTRACTION LOOP

4 MARKS

The question asks for **the output of each iteration and the final output**, which is a specific instruction: it wants the table, not just the number.

DRY RUN · $x = x - \text{NUMS}[i]$, STARTING FROM NUMS[0]

NUMS = {2, 4, 6, 8}, AND X STARTS AT NUMS[0] = 2

i	NUMS[i]	$x = x - \text{NUMS}[i]$	x AFTER
-	-	$x = \text{nums}[0]$	2
1	4	$2 - 4$	-2
2	6	$-2 - 6$	-8
3	8	$-8 - 8$	-16
Final output			-16

Two things the marker is watching for. First, **the loop starts at $i = 1$, not 0**, because x was already primed with nums[0]; start at 0 and you subtract 2 from itself and get -18. Second, **the loop prints nothing**: the println is outside it, so "the output of each iteration" is a trace of x, and the only thing the program actually prints is -16. Say both.

This is the "prime with the first element, then start the loop at 1" idiom you use for maximum and minimum (Unit 8.4). Here it is used for a running subtraction, which is the same shape with a different operator.

```
1 public class SubtractDown {
2     public static void main(String[] args) {
3         int[] nums = {2, 4, 6, 8};
4         int x = nums[0];
5         for (int i = 1; i < nums.length; i++) {
6             x = x - nums[i];
7         }
8         System.out.println(x);
9     }
10 }
```

OUTPUT

-16

Q2 D · TEN VALUES, EVENS, ODD SUM, AND THE MAXIMUM

7.5 MARKS

The question. Prompt for 10 values into an array. Write a method that **finds, counts and displays** the even numbers, and another that **sums and displays** the odd numbers. Then, **in main, use a for loop** to find and display the maximum.

Read the verbs: **find, count, display** is three things, and the even method must do all three. And the maximum must be found **in main with a for loop**, not in a third method: the question says where to put it.

```
1 import java.util.Scanner;
2
3 public class ArrayStats {
4     // ii a. find, count and display the even numbers
5     public static void evens(int[] values) {
6         int count = 0;
7         String found = "";
8         for (int i = 0; i < values.length; i++) {
9             if (values[i] % 2 == 0) {
10                 count++;
11                 found = found.equals("") ? "" + values[i] : found + ", " + values[i];
12             }
13         }
14         System.out.println("Even numbers: " + found);
15         System.out.println("How many even: " + count);
16     }
17
18     // ii b. sum and display the odd numbers
19     public static void odds(int[] values) {
20         int sum = 0;
21         for (int i = 0; i < values.length; i++) {
22             if (values[i] % 2 != 0) {
23                 sum += values[i];
24             }
25         }
26         System.out.println("Sum of the odd numbers: " + sum);
27     }
28
29     public static void main(String[] args) {
30         Scanner in = new Scanner(System.in);
31
32         // i. prompt for 10 values
33         int[] values = new int[10];
34         for (int i = 0; i < values.length; i++) {
35             values[i] = Integer.parseInt(in.nextLine());
36         }
37
38         evens(values);
39         odds(values);
40
41         // iii. a for loop in main to find and display the maximum
42         int max = values[0];
43         for (int i = 1; i < values.length; i++) {
44             if (values[i] > max) {
45                 max = values[i];
46             }
47         }
48         System.out.println("Maximum: " + max);
49     }
50 }
51 }
```

OUTPUT

```
Even numbers: 4, 12, 0, -8, 6
How many even: 5
Sum of the odd numbers: 39
Maximum: 15
```

Input is a Scanner here so the run can be printed. The ten values fed in were 4, 7, -3, 12, 9, 0, 15, -8, 6, 11.

● MUST-MEMORISE · TEST FOR ODD WITH `!= 0`, NEVER `== 1`

`if (v % 2 != 0)` is odd. **Always.**

`if (v % 2 == 1)` is odd **only for positives**, and silently wrong for negatives.

In Java, `%` keeps the sign of the **left** operand, so `-3 % 2` is **-1**, not 1. So `-3 % 2 == 1` is false and the `== 1` test declares -3 even.

The test data above proves it. The odd numbers are 7, -3, 9, 15 and 11, and they sum to **39**. Had the method used `== 1`, it would have skipped -3 and printed **42**: a wrong answer, from a program that runs cleanly and looks right. That is why the ten values include a negative odd number, and why yours should too. Unit 3.6 and Unit 8.4.

MEMORY HOOK: "not even" is safe; "equals one" is not.

TEACH · PRIMING THE MAXIMUM: WHY NOT START AT 0

The maximum loop starts `int max = values[0];` and then runs from `i = 1`. The tempting alternative is `int max = 0;`, and it is a bug waiting for the right data.

If every value is negative, `max = 0` never gets replaced and the program reports a maximum of **0**, a number that **is not in the array**. Priming with `values[0]` cannot do that: the answer starts as a real element and only ever moves to another real element.

The same reasoning as Unit 8.4's minimum, and the same reasoning that makes `Integer.MIN_VALUE` the other correct prime. Starting at 0 is correct only if you know the data contains a non-negative number, and "I know my data" is how bugs are born.

S.11 · 2024/25 · QUESTION THREE · 17.5 MARKS

Question Three, worked in full

Q3 A · DEFINE `PRINTWRITER`, `FILES.WRITE()`, AND `TRY-WITH-RESOURCES`

3 MARKS

Model answer.

1. **PrintWriter.** A character-output class that writes **formatted text** to a file or stream, giving you the same familiar methods as `System.out`: `print`, `println` and `printf`. It is a wrapper: you hand it a destination (a file name, or another `Writer`) and it adds the formatting.
2. **Files.write().** A static method on `java.nio.file.Files` that writes an entire byte array or collection of lines to a file **in a single call**, creating the file if needed and closing it for you. It is the one-line alternative to opening a writer, looping, and closing.
3. **try-with-resources.** A try with the resource declared in brackets, `try (PrintWriter out = ...) { ... }`, which **automatically closes** that resource when the block ends, whether it ends normally or by an exception. It removes the need for a `finally` block whose only job is closing. Unit 10.2.

Where the marks are: for each, say **what it is** and **what it saves you**. "PrintWriter writes formatted text" earns the mark; "PrintWriter is for files" does not distinguish it from `FileWriter`.

The code as printed:

```

1 java.io.FileWriter;
2
3 public class FileTestBroken {
4     public static void main(String[] args) {
5         FileWriter writer = FileWriter("note.txt");
6         writer.write("Hello from Java!");
7         writer.close
8     }
9 }

```

COMPILER OUTPUT

error: class, interface, enum, or record expected

The errors, listed.

1. **Line 1 is missing import.** `java.io.FileWriter`; on its own is not a statement, it is nothing at all, which is why `javac`'s first complaint is that it expected a class. Fix: `import java.io.FileWriter`;
2. **`FileWriter("note.txt")` is missing `new`.** Creating an object requires `new`: without it Java looks for a **method** called `FileWriter` and finds none. Fix: `new FileWriter("note.txt")`.
3. **`writer.close` is missing its brackets and semicolon.** A method call needs `()`. Fix: `writer.close()`; Naming a method without calling it is not a statement, so it cannot stand on a line by itself.
4. **The `IOException` is never handled.** This is the error the others hide, and it is the one worth the mark. `new FileWriter(...)`, `write(...)` and `close()` all throw `IOException`, which is **checked** (Unit 10.1), so the code **will not compile** until you either catch it or declare `throws IOException` on `main`. Fix all three syntax errors and it still does not compile.

Error 4 is the one that separates answers. Anyone can see a missing bracket; the examiner wants to know whether you understand that a checked exception is a **compile-time** obligation.

```

1 import java.io.FileWriter;
2 import java.io.IOException;
3
4 public class FileTest {
5     public static void main(String[] args) {
6         try (FileWriter writer = new FileWriter("note.txt")) {
7             writer.write("Hello from Java!");
8         } catch (IOException e) {
9             System.out.println("Could not write note.txt: " + e.getMessage());
10        }
11        System.out.println("note.txt written");
12    }
13 }

```

OUTPUT

note.txt written

`try-with-resources` answers error 3 and error 4 together: it closes the writer for you, so `close()` cannot be forgotten, and the catch discharges the checked exception. That is Q3 A part 3 being examined in part B.

Q3 C · WHAT IS THE OUTPUT OF THE DATA.TXT FILE?

4 MARKS

Note the question: not what the program prints on screen (it prints **nothing**), but what ends up **in the file**.

The answer:

```
DATA.TXT
Age: 25
Score: 89.46
```

Two lines. And three details carry the four marks.

● MUST-MEMORISE · PRINT, PRINTLN, PRINTF: THE THREE DIFFERENCES

```
print("Age: ") writes the text, no line break. So the 25 lands on the same line.
println(25) writes it and a line break. That break is what ends line 1.
printf("Score: %.2f", 89.456) formats, and adds no line break unless you write %n.
```

1. **%.2f rounds, it does not truncate.** 89.456 to two places is **89.46**, not 89.45. Unit 3.11.
2. **The file has no trailing newline.** printf did not add one, so the file ends immediately after the 6 of 89.46. The file is 21 bytes and its last character is not a line break.
3. **Nothing appears on screen.** Every call is on pw, not on System.out. A common wrong answer is to write the two lines as program output.

MEMORY HOOK: only println breaks the line. print and printf leave the cursor exactly where it stopped.

```

1  import java.io.*;
2
3  public class PrintWriterExample {
4      public static void main(String[] args) throws IOException {
5          PrintWriter pw = new PrintWriter("data.txt");
6          pw.print("Age: ");
7          pw.println(25);
8          pw.printf("Score: %.2f", 89.456);
9          pw.close();
10
11         // show exactly what landed in the file, bytes and all
12         BufferedReader in = new BufferedReader(new FileReader("data.txt"));
13         StringBuilder sb = new StringBuilder();
14         int c;
15         while ((c = in.read()) != -1) sb.append((char) c);
16         in.close();
17         System.out.println("--- data.txt verbatim ---");
18         System.out.print(sb);
19         System.out.println("\n--- as escaped bytes ---");
20         System.out.println(sb.toString().replace("\r", "\\r").replace("\n", "\\n"));
21         System.out.println("length = " + sb.length());
22     }
23 }

```

OUTPUT

```

--- data.txt verbatim ---
Age: 25
Score: 89.46
--- as escaped bytes ---
Age: 25
Score: 89.46
length = 21

```

The paper's four lines are exactly as printed; everything after `pw.close()` is ours, reading the file back so that this book can show you what landed in it. **Length 21** is the proof of the trailing-newline point: "Age: 25" is 7 characters, "Score: 89.46" is 12, which is 19, and the two remaining bytes are the single line break in the middle. There is nothing on the end.

Those two bytes are worth a word. `println` uses the **platform's** line separator, which on Windows is carriage-return plus line-feed (two characters) and on Linux is line-feed alone (one). Same program, 21 bytes here and 20 there. The two printed lines are the answer either way; if you are asked for a byte count, say which platform.

Q3 D · WRITE STUDENTS.TXT AS A FORMATTED, APPENDED TABLE

7.5 MARKS

The question. Create `students.txt` if it does not exist. Use **PrintWriter** to write John (20, 3.75), Lilian (21, 3.95) and Ken (19, 3.65) as a formatted table. Use **printf()** for column alignment. Ensure the file is **appended to if it already exists**. Handle exceptions properly.

● MUST-MEMORISE · PRINTWRITER CANNOT APPEND. THE FILEWRITER UNDER IT CAN

```
new PrintWriter("students.txt") truncates the file to empty. Every time. There is no append constructor.
```

```
new PrintWriter(new FileWriter("students.txt", true)) appends.
```

The `true` is the append flag, and it belongs to **FileWriter**, not to **PrintWriter**. So the answer to "append with **PrintWriter**" is to **wrap** one around a **FileWriter** opened in append mode, exactly as Unit 10.3 wraps a **BufferedReader** around a **FileReader** to get `readLine()`. Each wrapper adds what the other lacks.

And it satisfies part i for free. **FileWriter creates the file if it does not exist**, in either mode. So "create `students.txt` if it does not exist" needs no `if`, no `File.exists()`, no check at all: opening it in append mode does both jobs.

MEMORY HOOK: the flag is on the writer that touches the disk. **PrintWriter** only formats; **FileWriter** opens the file, so the mode is its business.

```

1  import java.io.BufferedReader;
2  import java.io.FileReader;
3  import java.io.FileWriter;
4  import java.io.IOException;
5  import java.io.PrintWriter;
6
7  public class StudentTable {
8      public static void main(String[] args) {
9          String[] names = {"John", "Lilian", "Ken"};
10         int[] ages = {20, 21, 19};
11         double[] gpas = {3.75, 3.95, 3.65};
12
13         // iv. APPEND, not overwrite. new PrintWriter("students.txt") would truncate
14         // the file on every run; the append flag belongs to the FileWriter underneath.
15         // i. FileWriter also creates students.txt if it does not already exist.
16         try (PrintWriter out = new PrintWriter(new FileWriter("students.txt", true))) {
17
18             // iii. printf for column alignment: %-8s is left-aligned in 8 columns.
19             out.printf("%-8s %-5s %-6s\n", "Name", "Age", "GPA");
20             for (int i = 0; i < names.length; i++) {
21                 out.printf("%-8s %-5d %-6.2f\n", names[i], ages[i], gpas[i]);
22             }
23
24         } catch (IOException e) {
25             // v. handle possible exceptions properly
26             System.out.println("Could not write students.txt: " + e.getMessage());
27         }
28
29         // Not part of the answer: read the file back so this book can show you what
30         // landed in it. A dot marks each trailing space, which printf's padding leaves.
31         try (BufferedReader in = new BufferedReader(new FileReader("students.txt"))) {
32             String line = in.readLine();
33             while (line != null) {
34                 System.out.println(line.replace(' ', '.'));
35                 line = in.readLine();
36             }
37         } catch (IOException e) {
38             System.out.println("Could not read students.txt back: " + e.getMessage());
39         }
40     }
41 }

```

STUDENTS.TXT, READ BACK, EVERY SPACE SHOWN AS A DOT

```

Name.....Age...GPA...
John.....20....3.75..
Lilian...21....3.95..
Ken.....19....3.65..

```

The dots are the marks. %-8s means: a String, left aligned, **padded to 8 columns**, and you can count the padding: "Name" plus four dots, "Lilian" plus two. The minus sign is what makes it left aligned; drop it and every column is pushed right and the table goes ragged. %-6.2f is the same idea for a double, fixed at 2 decimal places, which is why 3.75 prints as 3.75 and not 3.8. Unit 3.11.

Those trailing dots are real spaces, sitting at the end of every line. printf pads the last column just as faithfully as the others, because it cannot know it is last. Harmless in a text file, and worth knowing before you go looking for why two files that "look identical" are not.

The claim above is that the true is what makes it append and that without it the file is wiped. Here that claim is tested: the same write is done **twice** each way, and the lines are counted.

```

1  import java.io.BufferedReader;
2  import java.io.FileReader;
3  import java.io.FileWriter;
4  import java.io.IOException;
5  import java.io.PrintWriter;
6
7  public class AppendProof {
8
9      static void writeOnce(String file, boolean append) throws IOException {
10         try (PrintWriter out = append
11             ? new PrintWriter(new FileWriter(file, true)) // appends
12             : new PrintWriter(file)) { // TRUNCATES
13             out.printf("%-8s %-5d %-6.2f%n", "John", 20, 3.75);
14         }
15     }
16
17     static int lines(String file) throws IOException {
18         int n = 0;
19         try (BufferedReader in = new BufferedReader(new FileReader(file))) {
20             while (in.readLine() != null) n++;
21         }
22         return n;
23     }
24
25     public static void main(String[] args) throws IOException {
26         writeOnce("appended.txt", true);
27         writeOnce("appended.txt", true);
28         System.out.println("new PrintWriter(new FileWriter(f, true)) twice -> "
29             + lines("appended.txt") + " lines");
30
31         writeOnce("truncated.txt", false);
32         writeOnce("truncated.txt", false);
33         System.out.println("new PrintWriter(f) twice -> "
34             + lines("truncated.txt") + " lines");
35     }
36 }

```

OUTPUT

```

new PrintWriter(new FileWriter(f, true)) twice -> 2 lines
new PrintWriter(f) twice -> 1 lines

```

Two lines against one. The appending version kept both records; the plain `PrintWriter` threw the first away and left only the second. If you write `new PrintWriter("students.txt")` in the exam, your program passes every other part of this question and still fails part iv, and nothing in its output tells you.

Question Four, worked in full

Q4 A · DESCRIBE DECISION-MAKING, LOOPING AND BRANCHING

3 MARKS

Model answer. These are the three families of **control structure** (Unit 4.1): the statements that decide **which** code runs and **how often**, rather than doing the work themselves.

1. **Decision-making.** Choosing between paths by testing a condition, so that some code runs and some does not: if, if-else, the if-else ladder, and switch.
2. **Looping (iteration).** Repeating a block while a condition holds: for, while and do-while. The do-while is the one that always runs **at least once**, because it tests at the bottom.
3. **Branching (jumping).** Transferring control out of the normal flow: break leaves a loop or a switch case, continue skips to the next iteration, and return leaves the method entirely.

Where the marks are: name the actual keywords. "Looping means repeating" is the idea; "for, while, do-while, and do-while tests at the bottom so it runs at least once" is the answer. Units 4.1 to 4.7.

Q4 B · DEBUG THE WHILE LOOP

3 MARKS

The code as printed:

```

1 public class WhileDebugBroken {
2     public static void main(String[] args) {
3         int count = 1
4         float output = "";
5
6         while(count <=5); {
7             output = output + count + " ";
8             Count
9         }
10
11     System.out.println(output);
12 }
13 }
```

COMPILER OUTPUT

```
error: ';' expected
```

The errors, listed.

1. **int count = 1 has no semicolon.**
2. **float output = ""; is the wrong type.** A float is a number; "" is text. They are not interchangeable and Java will not convert one to the other. Fix: String output = "";
3. **while(count <=5); has a stray semicolon.** This is the one worth the mark. That semicolon **is** the loop body: an empty statement. The block below it then runs once, on its own, as an ordinary block. Since count never changes inside the empty body, the condition stays true forever and the program **hangs**.
4. **Count is not a statement.** Wrong case (Java is case sensitive) and, even spelled right, count alone does nothing. Fix: count++; which is what stops the loop.

⚠ TRAP · THE SEMICOLON THAT EATS THE LOOP BODY

`while (condition);` and `for (...);` are both **legal Java**. The semicolon is an empty statement, and an empty statement is a perfectly good loop body. So the compiler says nothing at all.

What you wrote:

```
while (count <= 5); { ... }
```

What Java read:

```
while (count <= 5) { } then, separately, { ... }
```

The braces you meant as the body became a free-floating block that runs **once**, after the loop finishes. Except the loop never finishes, because nothing inside an empty body can change `count`.

This exact trap appears **twice** on this paper: here, and again in Q6 C as for `(int i = 0; i <= n; i++);`. The examiner is not being subtle about how much they want you to notice it. **Whenever a loop misbehaves, look at the character immediately after the closing bracket.** Unit 4.5.

CODE · Q4 B CORRECTED

VERIFIED ON THE JVM

```
1 public class WhileDebug {
2     public static void main(String[] args) {
3         int count = 1;
4         String output = "";
5
6         while (count <= 5) {
7             output = output + count + " ";
8             count++;
9         }
10
11         System.out.println(output);
12     }
13 }
```

OUTPUT

1 2 3 4 5

There is a trailing space after the 5, because the loop appends `count + " "` every time including the last. Harmless, and worth writing down if you are asked for the output exactly.

Q4 C · STATE THE OUTPUT OF EACH ITERATION AND THE FINAL SUM

4 MARKS

The answer: Sum = 7.

DRY RUN · BREAK STOPS THE LOOP DEAD

INPUTS = {3, 4, -1, 2}

I	INPUTS[I]	NEGATIVE?	SUM += INPUTS[I]	SUM
0	3	no	0 + 3	3
1	4	no	3 + 4	7
2	-1	yes, so break	never runs	7
3	2	never examined at all		7

The 2 is the question. `break` leaves the loop **entirely**, so the 2 at index 3 is never seen and never added. Answer 9 and you have read `break` as `continue`, which is the whole point of the question.

break ends the loop. continue ends the iteration.

Swap them here and the sum is $3 + 4 + 2 = 9$, because `continue` would skip only the -1 and carry on to the 2.

Note also that this loop prints nothing per iteration: the `println` is after it. "The output of each iteration" is the trace of `sum`; the program's only output is `Sum = 7`. Unit 4.7.

```

1 public class BreakSum {
2     public static void main(String[] args) {
3         int sum = 0;
4         int[] inputs = {3, 4, -1, 2};
5
6         for (int i = 0; i < inputs.length; i++) {
7             if (inputs[i] < 0)
8                 break;
9             sum += inputs[i];
10        }
11        System.out.println("Sum = " + sum);
12    }
13 }

```

OUTPUT

Sum = 7

Q4 D · THE TEMPERATURECHECK CLASS

7.5 MARKS

The question. Define a class TemperatureCheck with a double temperature field, a constructor that sets it, and a method evaluateTemperature() using **if-else** to print "Too Cold" below 15, "Normal" between 15 and 30 **inclusive**, and "Too Hot" above 30. In main, prompt for a temperature with **JOptionPane**, create an object, and display the result.

● MUST-MEMORISE · "BETWEEN 15 AND 30 INCLUSIVE" NEEDS NO UPPER TEST

```

if (t < 15) "Too Cold";
else if (t <= 30) "Normal";
else "Too Hot";

```

The else if only runs when $t < 15$ was **false**, which already means $t \geq 15$. So writing else if ($t \geq 15 \ \&\& \ t \leq 30$) is not wrong, it is **redundant**: the ladder has already established the lower bound. Let the structure carry what it has proved (Unit 4.2).

The two boundaries are the marks. "Inclusive" means 15 and 30 are both Normal. Get $<$ and $\leq$ the wrong way round and the program is right for 99% of inputs and wrong on exactly the two values the examiner will test.

MEMORY HOOK: test the fences, not the field. When a spec names a boundary, that boundary is the test case.

```

1  import java.util.Scanner;
2
3  // i. the class, with its one data field
4  class TemperatureCheck {
5      private double temperature;
6
7      // ii. a constructor that sets the temperature
8      public TemperatureCheck(double temperature) {
9          this.temperature = temperature;
10     }
11
12     // iii. if-else evaluation
13     public void evaluateTemperature() {
14         if (temperature < 15) {
15             System.out.println("Too Cold");
16         } else if (temperature <= 30) {
17             System.out.println("Normal");
18         } else {
19             System.out.println("Too Hot");
20         }
21     }
22 }
23
24 public class TemperatureDemo {
25     public static void main(String[] args) {
26         Scanner in = new Scanner(System.in);
27         while (in.hasNextLine()) {
28             String line = in.nextLine();
29             if (line.isEmpty()) continue;
30             double t = Double.parseDouble(line);
31             System.out.print(t + " -> ");
32             TemperatureCheck check = new TemperatureCheck(t);
33             check.evaluateTemperature();
34         }
35     }
36 }

```

OUTPUT

```

5.0 -> Too Cold
15.0 -> Normal
22.0 -> Normal
30.0 -> Normal
31.0 -> Too Hot
40.0 -> Too Hot

```

Read rows 2 and 4: 15.0 and 30.0 are both Normal, which is exactly what "inclusive" demands, and 31.0 is the first Too Hot. The values were chosen to sit on the fences. In the hall this reads `double t = Double.parseDouble(JOptionPane.showInputDialog("Enter a temperature:"))`; and the result goes out through `showMessageDialog(F,11)`; the loop here exists only so one run can show you all six cases.

The field is private and set only by the constructor, which is Unit 6.4's encapsulation arriving without being asked for. The question says "data field (state)"; making it private costs one word and is never wrong.

Question Five, worked in full

Q5 A · DESCRIBE INHERITANCE, POLYMORPHISM AND METHOD OVERRIDING

3 MARKS

Model answer.

1. **Inheritance.** One class acquiring the fields and methods of another with extends, so the subclass has everything the superclass has without repeating it, and may add or replace behaviour. It models an **is-a** relationship: a Bicycle is a Vehicle.
2. **Polymorphism.** One reference, many behaviours: a variable of the superclass type may hold an object of any subclass, and a call through it runs the **object's own** version, decided at **run time**. "Many forms".
3. **Method overriding.** A subclass supplying its own version of a method **the superclass already has**, with the **same signature**, which replaces the inherited one for objects of that subclass.

Where the marks are: these three are one idea in three parts, and saying so earns the mark. **Inheritance** makes the family; **overriding** makes the members differ; **polymorphism** is what you get when you then hold any of them in one variable and call the method. Distinguish overriding (same signature, subclass, run time) from overloading (different parameters, same class, compile time), which is Q1 of this same paper. Units 6.6 and 6.7.

Q5 B · WRITE COUNTNEGATIVES

3 MARKS

The question. A method taking a single-dimensional int array and returning how many values are negative. The question explicitly says you do **not** need the class or main: just the method.

Read that concession and take it. Writing a whole class here wins no extra marks and costs you minutes you need for part D.

CODE · Q5 B

VERIFIED ON THE JVM

```

1 public class CountNegatives {
2
3     // Q5 B: the method the question asks for, and nothing else.
4     public static int countNegatives(int[] numbers) {
5         int count = 0;
6         for (int i = 0; i < numbers.length; i++) {
7             if (numbers[i] < 0) {
8                 count++;
9             }
10        }
11        return count;
12    }
13
14    // main is not part of the answer; it is here so the method can be run.
15    public static void main(String[] args) {
16        int[] a = {3, -1, 0, -7, 8, -2};
17        int[] b = {1, 2, 3};
18        int[] c = {};
19        System.out.println("{3, -1, 0, -7, 8, -2} -> " + countNegatives(a));
20        System.out.println("{1, 2, 3} -> " + countNegatives(b));
21        System.out.println("{} (empty) -> " + countNegatives(c));
22    }
23 }
```

OUTPUT

```

{3, -1, 0, -7, 8, -2} -> 3
{1, 2, 3} -> 0
{} (empty) -> 0
```

Only countNegatives is the answer; main is here so the method can be run. **Zero is not negative**, which is why the first array has a 0 in it and the count is 3, not 4: the test is `< 0`, never `<= 0`. And the empty array returns 0 without any special case, since the loop simply never runs.

The code as printed:

```

1 public class MiddleValueBroken {
2     public static void main(String[] args) {
3         int nums = {10, 20, 30};
4         System.out.println("Middle value is: " + nums[3]);
5         int sum = 0;
6         for(int i = 0; i <= nums.length; i) {
7             sum += nums[i];
8         }
9         System.out.println("Sum is: " + );
10    }
11 }

```

COMPILER OUTPUT

error: not a statement

The errors, listed.

1. **int nums = {10, 20, 30}; is not an array.** A list in braces needs an array type. Fix: `int[] nums = {10, 20, 30};`
2. **nums[3] does not exist.** Three elements means indexes 0, 1 and 2. And the label says "**Middle value**", so the intent is `nums[1]`, which is 20. Say both halves: the index is out of bounds **and** the middle of three is index 1.
3. **i <= nums.length is off by one** and would throw at `nums[3]`. Fix: `i < nums.length`.
4. **i never increments.** Fix: `i++`.
5. **"Sum is: " +); has nothing to add.** Fix: `"Sum is: " + sum`.

The output after correction:

OUTPUT

Middle value is: 20
Sum is: 60

```

1 public class MiddleValue {
2     public static void main(String[] args) {
3         int[] nums = {10, 20, 30};
4         System.out.println("Middle value is: " + nums[1]);
5         int sum = 0;
6         for (int i = 0; i < nums.length; i++) {
7             sum += nums[i];
8         }
9         System.out.println("Sum is: " + sum);
10    }
11 }

```

OUTPUT

Middle value is: 20
Sum is: 60

Errors 3 and 4 are the same two errors as Q2 B on this paper, character for character. **The off-by-one and the missing increment are this examiner's signature.** They are on both papers, in both years. Learn them once.

The question. A superclass `Vehicle` with an `int` `speed`, a constructor that initialises it, and `displayInfo()` printing "Vehicle speed: <speed> km/h". A subclass `Bicycle` overriding `displayInfo()` to print "Bicycle speed: <speed> km/h. This is an eco-friendly ride." In main, create a `Vehicle` at 80 and a `Bicycle` at 25 and call `displayInfo()` on both **to demonstrate polymorphism**.

CODE · Q5 D, COMPLETE

VERIFIED ON THE JVM

```

1 // i. the superclass
2 class Vehicle {
3     protected int speed;
4
5     public Vehicle(int speed) {
6         this.speed = speed;
7     }
8
9     public void displayInfo() {
10         System.out.println("Vehicle speed: " + speed + " km/h");
11     }
12 }
13
14 // ii. the subclass, overriding displayInfo()
15 class Bicycle extends Vehicle {
16     public Bicycle(int speed) {
17         super(speed);
18     }
19
20     @Override
21     public void displayInfo() {
22         System.out.println("Bicycle speed: " + speed
23             + " km/h. This is an eco-friendly ride.");
24     }
25 }
26
27 public class VehicleDemo {
28     public static void main(String[] args) {
29         // iii a/b. one of each
30         Vehicle car = new Vehicle(80);
31         Bicycle bike = new Bicycle(25);
32
33         // iii c. call displayInfo() on both
34         car.displayInfo();
35         bike.displayInfo();
36
37         // and the polymorphic form the question is really after
38         Vehicle[] fleet = {new Vehicle(80), new Bicycle(25)};
39         System.out.println("-- through a Vehicle reference --");
40         for (Vehicle v : fleet) {
41             v.displayInfo();
42         }
43     }
44 }

```

OUTPUT

```

Vehicle speed: 80 km/h
Bicycle speed: 25 km/h. This is an eco-friendly ride.
-- through a Vehicle reference --
Vehicle speed: 80 km/h
Bicycle speed: 25 km/h. This is an eco-friendly ride.

```

TEACH · THE LAST FOUR WORDS OF THE QUESTION CHANGE THE ANSWER

The question says: call `displayInfo()` on both objects **to demonstrate polymorphism**. Read literally, parts a to c are satisfied by:

```
Vehicle car = new Vehicle(80);
Bicycle bike = new Bicycle(25);
car.displayInfo();
bike.displayInfo();
```

That prints the two required lines, and it demonstrates **nothing**. Each variable's declared type already matches its object exactly, so the compiler could have picked both methods without any run-time decision. Overriding happened, but polymorphism was never tested.

Polymorphism needs the reference to be the SUPERCLASS type, so that the declared type and the actual object disagree and only the run time can settle it:

```
Vehicle[] fleet = {new Vehicle(80), new Bicycle(25)};
for (Vehicle v : fleet) v.displayInfo();
```

Now every element is declared `Vehicle`, the loop body is one identical call, and it still prints two different lines. **That** is the demonstration, and it is what the last four words are asking for. The listing above does both: the literal answer first, then the proof.

Same idea as Q4 part 3 v on the 2025/26 paper, where the reference is explicitly required to be a `BankAccount`. Two papers, two years, one point: **the reference decides what you may call, the object decides what runs**.

S.14 · 2024/25 · QUESTION SIX · 17.5 MARKS

Question Six, worked in full

Q6 A · SNIPPETS FOR THE THREE LOOP TYPES

3 MARKS

The question. A for printing 1 to 5; a while printing 5 down to 1; a do-while printing **only the number 10, once**.

Part iii is the one being marked. A do-while whose condition is false still runs **once**, and "print 10 once" is the cleanest possible demonstration of that. Choose a condition that is false immediately, and the body still prints.

```

1 public class ThreeLoopTypes {
2     public static void main(String[] args) {
3         // i. a for loop printing 1 to 5
4         for (int i = 1; i <= 5; i++) {
5             System.out.print(i + " ");
6         }
7         System.out.println();
8
9         // ii. a while loop printing 5 down to 1
10        int j = 5;
11        while (j >= 1) {
12            System.out.print(j + " ");
13            j--;
14        }
15        System.out.println();
16
17        // iii. a do-while printing only 10, once
18        int k = 10;
19        do {
20            System.out.print(k + " ");
21            k++;
22        } while (k <= 10);
23        System.out.println();
24    }
25 }

```

OUTPUT

```

1 2 3 4 5
5 4 3 2 1
10

```

In the do-while, k is 10, the body prints it, k becomes 11, and while (k <= 10) is **false**, so it stops. The body ran once **before** the condition was ever looked at, which is the whole difference between do-while and while. Write it as a while with the same condition and nothing prints at all. Unit 4.5.

Q6 B · WHAT DOES COMPUTE(13) RETURN? SHOW THE STEPS

3 MARKS

The answer: 3. And the question says **show the steps**, so the table is the answer, not the number.

DRY RUN · COMPUTE(13), STEP BY STEP

WHILE (X > 0): COUNT THE ODD STEPS, HALVE EACH TIME

X	X % 2	ODD, SO COUNT++?	X = X / 2	COUNT
13	1	yes	6	1
6	0	no	3	1
3	1	yes	1	2
1	1	yes	0	3
0	x > 0 is false, so the loop ends			3

What the method actually is. Repeatedly halving with $x / 2$ and testing $x \% 2$ walks the number's **binary digits**, from the right. So compute counts the **1 bits**. 13 in binary is **1101**, which has three 1s, so the answer is 3, and you can check it in one second without tracing anything.

Say that in your answer after showing the table. Naming what the code *is* ("it counts the set bits") is what separates a trace from understanding, and it costs one line.

```
1 public class ComputeBits {
2
3     // exactly as the paper prints it
4     public static int compute(int x) {
5         int count = 0;
6         while (x > 0) {
7             if (x % 2 == 1) count++;
8             x = x / 2;
9         }
10        return count;
11    }
12
13    public static void main(String[] args) {
14        System.out.println("compute(13) = " + compute(13));
15        System.out.println("13 in binary is " + Integer.toBinaryString(13));
16        for (int v : new int[]{1, 2, 3, 7, 8, 255}) {
17            System.out.println("compute(" + v + ") = " + compute(v)
18                               + "    binary " + Integer.toBinaryString(v));
19        }
20    }
21 }
```

OUTPUT

```
compute(13) = 3
13 in binary is 1101
compute(1) = 1    binary 1
compute(2) = 1    binary 10
compute(3) = 2    binary 11
compute(7) = 3    binary 111
compute(8) = 1    binary 1000
compute(255) = 8    binary 11111111
```

Every row is its own proof: the answer always equals the number of 1s printed beside it. `compute(255) = 8` and 255 is eight 1s; `compute(8) = 1` and 8 is 1000. The claim is not "this looks like a bit count", it is a bit count.

The code as printed:

```

1 public class EvenFinderBroken {
2     public static void main(String args) {
3         printEven(10.0)
4     }
5
6     public static void printEven(int) {
7         for (int i = 0; i <= n; i++);
8         if (i % 2 == 0)
9             System.out.print(i + " ");
10    }
11 }

```

COMPILER OUTPUT

error: ';' expected

The errors, listed.

1. **main(String args) is not main.** It must be `String[] args`. Without the brackets it is an ordinary method that the JVM will not recognise as the entry point.
2. **printEven(10.0) has no semicolon.**
3. **10.0 is a double.** The method takes an `int`, and Java will not narrow a double to an `int` on its own, because that loses information (Unit 3.3). Fix: `printEven(10);`
4. **printEven(int) has no parameter name.** Fix: `printEven(int n)`, which is also why `n` "does not exist" three lines later.
5. **for (...); has a stray semicolon.** The loop body is empty, so the `if` below is **not in the loop**. It runs once, after the loop, and it cannot even compile: `i` was declared in the `for`, so it is out of scope the moment the `for` ends.

Error 5 is Q4 B's stray semicolon again, on the same paper, in a `for` this time. The scope error is what gives it away here: when the compiler says "cannot find symbol: `i`" about a variable you can plainly see, the loop that declared it has already closed.

The output after correction: **0 2 4 6 8 10**

CODE · Q6 C CORRECTED

VERIFIED ON THE JVM

```

1 public class EvenFinder {
2     public static void main(String[] args) {
3         printEven(10);
4     }
5
6     public static void printEven(int n) {
7         for (int i = 0; i <= n; i++) {
8             if (i % 2 == 0) {
9                 System.out.print(i + " ");
10            }
11        }
12        System.out.println();
13    }
14 }

```

OUTPUT

0 2 4 6 8 10

0 is even, and `i <= n` means 10 is included, so the answer starts at 0 and ends at 10. Both ends are worth checking: this is the rare loop where `<=` is correct, because `n` is a value to reach, not a length.

The question. Write `hasEqualAdjacentPairs(int n)` returning **true** if the positive integer `n` contains **at least two pairs** of equal adjacent digits, and **false** otherwise. Write `main` to test at least two cases. The paper's examples:

112233 → true (pairs: 11, 22, 33). More than two adjacent pairs.
1223 → true (pairs: 22). Only one pair → return false

⚠ **TRAP · THE PAPER CONTRADICTS ITSELF, AND YOU MUST ANSWER ANYWAY**

Read the second example again. It says "**1223 - true**" and then, in the same line, "**Only one pair - return false**". Both cannot be right.

The **specification** says "at least two pairs". 1223 has **one** pair, the 22. One is not at least two.

The correct return is false. The leading "true" is a slip of the pen.

The examiner's own explanation ("Only one pair") agrees with the specification, so the specification and the reasoning outvote the stray word. This book shows you the paper exactly as printed and tells you what is true, and it never quietly corrects a source (compare the six errors in Module 3, which come from this course's own slides).

What to do in the hall. Do not agonise, and do not answer the typo. Implement the specification, and **write one line saying why**: "1223 has only one adjacent pair, so by the stated rule of at least two pairs the method returns false; the 'true' in the example appears to be a typographical error." That sentence costs ten seconds and it protects you whichever way the marking scheme reads, because it proves the answer was a decision and not a mistake.

MEMORY HOOK: when a question contradicts itself, follow the specification and say that you did.

```

1 public class AdjacentPairs {
2
3     // i/iii. true when n has AT LEAST TWO pairs of equal adjacent digits.
4     public static boolean hasEqualAdjacentPairs(int n) {
5         int pairs = 0;
6         int previous = n % 10;
7         n = n / 10;
8         while (n > 0) {
9             int current = n % 10;
10            if (current == previous) {
11                pairs++;
12            }
13            previous = current;
14            n = n / 10;
15        }
16        return pairs >= 2;
17    }
18
19    // iii b. main testing more than the two cases the question asks for
20    public static void main(String[] args) {
21        int[] tests = {112233, 1223, 1122, 111, 123, 7, 122333};
22        for (int t : tests) {
23            System.out.println(t + " -> " + hasEqualAdjacentPairs(t));
24        }
25    }
26 }

```

OUTPUT

```

112233 -> true
1223 -> false
1122 -> true
111 -> true
123 -> false
7 -> false
122333 -> true

```

The paper asks for at least two test cases; there are seven, and each earns its place. **112233** and **1223** are the paper's own. **1122** is the boundary: exactly two pairs, so exactly true. **111** asks what a triple counts as. **123** has no pairs and **7** has no adjacent digits at all, so the loop never runs and pairs stays 0.

● MUST-MEMORISE · COMPARE ADJACENT DIGITS BY REMEMBERING THE LAST ONE

Take the last digit **before** the loop and call it previous. Then each turn: take the new current, compare it with previous, and **make current the new previous**.

That is the whole technique, and it is the same % 10 and / 10 pair as Q6 A on the 2025/26 paper. The only new idea is carrying **one** value forward so that each digit can see its neighbour. You never need the digits in an array; you need exactly one of them remembered.

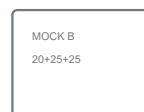
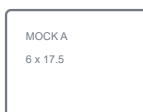
111 gives two pairs, not one. The digits are compared in overlapping pairs: positions 1-2 and 2-3. The middle 1 belongs to both. If you think a triple should count once, that is a different specification, and this one does not say so: it says pairs of equal adjacent digits, and 111 contains two of them.

MEMORY HOOK: to compare neighbours, remember one. Peel a digit, compare, then let it become the memory for the next.

PART XIII

Mock Papers

Two papers, in the two shapes this course has used, with no answers anywhere near them. Sit them closed-book, on paper, with a clock. Reading a solved question teaches you that you understood it; writing one under time pressure is the only way to find out whether you can. The answers are the next part, and they are worth nothing to you until you have written yours.



Mock Paper A, in the older shape

Six questions, any four, three hours. This is the 2024/25 shape, and it is the shape that rewards choosing well.

● MUST-MEMORISE · HOW TO SIT THIS PAPER

Time allowed: 3 hours. Attempt any four (4) questions. Each question carries 17.5 marks. Total: 70 marks.

Every question follows the same shape: **A** define (3), **B** debug (3), **C** dry run (4), **D** write a program (7.5).

MEMORY HOOK: four of six, and D is 7.5 of every 17.5. Nearly half of this paper is writing programs. Budget 45 minutes a question and spend half of it on D.

TEACH · SIT IT PROPERLY, OR DO NOT SIT IT

Book shut. Paper and pen. A clock you can see. Three hours, in one sitting, with no compiler. If you look up one method signature mid-paper, the result tells you nothing you can use, because the hall has no lookups.

Answer **four** questions, as instructed, and note which two you dropped. Then, afterwards and untimed, answer the other two anyway. The ones you avoid are a map of what you have not learned yet, and next year's paper is written by someone who does not know which two you like.

⚠ TRAP · CHOOSING FOUR IS ITSELF A SKILL, AND IT IS TIMED

Read all six before writing anything. Five minutes spent choosing is the best-paid five minutes on the paper, because part D is 7.5 marks and it is where a bad choice costs you twenty minutes you cannot get back.

Choose on **part D alone**. Parts A, B and C are 10 marks you can attempt on any topic; D is where a question is won or abandoned. If you cannot see the shape of D's program in your head within a minute, that is not your question.

M.2 · MOCK A · QUESTION ONE · 17.5 MARKS

Question One

Q1 A

3 MARKS

Define the following: a **formal parameter**, an **actual argument**, and the **return type** of a method.

Q1 B

3 MARKS

Debug this code snippet. List and correct the errors.

```
public class CubeCalcBroken {
    public static void main(String[] ) {
        int side = 4;
        double result = cube(side)
        System.out.println("Cube is: " + result);
    }

    public static int cube(int) {
        return s * s * s;
    }
}
```

Q1 C

4 MARKS

Dry run this method with the values **3** and **5**. State the value returned for each.

```
public static int tally(int n) {
    int total = 0;
    for (int k = 1; k <= n; k++) {
        total += (k + 1) / 2;
    }
    return total;
}
```

Q1 D

7.5 MARKS

Write a program that **overloads** a method named volume to compute:

1. the volume of a cube;
2. the volume of a cylinder;
3. the volume of a rectangular box.

Call these methods and print out their values in the main method.

M.3 · MOCK A · QUESTION TWO · 17.5 MARKS

Question Two

Q2 A

3 MARKS

Define the following: **array declaration** as distinct from **array creation**, the **index** of an array element, and **parallel arrays**.

Q2 B

3 MARKS

Debug this code snippet. List and correct the errors. The program is meant to copy the array into a second array in reverse order and print it.

```
public class ReverseArrayBroken {
    public static void main(String[] args) {
        int[] a = {1, 3, 5, 7};
        int b;
        for (int i = 0; i <= a.length; i) {
            b[i] = a[a.length - i];
        }
        System.out.println(b[i]);
    }
}
```

Q2 C

4 MARKS

Dry run this code. State the output for each iteration, and state the final output.

```
public class MaxOrDrop {
    public static void main(String[] args) {
        int[] data = {5, 3, 9, 1};
        int x = data[0];
        for (int i = 1; i < data.length; i++) {
            if (data[i] > x) {
                x = data[i];
            } else {
                x = x - data[i];
            }
            System.out.println("i = " + i + ", x = " + x);
        }
        System.out.println("Final x = " + x);
    }
}
```

Q2 D

7.5 MARKS

Write a program that does the following:

1. prompts the user to input 8 values into an array;
2. implements two methods:
 1. a method to **find**, **count** and **display** the multiples of 5;
 2. another method to **sum** and **display** the values that are not multiples of 5;
3. in the main method, uses a for loop to **find** and **display** the minimum value.

M.4 · MOCK A · QUESTION THREE · 17.5 MARKS

Question Three

Q3 A

3 MARKS

Define the following: **FileWriter**, the **BufferedReader** class, and a **checked exception**.

Q3 B

3 MARKS

Debug this code snippet. List and correct the errors.

```
import java.io.FileWriter;

public class NoteWriterBroken {
    public static void main(String[] args) {
        FileWriter out = FileWriter("log.txt");
        out.write("Session started");
        out.close();
    }
}
```

Q3 C

4 MARKS

What is written to the `report.txt` file? Give its exact contents, line by line.

```
import java.io.*;

public class ReportWriter {
    public static void main(String[] args) throws IOException {
        PrintWriter pw = new PrintWriter("report.txt");
        pw.print("Unit: ");
        pw.println(7);
        pw.printf("Rate: %.1f%n", 12.345);
        pw.printf("Total: %5.2f", 3.14159);
        pw.close();
    }
}
```

Q3 D

7.5 MARKS

Write a program that does the following:

1. **creates** a file named `books.txt` if it does not exist;
2. uses `PrintWriter` to **write** the following records to the file in a formatted table:

TITLE	COPIES	PRICE
Algebra	12	2500.00
Optics	4	1875.50
Ecology	7	990.25

3. uses `printf()` for proper column alignment;
4. ensures the file is **appended** to if it already exists;
5. **handles** possible exceptions properly.

M.5 · MOCK A · QUESTION FOUR · 17.5 MARKS

Question Four

Q4 A

3 MARKS

Describe the following: a **counter-controlled** loop, a **sentinel-controlled** loop, and the **continue** statement.

Q4 B

3 MARKS

Debug this code snippet. List and correct the errors. It is meant to build the string `1 2 3 4 5` and print it once.

```
public class CountUpBroken {
    public static void main(String[] args) {
        int count = 1
        float output = "";

        while (count <= 5); {
            output = output + count + " ";
            Count
        }

        System.out.println(output);
    }
}
```

Q4 C

4 MARKS

State the output of each iteration and the final sum.

```
public class SkipTotal {  
    public static void main(String[] args) {  
        int sum = 0;  
        int[] inputs = {4, -2, 6, -8, 5};  
  
        for (int i = 0; i < inputs.length; i++) {  
            if (inputs[i] < 0) {  
                continue;  
            }  
            sum += inputs[i];  
            System.out.println("i = " + i + ", sum = " + sum);  
        }  
        System.out.println("Sum = " + sum);  
    }  
}
```

Q4 D

7.5 MARKS

Write a program to define a class called SpeedCheck with the following specifications:

1. a data field (state): double speed;
2. a constructor method to **set** the speed;
3. a method called evaluateSpeed() that **uses if-else statements** to output:
 1. "Too Slow" if the speed is below 40;
 2. "Normal" if the speed is between 40 and 100 (inclusive);
 3. "Too Fast" if the speed is above 100;
4. in your main method, **prompt** the user to enter a speed, **create** an object of the class, and **display** the evaluation result. Use JOptionPane to accept input.

M.6 · MOCK A · QUESTION FIVE · 17.5 MARKS

Question Five

Q5 A

3 MARKS

Describe the following: the **super** keyword, the **@Override** annotation, and **runtime polymorphism**.

Q5 B

3 MARKS

Write a method called countAbove that accepts a single-dimensional array of double and a double limit, and returns how many values in the array are strictly greater than that limit. You do not need to write the full class or the main method, just the method definition.

Q5 C

4 MARKS

Identify the errors in this code snippet, and state the final output after correction.

```
public class TotalsBroken {
    public static void main(String[] args) {
        int values = {4, 8, 12, 16};
        System.out.println("Third value is: " + values[3]);
        int sum = 0;
        for (int i = 0; i <= values.length; i) {
            sum += values[i];
        }
        System.out.println("Sum is: " + );
    }
}
```

Q5 D

7.5 MARKS

Write a program using inheritance and method overriding as follows:

1. **create** a superclass called Employee with:
 1. an instance variable *salary* (int);
 2. a constructor that initialises *salary*;
 3. a method *displayInfo()* that **prints** "Employee salary: N<salary> per month";
2. **create** a subclass called Manager that **overrides** *displayInfo()* to print "Manager salary: N<salary> per month. This post carries a leadership allowance.";
3. in the main method:
 1. **create** an Employee with a salary of 80000;
 2. **create** a Manager with a salary of 250000;
 3. **call** *displayInfo()* on both objects to demonstrate polymorphism.

M.7 · MOCK A · QUESTION SIX · 17.5 MARKS

Question Six

Q6 A

3 MARKS

Write a code snippet to demonstrate each of the following loop types:

1. a for loop that prints the even numbers 2 to 10;
2. a while loop that prints the numbers from 9 down to 1;
3. a do-while loop that prints only the number 100, once.

Q6 B

3 MARKS

What will be the return value of `compute(2736)`? Show the steps to explain your answer.

```
public static int compute(int x) {
    int result = 0;
    while (x > 0) {
        int d = x % 10;
        if (d % 3 == 0) {
            result = result + d;
        }
        x = x / 10;
    }
    return result;
}
```

Q6 C

4 MARKS

Debug this code snippet. List the errors and state the correct output.

```
public class OddFinderBroken {
    public static void main(String args) {
        printOdd(9.0)
    }

    public static void printOdd(int) {
        for (int i = 1; i <= n; i++);
        if (i % 2 != 0)
            System.out.print(i + " ");
    }
}
```

Q6 D

7.5 MARKS

Write a program that includes a method called `hasAscendingDigits(int n)` which:

1. **returns** true if the digits of the positive integer `n` strictly increase from left to right;
2. **returns** false otherwise;
3. you must:
 1. **write** the method `hasAscendingDigits(int n)`;
 2. **write** the `main()` method to **test** at least two cases.

Examples

- ▶ 1359 gives true: $1 < 3 < 5 < 9$.
- ▶ 1354 gives false: 5 is not less than 4.
- ▶ 1223 gives false: 2 is not strictly less than 2.

TEACH · BEFORE YOU TURN TO THE ANSWERS

Mark your own paper first, from the solutions in the next part, and mark it as a stranger would: a program that does not compile scores what a program that does not compile scores. Note every mark you lost and write beside it **which of the four skills** it belonged to: define, debug, dry run, write.

Four ticks in one column is not four mistakes. It is one, and it has a name, and it is on the syllabus.

M.8 · MOCK B · BEFORE YOU START

Mock Paper B, in the current shape

Three sections, one question from each, three hours. This is the 2025/26 shape, and it does not let you choose your topics.

● MUST-MEMORISE · HOW TO SIT THIS PAPER

Time allowed: 3 hours. Attempt one question from each Section.

Section A: 20 marks, Q1 **or** Q2

Section B: 25 marks, Q3 **or** Q4

Section C: 25 marks, Q5 **or** Q6

Use comments in Section B and Section C to distinguish what task each block of code performs. That is an instruction, not advice.

MEMORY HOOK: $20 + 25 + 25 = 70$. Section A is the cheapest and the most predictable, so bank it first and fast.

⚠ **TRAP · THIS SHAPE REMOVES THE ESCAPE ROUTE MOCK A GAVE YOU**

On Mock A you may drop any two topics you dislike. Here you may not. Both Section C questions are long JOptionPane programs; both Section B questions are 25 marks of writing. If you cannot do file I/O **and** inheritance, Section B has no door for you, because Q3 is the first and Q4 is the second.

That is the real reason this book answers every question on both real papers rather than four of six.

The choice you get is between two questions, not between two topics you prepared and four you did not.

M.9 · MOCK B · SECTION A · QUESTION ONE · 20 MARKS

Section A, Question One

SECTION A (20 marks). Answer Question One **or** Question Two.

Q1 A

2 MARKS

Define a **constructor** in Java, and state precisely how the compiler tells a constructor apart from an ordinary method.

Q1 B

3 MARKS

State **three differences** between a while loop and a do-while loop in Java.

Q1 C

5 MARKS

This code is supposed to: loop through the array, and for each starting index *i*, find the first number (from *i* onward) that is a multiple of 3. Add (that number + 2) to sum, count it as a match, stop after 3 matches, then display the sum and the match count.

Find the errors if the correct output should be: Sum: 93.0 Matches: 3

```
1 import javax.swing.JOptionPane;
2 public class SnippetBroken {
3     public static void main(String[] args) {
4         int[] arr = {15.0, 22.0, 36.0, 41.0, 54.0, 60.0};
5         int sum = 0;
6         int matches = 0;
7         boolean stop = false;
8         for (int i = 0; i <= arr.length && !stop; i++) {
9             for (int j = i; j < arr.length; j++) {
10                 if (arr[j] % 3 == 0) {
11                     continue;
12                 }
13                 sum += arr[j] + 2;
14                 matches++;
15                 if (matches == 4) {
16                     stop = true;
17                 }
18                 break;
19             }
20         }
21         JOptionPane.showMessageDialog(null, "Sum: " + sum);
22         JOptionPane.showMessageDialog(null, "Matches: " + matches);
23         System.exit(0);
24     }
25 }
```

Q1 D

10 MARKS

An **automorphic number** is a positive integer whose square **ends in the number itself**. For example, 25 is automorphic because 25 squared is 625, which ends in 25. 6 is automorphic because 6 squared is 36, which ends in 6.

1. State the **five outputs** for this code. You may show your workings, but make the answer bold.
2. Retain everything else, but change the **For loop to a While loop** and the **While loops to For loops**.

```
public class AutomorphicChecker {
    public static void main(String[] args) {
        int[] numbers = {5, 6, 7, 25, 76};
        for (int idx = 0; idx < numbers.length; idx++) {
            int n = numbers[idx], digits = 0, temp = n;
            while (temp > 0) { digits++; temp /= 10; }
            int power = 1;
            for (int p = 0; p < digits; p++) power *= 10;
            int tail = (n * n) % power;
            System.out.println(n + " -> " + (tail == n ? "AUTOMORPHIC" : "NOT AUTOMORPHIC"));
        }
    }
}
```

M.10 · MOCK B · SECTION A · QUESTION TWO · 20 MARKS

Section A, Question Two

Q2 A

2 MARKS

Define **inheritance** as an OOP principle.

Q2 B

3 MARKS

State **three differences** between an **array** and an ordinary **variable** in Java.

This code is supposed to loop through an array of parcel weights, skip anything under 5, apply a 20% surcharge to weights of 5 and above, add the surcharged weight to the total, count it, and stop once 4 parcels have been surcharged. Then it displays the total and the count.

Find the errors if the correct output should be: Total: **84.6** Count: **4**

```

1  import javax.swing.JOptionPane;
2  public class DiscountBroken {
3  public static void main(String[] args) {
4  int[] weights = {3.5, 12.0, 4.25, 20.0, 8.5, 30.0};
5  int total = 0;
6  int count = 0;
7  boolean done = false;
8  for (int i = 0; i <= weights.length && !done; i++) {
9      if (weights[i] > 5) {
10         continue;
11     }
12     double surcharge = weights[i] * 0.2;
13     total += weights[i] + surcharge;
14     count++;
15     if (count == 5) {
16         done = true;
17     }
18 }
19 JOptionPane.showMessageDialog(null, "Total: " + total);
20 JOptionPane.showMessageDialog(null, "Count: " + count);
21 System.exit(0);
22 }
23 }

```

A **prime number** is a positive integer with exactly two distinct divisors: 1 and itself. A **composite number** has more than two. The number 1 has exactly one divisor, so it is **neither**.

1. State the output for this code. No need to show workings.
2. Change the **For loops to While loops**.

```

public class PrimeClassifier {
    public static void main(String[] args) {
        int primeCount = 0, compositeCount = 0, neitherCount = 0;
        for (int num = 1; num <= 30; num++) {
            int divisors = 0;
            for (int d = 1; d <= num; d++) {
                if (num % d == 0) divisors++;
            }
            if (divisors == 2) primeCount++;
            else if (divisors > 2) compositeCount++;
            else neitherCount++;
        }
        System.out.println("Prime: " + primeCount + ", Composite: " + compositeCount
            + ", Neither: " + neitherCount);
    }
}

```

Section B, Question Three

SECTION B (25 marks). Answer Question Three **or** Question Four. Use comments to say what each block of code does.

Q3 A

12 MARKS

A factory records a machine's vibration once an hour in an integer array `readings[]`, in the order the readings were taken. A **spike** is any position `i` that is strictly greater than **both** its neighbours. A **dip** is any position `i` that is strictly less than **both** its neighbours. The first and last positions have only one neighbour each and are never spikes or dips.

Given the array:

INDEX	0	1	2	3	4	5	6	7	8	9
READING	12	30	18	22	41	27	27	33	19	25

Write a program that:

1. creates a method that makes **one pass** through the array and returns the count of spikes;
2. creates a method that returns a single String listing, in original order and comma separated, every reading that is **either** a spike **or** a dip. Build this string using plain String concatenation only (no `StringBuilder`, and do not return an array of booleans);
3. using `JOptionPane`, displays **one** dialog showing (a) the spike count from part 1, and (b) the flagged-readings string from part 2.

[Hint: trace your algorithm by hand first. It must **not** flag 12, 22, 27, 27 or 25.]

Q3 B

13 MARKS

A library needs a program to record and review book details (book ID, title, rating) using basic file I/O. Using **only** `java.io.FileWriter`, `java.io.FileReader`, `java.io.BufferedReader`, `java.io.BufferedWriter` and `java.io.IOException` (or its subclasses), write a complete program that:

1. creates a file named `input.txt` and writes the details (book ID, title, rating) of at least three books into it using `FileWriter`, one book per line. Wrap the writing logic in a try-catch block that handles any exception that may occur;
2. reads book details from a file named `output.txt` using `FileReader` and prints each line to the console exactly as it appears in the file. Handle any exception that may occur while reading, **including the case where `output.txt` does not exist**;
3. creates a method that takes in a file name, opens the file, counts the number of lines (book records) it contains, and returns that count. This method must handle its own exceptions internally;
4. ensures every file resource opened is properly closed (either in a `finally` block or via `try-with-resources`) and that every catch block prints a clear, specific message describing what went wrong.

Console output (`System.out`) is acceptable throughout.

Section B, Question Four

A utility company wants a system representing different types of electricity meters using inheritance and polymorphism.

Q4 · PART 1: SUPERCLASS METERACCOUNT**9 MARKS**

1. create a class named `MeterAccount` with **private** fields to hold the meter number, the holder's name, and the units remaining. Provide a constructor to initialise all three;
2. create a method that adds a given number of units to the balance;
3. create a method that deducts a given number of units from the balance if there are sufficient units; otherwise the transaction should be rejected and a message displayed;
4. create a method that displays the meter's current details: its meter number, holder's name, and units remaining.

Q4 · PART 2: SUBCLASSES**10 MARKS**

1. `PrepaidMeter` extends `MeterAccount`. Override the deduction method so that a deduction is only allowed if the balance **after** it would still be at least **5.0 units**. Otherwise, reject it and display a message;
2. `PostpaidMeter` extends `MeterAccount`. Override the deduction method so that every **successful** deduction automatically adds a **1.0 unit** rebate to the balance afterwards.

Q4 · PART 3: MAIN METHOD**6 MARKS**

1. creates at least one `PrepaidMeter` and one `PostpaidMeter`;
2. adds units to each;
3. performs at least one successful **and** one unsuccessful/rebate-triggering deduction on each;
4. displays each meter's details before and after the transactions;
5. demonstrates polymorphism by storing each object in a `MeterAccount` reference and calling the deduction method through it, confirming that the correct overridden version runs.

Section C, Question Five

SECTION C (25 marks). Answer Question Five **or** Question Six. Use comments to say what each block of code does.

Q5 A · COURIER PRICING

12 MARKS

A courier needs a program (using JOptionPane only, for all input and output) that prices parcels across up to three delivery zones and produces a sales summary. Implement the following four methods.

1. create a method that takes in a zone number. Use a switch statement to return a base rate for Zone 1, Zone 2 or Zone 3 (pick any three distinct rates and state them clearly). If the zone number is not 1, 2 or 3, throw an `IllegalArgumentException`;
2. create a method that takes in a tier and a parcel weight in kg. Use **nested** if-else statements (not a switch, and not independent/separate if checks) to return a surcharge percentage:

Gold, over 20 kg, 5%

Gold, 20 kg or under, 0%

Silver, over 20 kg, 15%

Silver, 20 kg or under, 10%

Bronze (any weight), 25%

any tier string that is not "Gold" or "Silver" must be treated as "Bronze"

The nesting must reflect that the weight is only checked **within** each tier branch, not as a separate top-level check.

3. create a method that takes in a base rate total and a surcharge percentage and applies the surcharge to the base total. If the surcharged amount is above **500.0**, the method should return 500.0 instead (a price ceiling);
4. `main()`
 - ask the user for the number of parcels N (a positive integer);
 - loop exactly N times. In each iteration, ask for that parcel's number of zones crossed (an integer from 1 to 3, inclusive), its tier, and its weight in kg. For each zone from 1 up to that parcel's zone count, call the method from part (i) and add the result to that parcel's running base total (a running total that starts at 0 for each new parcel). Then call the method from part (ii) to get the surcharge percentage, and the method from part (iii) to get that parcel's final price;
 - accumulate: (a) total revenue across all parcels, and (b) a running count of how many parcels were Gold, Silver and Bronze;
5. after the loop, show **one** summary dialog with the total revenue and the three tier counts.

A factory records its shop-floor noise level once every hour, for 20 hours. Write a program, using **JOptionPane** only for input and output, that checks the log for sustained breaches of a safe limit. A reading is a **breach** if it is **above 85 decibels**.

- using a loop, prompt the user 20 times (one dialog per hour) for that hour's noise level. Each input must be validated: it must parse as a number between **0 and 140 inclusive**. If invalid, re-prompt for the **same** hour (using a nested loop/control statement) until a valid value is entered. Store all 20 valid readings in a 1-D array;
- scan the 20-hour array **once** to find every maximal run of consecutive breach-hours. Using only loops and conditionals (no extra arrays), determine:
 - the **total number of qualifying streaks**, where a qualifying streak is any consecutive run of **3 or more** breach-hours;
 - the **length in hours of the single longest** breach streak anywhere in the log;
- using nested if-else logic, classify the shift as exactly one of:
 - ⚡**CRITICAL**: longest streak of 6 hours or more
 - ⚡**WARNING**: longest streak of 3 to 5 hours inclusive
 - 🟢**SAFE**: no streak of 3 or more consecutive breach-hours occurred
- show **one** dialog stating the total number of qualifying streaks, the longest streak length, and the final classification.

M.14 · MOCK B · SECTION C · QUESTION SIX · 25 MARKS

Section C, Question Six

Design and implement a program, using **JOptionPane** only for input and output, that classifies a single user-supplied positive integer **N** according to its digit pattern.

Definition. A number is **monotone** if its digits never decrease from left to right. For example, 1237 and 1119 are monotone; 4821 is not. The **digit product** is every digit multiplied together, so the digit product of 1237 is 42.

Constraints. The entire task must be implemented inside `main()` only, using loops and nested/chained if-else and/or switch logic. **No arrays and no user-defined helper methods are permitted**; everything must be inline.

- using a loop built only on the % and / operators (no converting **N** to a String or an array), compute, **in the same loop**:
 - the **digit product** of **N**, and
 - the **largest digit** of **N**;
- determine, in that same loop, whether **N** is **monotone**;
- inline in `main()` (not a separate method), determine whether the **digit product** from step 1 is itself a **prime** number. Use a loop that tests divisibility only up to the **square root** of the digit product;
- using nested/chained conditional logic, classify **N** into **exactly one** of the following, checked strictly in this priority order:
 - ⚡**MONOTONE-PRIME-PRODUCT**: **N** is monotone AND its digit product is prime
 - ⚡**MONOTONE**: **N** is monotone but its digit product is **not** prime
 - ⚡**ZERO-PRODUCT**: **N** is **not** monotone but contains a digit 0
 - 🟢**STANDARD**: in every other case
- display **N**, its digit product, its largest digit, and its final classification in a **single** **JOptionPane** message dialog.

A library needs a program, using **JOptionPane only** for input and output, to process a batch of overdue books returned on the same day and calculate the total fines owed. The program must use clear separation of concerns: each task below must be its own method. **No collection classes** (for example ArrayList) are allowed anywhere.

- prompt for the number of overdue books, V. Then, using a loop, fill two **parallel arrays** of length V:
 - the number of days each book is late (an integer per book). A value of 0 or below is invalid; use a **nested control statement** to keep re-prompting until a valid value is entered;
 - the member type for each book (a String per book, either "Student" or "Public");
- create a method that takes in one book's days late and member type, and computes that book's fine using **nested if-else** logic (the rules below must be nested together, not checked as separate independent conditions):
 - Student:** the first 3 days are free; each additional day costs 20.00; capped at a maximum of 200.00;
 - Public:** no free period; the first 5 days cost a flat 150.00; each day beyond 5 costs 40.00; capped at a maximum of 500.00;
 - any member type that is not exactly "Student" must be treated as "Public";
- create a method that takes in the array of days late, loops over it and tallies the books into three bands: **SHORT** (1 to 3 days), **MEDIUM** (4 to 10 days), **EXTENDED** (more than 10 days). Return these three counts without using any collection class;
- in `main()`:
 - call the method from part (ii) for every book across both parallel arrays to accumulate the batch's total fines;
 - call the method from part (iii) once;
 - display **one final** JOptionPane showing the total fines and the three band counts.

TEACH · BEFORE YOU TURN TO THE ANSWERS

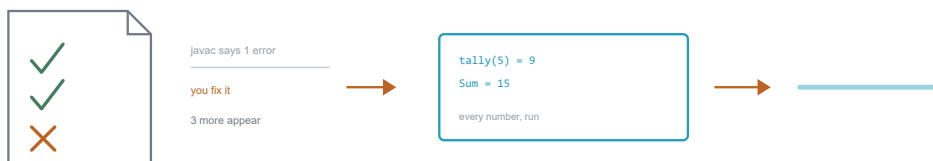
Mark this one against the clock as well as the scheme. Section A should have taken you about 35 minutes. If it took an hour, the problem is not Section A: it is that Sections B and C now have 2 hours between them and they need every minute.

Then compare your two papers. If Mock A scored better, it is not because the old format was easier. It is because you chose four questions and this one chose for you.

PART XIV

Mock Paper Solutions

Every question on both mocks, worked in full, with the marks accounted for line by line. Every program here was compiled and run, and every claim about what a compiler says was made by feeding it the code and reading the reply. Read this part only after you have written your own answers, and mark yourself like a stranger.



Marking Mock A, and the one thing it is really testing

Mark like a stranger. You know what you meant; the marker does not, and neither does the compiler.

● MUST-MEMORISE · THE COMPILER COUNTS ERRORS IN WAVES, NOT IN BUGS

The number of errors javac prints is not the number of bugs you have. Once the compiler cannot work out what a name or a type means, it stops looking inside it, so the bugs hiding behind that one are simply not reported yet. Fix the front row and a new row appears.

Every debug snippet on this mock was fed to a real compiler, and then fed to it again with the reported errors fixed. This is what came back:

SNIPPET	JAVAC SAYS	FIX THOSE, AND	NEVER MENTIONED AT ALL
Q1 B CubeCalcBroken	4 errors	3 more appear	
Q2 B ReverseArrayBroken	1 error	3 more appear	an out-of-bounds crash
Q3 B NoteWriterBroken	1 error	3 more appear	
Q4 B CountUpBroken	3 errors	2 more appear	an infinite loop
Q5 C TotalsBroken	2 errors	4 more appear	the wrong element printed
Q6 C OddFinderBroken	2 errors	3 more appear	a main that cannot run

Read the second column against the third. ReverseArrayBroken and NoteWriterBroken each report **one** error and each contain **four**. A candidate who lists what the compiler listed writes down one error out of four and loses the question.

MEMORY HOOK: the compiler is a **witness**, not an auditor. It reports what it tripped over first. The last column is the one that matters: those bugs are invisible to it entirely, and they are still bugs.

TEACH · WHY "LIST THE ERRORS" IS NEVER "LIST WHAT JAVAC PRINTED"

This is why every debug question on both real papers is worth 3 marks and contains four or five errors. The examiner is not counting your ability to read a compiler transcript. You do not have a compiler in the hall. You are being asked to read code the way the compiler cannot: all the way down, including the parts it gives up on.

So read for **faults**, not for messages, and use F.7's three kinds as your checklist rather than the compiler's list as your answer. **Compile-time** errors are the ones it reports, and it reports them late and in waves. **Run-time** errors it never mentions: ReverseArrayBroken's out-of-bounds crash is perfectly legal Java. **Logic** errors are invisible to the machine entirely, and two of the six snippets above contain one, which is exactly why the exam hands you the correct output: that output is the only instrument that detects them.

Question One, worked in full

Q1 A · FORMAL PARAMETER, ACTUAL ARGUMENT, RETURN TYPE

3 MARKS

Model answer.

A **formal parameter** is the variable named in a method's declaration, inside its brackets. It does not hold a value while the method is not running; it is a name waiting for one, and it exists only inside that method. In `public static int cube(int s)`, the formal parameter is `s`.

An **actual argument** is the value written at the **call site**, which is copied into the formal parameter when the call is made. In `cube(side)`, the actual argument is `side`, and what reaches `s` is a **copy** of its value (Unit 5.4).

The **return type** is the type of the value a method hands back, written immediately before the method's name. In `public static int cube(int s)` it is `int`. A method that returns nothing declares `void`. The return type is **not part of the method signature**, which is why two methods cannot differ by return type alone.

Where the marks are: one each. The word doing the work in the first two is **copy**: parameter is the name, argument is the value, and the value is copied in. Say that and both marks are yours.

Q1 B · DEBUG CUBECALCBROKEN

3 MARKS

Five errors.

1. **Line 2: `main(String[] )` has no parameter name.** Write `main(String[] args)`.
2. **Line 4: no semicolon** after `cube(side)`.
3. **Line 8: `cube(int)` has no parameter name.** Write `cube(int s)`.
4. **Line 9: `s` does not exist.** This is the symptom of error 3, not a separate fault: name the parameter and `s` exists. Say that out loud in your answer. It shows you found the cause rather than the squiggle.
5. **The closing brace is missing** for `cube` and for the class. Only one `}` is present where two are needed.

What the compiler actually said: four errors, and **not** the one about `s`. It reported the two missing names, the missing semicolon, and "reached end of file while parsing" for the brace. Only after those are fixed does it turn to line 9 and report `cannot find symbol: variable s` three times, once per use.

Three marks, five errors, and the fifth is invisible until the third is fixed. Do not stop at three.

CODE · Q1 B CORRECTED

VERIFIED ON THE JVM

```

1 public class CubeCalc {
2     public static void main(String[] args) {
3         int side = 4;
4         double result = cube(side);
5         System.out.println("Cube is: " + result);
6     }
7
8     public static int cube(int s) {
9         return s * s * s;
10    }
11 }
```

OUTPUT

```
Cube is: 64.0
```

64.0, not 64. `cube` returns an `int`, and that `int` is stored into a `double result`. Java **widens** it on the way in (Unit 5.7), so what is printed is a `double`, and a `double` always prints its point. Writing "Cube is: 64" throws away a mark that costs nothing.

Q1 C · DRY RUN TALLY WITH 3 AND 5

4 MARKS

The answers are 4 and 9. Not 5 and 10.5, which is what the arithmetic gives if you read $(k + 1) / 2$ as a fraction.

Both $k + 1$ and 2 are int, so $/$ is **integer division** and every term is chopped down to a whole number (Unit 3.6). Nothing rounds: $3/2$ is 1, not 1.5 and not 2.

DRY RUN · TALLY(5), ONE ROW PER ITERATION

K	K + 1	(K + 1) / 2	AS REAL ARITHMETIC	TOTAL
1	2	1	1.0	1
2	3	1	1.5	2
3	4	2	2.0	4
4	5	2	2.5	6
5	6	3	3.0	9

tally(3) stops after the third row: 1, 2, 4. **tally(5)** runs all five: 9.

Column four is not part of the answer. It is there to show you the trap: the two columns agree on rows 1, 3 and 5 and disagree on rows 2 and 4, so the loss is exactly 0.5 twice, and the real-arithmetic total of 10.5 becomes 9. A candidate who never notices the division is an int division still gets rows 1, 3 and 5 right, which is precisely why this trap survives: it is half invisible.

CODE · Q1 C, THE SAME METHOD WITH A MAIN THAT CALLS IT

VERIFIED ON THE JVM

```

1 public class TallyTrace {
2
3     public static int tally(int n) {
4         int total = 0;
5         for (int k = 1; k <= n; k++) {
6             total += (k + 1) / 2;
7         }
8         return total;
9     }
10
11    public static void main(String[] args) {
12        System.out.println("tally(3) = " + tally(3));
13        System.out.println("tally(5) = " + tally(5));
14    }
15 }

```

OUTPUT

```

tally(3) = 4
tally(5) = 9

```

Q1 D · OVERLOAD VOLUME

7.5 MARKS

Read the three shapes before you write anything. A cube needs one length, a cylinder needs a radius and a height, a box needs three lengths. One parameter, two parameters, three parameters: the three overloads differ by **arity**, so the compiler can always tell them apart.

```

1 public class VolumeOverload {
2
3     // i. a cube: one length is all it takes
4     public static double volume(double side) {
5         return side * side * side;
6     }
7
8     // ii. a cylinder: a radius and a height
9     public static double volume(double radius, double height) {
10        return Math.PI * radius * radius * height;
11    }
12
13    // iii. a box: a length, a width and a height
14    public static double volume(double length, double width, double height) {
15        return length * width * height;
16    }
17
18    public static void main(String[] args) {
19        System.out.printf("Cube, side 4:      %.2f\n", volume(4.0));
20        System.out.printf("Cylinder, r 3, h 5:  %.2f\n", volume(3.0, 5.0));
21        System.out.printf("Box, 2 by 3 by 4:   %.2f\n", volume(2.0, 3.0, 4.0));
22    }
23 }

```

OUTPUT

```

Cube, side 4:      64.00
Cylinder, r 3, h 5: 141.37
Box, 2 by 3 by 4:  24.00

```

⚠ TRAP · THIS QUESTION IS ANSWERABLE. ITS ANCESTOR WAS NOT

Compare this with the 2024/25 paper's Question One D, solved in S.9. That one asked for area overloaded for a circle, a rectangle and a triangle. A circle takes one length, but a rectangle takes two and a triangle takes two, so two of the three overloads have the **same signature**, `area(double, double)`, and the compiler rejects the class outright with "method `area(double,double)` is already defined". **That question, as printed, cannot be answered as printed.**

`volume` escapes because 1, 2 and 3 lengths are three different counts. That is luck, not design, and it is worth understanding rather than enjoying: **overloading resolves on the parameter list alone**. Not on the names you give the parameters, not on what you meant, and not on the return type. Two shapes that need the same numbers cannot be overloads, and the fix is to name them `rectangleArea` and `triangleArea`, or to make the lists genuinely differ.

RECALL AND CHECK

Why can `volume(double side)` and `volume(double radius, double height)` coexist, when `area(double, double)` twice cannot?

ANSWER Because the signature is the name plus the **parameter list**, and one parameter is a different list from two. The two area methods have identical lists, so they have identical signatures, so they are the same method declared twice. What the parameters *mean* to you is not in the signature.

Question Two, worked in full

Q2 A · DECLARATION, CREATION, INDEX, PARALLEL ARRAYS

3 MARKS

Model answer.

Array declaration announces a name and its type: `int[] a;`. It creates **no array**. It creates a reference variable capable of referring to one, and until something is assigned to it the variable refers to `null`.

Array creation is what actually makes the array and reserves its elements: `a = new int[4];`, or the shorthand `int[] a = {1, 3, 5, 7};` which declares and creates in one line. The length is fixed at the moment of creation and can never change afterwards.

The **index** is the position of an element, counted **from 0**. In an array of length 4 the valid indexes are 0, 1, 2 and 3, so the last index is always `length - 1` and `a[a.length]` is always wrong.

Parallel arrays are two or more arrays of the same length in which the same index refers to the same subject: `names[2]` and `ages[2]` are the name and the age of one person. They are the way this course models a record without a class and without a collection.

Where the marks are: the distinction in the first two is the whole point. "Declaration names it, creation makes it." A declaration that is never followed by a creation is exactly the bug in part B.

Q2 B · DEBUG REVERSEARRAYBROKEN

3 MARKS

Four errors. The compiler reports one.

1. **Line 5: i is not an update.** The third slot of a `for` must be a statement; `i` on its own is an expression that does nothing. Write `i++`. This is the only error `javac` reports: "not a statement".
2. **Line 4: `int b;` declares an int, not an array,** and never creates anything. Write `int[] b = new int[a.length];`.
3. **Line 5: `i <= a.length` is off by one.** Write `i < a.length`.
4. **Line 6: `a[a.length - i]` is out of bounds** the moment `i` is 0, because that is `a[4]` in an array of length 4. Reversing needs `a[a.length - 1 - i]`.
5. **Line 8: `i` does not exist here.** It was declared inside the `for`, so it dies with it. Printing the array needs its own loop.

What the compiler actually said: one error, "not a statement". Fix that alone and three more appear: "array required, but int found" twice, and "cannot find symbol: variable i". Error 4 is never reported by the compiler at all, because it is not a compile error: it is an `ArrayIndexOutOfBoundsException` that waits until the program is running, and the message it eventually gives is `Index 4 out of bounds for length 4`.

One reported error. Four real ones. This snippet is the whole argument of MS.1 in nine lines.

CODE · Q2 B CORRECTED

VERIFIED ON THE JVM

```

1 public class ReverseArray {
2     public static void main(String[] args) {
3         int[] a = {1, 3, 5, 7};
4         int[] b = new int[a.length];
5         for (int i = 0; i < a.length; i++) {
6             b[i] = a[a.length - 1 - i];
7         }
8         for (int i = 0; i < b.length; i++) {
9             System.out.println(b[i]);
10        }
11    }
12 }

```

OUTPUT

```

7
5
3
1

```

Q2 C · DRY RUN MAXORDROP

4 MARKS

The question asks for the output **of each iteration** and the final output, so a bare "8" is not the answer even though 8 is the final value. Write the four lines.

DRY RUN · MAXORDROP, ONE ROW PER ITERATION

I	DATA[I]	DATA[I] > X?	WHAT HAPPENS	X AFTER
<i>before the loop:</i> x = data[0]				5
1	3	3 > 5? no	x = x - 3	2
2	9	9 > 2? yes	x = 9	9
3	1	1 > 9? no	x = x - 1	8

The else is what makes this worth tracing. The variable is not a running maximum: when the new value does not beat it, it is **subtracted from it**, so x can fall as well as rise, and the final 8 is not any element of the array.

```
1 public class MaxOrDrop {
2     public static void main(String[] args) {
3         int[] data = {5, 3, 9, 1};
4         int x = data[0];
5         for (int i = 1; i < data.length; i++) {
6             if (data[i] > x) {
7                 x = data[i];
8             } else {
9                 x = x - data[i];
10            }
11            System.out.println("i = " + i + ", x = " + x);
12        }
13        System.out.println("Final x = " + x);
14    }
15 }
```

OUTPUT

```
i = 1, x = 2
i = 2, x = 9
i = 3, x = 8
Final x = 8
```

```

1  import java.util.Scanner;
2
3  public class ArrayReport {
4
5      // ii a. find, count and display the multiples of 5
6      public static void multiplesOfFive(int[] values) {
7          int count = 0;
8          String found = "";
9          for (int i = 0; i < values.length; i++) {
10             if (values[i] % 5 == 0) {
11                 count++;
12                 found = found.equals("") ? "" + values[i] : found + ", " + values[i];
13             }
14         }
15         System.out.println("Multiples of 5: " + found);
16         System.out.println("How many: " + count);
17     }
18
19     // ii b. sum and display everything that is not a multiple of 5
20     public static void sumOfTheRest(int[] values) {
21         int sum = 0;
22         for (int i = 0; i < values.length; i++) {
23             if (values[i] % 5 != 0) {
24                 sum += values[i];
25             }
26         }
27         System.out.println("Sum of the rest: " + sum);
28     }
29
30     public static void main(String[] args) {
31         Scanner in = new Scanner(System.in);
32
33         // i. prompt for 8 values
34         int[] values = new int[8];
35         for (int i = 0; i < values.length; i++) {
36             values[i] = Integer.parseInt(in.nextLine());
37         }
38
39         multiplesOfFive(values);
40         sumOfTheRest(values);
41
42         // iii. a for loop in main to find and display the minimum
43         int min = values[0];
44         for (int i = 1; i < values.length; i++) {
45             if (values[i] < min) {
46                 min = values[i];
47             }
48         }
49         System.out.println("Minimum: " + min);
50     }
51 }

```

OUTPUT, FOR THE INPUTS 12, 25, -4, 30, 7, 15, 3, 20

Multiples of 5: 25, 30, 15, 20

How many: 4

Sum of the rest: 18

Minimum: -4

Input is a Scanner here so the run can be printed on a page. In the hall, each prompt is `JOptionPane.showInputDialog` and each display is `showMessageDialog` (F.11). Everything else is unchanged.

⚠ TRAP · "-4 % 5" IS NOT WHAT YOU EXPECT, AND HERE IT DOES NOT MATTER

-4 % 5 in Java is **-4**, not 1: the remainder takes the sign of the **left** operand (Unit 3.6). So the test `values[i] % 5 == 0` still correctly rejects -4, because -4 is not 0 either way.

But had the question asked for **odd** numbers and you had written `values[i] % 2 == 1`, every negative odd number would have failed the test, because -7 % 2 is **-1**. That is why this book always writes `% 2 != 0` for odd. It is right for negatives; the other is not.

MS.4 · MOCK A SOLUTIONS · QUESTION THREE

Question Three, worked in full

Q3 A · FILEWRITER, BUFFEREDREADER, CHECKED EXCEPTION

3 MARKS

Model answer.

FileWriter is a class in `java.io` that writes characters to a file. Constructing one, `new FileWriter("log.txt")`, creates the file if it does not exist and **truncates it if it does**; the two-argument form `new FileWriter("log.txt", true)` appends instead. It writes characters one at a time, which is why it is usually wrapped in something more convenient, such as `PrintWriter` or `BufferedWriter`.

BufferedReader is a class in `java.io` that wraps another reader and reads it in blocks rather than a character at a time. It is worth naming for one method: `readLine()`, which returns the next line as a `String`, or `null` at end of file. That `null` is the loop's stopping condition.

A **checked exception** is one the compiler **insists** you deal with: you must either catch it or declare it with `throws`, and if you do neither the program does not compile. `IOException` is the example that matters here. The reasoning is that a missing file is not a programming mistake, it is a fact about the world, so the compiler forces you to have a plan for it. An **unchecked** exception, such as `ArrayIndexOutOfBoundsException`, is not policed this way, because it is a bug rather than a circumstance.

Where the marks are: for the third, the phrase "caught or declared, or it will not compile" is the mark. Part B is that sentence with the compiler saying it out loud.

Q3 B · DEBUG NOTEWRIERBROKEN

3 MARKS

Four errors, in two families. The compiler reports one.

1. **Line 5: `FileWriter("log.txt")` has no `new`.** As written, that is a call to a method named `FileWriter`, which does not exist. A constructor is only a constructor when `new` is in front of it. This is the only error `javac` reports: "cannot find symbol: method `FileWriter(String)`".
2. **Lines 5, 6 and 7 all throw `IOException`, which is checked**, and nothing here catches or declares it. Three errors, one cause. Wrap the work in `try (FileWriter out = new FileWriter("log.txt")) { ... } catch (IOException e) { ... }`, or declare `throws IOException` on `main`.
3. **The file is never closed on the failure path.** Even with a catch, a plain `out.close()` at the end is skipped if `write` throws. **`try-with-resources`** closes it either way, which is exactly why it exists.

What the compiler actually said: one error, about `new`. The three unreported exception `IOException` errors are invisible until it is fixed, because until then the compiler does not know what type `FileWriter("log.txt")` has, so it cannot know what that line throws.

This is the single best illustration on the paper of MS.1's rule. The **interesting** error, the one the question is really about, is the one the compiler will not tell you about until you have fixed the boring one.

```
1 import java.io.FileWriter;
2 import java.io.IOException;
3
4 public class NoteWriter {
5     public static void main(String[] args) {
6         try (FileWriter out = new FileWriter("log.txt")) {
7             out.write("Session started");
8         } catch (IOException e) {
9             System.out.println("Could not write log.txt: " + e.getMessage());
10        }
11        System.out.println("log.txt written");
12    }
13 }
```

OUTPUT

```
log.txt written
```

Q3 C · WHAT LANDS IN REPORT.TXT

4 MARKS

The answer, three lines:

```
REPORT.TXT
Unit: 7
Rate: 12.3
Total:  3.14
```

Four things are being tested here at once, and each is worth a mark.

1. **print then println.** `print("Unit: ")` adds no line break, so the 7 lands on the same line and **then** the line ends. One line, not two.
2. **%.1f of 12.345 is 12.3.** One decimal place means the 3 stays and everything after it decides whether it moves. What follows is 45, and 0.045 is less than half of 0.1, so it does not. See the box below, because the reason this is *ordinary* is itself worth a mark.
3. **%n ends the line** after the rate.
4. **%5.2f of 3.14159 is 3.14:** two decimals, and then **padded to a total width of 5**, which is 4 characters wide, so one space goes in front. The line reads `Total:` , a space from the literal, then a **second** space from the padding.

And there is no line break after the last one, because `printf("Total: %5.2f", ...)` has no `%n`. The file ends without a newline.

To answer any printf rounding question: **take the literal as printed, round it half up to the places asked for, and write that down.** That is the answer. Every time.

You may have been told the frightening version: that double is inexact, so `%.2f` of `1.005` comes out as `1.00` because `1.005` is really stored as `1.0049999999999999`. The first half is true. **The conclusion is false in Java**, and it is worth knowing why, because it is true in some other languages and the folklore travels.

Java's printf does not round the raw binary value. It first takes the **shortest decimal that uniquely identifies** that double, which for the literal `1.005` is the string `1.005`, and rounds **that**, half up. So it prints **1.01**, exactly as your paper arithmetic says.

This was measured, not assumed. Every three-decimal value from `0.001` to `99.999` was formatted with `%.2f` on a real JVM and compared against both rules:

RULE	VALUES TESTED	DISAGREEMENTS
round the digits you wrote, half up	99,999	0
round the stored binary value, half up	99,999	4,800

Zero. The scary rule is the one that is wrong, and it is wrong about one value in twenty: at `0.015`, printf prints **0.02** while the stored value sits just below `0.015` and the folklore predicts `0.01`.

MEMORY HOOK: **printf reads what you wrote, not what the hardware kept.** Unit 3.7's warning about inexact doubles is about **arithmetic**, where the error accumulates and is real. It is not about displaying a literal.

⚠ TRAP · THE INEXACTNESS IS REAL. IT JUST SHOWS UP SOMEWHERE ELSE

Do not read the box above as "doubles are fine". They are not, and Unit 3.7 means what it says. The difference is **where** the damage lands.

Displaying a literal is safe, because printf rounds the decimal you wrote. **Comparing** and **accumulating** are not: `0.1 + 0.2 == 0.3` is false, and no formatting rule saves you from that, because the damage happened in the addition, before printf ever saw it.

So the rule has a boundary, and it is this: trust printf to show a value faithfully; never trust `==` between two doubles.

```
1 import java.io.*;
2
3 public class ReportWriter {
4     public static void main(String[] args) throws IOException {
5         PrintWriter pw = new PrintWriter("report.txt");
6         pw.print("Unit: ");
7         pw.println(7);
8         pw.printf("Rate: %.1f%n", 12.345);
9         pw.printf("Total: %5.2f", 3.14159);
10        pw.close();
11
12        // Not part of the answer: read report.txt back so this book can show you
13        // exactly what landed in it. A dot marks every space, because %5.2f pads
14        // with spaces you cannot see on paper.
15        BufferedReader in = new BufferedReader(new FileReader("report.txt"));
16        String line = in.readLine();
17        int n = 0;
18        while (line != null) {
19            n++;
20            System.out.println("line " + n + ": " + line.replace(' ', '.'));
21            line = in.readLine();
22        }
23        in.close();
24    }
25 }
```

OUTPUT, WITH EVERY SPACE SHOWN AS A DOT

```
line 1: Unit:.7
line 2: Rate:.12.3
line 3: Total:..3.14
```

The read-back is not part of the answer; it is how this book proves the answer instead of asserting it. Count the dots on line 3: **two**. One is the space you typed inside "Total: " and the other is %5.2f padding 3.14 out to five columns. On line 2, **one** dot, and 12.3 after it.

```

1 import java.io.BufferedReader;
2 import java.io.FileReader;
3 import java.io.FileWriter;
4 import java.io.IOException;
5 import java.io.PrintWriter;
6
7 public class BookTable {
8     public static void main(String[] args) {
9         String[] titles = {"Algebra", "Optics", "Ecology"};
10        int[] copies = {12, 4, 7};
11        double[] prices = {2500.0, 1875.5, 990.25};
12
13        // iv. append, do not overwrite: the append flag belongs to the FileWriter
14        // underneath. i. FileWriter also creates books.txt if it does not exist.
15        try (PrintWriter out = new PrintWriter(new FileWriter("books.txt", true))) {
16
17            // iii. printf for column alignment: %-10s is left-aligned in 10 columns
18            out.printf("%-10s %-7s %-9s\n", "Title", "Copies", "Price");
19            for (int i = 0; i < titles.length; i++) {
20                out.printf("%-10s %-7d %-9.2f\n", titles[i], copies[i], prices[i]);
21            }
22
23        } catch (IOException e) {
24            // v. handle possible exceptions properly
25            System.out.println("Could not write books.txt: " + e.getMessage());
26        }
27
28        // Not part of the answer: read the file back so the book can show you what
29        // landed in it. A dot marks each trailing space that printf's padding leaves.
30        try (BufferedReader in = new BufferedReader(new FileReader("books.txt"))) {
31            String line = in.readLine();
32            while (line != null) {
33                System.out.println(line.replace(' ', '.'));
34                line = in.readLine();
35            }
36        } catch (IOException e) {
37            System.out.println("Could not read books.txt back: " + e.getMessage());
38        }
39    }
40 }

```

BOOKS.TXT AFTER ONE RUN, EVERY SPACE SHOWN AS A DOT

```

Title.....Copies..Price....
Algebra....12.....2500.00..
Optics....4.....1875.50..
Ecology....7.....990.25...

```

The dots are this book showing you the padding; the file has spaces. Read the alignment: `%-10s` puts the title in ten columns, left-aligned, and `%-9.2f` puts the price in nine, so the columns line up whatever the values are.

● MUST-MEMORISE · APPEND LIVES ON THE FILEWRITER, NOT THE PRINTWRITER

`new PrintWriter("books.txt")` **truncates**. There is no append flag on that constructor.
`new PrintWriter(new FileWriter("books.txt", true))` **appends**.

Part iv of this question says "ensure the file is appended to if it already exists", and it is the whole reason the two classes are stacked. The `true` belongs to the `FileWriter` underneath; `PrintWriter` is only there to give you `printf`.

MEMORY HOOK: the flag is on the writer that touches the disk. `PrintWriter` never touches the disk; it formats and hands down. Ask "who opens the file?" and the answer tells you where `true` goes.

Question Four, worked in full

Q4 A · COUNTER-CONTROLLED, SENTINEL-CONTROLLED, CONTINUE

3 MARKS

Model answer.

A **counter-controlled** loop repeats a number of times that is known before the loop starts. It counts. A for loop is its natural shape, because the start, the test and the step sit together on one line: `for (int i = 0; i < 8; i++)` runs exactly 8 times. It is also called a **definite** loop.

A **sentinel-controlled** loop repeats until a particular value arrives to say stop. That value is the **sentinel**, and it is not data: it is a signal, chosen so it cannot be mistaken for a real reading. Reading lines until `readLine()` returns `null` is sentinel-controlled, and so is "keep asking until the user types -1". The number of repetitions is not known in advance, which is why a while is its natural shape. It is also called an **indefinite** loop.

continue abandons the **rest of the current iteration** and goes straight to the next one. It does not leave the loop: that is **break**. In a for loop the update still runs, so **continue** is safe there; in a while loop it jumps straight back to the test, so if the increment sits below it, the loop never moves again and hangs (Unit 4.7).

Where the marks are: "known in advance" against "known only when it happens" carries the first two. For the third, the distinction from **break** is the mark: **continue skips an iteration, break leaves the loop**.

Q4 B · DEBUG COUNTUPBROKEN

3 MARKS

Four errors, and the worst one compiles.

1. **Line 3: no semicolon** after `int count = 1`.
2. **Line 4: float output = ""**; is the wrong type. A float is a number and "" is text; there is no conversion between them. It must be `String output = ""`;
3. **Line 6: while (count <= 5);** has a stray semicolon. That semicolon **is** the loop body: an empty statement. The block below it then runs once, on its own, as an ordinary block. Nothing inside the empty body ever changes `count`, so the test stays true forever and the program **hangs**. Delete the semicolon.
4. **Line 8: Count is not a statement**, and it is not `count` either, because Java is case sensitive. It should be `count++`;

What the compiler actually said: three errors: the missing semicolon, and `Count` twice ("not a statement" and "';' expected"). Fix those and **two more** appear, both incompatible types: `String` cannot be converted to `float`. And error 3, the stray semicolon, is **never reported at all**. It was compiled on its own to check: it is legal Java. It is an infinite loop, and the compiler has no opinion about those.

This snippet is the clearest case on the paper: **the most damaging bug in it is the one the compiler is happiest with**.

```

1 public class CountUp {
2     public static void main(String[] args) {
3         int count = 1;
4         String output = "";
5
6         while (count <= 5) {
7             output = output + count + " ";
8             count++;
9         }
10
11         System.out.println(output);
12     }
13 }

```

OUTPUT

1 2 3 4 5

There is a trailing space after the 5, because the loop appends `count + " "` every time round, including the last. The question asked for 1 2 3 4 5 and this is what that instruction produces. Do not invent a fix the question did not ask for.

Q4 C · STATE THE OUTPUT OF EACH ITERATION AND THE FINAL SUM

4 MARKS

Three lines, then the total. The loop runs five times and prints three, because `continue` jumps over the print.

DRY RUN · SKIPTOTAL, ONE ROW PER ITERATION

I	INPUTS[I]	NEGATIVE?	WHAT HAPPENS	PRINTED
0	4	no	sum = 0 + 4 = 4	i = 0, sum = 4
1	-2	yes	continue: skip the rest	<i>nothing</i>
2	6	no	sum = 4 + 6 = 10	i = 2, sum = 10
3	-8	yes	continue: skip the rest	<i>nothing</i>
4	5	no	sum = 10 + 5 = 15	i = 4, sum = 15

Look at the `i` values that print: **0, 2, 4**. The gaps are the evidence. A candidate who writes three lines numbered 0, 1, 2 has read `continue` as "do not add" rather than "skip the rest of this turn", and loses a mark on a detail they never looked at.

```
1 public class SkipTotal {
2     public static void main(String[] args) {
3         int sum = 0;
4         int[] inputs = {4, -2, 6, -8, 5};
5
6         for (int i = 0; i < inputs.length; i++) {
7             if (inputs[i] < 0) {
8                 continue;
9             }
10            sum += inputs[i];
11            System.out.println("i = " + i + ", sum = " + sum);
12        }
13        System.out.println("Sum = " + sum);
14    }
15 }
```

OUTPUT

```
i = 0, sum = 4
i = 2, sum = 10
i = 4, sum = 15
Sum = 15
```

⚠ TRAP · CONTINUE HERE, BREAK ON THE REAL PAPER, AND THE KEYWORD IS THE WHOLE QUESTION

The 2024/25 paper's Question Four C is this question with `break` instead of `continue`, over the array {3, 4, -1, 2}. It answers 7: the loop **stops** at -1, so the 2 sitting behind it is never reached and is silently missing from a sum that looks complete.

Here, `continue` only skips the negatives and carries on, so the 5 behind -8 **is** counted and the answer is 15. Same array shape, same question shape, one keyword, and a different answer.

Both have been asked. When you meet either keyword in a dry-run question, say which one it is out loud before you trace a single row.

```

1  import java.util.Scanner;
2
3  public class SpeedCheck {
4
5      // i. the state: one data field
6      private double speed;
7
8      // ii. a constructor that sets the temperature-equivalent: the speed
9      public SpeedCheck(double speed) {
10         this.speed = speed;
11     }
12
13     // iii. if-else, in one chain, so exactly one verdict can be reached
14     public void evaluateSpeed() {
15         if (speed < 40) {
16             System.out.println("Too Slow");
17         } else if (speed <= 100) {
18             System.out.println("Normal");
19         } else {
20             System.out.println("Too Fast");
21         }
22     }
23
24     public static void main(String[] args) {
25         Scanner in = new Scanner(System.in);
26
27         // iv. prompt, build the object, display the evaluation
28         double entered = Double.parseDouble(in.nextLine());
29         SpeedCheck check = new SpeedCheck(entered);
30         check.evaluateSpeed();
31     }
32 }

```

OUTPUT, FOR THE INPUT 75

Normal

Input is a Scanner here so the run can be printed on a page. In the hall it is `Double.parseDouble(JOptionPane.showInputDialog("Enter the speed:"))` and the verdict goes out through `showMessageDialog` (F.11).

● MUST-MEMORISE · "BETWEEN 40 AND 100 INCLUSIVE" IS ONE ELSE-IF, NOT TWO TESTS

```
if (speed < 40) ... else if (speed <= 100) ... else ...
```

The middle branch needs **no** lower bound. Reaching it already proves `speed >= 40`, because the first branch did not fire.

Writing `else if (speed >= 40 && speed <= 100)` is not wrong and costs only ink. But the same misunderstanding written as three separate `if` statements **is** wrong, because then nothing guarantees anything and every test runs on every value. Unit 4.4 is the paper's own obsession for this reason.

MEMORY HOOK: an else-if arrives already knowing what failed. Its job is only the upper bound. Every branch you reach carries the previous conditions' failure behind it, for free.

```

1 public class SpeedBench {
2
3     private double speed;
4
5     public SpeedBench(double speed) {
6         this.speed = speed;
7     }
8
9     public void evaluateSpeed() {
10        if (speed < 40) {
11            System.out.println(speed + " -> Too Slow");
12        } else if (speed <= 100) {
13            System.out.println(speed + " -> Normal");
14        } else {
15            System.out.println(speed + " -> Too Fast");
16        }
17    }
18
19    public static void main(String[] args) {
20        double[] probes = {39.9, 40.0, 75.0, 100.0, 100.1};
21        for (int i = 0; i < probes.length; i++) {
22            new SpeedBench(probes[i]).evaluateSpeed();
23        }
24    }
25 }

```

OUTPUT

```

39.9 -> Too Slow
40.0 -> Normal
75.0 -> Normal
100.0 -> Normal
100.1 -> Too Fast

```

Not part of the answer: this is how you check an if-else ladder. Both edges of "inclusive" are probed, 40.0 and 100.0, and both land in **Normal**. Test the boundaries, never the middle: 75 would have passed even with < written where <= belongs, and told you nothing.

MS.6 · MOCK A SOLUTIONS · QUESTION FIVE

Question Five, worked in full

Q5 A · SUPER, @OVERRIDE, RUNTIME POLYMORPHISM

3 MARKS

Model answer.

super is a keyword that refers to the superclass part of the current object. It has two uses. `super(...)`, as the **first statement** of a constructor, calls the superclass constructor, which is how a subclass gets the inherited fields initialised. `super.method()` calls the superclass version of a method that the subclass has overridden, which is how an override extends the original rather than replacing it.

@Override is an annotation placed above a method to state that it is meant to override an inherited one. It changes nothing at run time. Its whole value is that the **compiler checks the claim**: if the method does not actually override anything, because the name or the parameter list does not match, it is a compile error instead of a silent new method that never gets called.

Runtime polymorphism, also called dynamic method dispatch, is this: the method that runs is chosen by the **object's actual class at run time**, not by the type of the reference variable at compile time. So an Employee reference holding a Manager runs Manager's `displayInfo()`. It is what makes overriding useful, and it is the opposite of overloading, which is settled entirely at compile time (Unit 5.6).

Where the marks are: for the third, the sentence that earns it is "the **object** decides, not the reference". For the second, "the compiler checks it" is the mark; "it makes it override" is wrong, and that is the commonest answer.

The question asks for the method definition only, so this is the whole answer:

```
public static int countAbove(double[] values, double limit) {
    int count = 0;
    for (int i = 0; i < values.length; i++) {
        if (values[i] > limit) {
            count++;
        }
    }
    return count;
}
```

Three marks and four decisions: the **return type** is int, because it returns a count; the **parameters** are the array and the limit; the loop runs `i < values.length`; and the test is `>`, because the question said **strictly** greater. Write `>=` and you have answered a different question.

```
1 public class CountAboveDemo {
2
3     public static int countAbove(double[] values, double limit) {
4         int count = 0;
5         for (int i = 0; i < values.length; i++) {
6             if (values[i] > limit) {
7                 count++;
8             }
9         }
10        return count;
11    }
12
13    public static void main(String[] args) {
14        double[] marks = {45.0, 70.0, 39.5, 70.0, 88.25};
15        System.out.println("above 70.0: " + countAbove(marks, 70.0));
16        System.out.println("above 39.5: " + countAbove(marks, 39.5));
17        System.out.println("above 99.0: " + countAbove(marks, 99.0));
18    }
19 }
```

OUTPUT

```
above 70.0: 1
above 39.5: 4
above 99.0: 0
```

Not part of the answer, but read the first line: the array holds 70.0 twice and "above 70.0" is **1**, not 3. That is `>` doing exactly what was asked. The third line proves the method survives a limit nothing beats.

Five errors, and one of them prints the wrong answer without ever failing.

1. **Line 3: `int values = {4, 8, 12, 16};` is not an array.** Write `int[] values`. Everything else on the list is downstream of this one.
2. **Line 4: `values[3]` is not the third value.** Indexes count from 0, so the third value is `values[2]`, which is **12**. `values[3]` is 16, the fourth. This is a **logic** error: it compiles, it runs, and it prints a number that is simply wrong.
3. **Line 6: `i <= values.length` is off by one** and would throw `ArrayIndexOutOfBoundsException` at `values[4]`. Write `i < values.length`.
4. **Line 6: `i` is not an update.** Write `i++`.
5. **Line 9: `"Sum is: " + )` is missing its operand.** Write `"Sum is: " + sum`.

What the compiler actually said: two errors, "not a statement" and "illegal start of expression". Fix those and **four more** appear, all downstream of error 1: "illegal initializer for int", "array required, but int found" twice, and "int cannot be dereferenced" for `values.length`. And error 2 is **never reported at any stage**, because printing the wrong element is not illegal. It is just wrong.

The output after correction:

```
OUTPUT
Third value is: 12
Sum is: 40
```

$4 + 8 + 12 + 16 = 40$. If you answered "Third value is: 16" you corrected the compiler's errors and left the examiner's error in place, which is the one this question is actually about.

```
1 public class Totals {
2     public static void main(String[] args) {
3         int[] values = {4, 8, 12, 16};
4         System.out.println("Third value is: " + values[2]);
5         int sum = 0;
6         for (int i = 0; i < values.length; i++) {
7             sum += values[i];
8         }
9         System.out.println("Sum is: " + sum);
10    }
11 }
```

```
OUTPUT
Third value is: 12
Sum is: 40
```

```

1 // i. the superclass
2 class Employee {
3     protected int salary;
4
5     // i b. a constructor that initialises the state
6     public Employee(int salary) {
7         this.salary = salary;
8     }
9
10    // i c. the method the subclass will override
11    public void displayInfo() {
12        System.out.println("Employee salary: N" + salary + " per month");
13    }
14 }
15
16 // ii. the subclass overrides displayInfo()
17 class Manager extends Employee {
18
19     public Manager(int salary) {
20         super(salary);
21     }
22
23     @Override
24     public void displayInfo() {
25         System.out.println("Manager salary: N" + salary
26             + " per month. This post carries a leadership allowance.");
27     }
28 }
29
30 public class StaffDemo {
31     public static void main(String[] args) {
32         // iii a, b. one of each
33         Employee e = new Employee(80000);
34         Manager m = new Manager(250000);
35
36         // iii c. the same call, on two objects, reaching two different methods
37         e.displayInfo();
38         m.displayInfo();
39
40         // the same again through one superclass reference: this is the polymorphism
41         Employee[] staff = {new Employee(80000), new Manager(250000)};
42         for (int i = 0; i < staff.length; i++) {
43             staff[i].displayInfo();
44         }
45     }
46 }

```

OUTPUT

```

Employee salary: N80000 per month
Manager salary: N250000 per month. This post carries a leadership allowance.
Employee salary: N80000 per month
Manager salary: N250000 per month. This post carries a leadership allowance.

```

The last two lines are the point. They come from an `Employee[]` array, so at compile time both slots are just `Employee`. At run time each object runs **its own** `displayInfo()`. That is polymorphism, and the array is the shortest honest way to show it: one call site, two behaviours, chosen by the object.

⚠ TRAP · PROTECTED, NOT PRIVATE, OR THE SUBCLASS CANNOT SEE THE FIELD

salary is declared protected, and that is deliberate. Declare it private and Manager's displayInfo() **will not compile**: "salary has private access in Employee". A subclass inherits a private field in the sense that the object carries it, but it **cannot name it**.

Both ways are defensible in real code, and private plus a getter is the better habit, which is exactly what the 2025/26 Question Four demands. Here the question said "an instance variable salary" and the subclass must print it, so protected is the shortest correct reading. Say which you chose and why, in a comment. That is a mark for one line.

MS.7 · MOCK A SOLUTIONS · QUESTION SIX

Question Six, worked in full

CODE · QUESTION SIX, PART A

VERIFIED ON THE JVM

```
1 public class LoopDemo {
2     public static void main(String[] args) {
3         // i. a for loop that prints the even numbers 2 to 10
4         for (int i = 2; i <= 10; i += 2) {
5             System.out.print(i + " ");
6         }
7         System.out.println();
8
9         // ii. a while loop that prints 9 down to 1
10        int n = 9;
11        while (n >= 1) {
12            System.out.print(n + " ");
13            n--;
14        }
15        System.out.println();
16
17        // iii. a do-while that prints 100 exactly once
18        int k = 100;
19        do {
20            System.out.println(k);
21        } while (false);
22    }
23 }
```

OUTPUT

```
2 4 6 8 10
9 8 7 6 5 4 3 2 1
100
```

One mark each, and each one is a different loop **because of what it knows**. The for counts and steps by 2. The while counts down, and its n-- must be inside the body or it never ends. The do-while is the only loop that runs its body **before** its test, which is exactly why it can print once with a condition that is false from the very start.

● MUST-MEMORISE · DO-WHILE RUNS FIRST AND ASKS AFTERWARDS

do { ... } while (false); runs its body **exactly once**.
while (false) { ... } runs its body **exactly zero times**.

That single difference is the entire reason do-while exists, and "print only the number 10 once" is a question built to test it. Note the **semicolon** after while (false): a do-while is the one loop that ends in one, and leaving it off is a compile error.

MEMORY HOOK: do first, ask later. A do-while is a shop that lets you in and then checks your ticket. You are always getting in at least once.

Q6 B · THE RETURN VALUE OF COMPUTE(2736)

3 MARKS

The answer is 9. The method walks the digits from the right using $\% 10$ to take one off and $/ 10$ to drop it, and adds up only the digits that divide by 3.

The question says "show the steps", so the table **is** the answer. Two of the three marks are the working.

DRY RUN · COMPUTE(2736)

X	D = X % 10	D % 3 == 0?	RESULT	X = X / 10
2736	6	yes	0 + 6 = 6	273
273	3	yes	6 + 3 = 9	27
27	7	no	9	2
2	2	no	9	0
0	x > 0 is false: the loop ends and 9 is returned			

Watch the last row. $2 / 10$ is **0**, not 0.2: integer division is what terminates this loop, and every digit-walking loop in this course ends the same way (Unit 3.6). If $/$ gave 0.2 the loop would never stop.

CODE · Q6 B, RUN FOR REAL

VERIFIED ON THE JVM

```

1 public class ComputeTrace {
2
3     public static int compute(int x) {
4         int result = 0;
5         while (x > 0) {
6             int d = x % 10;
7             if (d % 3 == 0) {
8                 result = result + d;
9             }
10            x = x / 10;
11        }
12        return result;
13    }
14
15    public static void main(String[] args) {
16        System.out.println("compute(2736) = " + compute(2736));
17    }
18 }

```

OUTPUT

```
compute(2736) = 9
```

Five errors, and the first one compiles perfectly.

1. **Line 2: main(String args) is not a main method.** It needs String[] args. This is the one to notice. It is **legal Java**: it compiles without a murmur, and it is simply an ordinary method that happens to be called main. The failure waits until you run it, and the JVM says Error: Main method not found in class OddFinderBroken. It was compiled and run to confirm exactly that.
2. **Line 3: no semicolon** after printOdd(9.0).
3. **Line 3: 9.0 is a double** and the method takes an int. Java will not narrow it for you: "incompatible types: possible lossy conversion from double to int". Write printOdd(9).
4. **Line 6: printOdd(int) has no parameter name.** Write printOdd(int n).
5. **Line 7: for (...); has a stray semicolon.** The loop body is that empty statement, so the loop spins 9 times doing nothing, and then the if below runs **once**, outside the loop, where i no longer exists.

What the compiler actually said: two errors, the semicolon and the missing parameter name. Fix those and **three more** appear: the lossy conversion, and cannot find symbol: variable i twice, which is error 5 finally surfacing. Error 1 is never reported at any stage.

The output after correction:

```
OUTPUT
1 3 5 7 9
```

```
1 public class OddFinder {
2     public static void main(String[] args) {
3         printOdd(9);
4     }
5
6     public static void printOdd(int n) {
7         for (int i = 1; i <= n; i++) {
8             if (i % 2 != 0) {
9                 System.out.print(i + " ");
10            }
11        }
12        System.out.println();
13    }
14 }
```

```
OUTPUT
1 3 5 7 9
```

● MUST-MEMORISE · A STRAY SEMICOLON AFTER FOR OR WHILE IS THE BODY

for (int i = 1; i <= n; i++); is a **complete loop** with an empty body. The indented block below it is **not** inside it.

It appears twice on this mock, in Q4 B and Q6 C, and once on the real 2024/25 paper, in its Question Four B and Question Six C. That is not a coincidence: this lecturer plants it deliberately, because it is the purest example of code that compiles and lies.

Java does not care about your indentation. It cares about the semicolon, and a semicolon where a body should be **is** a body: an empty one.

MEMORY HOOK: a loop header never ends in a semicolon. If you see); at the end of a for or while line, that loop is already finished and everything you thought was inside it is not.

```

1 public class AscendingDigits {
2
3     // i, ii. true only if the digits strictly increase from left to right
4     public static boolean hasAscendingDigits(int n) {
5         int previous = n % 10;    // start at the rightmost digit
6         n = n / 10;
7         while (n > 0) {
8             int digit = n % 10;    // the digit to its left
9             if (digit >= previous) {
10                return false;    // not strictly increasing: stop at the first proof
11            }
12            previous = digit;
13            n = n / 10;
14        }
15        return true;
16    }
17
18    // iii b. test more than two cases
19    public static void main(String[] args) {
20        System.out.println("1359 -> " + hasAscendingDigits(1359));
21        System.out.println("1354 -> " + hasAscendingDigits(1354));
22        System.out.println("1223 -> " + hasAscendingDigits(1223));
23        System.out.println("7    -> " + hasAscendingDigits(7));
24    }
25 }

```

OUTPUT

```

1359 -> true
1354 -> false
1223 -> false
7     -> true

```

The method walks right to left, so it compares each digit with the one **to its right** and demands `digit < previous`. Reading right to left and testing "smaller" **is** reading left to right and testing "larger": the same fact, from the end that % 10 can actually reach.

⚠ TRAP · "STRICTLY" IS A WHOLE MARK, AND 1223 IS THE TEST FOR IT

1223 answers **false**, and the digits never go down. They stop going **up**: 2 is not **strictly** less than 2. Write `digit > previous` instead of `digit >= previous` and 1223 wrongly answers true, while every other example on the paper still passes. That is one character, one example, one mark.

The single digit 7 is the other test worth writing: the loop body never runs at all, and the method must still answer **true**. A method that cannot survive a one-digit input is a method that was never traced.

Choose your own test cases the way this book chose these four: one that passes, one that fails on a decrease, one that fails on a tie, and one edge. Two cases were asked for. Four cost you a minute and prove you thought.

RECALL AND CHECK

Of the six debug snippets on this paper, three contain a bug that the compiler never reports at any stage. Name the three bugs, and say what each would do to a running program.

ANSWER **Q4 B**: the stray semicolon after `while (count <= 5)` makes an empty body, so the program hangs forever. **Q5 C**: `values[3]` where `values[2]` was meant, so it prints 16 and calls it the third value: it runs perfectly and answers wrongly. **Q6 C**: `main(String args)` compiles as an ordinary method, so the JVM finds no main method and the program cannot start at all.

One hangs, one lies, one never starts. None of them is a compile error, and that is the exam's whole point.

Section A, Question One, worked in full

Q1 A · DEFINE A CONSTRUCTOR

2 MARKS

Model answer.

A **constructor** is a special member of a class that runs once, when an object of that class is created with new, and whose job is to put the new object's fields into a valid starting state.

The compiler tells it apart from an ordinary method by **two** facts, and both are needed for the mark: its **name is exactly the class name**, and it declares **no return type at all**, not even void. Those are not two ways of saying the same thing. Write `void MeterAccount(...)` and you have not written a constructor: you have written an ordinary method that happens to be called `MeterAccount`, it compiles, it is never called by `new`, and your fields stay at their defaults.

Where the marks are: one for what it is for, one for how it is recognised. "It has the same name as the class" alone is half the rule and often half the mark.

Q1 B · THREE DIFFERENCES: WHILE AGAINST DO-WHILE

3 MARKS

A three-row table beats a paragraph under time pressure, and it is easier to mark, which is the same thing.

	WHILE	DO-WHILE
When it tests	Before the body: it is a pre-test loop.	After the body: it is a post-test loop.
Fewest runs	Zero. If the condition is false at the start, the body never runs.	One. The body always runs at least once, whatever the condition.
Syntax	No semicolon after the closing brace.	Ends with <code>while (condition);</code> and that semicolon is required .

Where the marks are: one per row. The middle row is the one that decides which to use: "must this run at least once?" is the only question that matters, and it is why input validation is usually a do-while.

Q1 C · FIND THE ERRORS

5 MARKS

Five errors, one per mark, and they are the **same five kinds** that both real papers plant. Run the TTOIS checklist from S.1 and they fall out in about a minute.

- T for Type. Line 4:** `int[] arr` cannot hold 15.0. The required output is **93.0**, with a point, which is the question telling you the array is `double[]`. Fix: `double[] arr`.
- T for Total. Line 5:** `int sum` cannot accumulate decimals. Fix: `double sum`.
- O for Off-by-one. Line 8:** `i <= arr.length` would reach `arr[6]` in an array of 6. Fix: `i < arr.length`.
- I for Inverted. Line 10:** `if (arr[j] % 3 == 0) continue;` skips **exactly the multiples of 3 it was told to find**. Fix: `% 3 != 0`.
- S for Stop. Line 14:** `if (matches == 4)` stops after four, and the specification says three. Fix: `matches == 3`.

What the compiler actually said: errors 1 and 2 only, as six repeats of "incompatible types: possible lossy conversion from double to int". Errors 3, 4 and 5 are **not compile errors at all**. Fix the types and this program compiles, runs, and prints a confident wrong answer. That is 3 of the 5 marks sitting in code the compiler is perfectly happy with.

```

1 // JOptionPane replaced by println so the output can be printed here;
2 // in the exam, keep the dialogs exactly as the paper has them.
3 public class MB1C {
4     public static void main(String[] args) {
5         double[] arr = {15.0, 22.0, 36.0, 41.0, 54.0, 60.0}; // fix 1
6         double sum = 0; // fix 2
7         int matches = 0;
8         boolean stop = false;
9         for (int i = 0; i < arr.length && !stop; i++) { // fix 3
10             for (int j = i; j < arr.length; j++) {
11                 if (arr[j] % 3 != 0) { // fix 4
12                     continue;
13                 }
14                 sum += arr[j] + 2;
15                 matches++;
16                 if (matches == 3) { // fix 5
17                     stop = true;
18                 }
19                 break;
20             }
21         }
22         System.out.println("Sum: " + sum);
23         System.out.println("Matches: " + matches);
24     }
25 }

```

OUTPUT

Sum: 93.0
Matches: 3

Check it by hand, because the examiner gave you the answer and expects you to work back to it. The multiples of 3 are 15, 36, 54 and 60. Starting at $i=0$ the first one found is 15, giving 17. At $i=1$ it is 36, giving 38. At $i=2$ it is 36 again, giving 38, and that is the third match, so it stops. $17 + 38 + 38 = 93.0$, and 3 matches.

Q1 D I · THE FIVE OUTPUTS

10 MARKS, PART ONE

Square it, then keep as many digits off the right-hand end as the number itself has, and compare.

N	DIGITS	N * N	POWER = 10^DIGITS	TAIL = SQUARE % POWER	OUTPUT
5	1	25	10	25 % 10 = 5	AUTOMORPHIC
6	1	36	10	36 % 10 = 6	AUTOMORPHIC
7	1	49	10	49 % 10 = 9	NOT AUTOMORPHIC
25	2	625	100	625 % 100 = 25	AUTOMORPHIC
76	2	5776	100	5776 % 100 = 76	AUTOMORPHIC

% power is the whole trick: % 10 keeps the last digit, % 100 keeps the last two. "Ends in" is a remainder, and that is the idea the question is testing.

```

1 public class AutomorphicChecker {
2     public static void main(String[] args) {
3         int[] numbers = {5, 6, 7, 25, 76};
4         for (int idx = 0; idx < numbers.length; idx++) {
5             int n = numbers[idx], digits = 0, temp = n;
6             while (temp > 0) { digits++; temp /= 10; }
7             int power = 1;
8             for (int p = 0; p < digits; p++) power *= 10;
9             int tail = (n * n) % power;
10            System.out.println(n + " -> " + (tail == n ? "AUTOMORPHIC" : "NOT AUTOMORPHIC"));
11        }
12    }
13 }

```

OUTPUT

```

5 -> AUTOMORPHIC
6 -> AUTOMORPHIC
7 -> NOT AUTOMORPHIC
25 -> AUTOMORPHIC
76 -> AUTOMORPHIC

```

● **MUST-MEMORISE · LOOP CONVERSION IS MECHANICAL. MOVE THE THREE PARTS, CHANGE NOTHING ELSE**

A for is a while with its three parts collected onto one line:

```
for (INIT; TEST; UPDATE) { BODY }
```

is exactly

```
INIT; while (TEST) { BODY UPDATE }
```

So the conversion has three moves and no thinking: **INIT** goes above the loop, **TEST** stays in the brackets, **UPDATE** goes to the bottom of the body. Going the other way, collect all three back onto the header line. A for with an empty init and update, for (; temp > 0;), is a legal and honest way to write a While as a For.

MEMORY HOOK: INIT up, UPDATE down. Ten marks on the real paper and there is no idea in it at all, only care.

```

1 public class AutomorphicConverted {
2     public static void main(String[] args) {
3         int[] numbers = {5, 6, 7, 25, 76};
4
5         // the outer For became a While
6         int idx = 0;
7         while (idx < numbers.length) {
8             int n = numbers[idx], digits = 0, temp = n;
9
10            // this While became a For: same init, same test, same update
11            for (; temp > 0; ) { digits++; temp /= 10; }
12
13            // this For became a While: the update moves down into the body
14            int power = 1;
15            int p = 0;
16            while (p < digits) { power *= 10; p++; }
17
18            int tail = (n * n) % power;
19            System.out.println(n + " -> " + (tail == n ? "AUTOMORPHIC" : "NOT AUTOMORPHIC"));
20            idx++;
21        }
22    }
23 }

```

OUTPUT, IDENTICAL TO THE ORIGINAL

```

5 -> AUTOMORPHIC
6 -> AUTOMORPHIC
7 -> NOT AUTOMORPHIC
25 -> AUTOMORPHIC
76 -> AUTOMORPHIC

```

The output is byte for byte the output of the original, and that is the entire test. A conversion that changes the answer is not a conversion. Both versions were run and their outputs compared, which is exactly what you should do on paper: trace one line of each and check they agree.

⚠ TRAP · THE UPDATE MUST SURVIVE THE MOVE, ESPECIALLY PAST A CONTINUE

The commonest way to lose these ten marks is to move INIT up and then forget UPDATE, leaving while (idx < numbers.length) with no idx++ anywhere. It compiles. It hangs forever. The marker sees an infinite loop where a conversion was asked for.

And there is a sharper version, worth knowing because "INIT up, UPDATE down" is **not always safe**: it breaks the moment the body contains a continue. A for runs its update on every way round, including the ones continue cuts short. A while has no update slot, so the one you moved to the bottom of the body gets **jumped over**, and the loop stops advancing.

Both halves were compiled and run. This for, summing the odd numbers below 6 by skipping the even ones with continue, prints **total = 9**. Converted to a while by the rule above, with i++ at the bottom of the body, it **hangs forever**: at i = 0 the continue fires, i++ is never reached, and i is 0 for eternity.

Neither loop in this question contains a continue, so the rule is safe here. **Check that before you apply it.** If the body does have one, the update must be repeated before the continue as well, or the loop must stay a for.

Section A, Question Two, worked in full

Q2 A · DEFINE INHERITANCE

2 MARKS

Model answer.

Inheritance is the OOP principle by which one class, the **subclass** or child, acquires the fields and methods of another, the **superclass** or parent, using the `extends` keyword. The subclass gets everything the superclass has that it is allowed to see, and may then **add** members of its own or **override** inherited methods to behave differently.

Its purpose is to express an **is-a** relationship and to reuse code without copying it: a `PrepaidMeter` **is a** `MeterAccount`, so it needs no second copy of `topUp`. Because it is an is-a relationship, a subclass object may be used anywhere the superclass is expected, which is what makes runtime polymorphism possible.

Where the marks are: one for the mechanism (`extends`, and the subclass acquires the members), one for the point of it (reuse, and the is-a relationship). Note what inheritance does **not** give you: a subclass inherits a private field in the sense that the object carries it, but it cannot name it. That distinction is Question Four.

Q2 B · THREE DIFFERENCES: AN ARRAY AGAINST A VARIABLE

3 MARKS

	AN ORDINARY VARIABLE	AN ARRAY
How many values	Exactly one, of its declared type.	Many, all of the same type, reached by index.
What the name holds	The value itself, for a primitive: <code>int x = 5</code> holds 5.	A reference to an object elsewhere in memory. Declaring <code>int[] a;</code> creates no array at all, and <code>a</code> is <code>null</code> until <code>new</code> makes one.
Size	Not a question you can ask.	Fixed at creation and readable through <code>length</code> , which is a field , not a method: <code>a.length</code> , never <code>a.length()</code> .

Where the marks are: one per row. The second row is the one that earns respect, because it is the reason two array names can refer to the same array, and the reason passing an array to a method lets that method change it (Unit 5.4).

Q2 C · FIND THE ERRORS

5 MARKS

The same five kinds again. **TTTOIS**: Type, Total, Off-by-one, Inverted, Stop.

- T. Line 4:** `int[] weights` cannot hold 3.5. The required total, **84.6**, has a point in it. Fix: `double[] weights`.
- T. Line 5:** `int total` cannot accumulate decimals. Fix: `double total`.
- O. Line 8:** `i <= weights.length`. Fix: `i < weights.length`.
- I. Line 9:** `if (weights[i] > 5) continue;` skips **exactly the parcels it was told to charge** and charges the ones it was told to skip. Fix: `if (weights[i] < 5) continue;`.
- S. Line 15:** `if (count == 5)` stops after five, and the specification says four. Fix: `count == 4`.

What the compiler actually said: errors 1 and 2, as six repeats of "incompatible types". Once again the three interesting errors compile perfectly.

```

1 // JOptionPane replaced by println so the output can be printed here;
2 // in the exam, keep the dialogs exactly as the paper has them.
3 public class MB2C {
4     public static void main(String[] args) {
5         double[] weights = {3.5, 12.0, 4.25, 20.0, 8.5, 30.0}; // fix 1
6         double total = 0; // fix 2
7         int count = 0;
8         boolean done = false;
9         for (int i = 0; i < weights.length && !done; i++) { // fix 3
10             if (weights[i] < 5) { // fix 4
11                 continue;
12             }
13             double surcharge = weights[i] * 0.2;
14             total += weights[i] + surcharge;
15             count++;
16             if (count == 4) { // fix 5
17                 done = true;
18             }
19         }
20         System.out.println("Total: " + total);
21         System.out.println("Count: " + count);
22     }
23 }

```

OUTPUT

Total: 84.6
Count: 4

Work back to it: the weights of 5 and above are 12.0, 20.0, 8.5 and 30.0, which is four, so the stop-at-four fires on the last one. With a 20% surcharge they cost 14.4, 24.0, 10.2 and 36.0, and those add to **84.6**.

⚠ TRAP · "SKIP ANYTHING UNDER 5" IS NOT "CONTINUE IF OVER 5"

Error 4 is the one this lecturer plants every year, in both questions, in both papers, and it is worth understanding rather than memorising. The English says **skip the small ones**. The code says `if (weights[i] > 5) continue;`, which skips the **big** ones. It is not a typo for `<`: it is the condition **inverted**, and inverted code reads perfectly.

The defence is mechanical. Whenever you see `continue`, say out loud: "**this line names the ones I am throwing away.**" Then check that sentence against the specification. Here the specification throws away the ones under 5, so the condition after `if` must be the under-5 test, and it is not.

Q2 D I · THE OUTPUT

10 MARKS, PART ONE

Prime: 10, Composite: 19, Neither: 1

The inner loop counts **divisors**, testing every `d` from 1 to `num` inclusive, so it counts 1 and `num` themselves.

- ▶ 1 has exactly one divisor, itself. Not 2, not more than 2, so it falls to the else: **Neither: 1**.
- ▶ **Primes** up to 30 are 2, 3, 5, 7, 11, 13, 17, 19, 23, 29: **10** of them.
- ▶ Everything else is composite: 30 numbers, minus 1, minus 10, is **19**.

The check that costs nothing: $10 + 19 + 1 = 30$, and the loop ran 30 times. Every number landed in exactly one bucket, so no number was counted twice or dropped. Do that sum in the hall.

CODE · Q2 D, THE ORIGINAL, RUN FOR REAL

VERIFIED ON THE JVM

```

1 public class PrimeClassifier {
2     public static void main(String[] args) {
3         int primeCount = 0, compositeCount = 0, neitherCount = 0;
4         for (int num = 1; num <= 30; num++) {
5             int divisors = 0;
6             for (int d = 1; d <= num; d++) {
7                 if (num % d == 0) divisors++;
8             }
9             if (divisors == 2) primeCount++;
10            else if (divisors > 2) compositeCount++;
11            else neitherCount++;
12        }
13        System.out.println("Prime: " + primeCount + ", Composite: " + compositeCount
14            + ", Neither: " + neitherCount);
15    }
16 }

```

OUTPUT

Prime: 10, Composite: 19, Neither: 1

CODE · Q2 D II, BOTH FOR LOOPS CONVERTED TO WHILE LOOPS

VERIFIED ON THE JVM

```

1 public class PrimeClassifierWhile {
2     public static void main(String[] args) {
3         int primeCount = 0, compositeCount = 0, neitherCount = 0;
4
5         // the outer For became a While
6         int num = 1;
7         while (num <= 30) {
8             int divisors = 0;
9
10            // the inner For became a While too
11            int d = 1;
12            while (d <= num) {
13                if (num % d == 0) divisors++;
14                d++;
15            }
16
17            if (divisors == 2) primeCount++;
18            else if (divisors > 2) compositeCount++;
19            else neitherCount++;
20            num++;
21        }
22        System.out.println("Prime: " + primeCount + ", Composite: " + compositeCount
23            + ", Neither: " + neitherCount);
24    }
25 }

```

OUTPUT, IDENTICAL TO THE ORIGINAL

Prime: 10, Composite: 19, Neither: 1

Both loops convert, and the inner one is where the marks are lost. Its `int d = 1` must be reset **inside** the outer loop, once per num, because that is where the for version's init lived. Hoist it above the outer loop, as people do when they are converting in a hurry, and `d` keeps the value it reached on the previous number. It was compiled and run: the answer becomes **Prime: 0, Composite: 0, Neither: 30**. Every number falls into the same bucket, because the inner loop now runs at most once and finds one divisor every time. Same output as the original, or the conversion is wrong.

Section B, Question Three, worked in full

Q3 A · READ THE DEFINITION BEFORE YOU WRITE A LINE

12 MARKS

Everything here is decided by two words in the question: **strictly**, and **both**. A spike beats **both** neighbours, **strictly**. Get those into the condition and the program is fifteen lines.

The hint is the specification restated, and it is worth trusting: the answer must not flag 12, 22, 27, 27 or 25. Work out why each is excluded **before** you code, because each one is excluded for a different reason.

INDEX	READING	NEIGHBOURS	VERDICT
0	12	only one, on the right	never checked: an end
1	30	12 and 18	SPIKE : beats both
2	18	30 and 22	DIP : below both
3	22	18 and 41	above 18 but below 41: neither
4	41	22 and 27	SPIKE : beats both
5	27	41 and 27	below 41, but equal to 27: not strictly below both
6	27	27 and 33	equal to 27, below 33: neither
7	33	27 and 19	SPIKE : beats both
8	19	33 and 25	DIP : below both
9	25	only one, on the left	never checked: an end

So: 3 spikes, and 5 flagged readings: 30, 18, 41, 33, 19.

```

1 public class SpikeReport {
2
3     // i. ONE pass through the array: how many readings beat both neighbours
4     public static int countSpikes(int[] readings) {
5         int count = 0;
6         for (int i = 1; i < readings.length - 1; i++) {
7             if (readings[i] > readings[i - 1] && readings[i] > readings[i + 1]) {
8                 count++;
9             }
10        }
11        return count;
12    }
13
14    // ii. every spike or dip, in original order, comma separated, plain String
15    // concatenation only: no StringBuilder, no array of booleans returned
16    public static String flagged(int[] readings) {
17        String out = "";
18        for (int i = 1; i < readings.length - 1; i++) {
19            boolean spike = readings[i] > readings[i - 1] && readings[i] > readings[i + 1];
20            boolean dip = readings[i] < readings[i - 1] && readings[i] < readings[i + 1];
21            if (spike || dip) {
22                out = out.equals("") ? "" + readings[i] : out + ", " + readings[i];
23            }
24        }
25        return out;
26    }
27
28    public static void main(String[] args) {
29        int[] readings = {12, 30, 18, 22, 41, 27, 27, 33, 19, 25};
30
31        // iii. one dialog carrying both answers; println here so it can be printed
32        System.out.println("Spikes: " + countSpikes(readings));
33        System.out.println("Flagged: " + flagged(readings));
34    }
35 }

```

OUTPUT

Spikes: 3

Flagged: 30, 18, 41, 33, 19

Both methods loop `i = 1` to `readings.length - 2`, and that is the whole answer to "the ends are never spikes": do not test them, rather than test them and guard. `i - 1` and `i + 1` are then always in bounds, by construction, and no `ArrayIndexOutOfBoundsException` is possible. `println` here so the run can be printed; in the hall it is one `showMessageDialog` carrying both lines (F.11).

⚠ TRAP · THE EQUAL PAIR AT INDEXES 5 AND 6 IS THE WHOLE QUESTION

The readings 27 and 27 sit side by side, and they are there on purpose. Write `>=` where the question said **strictly** greater and index 6 becomes a spike, because `27 >= 27`. Write `<=` for the dip and index 5 becomes a dip, for the same reason. Every other reading in the array answers identically either way.

That is what a planted equal pair is for. It is the only element of the array that can tell a marker whether you read the word "strictly" or skimmed past it. This lecturer plants one in every array question: the 2025/26 Question Three does the same with its consecutive duplicates.

● MUST-MEMORISE · BUILDING A COMMA-SEPARATED STRING WITHOUT A TRAILING COMMA

```
out = out.equals("") ? "" + value : out + ", " + value;
```

The separator goes **before** every item except the first, so ask "am I first?" instead of trimming a comma off the end afterwards.

The obvious way, `out = out + value + ", ";`, leaves `30, 18, 41, 33, 19,` with a comma and a space hanging off the end, and the marker can see it. The obvious repair, chopping the last two characters off, breaks when the list is empty and there are no two characters to chop.

The empty check is not decoration: it is the base case. And note `"" + value`, which is how you turn an `int` into a `String` with no method call at all.

MEMORY HOOK: the comma belongs to the item on its left. So the first item does not get one, and no item ever gets one it has to give back.

Q3 B · THE RESTRICTED API LIST IS THE ANSWER PLAN

13 MARKS

The question names five classes and permits nothing else: `FileWriter`, `FileReader`, `BufferedReader`, `BufferedWriter`, `IOException`. Reach for `PrintWriter` or `Files.write()` here and you lose marks for code that works, because the restriction **is** the question.

Read part ii carefully: it says read `output.txt`, and nothing in this program ever creates `output.txt`. That is not an oversight in the paper. **The missing file is the test**, and your catch block firing is the right answer, not a bug.

```

1  import java.io.BufferedReader;
2  import java.io.BufferedWriter;
3  import java.io.FileReader;
4  import java.io.FileWriter;
5  import java.io.IOException;
6
7  public class LibraryRecords {
8
9      // iii. takes a file name, counts the lines, returns the count, and handles its
10     // own exceptions internally: the caller is never asked to deal with them
11     public static int countRecords(String fileName) {
12         int lines = 0;
13         try (BufferedReader in = new BufferedReader(new FileReader(fileName))) {
14             while (in.readLine() != null) {
15                 lines++;
16             }
17         } catch (IOException e) {
18             System.out.println("Could not count " + fileName + ": it could not be opened.");
19             return 0;
20         }
21         return lines;
22     }
23
24     public static void main(String[] args) {
25
26         // i. write three book records to input.txt, one per line
27         try (BufferedWriter out = new BufferedWriter(new FileWriter("input.txt"))) {
28             out.write("L001,Things Fall Apart,4.80");
29             out.newLine();
30             out.write("L002,The Concubine,3.95");
31             out.newLine();
32             out.write("L003,Efuru,4.25");
33             out.newLine();
34             System.out.println("input.txt written");
35         } catch (IOException e) {
36             System.out.println("Could not write input.txt: " + e.getMessage());
37         }
38
39         // ii. read output.txt and print every line exactly as it appears. The file
40         // does not exist, and that is the case the question asks you to handle.
41         try (BufferedReader in = new BufferedReader(new FileReader("output.txt"))) {
42             String line = in.readLine();
43             while (line != null) {
44                 System.out.println(line);
45                 line = in.readLine();
46             }
47         } catch (IOException e) {
48             System.out.println("Could not read output.txt: it does not exist yet.");
49         }
50
51         // iii. used
52         System.out.println("input.txt holds " + countRecords("input.txt") + " records");
53         System.out.println("output.txt holds " + countRecords("output.txt") + " records");
54     }
55 }

```

OUTPUT, RUN IN A FOLDER WITH NO OUTPUT.TXT

```

input.txt written
Could not read output.txt: it does not exist yet.
input.txt holds 3 records
Could not count output.txt: it could not be opened.
output.txt holds 0 records

```

Read the last two lines together. The counting method met a file that does not exist, **handled it itself**, said so, and returned 0. It did not throw, and main was never asked to have an opinion. That is exactly what part iii means by "must handle its own exceptions internally", and it is why the method returns an int and not a wobble.

⚠ TRAP · DO NOT PRINT E.GETMESSAGE() AND CALL IT A SPECIFIC MESSAGE

Part iv asks every catch block to print "a clear, specific message describing what went wrong", and the reflex is `System.out.println(e.getMessage())`. Resist it. For a missing file the JVM fills that message with the **operating system's own words**: on the machine this book was built on it reads `output.txt` (The system cannot find the file specified), while on a different system it reads `output.txt` (No such file or directory).

So the same program prints different text on different machines, and neither sentence says what **your program** was trying to do when it failed. "Could not count output.txt" does. Name the operation and the file; let the exception object stay in the log where it belongs.

● MUST-MEMORISE · TRY-WITH-RESOURCES CLOSES IT, ON EVERY PATH, INCLUDING THE ONE YOU FORGOT

```
try (BufferedReader in = new BufferedReader(new FileReader(f))) { ... }
```

closes in when the block ends, **whether it ends normally or by throwing**. No finally, no close(), nothing to forget.

Part iv accepts a finally block, and a finally is a correct answer. But it is three more lines, and its own close() can itself throw, so the honest version needs a try inside the finally. The resource form is shorter **and** harder to get wrong, which is the only kind of shorter worth having.

MEMORY HOOK: the brackets after try own the thing inside them. Whatever is declared there is closed when the block ends, in the reverse of the order it was opened.

MS.11 · MOCK B SOLUTIONS · SECTION B, QUESTION FOUR

Section B, Question Four, worked in full

Q4 · WHAT THE THREE PARTS ARE REALLY ASKING

25 MARKS

The marks split 9 / 10 / 6, and the shape of the answer is dictated:

- ▶ **Part 1 (9):** a superclass with **private** fields. The word private is doing real work and it is examined in Part 2.
- ▶ **Part 2 (10):** two subclasses, each **overriding the same method** differently. This is where the marks are.
- ▶ **Part 3 (6):** a main that **proves** the overrides run, including through a superclass reference.

Part 3 v is the sentence that decides your grade: "**confirming that the correct overridden version runs**". A main that calls `pre.consume(...)` and `post.consume(...)` directly does not confirm anything: of course they run their own methods. The confirmation is calling it through a `MeterAccount` reference, where the compiler cannot know which one it will be.

```

1 // Part 1: the superclass, with PRIVATE state
2 class MeterAccount {
3     private String meterNumber;
4     private String holderName;
5     private double units;
6
7     public MeterAccount(String meterNumber, String holderName, double units) {
8         this.meterNumber = meterNumber;
9         this.holderName = holderName;
10        this.units = units;
11    }
12
13    // the subclasses cannot touch `units` directly, so they go through these
14    public double getUnits() { return units; }
15    protected void setUnits(double units) { this.units = units; }
16    public String getMeterNumber() { return meterNumber; }
17
18    // Part 1 ii: add units
19    public void topUp(double amount) {
20        units += amount;
21        System.out.printf("%s topped up %.1f units. Balance: %.1f%n",
22            meterNumber, amount, units);
23    }
24
25    // Part 1 iii: deduct if there are enough, otherwise reject and say so
26    public void consume(double amount) {
27        if (amount <= units) {
28            units -= amount;
29            System.out.printf("%s used %.1f units. Balance: %.1f%n",
30                meterNumber, amount, units);
31        } else {
32            System.out.printf("%s REJECTED: %.1f units requested, only %.1f available.%n",
33                meterNumber, amount, units);
34        }
35    }
36
37    // Part 1 iv: display the account
38    public void display() {
39        System.out.printf("Meter %s | %s | %.1f units%n", meterNumber, holderName, units);
40    }
41 }
42
43 // Part 2 i: a prepaid meter must keep a 5.0 unit reserve
44 class PrepaidMeter extends MeterAccount {
45     public PrepaidMeter(String meterNumber, String holderName, double units) {
46         super(meterNumber, holderName, units);
47     }
48
49     @Override
50     public void consume(double amount) {
51         if (getUnits() - amount >= 5.0) {
52             super.consume(amount);
53         } else {
54             System.out.printf("%s PREPAID REJECTED: %.1f units would leave less than "
55                 + "the 5.0 unit reserve.%n", getMeterNumber(), amount);
56         }
57     }
58 }
59
60 // Part 2 ii: a postpaid meter earns a 1.0 unit rebate on every successful use
61 class PostpaidMeter extends MeterAccount {
62     public PostpaidMeter(String meterNumber, String holderName, double units) {
63         super(meterNumber, holderName, units);
64     }

```

```

65
66     @Override
67     public void consume(double amount) {
68         double before = getUnits();
69         super.consume(amount);
70         if (getUnits() < before) {           // it actually went through
71             setUnits(getUnits() + 1.0);
72             System.out.printf("%s POSTPAID rebate: 1.0 unit added. Balance: %.1f%n",
73                             getMeterNumber(), getUnits());
74         }
75     }
76 }
77
78 public class MeterSystem {
79     public static void main(String[] args) {
80         // Part 3 i: one of each
81         PrepaidMeter pre = new PrepaidMeter("PRE-001", "Ada", 50.0);
82         PostpaidMeter post = new PostpaidMeter("POST-002", "Bola", 50.0);
83
84         System.out.println("--- before ---");
85         pre.display();
86         post.display();
87
88         // Part 3 ii: deposit into each
89         pre.topUp(20.0);
90         post.topUp(20.0);
91
92         // Part 3 iii: one that works and one that does not, on each
93         System.out.println("--- transactions ---");
94         pre.consume(30.0);      // leaves 40.0, comfortably above the reserve
95         pre.consume(38.0);      // would leave 2.0: rejected by the override
96         post.consume(30.0);     // works, and earns the rebate
97         post.consume(100.0);    // rejected by the superclass: not enough units
98
99         System.out.println("--- after ---");
100        pre.display();
101        post.display();
102
103        // Part 3 v: the same call through a superclass reference. The reference is
104        // MeterAccount for both, so the compiler cannot know which consume() runs.
105        System.out.println("--- through a MeterAccount reference ---");
106        MeterAccount[] meters = {pre, post};
107        for (int i = 0; i < meters.length; i++) {
108            meters[i].consume(10.0);
109        }
110
111        pre.display();
112        post.display();
113    }
114 }

```

OUTPUT

```

--- before ---
Meter PRE-001 | Ada | 50.0 units
Meter POST-002 | Bola | 50.0 units
PRE-001 topped up 20.0 units. Balance: 70.0
POST-002 topped up 20.0 units. Balance: 70.0
--- transactions ---
PRE-001 used 30.0 units. Balance: 40.0
PRE-001 PREPAID REJECTED: 38.0 units would leave less than the 5.0 unit reserve.
POST-002 used 30.0 units. Balance: 40.0
POST-002 POSTPAID rebate: 1.0 unit added. Balance: 41.0
POST-002 REJECTED: 100.0 units requested, only 41.0 available.
--- after ---
Meter PRE-001 | Ada | 40.0 units
Meter POST-002 | Bola | 41.0 units

```

```

--- through a MeterAccount reference ---
PRE-001 used 10.0 units. Balance: 30.0
POST-002 used 10.0 units. Balance: 31.0
POST-002 POSTPAID rebate: 1.0 unit added. Balance: 32.0
Meter PRE-001 | Ada | 30.0 units
Meter POST-002 | Bola | 32.0 units

```

Read the last block. One call site, `meters[i].consume(10.0)`, inside one loop, over an array whose type is `MeterAccount`. The prepaid meter checked its reserve; the postpaid meter paid its rebate. The compiler could not have chosen either: **the objects did**.

● MUST-MEMORISE · A SUBCLASS CANNOT SEE A PRIVATE FIELD. THAT IS NOT A BUG IN YOUR CODE

Part 1 says **private** fields. So `PrepaidMeter` may **not** write units. It must go through `getUnits()` and `setUnits()`.

Write `units - amount >= 5.0` inside the subclass and it is a compile error: `units` has private access in `MeterAccount`.

The object **has** the field: a `PrepaidMeter` genuinely carries its own units, and `topUp` changes it. The subclass simply cannot **name** it. Inheritance gives you the state; it does not give you permission.

This is the one place where "just make it protected" is the wrong instinct in an exam: the question **said** private, so the accessor is the answer, and reaching for protected to dodge the compiler is answering a question nobody asked.

MEMORY HOOK: **private means private to the class, not to the object**. The child owns the box and still cannot read the label.

⚠ TRAP · THE OVERRIDE MUST CALL SUPER, OR YOU HAVE REWRITTEN THE PARENT BADLY

Both overrides here do their own check and then hand the actual work to `super.consume(amount)`. That is deliberate, and it is worth a mark you will not see itemised.

Rewrite the deduction inside `PrepaidMeter` instead, and you now have two copies of "take the units off and print the balance". They agree today. They will not agree the moment anything changes, and the marker has seen that answer a hundred times.

`PostpaidMeter` shows the sharper reason. It has to know whether the deduction **actually happened**, because a rejected one earns no rebate. It cannot know that by re-checking the balance rule, because that would be a third copy of it. So it reads the balance before, calls `super`, and reads it after. **Ask the parent what it did; do not re-derive it**.

RECALL AND CHECK

In the last block, `meters` is declared `MeterAccount[]`. So why does `meters[1].consume(10.0)` pay a rebate, when `MeterAccount.consume` contains no rebate at all?

ANSWER Because the method that runs is chosen by the **object**, at run time, not by the reference's type at compile time. `meters[1]` refers to a `PostpaidMeter` object, so `PostpaidMeter.consume` runs. The declared type `MeterAccount` only decides what the compiler will **let** you call; it does not decide which version answers. That is runtime polymorphism, and it is the entire point of Part 3 v.

Section C, Question Five, worked in full

● MUST-MEMORISE · SECTION C TELLS YOU WHICH CONSTRUCT TO USE, AND MARKS YOU ON IT

Section C does not merely ask for a working program. It **names the construct**, and using a different one loses the mark even when the output is right.

"Use a switch" · "use **nested** if-else (not a switch, and not independent/separate if checks)" · "using a nested control statement" · "no arrays and no user-defined helper methods" · "no collection classes" · "JOptionPane only" · "each task must be its own method".

And from the front page: use comments in Section B and Section C to say what each block does. That is an instruction, and it is free marks.

MEMORY HOOK: underline every noun that names a construct before you write anything. Those nouns are the marking scheme.

```

1  import java.util.Scanner;
2
3  public class CourierPricing {
4
5      // Part i: a switch returning the base rate for a zone. Any other zone is illegal.
6      static double baseRateForZone(int zone) {
7          switch (zone) {
8              case 1: return 120.0;
9              case 2: return 200.0;
10             case 3: return 260.0;
11             default: throw new IllegalArgumentException(
12                 "Zone must be 1, 2 or 3 but was " + zone);
13         }
14     }
15
16     // Part ii: NESTED if-else. Weight is only examined inside a tier branch.
17     static double surchargePercent(String tier, double weightKg) {
18         if (tier.equals("Gold")) {
19             if (weightKg > 20.0) {
20                 return 5.0;
21             } else {
22                 return 0.0;
23             }
24         } else if (tier.equals("Silver")) {
25             if (weightKg > 20.0) {
26                 return 15.0;
27             } else {
28                 return 10.0;
29             }
30         } else {
31             return 25.0;
32         }
33     }
34
35     // Part iii: apply the surcharge, with a price CEILING of 500.0
36     static double applySurcharge(double baseTotal, double percent) {
37         double charged = baseTotal + (baseTotal * percent / 100.0);
38         if (charged > 500.0) {
39             return 500.0;
40         }
41         return charged;
42     }
43
44     public static void main(String[] args) {
45         Scanner in = new Scanner(System.in);
46
47         int parcels = Integer.parseInt(in.nextLine());
48
49         double totalRevenue = 0;
50         int goldCount = 0, silverCount = 0, bronzeCount = 0;
51
52         for (int p = 1; p <= parcels; p++) {
53             int zones = Integer.parseInt(in.nextLine());
54             String tier = in.nextLine();
55             double weightKg = Double.parseDouble(in.nextLine());
56
57             double baseTotal = 0;
58             for (int z = 1; z <= zones; z++) {
59                 baseTotal += baseRateForZone(z);
60             }
61
62             double percent = surchargePercent(tier, weightKg);
63             double finalPrice = applySurcharge(baseTotal, percent);
64             totalRevenue += finalPrice;

```

```

66         if (tier.equals("Gold")) {
67             goldCount++;
68         } else if (tier.equals("Silver")) {
69             silverCount++;
70         } else {
71             bronzeCount++;
72         }
73         System.out.printf("Parcel %d: %s, %d zone(s), %.1f kg, base %.1f, "
74             + "surcharge %.1f%%, pays %.2f%n",
75             p, tier, zones, weightKg, baseTotal, percent, finalPrice);
76     }
77 }
78
79 System.out.println("SUMMARY");
80 System.out.printf("Total revenue: %.2f%n", totalRevenue);
81 System.out.println("Gold: " + goldCount + ", Silver: " + silverCount
82     + ", Bronze: " + bronzeCount);
83 }
84 }

```

OUTPUT, FOR 5 PARCELS

```

Parcel 1: Gold, 3 zone(s), 25.0 kg, base 580.0, surcharge 5.0%, pays 500.00
Parcel 2: Gold, 1 zone(s), 10.0 kg, base 120.0, surcharge 0.0%, pays 120.00
Parcel 3: Silver, 2 zone(s), 30.0 kg, base 320.0, surcharge 15.0%, pays 368.00
Parcel 4: Bronze, 2 zone(s), 5.0 kg, base 320.0, surcharge 25.0%, pays 400.00
Parcel 5: Platinum, 3 zone(s), 5.0 kg, base 580.0, surcharge 25.0%, pays 500.00
SUMMARY
Total revenue: 1888.00
Gold: 2, Silver: 1, Bronze: 2

```

The test data is chosen so every rule fires visibly. Parcels 1 and 5 would both have paid more than 500 and both paid exactly 500.00, so the ceiling is working. Parcel 5 is "Platinum", which is not Gold and not Silver: it got the 25% Bronze surcharge **and** was counted as Bronze, so the two Bronze rules agree with each other. Input is a Scanner here so the run can be printed; in the hall every prompt is `JOptionPane.showInputDialog` and the summary is one `showMessageDialog` (F.11).

⚠ TRAP · IF PLATINUM IS PRICED AS BRONZE, IT MUST BE COUNTED AS BRONZE

The question says any tier that is not "Gold" or "Silver" is treated as Bronze. That rule has to be obeyed **twice**: once when pricing, and once when counting for the summary. It is easy to write the pricing branch correctly and then count the tiers with a separate chain that has a Platinum case, or that counts nothing at all for it.

Then your dialog reports Gold 2, Silver 1, Bronze 1, and four parcels in a summary of five. The counts must add up to N. **Check that they do before you put your pen down:** $2 + 1 + 2 = 5$, and there were 5 parcels.

● MUST-MEMORISE · A SWITCH THAT RETURNS NEEDS NO BREAK

case 1: return 120.0; has no break, and needs none.

return leaves the whole **method**, so it certainly leaves the switch. There is nothing to fall through to.

The default throwing `IllegalArgumentException` is what makes the method total: every zone either returns a price or raises an error, and none silently returns 0. That is the question's own instruction, and it is also what stops a bad zone from quietly costing nothing.

MEMORY HOOK: break or return, never neither, never both. A switch that assigns needs breaks; a switch that returns does not.

```

1  import java.util.Scanner;
2
3  public class NoiseLog {
4      public static void main(String[] args) {
5          Scanner in = new Scanner(System.in);
6
7          // i. 20 prompts. Each is validated by a NESTED loop that re-prompts the SAME
8          // hour until a number between 0 and 140 arrives.
9          double[] readings = new double[20];
10         for (int h = 0; h < readings.length; h++) {
11             boolean valid = false;
12             while (!valid) {
13                 String entered = in.nextLine();
14                 try {
15                     double value = Double.parseDouble(entered);
16                     if (value >= 0.0 && value <= 140.0) {
17                         readings[h] = value;
18                         valid = true;
19                     } else {
20                         System.out.println("Hour " + h + ": " + entered
21                             + " is outside 0 to 140. Enter it again.");
22                     }
23                 } catch (NumberFormatException e) {
24                     System.out.println("Hour " + h + ": " + entered
25                         + " is not a number. Enter it again.");
26                 }
27             }
28         }
29
30         // ii. ONE scan, no extra arrays: count the streaks of 3 or more breach-hours
31         // and measure the longest streak of any length.
32         int streaks = 0, longest = 0, run = 0;
33         for (int h = 0; h < readings.length; h++) {
34             if (readings[h] > 85.0) {
35                 run++;
36                 if (run > longest) {
37                     longest = run;
38                 }
39                 if (run == 3) {           // count each qualifying streak once, as it qualifies
40                     streaks++;
41                 }
42             } else {
43                 run = 0;                 // the streak is broken
44             }
45         }
46
47         // iii. nested if-else: the length is only examined once we know a streak exists
48         String verdict;
49         if (longest >= 3) {
50             if (longest >= 6) {
51                 verdict = "CRITICAL";
52             } else {
53                 verdict = "WARNING";
54             }
55         } else {
56             verdict = "SAFE";
57         }
58
59         // iv. one dialog with all three facts
60         System.out.println("Qualifying streaks (3 or more hours): " + streaks);
61         System.out.println("Longest breach streak: " + longest + " hour(s)");
62         System.out.println("Classification: " + verdict);
63     }

```

OUTPUT, FOR A LOG WITH TWO INVALID ENTRIES AT HOUR 0

```
Hour 0: 999 is outside 0 to 140. Enter it again.
Hour 0: abc is not a number. Enter it again.
Qualifying streaks (3 or more hours): 2
Longest breach streak: 6 hour(s)
Classification: CRITICAL
```

The 20 readings after the two rejects are 80, 82, 90, 91, 88, 70, 86, 87, 90, 92, 95, 88, 60, 90, 91, 70, 71, 72, 73, 74. The breaches (above 85) form runs of **3** (90, 91, 88), then **6** (86, 87, 90, 92, 95, 88), then **2** (90, 91). So two runs reach 3 and qualify, the run of 2 does not, and the longest is 6, which is CRITICAL. Both rejects were for hour **0**, and the third value became hour 0: the loop re-prompts the **same** hour, which is what "re-prompt for the same hour" means.

● MUST-MEMORISE · COUNT A STREAK WHEN IT REACHES THE THRESHOLD, NOT WHEN IT ENDS

```
if (run == 3) streaks++;
Exactly == 3, not >= 3.
```

This single line is the elegant half of the question. A run of 6 passes through `run == 3` once, on its third hour, and is counted once. Write `>= 3` and that run of 6 is counted **four** times, once for each hour from the third onward, and the answer becomes 5 instead of 2.

The alternative is to wait until the run breaks and count it then, but that needs a second check after the loop for a streak still running at the last hour, and forgetting it is the classic bug. Counting at the threshold has no end case at all.

MEMORY HOOK: **count it as it qualifies, not as it dies.** The moment it crosses the line is unique; the moment it ends may never come.

⚠ TRAP · VALIDATION MUST CATCH BOTH KINDS OF BAD INPUT, AND ONE OF THEM IS NOT A NUMBER

"It must parse as a number between 0 and 140" is **two** requirements, and the marks are split. 999 is a perfectly good number that is out of range: an `if` catches it. `abc` is not a number at all: `Double.parseDouble("abc")` **throws** `NumberFormatException`, and no `if` in the world catches that. It needs a `try-catch`.

A solution with only the range check compiles, passes every test you would think to run by hand, and dies the first time a marker types a letter. Both entries are in this book's test data on purpose, and the output above shows both messages, because an untested validation is a guess.

MS.13 · MOCK B SOLUTIONS · SECTION C, QUESTION SIX

Section C, Question Six, worked in full

Q6 A · THE CONSTRAINTS ARE HALF THE MARKS

12 MARKS

Read what is forbidden: **no arrays**, **no helper methods**, everything **inline in main**, the digit loop built **only** on `%` and `/`, and the primality test limited to the **square root**. A beautiful, well-factored answer with four private static methods scores badly here, and that is not unfair: the question is testing whether you can hold a whole algorithm in one scope, which is exactly what a dry run asks of you.

Both facts in step 1 come out of **the same single loop**, and step 2 rides along in it too. Three answers, one pass.

DRY RUN · N = 1237, ONE ROW PER TURN OF THE DIGIT LOOP

TEMP	DIGIT	PRODUCT	LARGEST	DIGIT > PREVIOUS?	PREVIOUS
1237	7	1 x 7 = 7	7	first digit: nothing to compare	7
123	3	7 x 3 = 21	7	3 > 7? no	3
12	2	21 x 2 = 42	7	2 > 3? no	2
1	1	42 x 1 = 42	7	1 > 2? no	1
0	temp > 0 is false: the loop ends. product 42, largest 7, monotone stayed true				

The loop walks **right to left**, because % 10 can only reach the last digit. So previous holds the digit to the **right** of the one in hand, and "never decreases left to right" becomes "each digit is not greater than the one to its right". Same fact, read from the end you can actually reach.

Then: is 42 prime? d = 2, and 2 * 2 <= 42, and 42 % 2 == 0, so no, immediately. N is monotone but its product is not prime, so the second branch fires: **MONOTONE**.

```

1  import java.util.Scanner;
2
3  public class DigitClassifier {
4      public static void main(String[] args) {
5          Scanner in = new Scanner(System.in);
6          int n = Integer.parseInt(in.nextLine());
7
8          // i. ONE loop, built only on % and /, computing the digit product and the
9          // largest digit. ii. and, in the same loop, whether the digits never
10         // decrease from left to right.
11         int product = 1;
12         int largest = 0;
13         boolean monotone = true;
14         int previous = -1;          // the digit to the RIGHT of the current one
15         int temp = n;
16         while (temp > 0) {
17             int digit = temp % 10;
18             product = product * digit;
19             if (digit > largest) {
20                 largest = digit;
21             }
22             if (previous != -1 && digit > previous) {
23                 monotone = false;
24             }
25             previous = digit;
26             temp = temp / 10;
27         }
28
29         // iii. is the digit product prime? Tested inline, dividing only up to the
30         // square root: d * d <= product is the same test as d <= sqrt(product),
31         // and it needs no Math call.
32         boolean prime = product > 1;
33         for (int d = 2; d * d <= product; d++) {
34             if (product % d == 0) {
35                 prime = false;
36                 break;
37             }
38         }
39
40         // the product is 0 exactly when one of the digits is 0
41         boolean hasZero = (product == 0);
42
43         // iv. classify, strictly in this priority order
44         String verdict;
45         if (monotone && prime) {
46             verdict = "MONOTONE-PRIME-PRODUCT";
47         } else if (monotone) {
48             verdict = "MONOTONE";
49         } else if (hasZero) {
50             verdict = "ZERO-PRODUCT";
51         } else {
52             verdict = "STANDARD";
53         }
54
55         // v. one dialog with everything
56         System.out.println("N = " + n);
57         System.out.println("Digit product = " + product);
58         System.out.println("Largest digit = " + largest);
59         System.out.println("Classification = " + verdict);
60     }
61 }

```

OUTPUT, FOR THE INPUT 1237

```
N = 1237
Digit product = 42
Largest digit = 7
Classification = MONOTONE
```

● **MUST-MEMORISE · $D * D \leq N$ IS THE SQUARE ROOT TEST, AND IT NEEDS NO MATH**

```
for (int d = 2; d * d <= product; d++)
```

is the same test as `d <= sqrt(product)`, with no `Math.sqrt`, no `double`, and no rounding to argue about.

Why the square root at all: if n has a divisor bigger than its square root, it must also have the matching one **below** it, because the two multiply to n . So a divisor above the root can never be the first one found, and testing past the root can never discover anything new.

Prefer `d * d <= product` to `d <= Math.sqrt(product)`. It stays in integers, so there is no question about whether the root of 49 comes back as 6.999999 and drops a divisor.

MEMORY HOOK: divisors come in pairs, and the root is the pivot. Every pair has one member on each side, so one side is enough.

```

1  import java.util.Scanner;
2
3  public class DigitClassifierBench {
4      public static void main(String[] args) {
5          Scanner in = new Scanner(System.in);
6          int cases = Integer.parseInt(in.nextLine());
7
8          for (int c = 0; c < cases; c++) {
9              int n = Integer.parseInt(in.nextLine());
10
11              int product = 1;
12              int largest = 0;
13              boolean monotone = true;
14              int previous = -1;
15              int temp = n;
16              while (temp > 0) {
17                  int digit = temp % 10;
18                  product = product * digit;
19                  if (digit > largest) {
20                      largest = digit;
21                  }
22                  if (previous != -1 && digit > previous) {
23                      monotone = false;
24                  }
25                  previous = digit;
26                  temp = temp / 10;
27              }
28
29              boolean prime = product > 1;
30              for (int d = 2; d * d <= product; d++) {
31                  if (product % d == 0) {
32                      prime = false;
33                      break;
34                  }
35              }
36
37              boolean hasZero = (product == 0);
38
39              String verdict;
40              if (monotone && prime) {
41                  verdict = "MONOTONE-PRIME-PRODUCT";
42              } else if (monotone) {
43                  verdict = "MONOTONE";
44              } else if (hasZero) {
45                  verdict = "ZERO-PRODUCT";
46              } else {
47                  verdict = "STANDARD";
48              }
49
50              System.out.printf("%-6d product %-4d largest %d  -> %s\n",
51                  n, product, largest, verdict);
52          }
53      }
54  }

```

OUTPUT

```

113  product 3    largest 3  -> MONOTONE-PRIME-PRODUCT
1237  product 42   largest 7  -> MONOTONE
405   product 0    largest 5  -> ZERO-PRODUCT
4821  product 64   largest 8  -> STANDARD
7     product 7    largest 7  -> MONOTONE-PRIME-PRODUCT
1119  product 9    largest 9  -> MONOTONE

```

Not part of the answer: this is the same inline logic run over six numbers, so that **all four** classifications appear and the priority order is visible. 7 is the edge worth writing: a single digit, the loop runs once, and it is monotone by definition because there is no pair of digits to go the wrong way. 1119 shows that "never decreases" allows **equal** digits, which is why the test is `digit > previous` and not `>=`.

⚠ **TRAP · THE DIGIT PRODUCT STARTS AT 1, AND A ZERO DIGIT IS WHY ZERO-PRODUCT EXISTS**

Two initialisation traps sit next to each other, and they pull in opposite directions.

`int product = 1;`, not 0. A product accumulates by multiplying, and anything times 0 is 0, so starting at 0 makes every answer 0. Meanwhile `int largest = 0;` is right, because a largest accumulates by comparing and every digit is at least 0.

And 405 shows why the question has a ZERO-PRODUCT class at all: one 0 digit drags the whole product to 0, permanently. So `product == 0` is not a special case to be guarded, it is a **reliable test** for "N contains a zero", and the classification uses it as one. That is the question rewarding you for noticing what multiplication already knows.

```

1  import java.util.Scanner;
2
3  public class LibraryFines {
4
5      // ii. one book's fine, with the rules NESTED: the member type is decided first,
6      // and the days are only examined inside that branch.
7      public static double fineFor(int daysLate, String memberType) {
8          double fine;
9          if (memberType.equals("Student")) {
10             if (daysLate <= 3) {
11                 fine = 0.0; // the first 3 days are free
12             } else {
13                 fine = (daysLate - 3) * 20.0;
14                 if (fine > 200.0) {
15                     fine = 200.0; // capped
16                 }
17             }
18         } else { // anything not exactly "Student"
19             if (daysLate <= 5) {
20                 fine = 150.0; // a flat charge for the first 5
21             } else {
22                 fine = 150.0 + (daysLate - 5) * 40.0;
23                 if (fine > 500.0) {
24                     fine = 500.0; // capped
25                 }
26             }
27         }
28         return fine;
29     }
30
31     // iii. tally the three bands and return all three counts, with no collection class
32     public static int[] tallyBands(int[] daysLate) {
33         int shortBand = 0, mediumBand = 0, extendedBand = 0;
34         for (int i = 0; i < daysLate.length; i++) {
35             if (daysLate[i] <= 3) {
36                 shortBand++;
37             } else if (daysLate[i] <= 10) {
38                 mediumBand++;
39             } else {
40                 extendedBand++;
41             }
42         }
43         return new int[] {shortBand, mediumBand, extendedBand};
44     }
45
46     public static void main(String[] args) {
47         Scanner in = new Scanner(System.in);
48
49         // i. two PARALLEL arrays: index i is one book in both
50         int books = Integer.parseInt(in.nextLine());
51         int[] daysLate = new int[books];
52         String[] memberType = new String[books];
53
54         for (int i = 0; i < books; i++) {
55             int days = 0;
56             boolean valid = false;
57             while (!valid) { // the nested control statement
58                 days = Integer.parseInt(in.nextLine());
59                 if (days > 0) {
60                     valid = true;
61                 } else {
62                     System.out.println("Book " + (i + 1) + ": " + days
63                         + " is not a positive number of days. Enter it again.");
64                 }
65             }
66         }
67     }
68 }

```

```

64         }
65     }
66     daysLate[i] = days;
67     memberType[i] = in.nextLine();
68 }
69
70 // iv. every vehicle across both arrays, accumulating the batch total
71 double totalFines = 0;
72 for (int i = 0; i < books; i++) {
73     double fine = fineFor(daysLate[i], memberType[i]);
74     totalFines += fine;
75     System.out.printf("Book %d: %s, %d day(s) late, fine %.2f%n",
76         i + 1, memberType[i], daysLate[i], fine);
77 }
78
79 int[] bands = tallyBands(daysLate);
80
81 System.out.println("SUMMARY");
82 System.out.printf("Total fines: %.2f%n", totalFines);
83 System.out.println("SHORT (1-3): " + bands[0] + ", MEDIUM (4-10): " + bands[1]
84     + ", EXTENDED (more than 10): " + bands[2]);
85 }
86 }

```

OUTPUT, FOR 6 BOOKS WITH ONE INVALID ENTRY

```

Book 1: 0 is not a positive number of days. Enter it again.
Book 1: Student, 2 day(s) late, fine 0.00
Book 2: Student, 5 day(s) late, fine 40.00
Book 3: Student, 20 day(s) late, fine 200.00
Book 4: Public, 3 day(s) late, fine 150.00
Book 5: Public, 12 day(s) late, fine 430.00
Book 6: Staff, 30 day(s) late, fine 500.00
SUMMARY
Total fines: 1320.00
SHORT (1-3): 2, MEDIUM (4-10): 1, EXTENDED (more than 10): 3

```

Six books, and every rule fires exactly once. Book 1: inside the free 3 days, so 0.00. Book 2: $(5 - 3) \times 20 = 40.00$. Book 3: $(20 - 3) \times 20 = 340$, **capped** to 200.00. Book 4: inside the flat 5 days, 150.00. Book 5: $150 + (12 - 5) \times 40 = 430.00$. Book 6: "Staff" is not "Student", so it is charged as Public, $150 + 25 \times 40 = 1150$, **capped** to 500.00. Both caps, both free periods, the fallback, and the re-prompt: if your test data cannot show all of that, it cannot tell you your program works.

● MUST-MEMORISE · RETURNING THREE NUMBERS WITHOUT A COLLECTION CLASS

```
return new int[] {shortBand, mediumBand, extendedBand};
```

A method returns **one** thing. When the question wants three and bans ArrayList, that one thing is a small array.

The alternatives are worse and the marker knows all of them: three separate methods each looping the array again, which is three passes for one question that said one method; or three static fields quietly written as a side effect, which works and is unreadable.

MEMORY HOOK: one return, so box them. `new int[] {a, b, c}` is a whole array written inline, with no separate declaration and no `new int[3]` followed by three assignments.

⚠ TRAP · PARALLEL ARRAYS MUST BE FILLED IN THE SAME ITERATION, OR THE PAIRING ROTS

`daysLate[i]` and `memberType[i]` are the same book, and the only thing making that true is that they are written in the same turn of the same loop. Fill them in two separate loops, or let the re-prompt loop advance `i`, and index 3's days belong to index 4's member type. Nothing throws. Nothing warns. Every fine is simply wrong.

Look at where `i` changes in the answer: **only** in the outer `for`. The validation while re-reads days and never touches `i`, which is precisely why book 1's rejected 0 does not cost book 1 its place.

This is why the paper bans `ArrayList` and asks for parallel arrays: not because collections are bad, but because holding two arrays in step by hand is the skill it wants to see you have.

RECALL AND CHECK

Book 6's member type is "Staff". The fine method has no Staff branch anywhere. Why does it charge 500.00 rather than crash, or charge nothing?

ANSWER Because the method asks only one question, `memberType.equals("Student")`, and everything that is not Student falls into the `else`, which is the Public rule. There is no list of valid types to fail against. The question demanded exactly this: "any member type that is not exactly Student must be treated as Public". An `else` with no condition cannot miss a case, which is what makes it the right shape for a catch-all rule, and it is why the answer never needs to know that "Staff" exists.

PART XV

Reference

Everything this course uses, in one place, so that the promise made on the first page holds on the last: you need nothing else. Every entry here is something the book itself uses, and everything the book uses is here. That is not a claim, it is a gate: a script compares this reference against every listing in the manual and fails if either half drifts.



The API card: what you must have cold

Every call below has appeared on a real COS221 paper. This is the short list. If you know only one page of this reference, know this one.

● MUST-MEMORISE · INPUT AND OUTPUT, THE LECTURER'S WAY

Every write-a-program question on both papers says **JOptionPane**. It is two calls and one import.

CALL	WHAT IT DOES, AND WHAT TO WATCH
<code>import javax.swing.JOptionPane;</code>	The import. javax, not java: the single commonest typo in the hall.
<code>JOptionPane.showInputDialog(null, "Enter age:");</code>	Asks, and returns what was typed as a String , always. Returns null if the user presses Cancel.
<code>JOptionPane.showMessageDialog(null, "Total: " + t)</code>	Shows a message. Returns nothing. The null is the parent window: there isn't one.
<code>System.exit(0)</code>	Ends the program. 0 means "ended normally". See F.11: it is insurance, not a rescue.

MEMORY HOOK: *showInput asks, showMessage tells*. Input comes back as text, so it always needs converting.

● MUST-MEMORISE · TEXT TO NUMBER, BECAUSE A DIALOG ONLY EVER GIVES YOU TEXT

CALL	WHAT IT DOES, AND WHAT TO WATCH
<code>Integer.parseInt(s)</code>	String to int. Throws <code>NumberFormatException</code> if <code>s</code> is not a whole number.
<code>Double.parseDouble(s)</code>	String to double. Same exception on nonsense.

The full shape, which you will write a dozen times on one paper:

```
int age = Integer.parseInt(JOptionPane.showInputDialog(null, "Age:"));
```

MEMORY HOOK: *the dialog hands you a String even when you asked for a number*. Every input is two steps: ask, then convert.

● MUST-MEMORISE · THE FILE I/O THE PAPER ALLOWS

The 2025/26 Question Three names **five** classes and permits nothing else: `FileWriter`, `FileReader`, `BufferedReader`, `BufferedWriter`, `IOException`.

CALL	WHAT IT DOES, AND WHAT TO WATCH
<code>new FileWriter("f.txt")</code>	Opens for writing. Creates the file if absent, and truncates it if present .
<code>new FileWriter("f.txt", true)</code>	The same, but appends . This second argument is the only way to append, and it belongs here, on the writer that touches the disk.
<code>new BufferedWriter(new FileWriter("f.txt"))</code>	Wraps a writer and writes in blocks. Gives you <code>newLine()</code> .
<code>new PrintWriter(new FileWriter("f.txt", true))</code>	Wraps a writer and gives you <code>printf()</code> and <code>println()</code> on a file. Note <code>new PrintWriter("f.txt")</code> on its own truncates and cannot append .
<code>new BufferedReader(new FileReader("f.txt"))</code>	Opens for reading, in blocks. Gives you <code>readLine()</code> .
<code>readLine()</code>	The next line, or null at the end of the file. That null is the loop's stopping condition.
<code>try (BufferedReader in = ...) { }</code>	try-with-resources. Closes it whether the block ends normally or by throwing. Nothing to forget.
<code>catch (IOException e)</code>	Checked: catch it or declare throws, or it will not compile.

MEMORY HOOK: Reader in, Writer out; Buffered wraps; Print formats. The one that touches the disk carries the append flag.

● MUST-MEMORISE · PRINTF, AND THE FOUR FORMATS THAT COVER EVERY PAPER

FORMAT	MEANS
<code>%d</code>	a whole number
<code>%f</code>	a decimal. <code>%.2f</code> means two decimal places.
<code>%s</code>	a string. <code>%-8s</code> means left-aligned in 8 columns.
<code>%n</code>	end the line. Use this, not <code>\n</code> .

`printf` rounds **the digits you wrote**, half up, so your paper arithmetic is trustworthy: `%.2f` of 1.005 really is 1.01. See MS.4, where that was measured against 99,999 values.

MEMORY HOOK: width before the point, precision after. `%-8.2f` is "left-aligned, eight wide, two decimals".

The method reference, by class

Every library call this book makes, grouped by the class it belongs to. If you met it in a listing, it is here.

TEACH · SYSTEM: PRINTING, AND TWO THINGS THAT ARE NOT PRINTING

CALL	WHAT IT DOES
<code>System.out.println(x)</code>	Prints x, then ends the line.
<code>System.out.print(x)</code>	Prints x and stays on the line .
<code>System.out.printf(fmt, ...)</code>	Formatted printing. Ends no line unless you write <code>%n</code> .
<code>System.exit(0)</code>	Stops the JVM at once. 0 means normal.
<code>System.arraycopy(src, a, dst, b, n)</code>	Copies n elements from src at a into dst at b. Note the lower-case c: <code>arrayCopy</code> does not exist.
<code>System.console()</code>	The terminal, or null when there isn't one. Returns a <code>Console</code> .
<code>System.in</code>	The keyboard, as a stream. Only ever seen inside <code>new Scanner(System.in)</code> .

TEACH · MATH: NO IMPORT NEEDED, AND EVERY METHOD IS STATIC

CALL	WHAT IT DOES
<code>Math.PI</code>	3.141592653589793. A constant, so no brackets.
<code>Math.abs(x)</code>	The size of x, sign removed.
<code>Math.max(a, b)</code>	The larger.
<code>Math.min(a, b)</code>	The smaller.
<code>Math.pow(a, b)</code>	a to the power b, always a double . See Unit 3.6: it is exact for the sizes this course uses.
<code>Math.sqrt(x)</code>	The square root, as a double. For a primality test prefer <code>d * d <= n</code> , which stays in integers.
<code>Math.round(x)</code>	Nearest whole number, halves going up. Returns long for a double.

TEACH · STRING: THE TEXT TYPE, AND IT NEVER CHANGES

Every method here **returns a new String**. None of them alters the one you called it on, because a String cannot be altered (Unit 7.1). If you do not keep the answer, nothing happened.

CALL	WHAT IT DOES
<code>length()</code>	How many characters. A method on a String, unlike an array's length.
<code>charAt(i)</code>	The character at index <code>i</code> , counting from 0.
<code>substring(from)</code>	From <code>from</code> to the end.
<code>substring(from, to)</code>	From <code>from</code> up to but not including <code>to</code> .
<code>indexOf(s)</code>	Where <code>s</code> starts, or -1 if it is absent.
<code>contains(s)</code>	<code>true</code> if <code>s</code> is in there.
<code>startsWith(s)</code>	<code>true</code> if it begins with <code>s</code> .
<code>endsWith(s)</code>	<code>true</code> if it ends with <code>s</code> .
<code>toUpperCase()</code>	A new String, in capitals.
<code>isEmpty()</code>	<code>true</code> only if there are no characters. A space is a character.
<code>concat(s)</code>	A new String with <code>s</code> joined on. The <code>+</code> operator does the same.
<code>replace(a, b)</code>	A new String with every <code>a</code> swapped for <code>b</code> .
<code>equals(s)</code>	Compares the characters . This is the one you want (Unit 7.3).
<code>equalsIgnoreCase(s)</code>	The same, treating "JAVA" and "java" as equal.
<code>compareTo(s)</code>	Negative, zero or positive, by dictionary order.
<code>new String("hi")</code>	Forces a brand new object. It exists to demonstrate why <code>==</code> is wrong for text, and for no other reason.

TEACH · STRINGBUILDER: THE TEXT TYPE THAT DOES CHANGE

The mirror image of String: every method here **alters the builder itself** and you do not keep a return value (Unit 7.4).

CALL	WHAT IT DOES
<code>new StringBuilder("Java")</code>	A builder holding "Java".
<code>append(x)</code>	Adds <code>x</code> to the end.
<code>insert(i, s)</code>	Pushes <code>s</code> in so it starts at <code>i</code> , sliding the rest right. Nothing is lost.
<code>delete(a, b)</code>	Removes from <code>a</code> up to but not including <code>b</code> .
<code>replace(a, b, s)</code>	Removes <code>a</code> to <code>b</code> , then puts <code>s</code> there. Can change the length.
<code>reverse()</code>	Turns it back to front.
<code>toString()</code>	The finished text, as a String.

TEACH · ARRAYS AND THE ARRAY ITSELF

CALL	WHAT IT DOES
Arrays	The toolkit class. <code>import java.util.Arrays;</code> Every method on it is static.
length	The size of an array. A field : <code>a.length</code> , never <code>a.length()</code> . The last index is always <code>length - 1</code> .
<code>Arrays.toString(a)</code>	The array as readable text. Printing <code>a</code> directly gives you a hash code.
<code>Arrays.sort(a)</code>	Sorts <code>a</code> in place . Returns nothing.
<code>Arrays.copyOf(a, n)</code>	A new array of length <code>n</code> , padded or trimmed.
<code>Arrays.equals(a, b)</code>	Compares the contents . <code>a == b</code> compares the references.
<code>Arrays.asList(x, y)</code>	The arguments as a list, for the one place this course needs one: <code>Files.write</code> .

The exam bans collection classes and asks for **parallel arrays** instead: two arrays of the same length where index `i` means the same subject in both.

TEACH · READING THE KEYBOARD: SCANNER, AND THE TWO ALTERNATIVES

CALL	WHAT IT DOES
Scanner	The class. <code>import java.util.Scanner;</code>
<code>new Scanner(System.in)</code>	A scanner on the keyboard.
<code>nextLine()</code>	The whole next line, as a String.
<code>nextInt()</code>	The next whole number. Leaves the newline behind , which is why a following <code>nextLine()</code> returns empty (Unit 3.10).
<code>hasNextLine()</code>	<code>true</code> while a line is waiting. Ask before you take, instead of comparing against a sentinel.
<code>close()</code>	Releases it. Every reader and writer has one.
<code>new InputStreamReader(System.in)</code>	Turns the raw keyboard stream into something a reader can read.
<code>new BufferedReader(new InputStreamReader(System.in))</code>	The older way to read the keyboard. Its <code>readLine()</code> returns a String.
Console	The class <code>System.console()</code> hands back. It is <code>null</code> in every IDE, which is the whole lesson of Unit 3.10.
<code>readPassword()</code>	On a Console: reads without echoing. Returns a <code>char[]</code> , not a String.

TEACH · FILES: THE CLASSIC WAY, WHICH THE PAPER DEMANDS

CALL	WHAT IT DOES
<code>new FileWriter(name)</code>	Opens for writing. Creates, or truncates .
<code>new FileWriter(name, true)</code>	Opens for appending .
<code>new FileReader(name)</code>	Opens for reading.
<code>new BufferedWriter(w)</code>	Wraps a writer, writing in blocks.
<code>new BufferedReader(r)</code>	Wraps a reader, reading in blocks.
<code>new PrintWriter(w)</code>	Wraps a writer and adds <code>printf</code> and <code>println</code> .
<code>new File(name)</code>	A file name , not a file. Creating one opens nothing.
<code>write(s)</code>	Writes text. On a <code>FileWriter</code> or <code>BufferedWriter</code> .
<code>newLine()</code>	Ends the line, on a <code>BufferedWriter</code> .
<code>read()</code>	One character, as an <code>int</code> , or -1 at the end of the file.

TEACH · FILES: THE MODERN ONE-LINERS, AND THE 24/25 QUESTION THAT NAMES THEM

CALL	WHAT IT DOES
<code>Paths.get("f.txt")</code>	A <code>Path</code> : a file name in the form the <code>Files</code> methods want.
<code>Files.write(path, lines)</code>	Writes every line and closes the file, in one call.
<code>Files.readAllLines(path)</code>	The whole file, as a list of lines.
<code>Files.readAllBytes(path)</code>	The whole file, as raw bytes.

The 2024/25 Question Three A asks you to define `Files.write()`. Learn these four, and then use the classic classes in Section B, because the 2025/26 paper bans everything but them.

TEACH · EXCEPTIONS, AND THE WRAPPERS THAT THROW THEM

NAME	WHAT IT IS
Exception	The parent of nearly all of them. <code>catch (Exception e)</code> catches everything below it, which is why it must come last , and why catching it instead of the specific one is a way of not saying what you expected to go wrong.
IOException	Checked. Anything went wrong with a file. Catch it or declare it.
FileNotFoundException	Checked. A subclass of <code>IOException</code> , so catch it first or it is unreachable.
NumberFormatException	Unchecked. <code>parseInt</code> met something that is not a number.
ArithmeticException	Unchecked. Integer division by zero. Decimal division by zero gives Infinity instead.
ArrayIndexOutOfBoundsException	Unchecked. An index below 0 or at length or above.
NullPointerException	Unchecked. A method was called on <code>null</code> .
<code>new IllegalArgumentException("why")</code>	Unchecked. Throw it yourself when an argument makes no sense.
<code>new ArithmeticException("why")</code>	Throwing one yourself, to show throw against throws.
<code>getMessage()</code>	The text carried by the exception.
<code>getClass()</code>	The object's class, at run time.
<code>getSimpleName()</code>	On a class: its name without the package, so <code>IOException</code> rather than <code>java.io.IOException</code> .

TEACH · THE WRAPPER CLASSES AND THEIR CONSTANTS

CALL	WHAT IT DOES
<code>Integer.parseInt(s)</code>	String to int.
<code>Double.parseDouble(s)</code>	String to double.
<code>Integer.MAX_VALUE</code>	2147483647. Use the constant, not the digits: your course's own table mistyped them.
<code>Integer.MIN_VALUE</code>	-2147483648.
<code>Long.MAX_VALUE</code>	9223372036854775807.
<code>Long.MIN_VALUE</code>	-9223372036854775808.
<code>Integer.toString(n)</code>	<code>n</code> written in binary, as text.

The keyword index

Every Java keyword this course uses, in one line each. These are reserved: you can never name a variable after one.

TEACH · TYPES

KEYWORD	WHAT IT MEANS
byte	A whole number, 8 bits, -128 to 127.
short	A whole number, 16 bits, -32768 to 32767.
int	A whole number, 32 bits. The default for any whole number you type.
long	A whole number, 64 bits. A big literal needs an L: 9000000000L.
float	A decimal, 32 bits. A literal needs an f: 3.14f.
double	A decimal, 64 bits. The default for any decimal you type.
char	One character, in single quotes: 'J'.
boolean	true or false, and nothing else. Not 0 and 1.
true	One of the two boolean values.
false	The other.
null	"Refers to no object." Not zero, and not an empty string.
void	As a return type: this method gives nothing back.

TEACH · CONTROL FLOW

KEYWORD	WHAT IT MEANS
if	Run the block only when the condition is true.
else	Otherwise. It arrives already knowing what failed.
switch	Choose a branch by matching a value against cases.
case	One value a switch matches.
default	The switch branch for everything unmatched.
for	A counting loop: init, test and update on one line.
while	A loop that tests before the body, so it can run zero times.
do	With while: a loop that tests after , so it always runs once.
break	Leaves the loop at once. In a switch, ends that case.
continue	Skips the rest of this turn and goes to the next. It does not leave the loop.
return	Leaves the method, carrying a value unless the method is void.

TEACH · CLASSES, OBJECTS AND ACCESS

KEYWORD	WHAT IT MEANS
class	Declares a class: the blueprint an object is made from.
new	Makes an object and runs its constructor.
this	The object the method was called on. Used to tell a field from a parameter of the same name.
static	Belongs to the class , not to any object. Called as <code>ClassName.method()</code> .
final	Cannot be changed after it is set. The way to write a constant.
public	Visible everywhere.
private	Visible only inside this class. Not even a subclass can name it.
protected	Visible inside this class and its subclasses.
extends	This class inherits from that one.
super	The superclass part of this object: <code>super(...)</code> calls its constructor, <code>super.m()</code> its method.
abstract	Incomplete on purpose: it cannot be instantiated, and a subclass must finish it.
interface	A contract of method signatures with no bodies.
implements	This class promises to supply an interface's methods.
import	Names a class from another package so you can use its short name.

TEACH · EXCEPTIONS

KEYWORD	WHAT IT MEANS
try	Guard this block. With brackets, it also closes what it opened.
catch	Handle an exception of the named type. Catch subclasses first.
finally	Runs whatever happens, exception or not.
throw	Raises an exception now: <code>throw new IllegalArgumentException("...")</code> .
throws	Declares that this method may raise one, and refuses to handle it.

MEMORY HOOK: **throw does, throws warns.** The s is a signature, not an action.

⚠ TRAP · THE KEYWORDS THIS COURSE DOES NOT USE ARE STILL RESERVED

Java has fifty reserved words. This book uses the forty-two above, because they are the ones your course teaches and your papers examine. The rest are real and you may not use them as names either:

`assert, const, enum, goto, instanceof, native, package, strictfp, synchronized, transient, volatile.`

Two of those are curiosities worth ten seconds: `const` and `goto` are reserved and **do nothing at all**.

They exist only so that the compiler can give you a proper error instead of a confusing one if you arrive from a language that has them. Java's constant is `final`.

They are listed here rather than tabled above for one reason: **this book documents what it uses**. An entry claiming to teach you `synchronized` would be a promise no page of this manual keeps.

The glossary

The vocabulary, defined once. Both papers open with "define" and "describe" questions, and those are the cheapest marks on either paper: they are the only ones you can be certain of before you walk in.

TEACH · THE MACHINE AND THE LANGUAGE

TERM	DEFINITION
Source code	The Java you type, in a .java file. Text, readable by people.
Compiler (javac)	Translates source into bytecode, and refuses if the source is not legal Java.
Bytecode	The .class file. Not machine code, and not tied to any one processor.
JVM	The Java Virtual Machine: the program that runs bytecode. The reason Java runs anywhere.
JDK	The kit: compiler, JVM, and libraries. You need it to write Java, not merely to run it.
Compile-time error	The code is not legal Java. Nothing runs. The compiler tells you the line.
Run-time error	Legal code that hits something impossible while running. Java throws an exception.
Logic error	It compiles, it runs, it finishes, and the answer is wrong. Nothing warns you. Only a dry run finds it.
Dry run	Executing a program by hand, one row per step, writing down every variable as it changes. Your lecturer's own term, and an exam skill.

TEACH · DATA

TERM	DEFINITION
Variable	A named box holding one value of a declared type.
Primitive type	One of the eight built-in types. The variable holds the value itself.
Reference type	Everything else. The variable holds a reference to an object stored elsewhere.
Literal	A value written directly in the code: 42, 3.14, "hi". Its type is fixed when you type it, before anything is assigned.
Constant	A final variable: set once, never changed.
Casting	Telling the compiler to convert a type, taking responsibility for the loss: (int) 3.9 is 3.
Widening	A smaller type flowing into a bigger one automatically, losing nothing: an int into a double.
Narrowing	The reverse, which loses data, so Java never does it silently.
Integer division	/ between two whole numbers, which chops the remainder away. 7 / 2 is 3. It does not round.
Array	A fixed-length run of values of one type, reached by index from 0 .
Index	An element's position. The last is always length - 1.
Parallel arrays	Two or more arrays of equal length where the same index means the same subject. This course's substitute for a collection.

TEACH · METHODS

TERM	DEFINITION
Method	A named, reusable block that may take values in and may give one back.
Formal parameter	The variable named in the method's declaration. A name waiting for a value.
Actual argument	The value written at the call, which is copied into the parameter.
Method signature	The name plus the parameter list . The return type is not part of it.
Return type	The type of what comes back, or void for nothing.
Overloading	Several methods, same name, different parameter lists . Resolved at compile time . Compile-time polymorphism.
Overriding	A subclass replacing an inherited method with the same signature. Resolved at run time .
Pass by value	Java's only mechanism. The value is copied. For a reference type, the reference is copied, so the method can change the object but cannot make your variable point elsewhere.
Recursion	A method that calls itself, with a base case that stops it.

TEACH · OBJECTS

TERM	DEFINITION
Class	The blueprint: what an object of this kind knows and can do.
Object	One thing made from that blueprint, with its own copy of the state.
Attribute (field)	A variable belonging to an object: what it knows .
Constructor	Runs once, at new, to put a new object into a valid state. Named exactly like the class, and declares no return type at all .
Instance method	Belongs to an object. Called on one: <code>account.deposit(500)</code> .
Static method	Belongs to the class. Called on it: <code>Math.abs(-3)</code> .
Encapsulation	Keeping state private and exposing it through methods, so an object controls its own data.
Inheritance	A subclass acquiring the superclass's members with <code>extends</code> . An is-a relationship.
Polymorphism	One name, several behaviours. Compile-time is overloading; run-time is overriding.
Dynamic method dispatch	The run-time half: the object decides which override runs, not the reference's declared type.
Abstraction	Exposing what a thing does and hiding how. In Java: abstract classes and interfaces.

TEACH · EXCEPTIONS AND FILES

TERM	DEFINITION
Exception	An object describing something that went wrong, thrown at the point of failure.
Checked exception	The compiler insists you catch it or declare it. <code>IOException</code> . A fact about the world, not a bug.
Unchecked exception	Not policed. <code>NullPointerException</code> . Usually a bug, so the fix is to correct the code, not to catch it.
try-with-resources	<code>try (Reader r = ...)</code> : closes what it opened, on every path, including the throwing one.
Stream	A one-way flow of data. A reader reads one; a writer writes one.
Buffering	Collecting data and moving it in blocks rather than one character at a time.
Sentinel	A value that means "stop", chosen so it cannot be real data: <code>readLine()</code> gives <code>null</code> , <code>read()</code> gives <code>-1</code> .

● MUST-MEMORISE · THE FIVE DEFINITIONS BOTH PAPERS KEEP ASKING FOR

Across 2024/25 and 2025/26, these came up again and again. If you learn five definitions, learn these, in these words:

- 1. Method signature:** the method's **name plus its parameter list**. The return type is not part of it.
- 2. Overloading against overriding:** same name different parameters, settled at compile time; against same signature in a subclass, settled at run time.
- 3. Checked against unchecked:** the compiler forces you to handle a checked one, or it will not compile.
- 4. Encapsulation:** private state, public methods, so the object owns its own data.
- 5. Dry run:** hand-execute the code, one row per step.

MEMORY HOOK: every one of these is a **distinction**, not a fact. The mark is in the contrast, so answer in two halves and say which is which.

TEACH · THAT IS THE WHOLE COURSE

You have the machine underneath, the ten modules, both real papers solved, two mocks sat and marked, and this reference. Nothing in this book was taken on trust: every listing was compiled and run, every printed output came off a real JVM, and every claim about what the compiler says was made by asking it.

Where your course materials and the compiler disagreed, this book says so and shows you the run. That is not disrespect. It is the only habit that matters in programming: **the machine is the authority, and it will tell you, and you can ask it any time you like.**

Go and ask it. Then go and pass.